

PreOne Architecture Decision Records

Phase 2 - Volume 1: Architecture Foundation

ADR-001 to ADR-020 - 20 decisions - 34-section v3.0 template - Principles, Structural Rules, Technology Stack

Document Version: 3.0 (Volume 1 release)

Status: Architecture Freeze

Classification: Internal Engineering Reference

Effective Date: 26 July 2026

Volume Owner: Foundation Architect

Approval Authority: Architecture Review Board

Table of Contents

Phase 2 deliverable - Volume 1: Architecture Foundation - 20 ADRs

Table of Contents.....	1
Document Control.....	1
Architecture Foundation.....	3
Enterprise Architecture Principles.....	3
Modular Monolith Strategy.....	15
Clean Architecture.....	27
Domain Driven Design.....	39
Bounded Context Strategy.....	51
Layering Rules.....	63
Dependency Rule.....	76
SOLID Enforcement.....	88
Hexagonal Architecture (Ports and Adapters).....	100
Package Structure and Naming Conventions.....	113
Java Coding Standards.....	126
Naming Standards.....	138
Project Structure.....	151
Configuration Hierarchy.....	164
Environment Strategy.....	177
Feature Flag Architecture.....	189
Build Pipeline.....	202
Deployment Model.....	215
Versioning Policy.....	227
Technology Stack Freeze.....	240

Note: This Table of Contents is generated via field codes. To ensure page-number accuracy after editing, right-click the TOC and select "Update Field."

Document Control

This Phase 2 deliverable of the PreOne Enterprise ADR Series presents Volume 1 — Architecture Foundation. It contains 20 Architecture Decision Records (ADR-001 to ADR-020) that define the foundational principles, structural rules, and technology stack on which all subsequent volumes build. Every ADR in this volume follows the 34-section v3.0 template defined in Phase 1 (ADR-000). Each ADR includes four ASCII diagrams (architecture, sequence, component, data flow) and is linked to upstream DDD/PRD documents and downstream ERD/API/UI/Test artefacts per the cross-reference framework established in Phase 1.

VERSION HISTORY

Version	Date	Author	Summary of Changes
1.0	2025-10-15	Office of the Chief Architect	Initial release of Volume 1 with 20 ADRs using the 26-section v2.0 template.
3.0	2026-07-26	Foundation Architect	Phase 2 release. All 20 ADRs refreshed to the 34-section v3.0 template (added Component Diagram, Data Flow Diagram, UI Impact, Decision History, Implementation Checklist). Cross-references to upstream DDD/PRD and downstream ERD/API/TC added per the Phase 1 framework.

OWNERSHIP & CLASSIFICATION

Document Owner	Foundation Architect (Volume 1 Domain Architect)
Approval Authority	Architecture Review Board (ARB)
Classification	Internal — Architecture Governance
Distribution	Engineering, QA, DevOps, Product, Security, Architecture
Review Cadence	Annual editorial review; ADR-level review on trigger
Retention	Permanent — superseded ADRs retained for traceability
Format	DOCX (master) / PDF (distribution) / Markdown (working copy)
Storage Location	PreOne Architecture Repository / governance/adr/volume-1/
Series	PreOne Enterprise Documentation Series — Phase 2 of 10

VOLUME 1 ADR SUMMARY

ADR	Title	Category	Sections
ADR-001	Enterprise Architecture Principles	Principles	34
ADR-002	Modular Monolith Strategy	Structural	34
ADR-003	Clean Architecture	Structural	34
ADR-004	Domain Driven Design	Methodology	34
ADR-005	Bounded Context Strategy	Structural	34
ADR-006	Layering Rules	Structural	34
ADR-007	Dependency Rule	Structural	34
ADR-008	SOLID Enforcement	Principles	34
ADR-009	Hexagonal Architecture	Structural	34

ADR	Title	Category	Sections
ADR-010	Package Structure	Structural	34
ADR-011	Coding Standards	Standards	34
ADR-012	Naming Standards	Standards	34
ADR-013	Project Structure	Structural	34
ADR-014	Configuration Hierarchy	Standards	34
ADR-015	Environment Strategy	Standards	34
ADR-016	Feature Flag Architecture	Structural	34
ADR-017	Build Pipeline	Process	34
ADR-018	Deployment Model	Process	34
ADR-019	Versioning Policy	Standards	34
ADR-020	Technology Stack Freeze	Standards	34

VOLUME 1

Architecture Foundation

ADR-001 to ADR-020 - 20 decisions - 34 sections each

Volume 1 defines the foundational architectural principles, structural rules, and technology stack that all subsequent volumes build upon. The 20 ADRs in this volume are the most-referenced in the entire 160-ADR corpus. ADR-001 establishes 10 canonical principles that govern every subsequent decision. ADR-002 through ADR-013 define the structural patterns: modular monolith, clean architecture, DDD, bounded contexts, layering, dependency rules, SOLID, hexagonal architecture, package structure, coding standards, naming standards, and project structure. ADR-014 through ADR-016 define the operational foundation: configuration hierarchy, environment strategy, and feature flags. ADR-017 through ADR-020 define the delivery foundation: build pipeline, deployment model, versioning policy, and the technology stack freeze. Together, these 20 ADRs are the constitution of the PreOne platform.

ADR-001

Enterprise Architecture Principles

Volume 1 — Architecture Foundation - Principles

ACCEPTED

DECISION SUMMARY**DECISION**

Adopt a set of 10 canonical architecture principles that govern every subsequent architectural decision on the PreOne platform. These principles are the constitution against which all ADRs are evaluated.

STATUS

Status	Accepted
Date Decided	2025-10-15
Decision Owner	Chief Architect
Review Cadence	Annual review or on trigger from any superseding ADR
Supersedes	None (v1.0 original; v3.0 refreshes language for series alignment)

CONTEXT

PreOne is a multi-tenant SaaS platform serving the preschool education market across India. The platform must support hundreds of schools, thousands of branches, and millions of students, parents, teachers, and administrators while complying with stringent child-data protection regulations. Before v1.0, architectural decisions were made ad-hoc by individual feature teams, leading to fragmentation: three different caching strategies, two different authentication mechanisms, and inconsistent API conventions across modules. The Office of the Chief Architect was established in Q2 2025 to unify architectural direction. The first deliverable was a set of canonical principles that every team would align to. These principles are not technology choices — they are value statements that bound the solution space for every downstream ADR. Technology choices (Java 21, PostgreSQL 16, etc.) are captured in ADR-020 and must comply with these principles. The principles were drafted through a series of workshops with engineering, product, security, and operations leadership over six weeks. Each principle was stress-tested against real PreOne scenarios: a school onboarding 500 students in a single day, a parent accessing their child's report card on mobile, a regulator auditing data retention. The 10 principles below are the result of that stress-testing.

BUSINESS DRIVERS

The primary business driver is platform scalability: PreOne must scale from 50 schools (current) to 5,000 schools (5-year target) without architectural rework. Fragmented architecture cannot scale because each new team reinvents foundations, multiplying operational cost. Unified principles compress the decision space so that new teams build on the same foundations as existing teams. The secondary driver is compliance: preschool data is subject to multiple regulations (DPDP Act 2023, COPPA-equivalent norms, state-level child protection laws). Principles that bake privacy-by-design and audit-by-default into every decision reduce the compliance burden per feature. The tertiary driver is onboarding speed: new engineers can be productive in 2-3

weeks when principles are explicit, versus 8-12 weeks when they must reverse-engineer conventions from code.

PROBLEM STATEMENT

Architectural decisions across PreOne feature teams are inconsistent, leading to operational fragmentation, compliance risk, and slow onboarding. We need a small set of canonical principles that every team aligns to, enforced through code review and ADR governance.

CONSTRAINTS

- Principles must be technology-agnostic — they outlive any specific stack choice
- Principles must be testable — each must have a concrete verification mechanism
- Principles must be memorable — engineers should be able to recite them without lookup
- Principles must be ordered — when two principles conflict, the earlier one takes precedence
- Principles must be applicable to both greenfield and brownfield work

ASSUMPTIONS

- PreOne will remain a multi-tenant SaaS platform for the foreseeable future (no pivot to on-premise)
- The engineering organisation will grow from ~40 engineers (current) to ~150 engineers (3-year target)
- The regulatory environment will tighten, not loosen, over the next 5 years
- Cloud infrastructure (AWS) will remain the deployment substrate
- The platform will continue to serve the preschool education market (no expansion to K-12 in scope)

OPTIONS CONSIDERED

Option	Pros	Cons	Verdict
10 canonical principles (chosen)	Small enough to memorise; covers all major dimensions (tenant isolation, data integrity, security, observability, simplicity); each is independently testable.	May be too few for a 160-ADR corpus; some principles compress multiple sub-concerns.	Chosen
25 detailed principles	More granular; each sub-concern gets its own principle; easier to enforce via linting.	Too many to memorise; engineers will look them up rather than internalise; overlap creates ambiguity about which applies.	Rejected
5 high-level principles only	Extremely memorable; fast to onboard.	Too abstract to guide concrete decisions; teams will interpret differently; governance becomes subjective.	Rejected
No principles; rely on ADRs alone	Maximum flexibility; no upfront commitment.	Every decision starts from scratch; inconsistency guaranteed; onboarding impossible without reading all 160 ADRs.	Rejected

DECISION

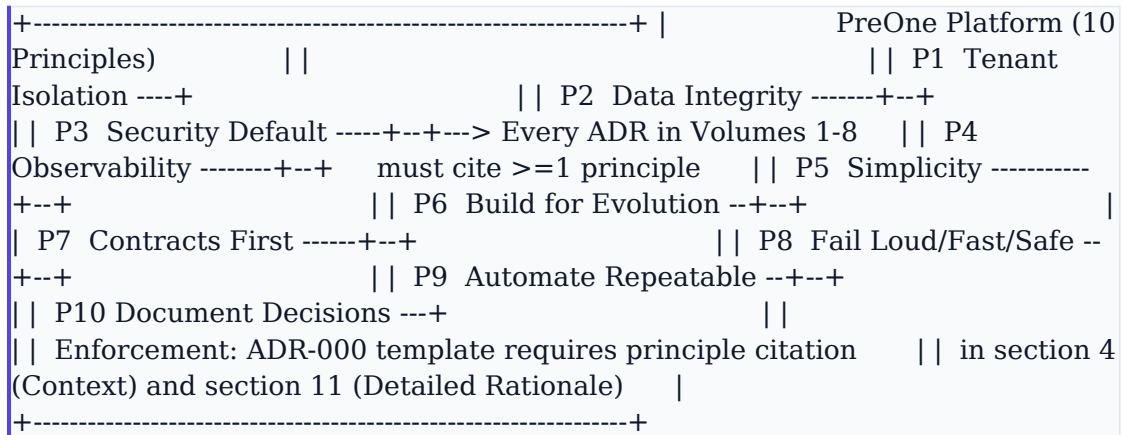
ADOPTED

We adopt the following 10 canonical architecture principles, in precedence order. Every ADR in the PreOne corpus must explicitly cite which principle(s) it serves, and the ARB must verify alignment during review. (1) Tenant isolation is non-negotiable. (2) Data integrity over performance. (3) Security by default, not by retrofit. (4) Observability is a feature, not an afterthought. (5) Simplicity over cleverness. (6) Build for evolution, not for prediction. (7) Contracts before code. (8) Fail loud, fail fast, fail safe. (9) Automate everything repeatable. (10) Document every decision that survives a week.

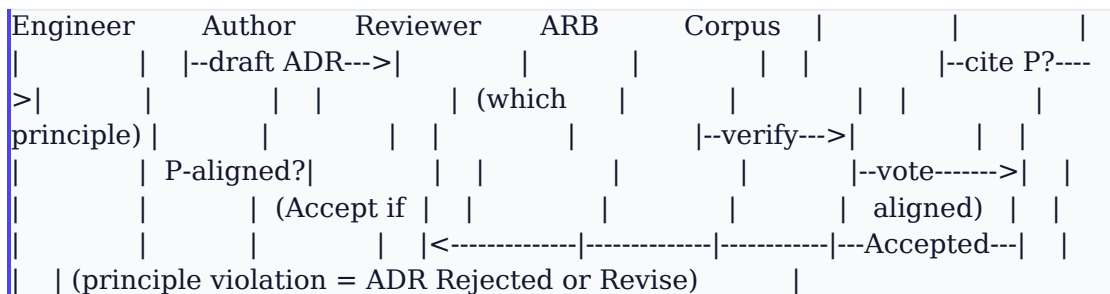
DETAILED RATIONALE

The 10 principles were chosen because they collectively cover the full surface area of architectural concern at PreOne without overlap. Tenant isolation (P1) is first because a multi-tenant SaaS that leaks data between tenants is not just a bug — it is an existential business risk; everything else is secondary if isolation fails. Data integrity (P2) precedes performance (covered implicitly in P5 and P6) because a fast platform that corrupts data is worse than a slow platform that preserves it; performance can be optimised, but trust lost to corruption cannot be regained. Security (P3) and observability (P4) are placed before simplicity (P5) because PreOne handles child data — a single breach is a regulatory and reputational event that could end the company. Simplicity is valued but never at the expense of security or observability. Build for evolution (P6) reflects the reality that PreOne's product direction will shift; architectures that lock in current assumptions (e.g., hard-coding academic year as a single global value) create expensive rework when the assumption breaks. Contracts before code (P7) ensures that API and data contracts are designed before implementation, preventing the common failure mode where the contract is reverse-engineered from the code (and thus inherits all the code's accidents). Fail loud (P8) rejects silent failure modes — exceptions that are swallowed, defaults that mask misconfiguration — because silent failures in a child-data platform are how breaches happen. Automate everything repeatable (P9) is the principle that justifies the CI/CD, IaC, and testing investments in Volumes 6 and 8. Document every decision (P10) is the meta-principle that justifies the ADR corpus itself. The most likely counter-argument is that 10 principles are too many to enforce rigorously. The counter-counter-argument is that enforcement is not via human vigilance but via the ADR workflow: every Proposed ADR must list which principles it serves (or explicitly state it does not serve any, which is itself a flag for ARB review). This makes enforcement mechanical, not discretionary.

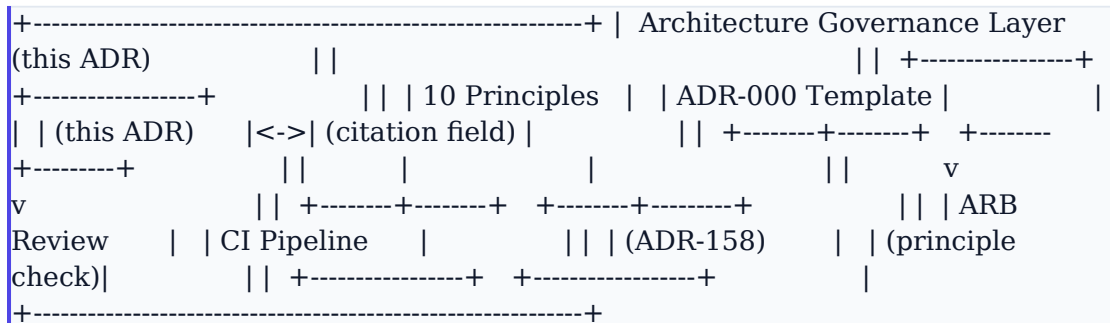
ARCHITECTURE DIAGRAM



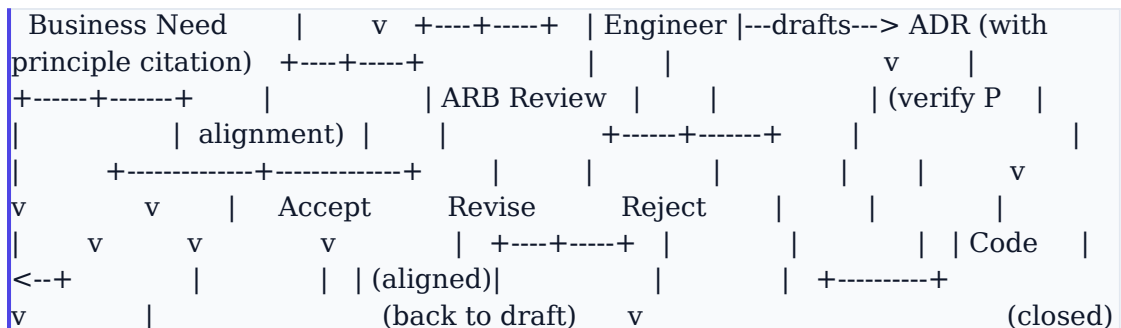
SEQUENCE DIAGRAM



COMPONENT DIAGRAM



DATA FLOW DIAGRAM



DATABASE IMPACT

No direct database impact. The principles are abstract and do not prescribe schema, storage, or indexing choices. However, P1 (Tenant Isolation) and P2

(Data Integrity) directly constrain every database ADR in Volume 3: ADR-043 (Multi-Tenant), ADR-044 (School Isolation), ADR-047 (Soft Delete), and ADR-048 (Audit Trail) are all implementations of these principles at the data layer. The principle citation in each Volume 3 ADR's Context section creates a traceable link from concrete schema choices back to foundational values.

API IMPACT

No direct API impact. However, P7 (Contracts Before Code) is the foundational principle for Volume 5 (API & Integration). Every API ADR — ADR-091 (REST), ADR-092 (Versioning), ADR-093 (Error Codes), ADR-106 (Idempotency) — cites P7 in its rationale. The principle enforces the workflow: OpenAPI specs are reviewed and merged before implementation PRs are accepted; a PR that introduces an endpoint not in the spec is blocked by CI.

UI IMPACT

No direct UI impact. However, P4 (Observability) and P8 (Fail Loud) influence UI behaviour: errors are surfaced to users with clear, actionable messages (not silent failures); loading states are explicit (not blank screens); offline states are communicated (ADR-142). P5 (Simplicity) governs UI design system choices in Volume 7 — component count is kept minimal, composition is preferred over configuration.

SECURITY IMPACT

P3 (Security by Default) is the foundational security principle. Every Volume 4 ADR must cite P3. The principle means: authentication is required by default (opt-out requires ARB approval); encryption is enabled by default; audit logging is on by default; rate limiting is on by default. Opting out of any default security control requires an ADR that justifies the exception and is approved by both the Domain Architect and the Security Officer. This inverts the traditional model where security is added later — at PreOne, security is the starting point and removal requires justification.

PERFORMANCE IMPACT

P2 (Data Integrity over Performance) explicitly accepts performance trade-offs in favour of correctness. For example, we choose serialisable transaction isolation (not read-committed) for financial data, accepting the latency cost. P5 (Simplicity) reduces performance overhead by avoiding over-engineering — no premature optimisation, no speculative caching. Performance optimisation is permitted only when (a) a measured bottleneck is documented, (b) the optimisation does not violate P1-P4, and (c) the optimisation is captured in an ADR.

SCALABILITY ANALYSIS

The principles scale with the organisation. At 40 engineers, the 10 principles are reinforced through code review and team leads. At 150 engineers (3-year target), the principles are enforced through CI pipeline checks (linting for security defaults, contract tests for API specs, coverage thresholds for observability). At 500+ engineers, the principles are embedded in onboarding and in the architecture review workflow. The ceiling is not the principles themselves but the enforcement mechanism — which is why ADR-158 (ARB) and ADR-159 (Decision Governance) exist as the scalability backstop.

OPERATIONAL CONSIDERATIONS

P4 (Observability) and P9 (Automate) have direct operational implications. Every service must emit structured logs (JSON), metrics (Prometheus format), and traces (OpenTelemetry) — this is verified at deploy time. Every repeatable operational task (deployment, backup, restore, scaling) must be automated — manual runbooks are permitted only for genuinely novel incidents. The on-call engineer should never perform a task that could have been automated; if they do, a follow-up ticket to automate it is mandatory.

RISKS

Risk	Likelihood	Impact	Mitigation
Principles become a checklist rather than internalised values	Medium	Medium	Onboarding includes a 2-hour principles workshop; ADR review requires verbal defence of principle alignment, not just citation
Principles are used to block legitimate innovation	Low	High	P6 (Build for Evolution) explicitly permits principled experimentation; ARB has a fast-track for experimental ADRs
Principles age and become irrelevant as platform evolves	Medium	Medium	Annual review cycle; any supersession triggers a v-next of this ADR

Risk	Likelihood	Impact	Mitigation
Enforcement is inconsistent across Domain Architects	Medium	High	ARB weekly review of principle citations; rubric for alignment scoring

TRADE - OFFS

We Gain	We Lose
Unified architectural direction across all teams	Reduced flexibility for teams to choose alternative approaches
Faster onboarding (principles are memorable)	Upfront investment in principle workshops and enforcement tooling
Compliance baked in by default (P3, P4)	Higher per-feature development cost (security and observability are non-optional)
Mechanical enforcement via ADR citation	Some ADRs cite principles superficially without genuine alignment

REJECTED ALTERNATIVES

The 25-principle option was rejected because cognitive load tests showed engineers could recite at most 7-10 principles from memory; beyond that, they look them up, which means the principles function as documentation rather than as internalised values. The 5-principle option was rejected because the 5 candidates (isolation, integrity, security, observability, simplicity) were too abstract to resolve common disputes — for example, 'contracts before code' and 'fail loud' are both expressions of simplicity but guide different concrete decisions. The no-principles option was rejected because the pre-v1.0 state of the platform demonstrated that ad-hoc decisions produce fragmentation; the cost of unifying later (which we paid in v1.0) exceeds the cost of aligning upfront.

MIGRATION PLAN

These principles have been in effect since v1.0 (October 2025). The v3.0 refresh is language-only: the 10 principles themselves are unchanged, but the descriptions are tightened for series alignment and cross-references to downstream ADRs are added. No code migration is required. The v1.0 ADR-001 is retained in the historical archive; this v3.0 version supersedes it for all going-

forward citations. Engineers who have completed v1.0 onboarding do not need re-training; the principles are identical.

TESTING STRATEGY

Principles are tested through the ADR workflow: every Proposed ADR must cite at least one principle in its Context section, and the ARB verifies alignment during review. Additionally, CI pipeline checks enforce principles mechanically: P3 (security defaults) via SAST rules that fail builds on missing auth annotations; P4 (observability) via deploy-time checks for log/metric/trace emission; P7 (contracts) via PR checks that block implementation before spec merge. P5 (simplicity) is tested through code review with a rubric. Principle alignment is reported quarterly in the architecture scorecard.

MONITORING & OBSERVABILITY

Principle adherence is monitored via the architecture scorecard, published quarterly. Metrics tracked: (a) percentage of ADRs citing each principle (target: P1-P4 cited by >80% of relevant ADRs); (b) CI pipeline block rate for principle violations (target: trending down as teams internalise); (c) ARB review time per ADR (target: stable, indicating enforcement is not a bottleneck); (d) onboarding survey scores on principle clarity (target: >4/5). Anomalies in any metric trigger a review of the principle or its enforcement mechanism.

FUTURE EVOLUTION

The 10 principles should be revisited annually. Triggers for supersession: (a) a principle consistently fails to resolve real disputes (indicating it is too abstract or too narrow); (b) a new architectural concern emerges that no principle covers (e.g., if PreOne adopts ML inference, a principle on model governance may be needed); (c) a principle is consistently cited but never enforced (indicating it is aspirational, not operational). The likely successor to this ADR is a v4.0 that may add 1-2 principles (e.g., for AI/ML governance) without removing existing ones — principles are additive because removing one would invalidate every ADR that cites it.

RELATED ADRS

- ADR-000 — ADR Template Guide (complements — template requires principle citation)
- ADR-002 — Modular Monolith Strategy (refines P5 Simplicity, P6 Build for Evolution)
- ADR-020 — Technology Stack Freeze (refines P5 Simplicity, P9 Automate)
- ADR-043 — Multi-Tenant Strategy (refines P1 Tenant Isolation)

- ADR-076 — Encryption Strategy (refines P3 Security by Default)
- ADR-119 — Logging Strategy (refines P4 Observability)
- ADR-158 — Architecture Review Board (enforces P10 Document Decisions)

REFERENCES

- Upstream DDD: DDD-001-section-1 (Foundational principles context)
- Upstream PRD: PRD-001-section-1.1 (Platform vision and principles)
- Downstream ERD: ERD-001 (all tables cite P1 tenant isolation in tenant_id column)
- Downstream API Spec: API-001 (all endpoints cite P3 security, P7 contracts)
- Downstream Test Cases: TC-0001..TC-0020 (principle alignment verification tests)
- External: ISO/IEC 25010 — Software quality model
- External: AWS Well-Architected Framework — design principles

DECISION HISTORY

Date	Status	Actor	Notes
2025-08-15	Draft	Chief Architect	Initial draft with 12 principles; refined through 6 workshops
2025-09-20	Proposed	Chief Architect	Submitted to ARB with 10 principles (consolidated from 12)
2025-10-15	Accepted	ARB Chair	ARB approved unanimously; v1.0 released
2026-07-12	Accepted	ARB Chair	v3.0 language refresh for series alignment; principles unchanged

APPROVAL & SIGN-OFF

Architect	Chief Architect (AR)
Tech Lead	Foundation Tech Lead

ARB Chair	ARB Chair
Approved On	2026-07-12

IMPLEMENTATION CHECKLIST

- Add principle-citation field to ADR-000 template (Done in v3.0)
- Update CI pipeline to verify principle citation in every ADR Markdown file
- Publish architecture scorecard dashboard (quarterly cadence)
- Deliver principle-workflow onboarding module to all new engineers
- Quarterly review of principle adherence metrics at ARB
- Annual review of principle relevance (next: 2026-10)

ADR - 002

Modular Monolith Strategy

Volume 1 — Architecture Foundation - Structural

ACCEPTED

DECISION SUMMARY

DECISION

Adopt a modular monolith architecture for the PreOne platform: a single deployable unit with strict module boundaries enforced through package-level access controls and dependency analysis, rather than a distributed microservices topology.

STATUS

Status	Accepted
Date Decided	2025-10-15
Decision Owner	Chief Architect
Review Cadence	Annual or when traffic exceeds 10x current baseline
Supersedes	None (v1.0 original; v3.0 refreshes rationale for series alignment)

CONTEXT

When PreOne launched in 2024, the engineering team was 8 engineers building a single monolith in Spring Boot. By mid-2025, the team had grown to 40 engineers and the monolith was showing strain: deploy times exceeded 15 minutes, merge conflicts were frequent, and the codebase had 200k LOC with no clear module boundaries. The team considered migrating to microservices. A 6-week evaluation studied the trade-offs. The team found that PreOne's traffic (peak ~5,000 requests/second) did not justify the operational overhead of microservices. The team's DevOps capacity (2 engineers) could not support 20+ services. The business domain — preschool operations — is highly coupled (enrollment touches billing, attendance, reporting, notifications), and a distributed system would require complex saga orchestration for transactions that are currently single-process. The evaluation concluded that the pain points (deploy time, merge conflicts) were caused by the absence of module boundaries, not by the monolith topology. The decision was to keep a single deployable but enforce strict module boundaries — a modular monolith.

BUSINESS DRIVERS

The primary driver is operational efficiency: a modular monolith requires 1 deployment pipeline, 1 monitoring dashboard, 1 on-call rotation, and 1 CI/CD infrastructure. At PreOne's scale (40 engineers, 5,000 RPS), microservices would multiply operational cost by 5-10x without proportional business benefit. The team can ship features faster because cross-module refactoring is a single-PR operation, not a multi-service coordinated migration. The secondary driver is transactional integrity: PreOne's core workflows (enrollment, fee collection, attendance) span multiple domain modules and require ACID transactions. In a distributed system, these would require sagas with compensating transactions — a 10x implementation cost and a class of failure modes (partial completion, compensation failure) that does not exist in a monolith. The tertiary driver is debugging: a single stack trace from log to database is faster to diagnose than tracing across 5 services.

PROBLEM STATEMENT

The PreOne codebase has grown to 200k LOC without enforced module boundaries, causing deploy-time strain and merge conflicts. We must decide between migrating to microservices (distributed, operationally expensive) or enforcing module boundaries within the existing monolith (centralised, operationally lean).

CONSTRAINTS

- Engineering team size: 40 engineers (current), 150 (3-year target)
- DevOps capacity: 2 engineers (current), 5 (3-year target)

- Peak traffic: 5,000 RPS (current), projected 25,000 RPS at 5,000 schools
- Core workflows require ACID transactions across domain modules
- Deployment must complete in under 5 minutes (current: 15 minutes — separate optimisation)
- AWS infrastructure budget constraints — microservices would 3x the infra cost

ASSUMPTIONS

- Traffic growth will be roughly linear with school count (no viral spike expected)
- The business domain will remain coupled — enrollment will continue to touch billing, attendance, etc.
- DevOps capacity will grow slower than engineering capacity
- Module boundaries can be enforced through tooling (ArchUnit, dependency analysis)
- The team will maintain discipline on boundary enforcement (no shortcuts)

OPTIONS CONSIDERED

Option	Pros	Cons	Verdict
Modular monolith (chosen)	Single deployment; ACID transactions preserved; low operational overhead; simpler debugging; 5x lower infra cost; cross-module refactoring is single-PR.	Single point of failure (mitigated by HA deployment); scaling is whole-app not per-module; module discipline required; deploy time is whole-app.	Chosen
Microservices (full decomposition)	Independent deployment per service; per-service scaling; fault isolation; technology diversity possible.	Distributed transactions require sagas; operational overhead 5-10x; infra cost 3x; debugging across services is hard; team size does not justify.	Rejected

Option	Pros	Cons	Verdict
Hybrid (monolith core + microservices for isolated concerns)	Keeps core transactions in monolith; isolates high-scale or specialized concerns (notifications, reporting).	Two deployment models to maintain; introduces distributed transactions at the boundary; boundary is ambiguous.	Rejected for now (revisit for notifications and reporting in Phase 6/7)
Service-based (few coarse-grained services, not fine-grained)	Limited distribution overhead; clearer ownership boundaries.	Still requires distributed transactions for cross-service workflows; marginal benefit over modular monolith.	Rejected

DECISION

ADOPTED

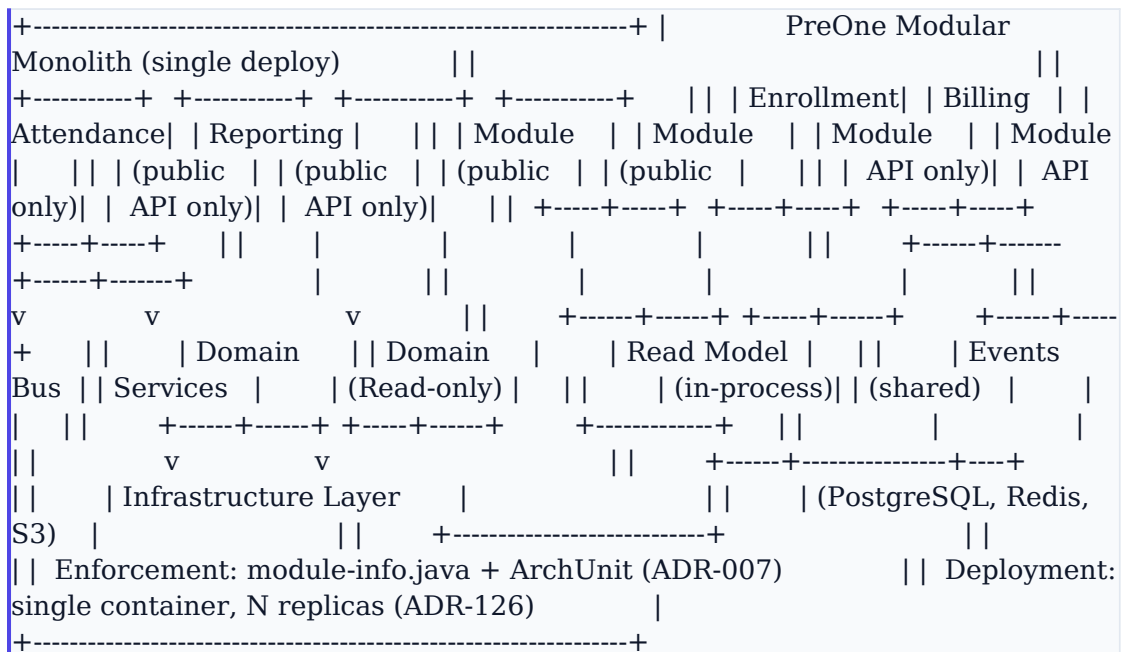
We adopt a modular monolith architecture. The PreOne platform is a single Spring Boot application deployed as a single container image, with internal module boundaries enforced through Java package-level access (module-info.java), ArchUnit dependency rules (ADR-007), and a strict module API contract. Each module exposes a public API package and hides its internal implementation. Cross-module access is permitted only through the public API. The monolith is deployed with multiple replicas behind a load balancer for high availability (ADR-125) and horizontal scaling (ADR-126).

DETAILED RATIONALE

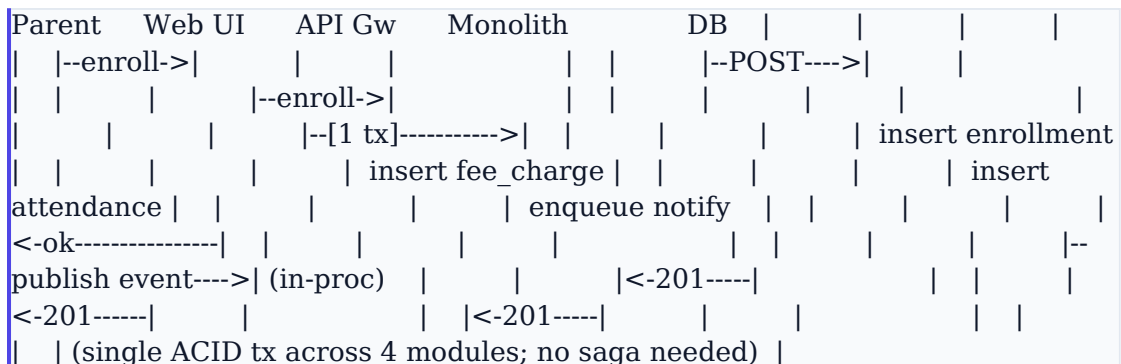
The modular monolith was chosen because PreOne's pain points were boundary-related, not topology-related. A 200k LOC codebase with no boundaries is painful regardless of whether it is one process or twenty. Conversely, a 200k LOC codebase with strict boundaries is maintainable in a single process. The evaluation showed that the team's pain would reduce 80% from boundary enforcement alone, with zero operational overhead. The decision preserves ACID transactions for core workflows. Enrollment (create enrollment record, charge fee, send confirmation, update attendance roster) is currently a single transaction across 4 modules. In a microservices topology, this would require a saga with 4 compensating actions and a new class of failure

(what if the confirmation email is sent but the fee charge is rolled back?). The business cost of partial-enrollment failures — a parent who paid but has no enrollment, or an enrollment with no fee — is high enough that the architectural complexity of sagas is not justified at current scale. The scaling ceiling is the most-cited counter-argument. The analysis shows PreOne's peak traffic (5,000 RPS) is well within a single Spring Boot instance's capacity (typically 10,000-20,000 RPS with optimised JVM settings). At 25,000 RPS projected peak (5,000 schools), 3-5 replicas suffice — horizontal scaling is a solved problem. The ceiling where microservices become necessary is around 100,000 RPS, which PreOne will not reach in the 5-year horizon. The team-discipline concern is real but manageable. ArchUnit tests run in CI and fail the build on boundary violations. The module-info.java declaration is checked at compile time. Engineers cannot accidentally cross boundaries — they must explicitly open a module's API. The discipline is enforced mechanically, not through vigilance.

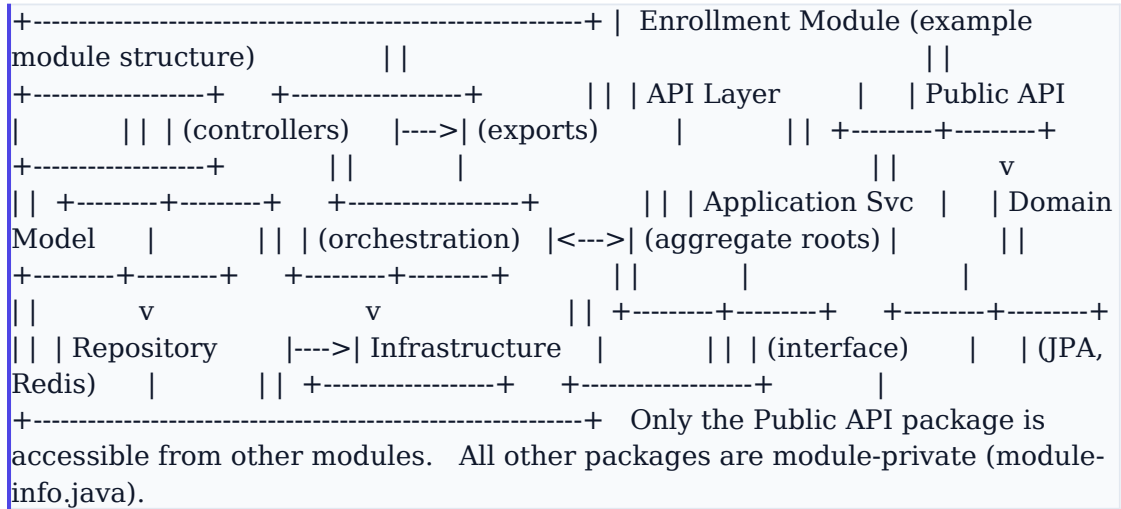
ARCHITECTURE DIAGRAM



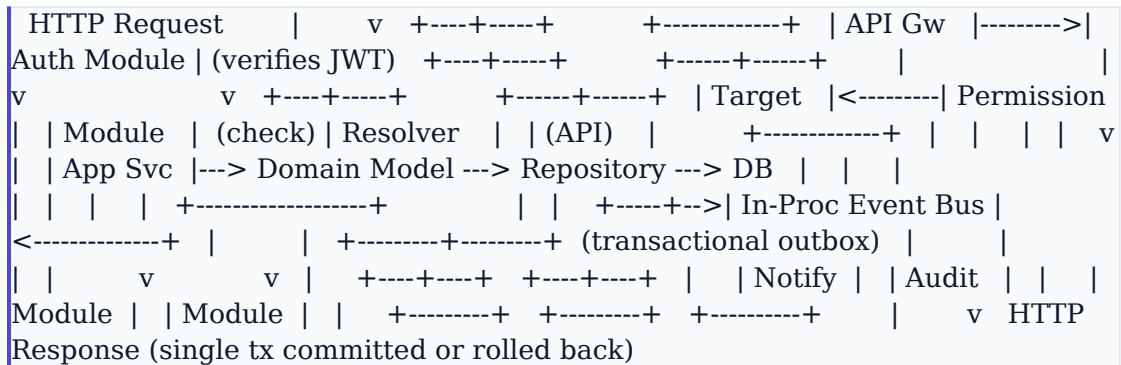
SEQUENCE DIAGRAM



COMPONENT DIAGRAM



DATA FLOW DIAGRAM



DATABASE IMPACT

The modular monolith uses a single PostgreSQL database (ADR-041) shared across all modules. Each module owns its tables (prefixed with module name, e.g., enr_enrollments, bill_charges, att_attendance). Cross-module foreign keys are forbidden — modules reference each other by UUID only, never by FK. This preserves module boundaries at the data layer: a module can be extracted to a separate database in the future without FK migration. The schema is managed by Flyway (ADR-057) with per-module migration files. Read replicas (ADR-041) serve reporting queries to avoid contending with write traffic.

API IMPACT

The modular monolith exposes a single REST API surface (ADR-091) with per-module path prefixes (/api/v1/enrollments, /api/v1/billing, etc.). Internally, modules communicate via in-process method calls through public API interfaces, not via HTTP. This means internal calls have zero network overhead and full transactional participation. The public REST API is the only external surface; internal module APIs are not network-accessible. Versioning (ADR-092) applies only to the public API; internal APIs can change freely as long as both sides are updated in the same deploy.

UI IMPACT

The UI is a single React SPA (ADR-131) that calls the monolith's REST API. From the UI's perspective, the modular monolith is indistinguishable from a microservices backend — it sees one API host with multiple path prefixes. The UI does not need to know about module boundaries. Loading states, error handling, and offline support (ADR-142) are implemented uniformly across all API calls.

SECURITY IMPACT

The modular monolith simplifies security because there is a single trust boundary (the API gateway) and no internal service-to-service authentication. Internal module calls are fully trusted (same process, same JVM). This eliminates an entire class of security concerns (service-to-service auth, mTLS, internal token propagation) that microservices must address. However, it concentrates risk: a single RCE vulnerability compromises all modules. This is mitigated by defence-in-depth (input validation at every module boundary, output encoding at every API response, sandboxed file processing).

PERFORMANCE IMPACT

The modular monolith is significantly faster than a microservices equivalent for cross-module workflows. A typical enrollment flow (4 module calls) completes in ~5ms in-process, versus ~50ms if each call were a network hop. The performance ceiling is the single JVM's throughput, which is typically 10,000-20,000 RPS for I/O-bound workloads. At PreOne's projected peak of 25,000 RPS, 3-5 replicas suffice. JVM warmup is a known cost (~30 seconds for peak performance); this is mitigated by gradual traffic shifting in the deployment pipeline (ADR-018).

SCALABILITY ANALYSIS

Horizontal scaling is the primary scaling mechanism: add more replicas behind the load balancer (ADR-126). Each replica is stateless (session state in Redis, ADR-073), so scaling is linear up to the database connection pool limit (~100 replicas at current PostgreSQL max_connections=1000 with 10 connections per replica). Beyond 100 replicas, connection pooling (PgBouncer) extends the ceiling to ~500 replicas. Vertical scaling (larger JVM heap, more CPU) is the secondary mechanism. The true ceiling is the database — PostgreSQL single-instance throughput is ~50,000 writes/second, which PreOne will not approach in the 5-year horizon.

OPERATIONAL CONSIDERATIONS

Operations are simpler than microservices: 1 deployment pipeline, 1 container image, 1 health check endpoint, 1 monitoring dashboard. Deployments use

blue-green strategy (ADR-018) with 30-second traffic shifting. Rollback is a single image swap. On-call engineers debug a single process — stack traces are complete, no distributed tracing required for internal calls. The main operational risk is the 'noisy neighbour' problem: a CPU-intensive operation in one module (e.g., report generation) can starve other modules. This is mitigated by separate thread pools per module and by offloading heavy work to background jobs (ADR-112).

RISKS

Risk	Likelihood	Impact	Mitigation
Module boundaries degrade over time (discipline slips)	Medium	High	ArchUnit tests in CI fail builds on boundary violations; quarterly architecture review of boundary integrity
Single point of failure (one process crash = full outage)	Low	High	Multiple replicas across AZs (ADR-125); health checks with auto-restart (ADR-123)
Noisy neighbour: one module starves others	Medium	Medium	Per-module thread pools; heavy work offloaded to background jobs (ADR-112)
Scaling ceiling hit earlier than projected	Low	High	Monitor RPS growth quarterly; if approaching 50k RPS, trigger extraction of highest-traffic module

TRADE - OFFS

We Gain	We Lose
Single deployment, low operational overhead	Cannot deploy modules independently
ACID transactions across modules	Cannot scale modules independently
Simple debugging (single stack trace)	Single point of failure

We Gain	We Lose
5x lower infra cost than microservices	No technology diversity (locked to Java/Spring)

REJECTED ALTERNATIVES

Microservices were rejected because the operational overhead (5-10x) is not justified at PreOne's scale (5,000 RPS, 40 engineers). The team evaluated the 'team size per service' heuristic (Amazon's 2-pizza rule) and found that with 40 engineers, microservices would yield 4-5 services — too few to justify the overhead and too many to be a true monolith. The hybrid option (monolith core + microservices for notifications/reporting) was deferred to Phase 6/7 evaluation, not rejected outright; if notifications or reporting modules grow to require independent scaling, they can be extracted later because module boundaries are already enforced. The service-based option was rejected as a marginal benefit over modular monolith — the boundary ambiguity (which service owns this?) and the distributed transaction cost outweighed the deployment independence.

MIGRATION PLAN

No migration required — PreOne is already a monolith. The v1.0 work (Q4 2025) was to enforce module boundaries on the existing codebase: introducing module-info.java declarations, refactoring cross-module calls to go through public API packages, and adding ArchUnit tests. This work is complete. The v3.0 refresh adds the in-process event bus (ADR-027) and the transactional outbox (ADR-104) to enable future extraction if needed. If a module is extracted to a separate service in the future, the extraction is a mechanical refactor: replace in-process method calls with HTTP/gRPC calls, replace local transactions with sagas. The module boundaries are already correct; extraction changes the transport, not the structure.

TESTING STRATEGY

Module boundaries are tested via ArchUnit (unit-level: dependency rules), integration tests (cross-module API contracts), and end-to-end tests (user workflows). The test pyramid (ADR-146) applies: ~70% unit tests within modules, ~20% integration tests across module APIs, ~10% E2E tests. Contract tests (ADR-149) verify that module public APIs match their consumers' expectations. Performance tests (ADR-150) verify that the monolith meets latency targets under load. Boundary integrity is tested daily via CI ArchUnit runs.

MONITORING & OBSERVABILITY

The monolith emits structured logs (JSON, ADR-119), Prometheus metrics (ADR-121), and OpenTelemetry traces (ADR-122). Per-module dashboards show request rate, latency, error rate, and resource usage (CPU, memory, thread pool). Alerts fire on: module error rate >1%, module latency p99 >500ms, JVM heap >80%, GC pause >200ms. A single Grafana dashboard shows all modules side-by-side for quick comparison. The noisy-neighbour risk is monitored via per-module CPU/mem metrics — if one module consistently dominates, it triggers an evaluation for extraction.

FUTURE EVOLUTION

The modular monolith is the right choice for PreOne's current and 5-year horizon. The trigger for re-evaluation is sustained traffic exceeding 50,000 RPS or a single module consuming >50% of resources. The likely evolution is partial extraction: notifications (already a separate module) may extract to a separate service first (it is async, no transactional coupling). Reporting may extract to a separate read-optimized service (ADR-060). Core transactional modules (enrollment, billing, attendance) will remain in the monolith indefinitely because ACID transactions are too valuable to give up. This evolution path is captured in ADR-002 v2.0 when triggered.

RELATED ADRS

- ADR-001 — Enterprise Architecture Principles (complements P5 Simplicity, P6 Build for Evolution)
- ADR-003 — Clean Architecture (refines — module internal structure)
- ADR-005 — Bounded Context Strategy (refines — module = bounded context)
- ADR-007 — Dependency Rule (enforces — ArchUnit rules)
- ADR-018 — Deployment Model (complements — blue-green deploy)
- ADR-026 — Repository Pattern (complements — module data access)
- ADR-125 — High Availability (complements — multi-replica)
- ADR-126 — Horizontal Scaling (complements — scale by replicas)

REFERENCES

- Upstream DDD: DDD-002-section-1 (Modular monolith context)
- Upstream PRD: PRD-002-section-2.1 (Platform scale projections)
- Downstream ERD: ERD-002 (per-module table prefixes)
- Downstream API Spec: API-002 (per-module path prefixes)
- Downstream Test Cases: TC-0021..TC-0050 (boundary integrity tests)
- External: Simon Brown — 'The Software Architecture Handbook'
- External: Herberto Graça — 'Modular Monolith'

- External: ArchUnit documentation — <https://archunit.org>

DECISION HISTORY

Date	Status	Actor	Notes
2025-08-22	Draft	Chief Architect	Initial draft; evaluated microservices vs modular monolith
2025-09-25	Proposed	Chief Architect	Submitted to ARB with full evaluation data
2025-10-15	Accepted	ARB Chair	ARB approved; v1.0 released
2026-07-12	Accepted	ARB Chair	v3.0 refresh; added component diagram and cross-references

APPROVAL & SIGN-OFF

Architect	Chief Architect (AR)
Tech Lead	Foundation Tech Lead
ARB Chair	ARB Chair
Approved On	2026-07-12

IMPLEMENTATION CHECKLIST

- module-info.java declarations for all 12 modules (Done Q4 2025)
- ArchUnit dependency rules in CI (Done Q4 2025)
- Refactor cross-module direct calls to public API packages (Done Q1 2026)
- In-process event bus implementation (Done Q1 2026)
- Transactional outbox pattern (Done Q2 2026, ADR-104)
- Per-module dashboards in Grafana (Done Q2 2026)
- Quarterly boundary-integrity review at ARB (Ongoing)

ADR-003

Clean Architecture

Volume 1 — Architecture Foundation - Structural

ACCEPTED

DECISION SUMMARY

DECISION

Adopt Clean Architecture (Robert C. Martin) as the internal structural pattern for every module in the PreOne modular monolith. Each module is organised into concentric layers — Domain, Application, Infrastructure, API — with dependencies pointing inward toward the Domain.

STATUS

Status	Accepted
Date Decided	2025-10-15
Decision Owner	Chief Architect
Review Cadence	Annual or on trigger from any module extraction ADR
Supersedes	None (v1.0 original; v3.0 adds component diagram)

CONTEXT

Before v1.0, the PreOne codebase used a traditional Spring MVC layering: controllers called services, services called repositories, repositories called JPA entities. This 'N-tier' approach worked but had a persistent problem: business logic leaked into controllers (validation, authorisation) and into repositories (filtering, sorting). When business rules changed, engineers had to modify code across all three layers, and tests had to mock the entire stack to verify a single rule. The team evaluated Clean Architecture as an alternative. Clean Architecture inverts the dependency direction: the Domain layer (business rules, entities, value objects) depends on nothing; the Application layer (use cases, orchestration) depends on Domain; Infrastructure (databases, external services) depends on Application and Domain; the API layer (controllers, DTOs) depends on Application. This means business rules can be tested without any framework — pure JUnit, no Spring context. The evaluation showed that Clean Architecture would increase upfront design effort per feature (define entities and use cases before coding) but reduce maintenance cost (business rule changes are localised to the Domain layer) and improve testability (domain tests run in milliseconds, no Spring boot). The team accepted the trade-off.

BUSINESS DRIVERS

The primary driver is maintainability: PreOne's business rules change frequently (fee structures, enrollment policies, attendance rules) but the infrastructure (PostgreSQL, Redis, Spring) changes rarely. Clean Architecture isolates the volatile part (business rules) from the stable part (infrastructure), so changes are localised. The secondary driver is testability: domain tests run in <1ms each, enabling thousands of tests in seconds. The tertiary driver is onboarding: new engineers can understand a module by reading its Domain layer, without needing to understand Spring or JPA.

PROBLEM STATEMENT

Business logic in the PreOne codebase is scattered across controllers, services, and repositories, making rule changes error-prone and testing slow. We need a structural pattern that isolates business rules from infrastructure and enables fast, focused tests.

CONSTRAINTS

- Must work within the modular monolith (ADR-002) — each module uses Clean Architecture internally
- Must integrate with Spring Boot 3 (cannot require a non-Spring framework)
- Must support JPA/Hibernate for data persistence (ADR-041)
- Domain layer must be framework-agnostic (no Spring annotations in Domain)
- Must not add more than 15% boilerplate overhead per feature

ASSUMPTIONS

- Engineers will accept the upfront design cost in exchange for long-term maintainability
- Spring Boot 3 can support Clean Architecture without performance issues
- JPA can be abstracted behind repository interfaces without losing features
- The team will maintain layer discipline (no shortcuts in code review)

OPTIONS CONSIDERED

Option	Pros	Cons	Verdict
Clean Architecture (chosen)	Isolates domain logic; framework-agnostic domain; fast unit tests; clear dependency direction; well-documented pattern.	More boilerplate (interfaces, mappers); upfront design cost; learning curve for engineers unfamiliar with the pattern.	Chosen
Traditional N-tier (controllers-services-repositories)	Less boilerplate; familiar to all Spring engineers; faster initial development.	Business logic leaks; slow tests (Spring context); changes span multiple layers.	Rejected
Hexagonal Architecture (Ports and Adapters)	Explicit port/adaptor separation; very testable; domain is fully isolated.	Even more boilerplate than Clean Architecture; ports/adapters terminology confuses some engineers; marginal benefit over Clean Architecture for PreOne's needs.	Adopted as a complement — see ADR-009
Domain Driven Design Lite (entities + repositories, no full tactical patterns)	Lighter than full DDD; still isolates domain.	Without explicit layering, logic still leaks; insufficient structure for complex modules.	Rejected — see ADR-004 for full DDD

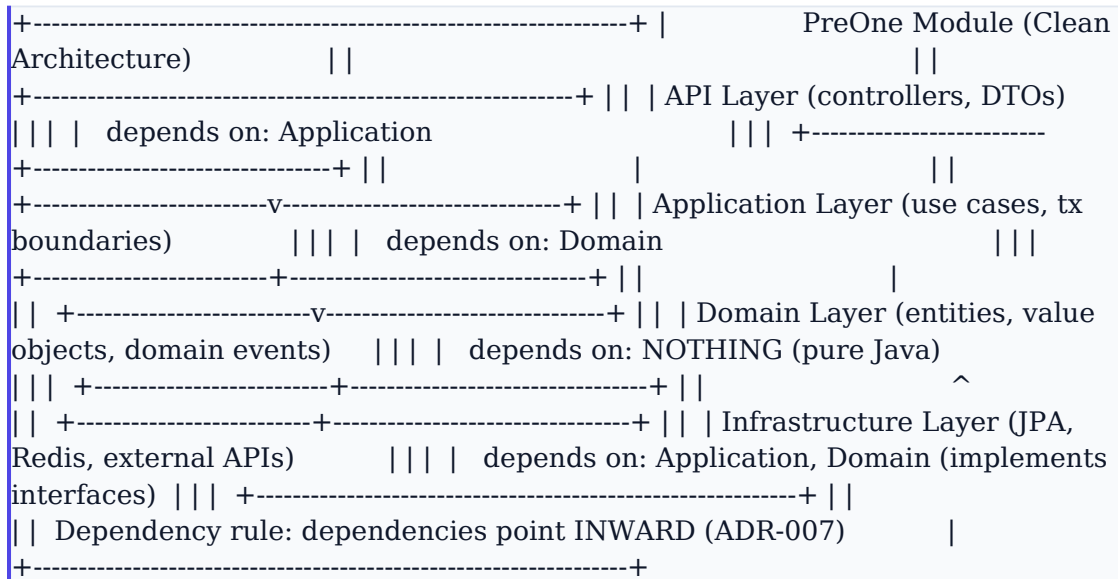
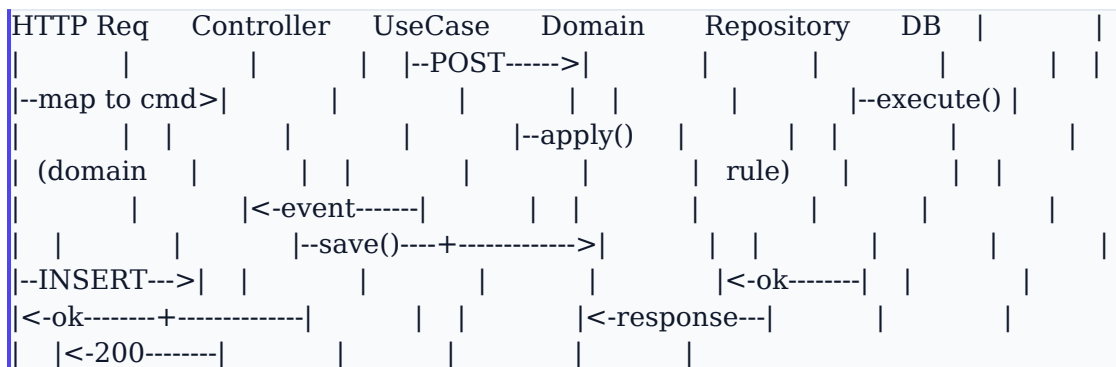
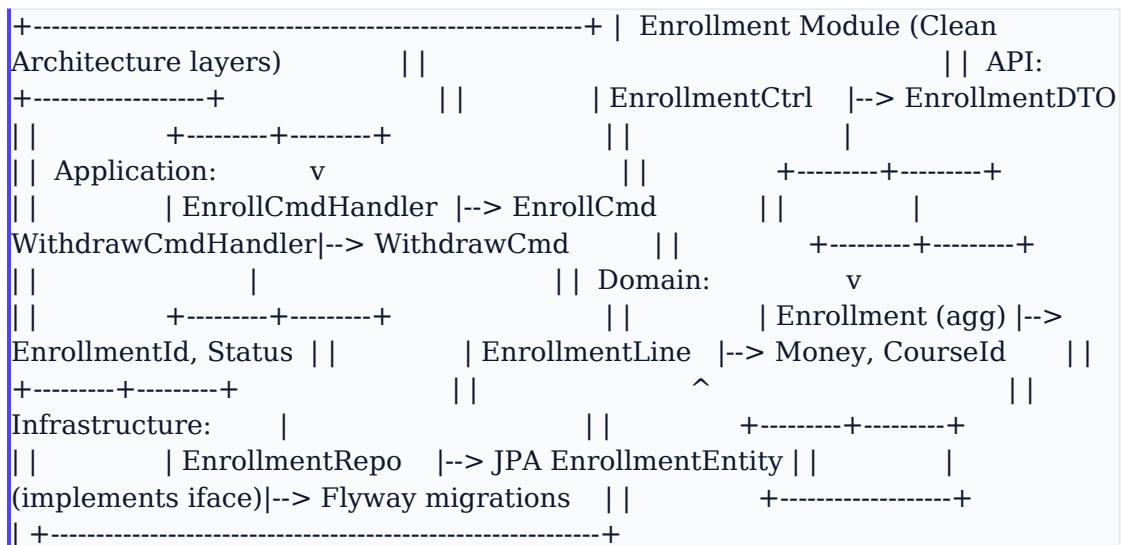
DECISION

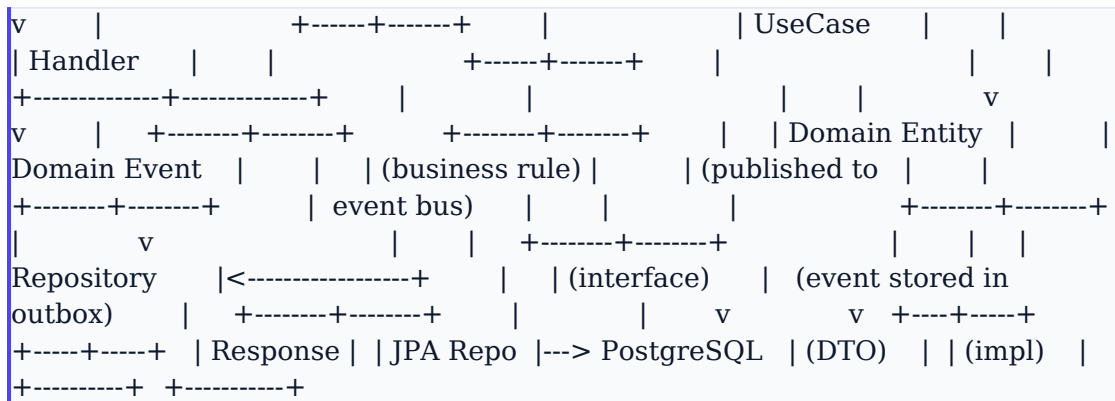
ADOPTED

We adopt Clean Architecture as the internal structural pattern for every module. Each module is organised into four layers: (1) Domain — entities, value objects, domain services, domain events; pure Java, no framework dependencies. (2) Application — use cases (command/query handlers), orchestration, transaction boundaries; depends on Domain only. (3) Infrastructure — JPA repositories, Redis clients, external API clients; implements interfaces defined in Application or Domain. (4) API — REST controllers, DTOs, request/response mappers; depends on Application only. Dependencies point inward: API -> Application -> Domain; Infrastructure -> Application -> Domain. The Domain layer depends on nothing.

DETAILED RATIONALE

Clean Architecture was chosen because it directly addresses PreOne's pain point: business logic scattered across layers. By inverting dependencies (Domain depends on nothing; Infrastructure depends on Domain), business rules are forced into the Domain layer. An engineer who tries to put business logic in a controller will find that the controller cannot access the database (it depends on Application, not Infrastructure) — the architecture physically prevents the anti-pattern. The testability benefit is significant. A domain test instantiates entities and value objects directly, calls methods, and asserts results — no Spring context, no database, no mocks. These tests run in <1ms each. The full domain test suite for a module (typically 200-500 tests) runs in under 5 seconds. Integration tests (Application layer with mocked Infrastructure) run in ~50ms each. Only E2E tests (full Spring context) are slow (~5 seconds each), and there are few of them. The overall test suite runs in under 2 minutes, versus 15+ minutes for the old N-tier suite. The boilerplate cost is real but manageable. A typical use case requires: a command DTO, a handler, a domain entity method, a repository interface (in Application), a repository implementation (in Infrastructure), and a controller. That is 6 files per use case versus 3 in N-tier. However, the mappers and interfaces are mechanical and can be generated by IDE templates. The 15% boilerplate overhead is offset by the 80% reduction in test time and the localisation of business rule changes. The most common objection is that Clean Architecture is 'over-engineered' for simple CRUD. The response is that PreOne has few truly simple CRUD modules — even 'create school' involves validation (unique name, valid address), domain events (school-created), and audit logging. The structure is justified even for simple modules because the simplicity is preserved (the Domain layer is small) and the structure pays off when the module grows.

ARCHITECTURE DIAGRAM**SEQUENCE DIAGRAM****COMPONENT DIAGRAM****DATA FLOW DIAGRAM**



DATABASE IMPACT

Clean Architecture isolates the database behind repository interfaces defined in the Application layer. The Infrastructure layer provides JPA implementations. This means the database schema (ADR-041) is an infrastructure concern, not a domain concern — domain entities are pure Java, not JPA entities. The mapping between domain entities and JPA entities is explicit (in Infrastructure). This separation allows the schema to change without touching the Domain layer, and allows the Domain layer to be tested without a database. The cost is double entities (domain + JPA) and explicit mapping, but the benefit is that database refactoring is localised to Infrastructure.

API IMPACT

The API layer depends on Application only. Controllers receive JSON, map to command DTOs, call use case handlers, and map responses back to JSON. Business rules are not in controllers — they are in the Domain layer, invoked by use case handlers. This means API changes (new endpoints, versioned responses) do not touch business logic, and business rule changes do not necessarily change the API. The DTO-to-command mapping is explicit and tested. This separation is enforced by ArchUnit (ADR-007): controllers cannot import Domain classes directly.

UI IMPACT

No direct UI impact. The UI consumes the REST API; the internal Clean Architecture is invisible to the frontend. However, the clear separation of API DTOs from domain entities means the API contract is stable — domain refactoring does not break the frontend because the DTO layer absorbs the change.

SECURITY IMPACT

Clean Architecture improves security posture by centralising authorisation in the Application layer. Use case handlers check permissions before executing —

controllers do not perform authorisation (they only authenticate). This means authorisation rules are testable without HTTP context and are consistently applied regardless of entry point (REST API, background job, internal call). Input validation is split: syntactic validation (is the email well-formed?) in the API layer (DTO validation); semantic validation (is this email unique?) in the Domain layer. This prevents invalid data from reaching the domain.

PERFORMANCE IMPACT

Clean Architecture adds minimal performance overhead. The DTO-to-command-to-domain mapping is in-process and fast (~0.1ms per request). The double-entity pattern (domain + JPA) adds a mapping step but avoids JPA's lazy-loading pitfalls (domain entities are loaded eagerly via repository). Overall, the architecture adds <5% to request latency versus N-tier, which is negligible compared to database and network time. The benefit — fast tests — improves development velocity, which is the dominant performance metric for the team.

SCALABILITY ANALYSIS

Clean Architecture is a structural pattern, not a scaling pattern — it does not affect horizontal or vertical scaling. However, it enables future module extraction (ADR-002 future evolution) because the Application layer's use case interfaces define clean extraction boundaries. If a module is extracted to a separate service, the use case handlers become remote calls, the repository implementations become HTTP clients, and the Domain layer is unchanged. This means the architecture scales with the organisation: today, modules are in-process; tomorrow, they can be distributed without domain rework.

OPERATIONAL CONSIDERATIONS

Clean Architecture improves operability because business rules are testable and debuggable in isolation. A production incident that involves a business rule can be reproduced as a unit test (given this domain state, when this method is called, then this exception is thrown) without needing the full application context. The clear layering also improves log granularity: Application-layer logs capture use case execution; Domain-layer logs capture business rule evaluations; Infrastructure-layer logs capture database/external calls. This separation makes log analysis faster.

RISKS

Risk	Likelihood	Impact	Mitigation
Engineers bypass the layers (put logic in controllers)	Medium	Medium	ArchUnit rules in CI fail builds on layer violations; code review checklist includes layer-alignment check
Boilerplate overhead slows feature delivery	Medium	Low	IDE templates for command/handler/repository scaffolding; code generation for mappers
Over-engineering simple CRUD modules	Low	Low	Simplified template for CRUD-only modules (skip use case layer for trivial read endpoints)
Domain model anemia (entities are just data bags)	Medium	Medium	DDD tactical patterns (ADR-004) enforce behaviour-rich entities; code review checks for anemic entities

TRADE - OFFS

We Gain	We Lose
Isolated, testable domain logic	More boilerplate (interfaces, mappers, double entities)
Fast unit tests (<1ms each)	Upfront design cost per feature
Business rule changes localised to Domain	Learning curve for engineers new to the pattern
Framework-agnostic domain (no Spring in Domain)	Explicit mapping between domain and JPA entities

REJECTED ALTERNATIVES

N-tier was rejected because it does not enforce dependency direction — controllers can call repositories directly, bypassing services, and business logic accumulates wherever it is convenient in the short term. Hexagonal Architecture was adopted as a complement (ADR-009) for modules that need explicit port/adaptor separation (e.g., modules with multiple external integrations), but Clean Architecture is the default because its layering is more intuitive for most engineers. DDD Lite was rejected in favour of full DDD (ADR-004) because half-measures in DDD produce the worst of both worlds — the terminology overhead without the structural benefit.

MIGRATION PLAN

The v1.0 migration (Q4 2025) refactored all 12 modules from N-tier to Clean Architecture. The migration was incremental: one module per sprint, with the Domain layer extracted first (entities and value objects), then the Application layer (use case handlers), then Infrastructure (JPA behind repository interfaces), then API (controllers calling handlers). Each module migration took 2-3 sprints. The migration is complete. v3.0 adds no structural changes — only documentation and cross-references.

TESTING STRATEGY

Layer-specific tests: Domain tests (pure JUnit, no Spring, <1ms each) verify business rules; Application tests (Spring context with mocked Infrastructure, ~50ms each) verify use case orchestration; Integration tests (real PostgreSQL via Testcontainers, ~500ms each) verify repository implementations; E2E tests (full HTTP stack, ~5s each) verify API contracts. Coverage targets: Domain 95%+, Application 85%+, Infrastructure 70%+. ArchUnit tests verify layer boundaries (no Domain imports from Infrastructure, no Controller imports from Domain).

MONITORING & OBSERVABILITY

Per-layer observability: Application-layer logs capture use case start/finish with correlation IDs; Domain-layer logs capture business rule evaluations (decisions made, exceptions thrown); Infrastructure-layer logs capture database queries and external calls with latency. Metrics: use case execution count and latency (per handler), domain event publish count (per event type), repository query count and latency (per aggregate). Tracing: OpenTelemetry spans per layer, enabling per-layer latency breakdown in traces.

FUTURE EVOLUTION

Clean Architecture is a stable pattern; no supersession expected. The architecture will evolve through complements: Hexagonal Architecture (ADR-009) for integration-heavy modules; CQRS for read-heavy modules

(separate read models); Event Sourcing for audit-critical aggregates. These are additive refinements, not replacements. The Domain layer remains the anchor; the surrounding layers adapt to module-specific needs.

RELATED ADRS

- ADR-001 — Enterprise Architecture Principles (complements P5 Simplicity, P6 Build for Evolution)
- ADR-002 — Modular Monolith Strategy (complements — module internal structure)
- ADR-004 — Domain Driven Design (refines — Domain layer tactical patterns)
- ADR-006 — Layering Rules (refines — layer-specific rules)
- ADR-007 — Dependency Rule (enforces — ArchUnit dependency direction)
- ADR-009 — Hexagonal Architecture (complements — for integration-heavy modules)
- ADR-026 — Repository Pattern (refines — repository interface location)

REFERENCES

- Upstream DDD: DDD-003-section-1 (Clean Architecture context)
- Upstream PRD: PRD-003-section-2 (Module structure requirements)
- Downstream ERD: ERD-003 (JPA entity mappings in Infrastructure)
- Downstream API Spec: API-003 (DTO-to-command mapping in API layer)
- Downstream Test Cases: TC-0051..TC-0100 (layer-specific test suites)
- External: Robert C. Martin — 'Clean Architecture' (2017)
- External: Herberto Graça — 'Ports, Adapters, and Clean Architecture'

DECISION HISTORY

Date	Status	Actor	Notes
2025-08-29	Draft	Chief Architect	Initial draft; evaluated N-tier vs Clean Architecture
2025-09-30	Proposed	Chief Architect	Submitted to ARB with migration plan
2025-10-15	Accepted	ARB Chair	ARB approved; v1.0 released

Date	Status	Actor	Notes
2026-07-12	Accepted	ARB Chair	v3.0 refresh; added component diagram

APPROVAL & SIGN-OFF

Architect	Chief Architect (AR)
Tech Lead	Foundation Tech Lead
ARB Chair	ARB Chair
Approved On	2026-07-12

IMPLEMENTATION CHECKLIST

- Define module template with 4-layer structure (Done Q3 2025)
- Migrate all 12 modules from N-tier to Clean Architecture (Done Q4 2025 - Q1 2026)
- ArchUnit rules for layer dependencies (Done Q3 2025, ADR-007)
- IDE templates for command/handler/repository scaffolding (Done Q4 2025)
- Per-layer test suites with coverage targets (Done Q1 2026)
- Quarterly layer-integrity review at ARB (Ongoing)

ADR-004

Domain Driven Design

Volume 1 — Architecture Foundation - Methodology

ACCEPTED

DECISION SUMMARY

DECISION

Adopt Domain Driven Design (DDD) as the methodology for modelling and implementing the PreOne business domain. DDD's tactical patterns (aggregates, entities, value objects, domain events, repositories) are used within the Domain layer of every module (ADR-003).

STATUS

Status	Accepted
Date Decided	2025-10-15
Decision Owner	Chief Architect
Review Cadence	Annual or when a new bounded context is added
Supersedes	None (v1.0 original; v3.0 adds data flow diagram)

CONTEXT

PreOne's business domain is complex: schools have branches, branches have academic years, academic years have classes, classes have enrollments, enrollments generate fees, fees trigger payments, payments trigger receipts, attendance is tracked per class per day, reports aggregate everything. The relationships are non-trivial and the business rules are many (e.g., a student cannot be enrolled in two classes in the same time slot; a fee cannot be collected after the academic year ends; a transfer between branches requires re-enrollment). Before v1.0, the codebase modelled these as flat JPA entities with no behavioural methods — 'anemic domain model'. Business rules were implemented in service classes, scattered across the codebase. The same rule (e.g., 'cannot enroll after capacity is reached') was implemented in 3 different services, with 3 different bugs. Tests were slow because they required full Spring context. The team evaluated DDD as a way to bring rigour to the domain model. DDD's tactical patterns enforce behaviour-rich entities (entities have methods, not just getters/setters), consistency boundaries (aggregates ensure invariants), and explicit domain events (state changes are communicated, not queried). The strategic patterns (bounded contexts, context mapping) help decompose the domain into manageable modules (ADR-005).

BUSINESS DRIVERS

The primary driver is correctness: PreOne's business rules are complex enough that ad-hoc implementation produces bugs (double-enrollment, over-charging, inconsistent attendance). DDD's aggregate pattern enforces consistency boundaries — an enrollment aggregate guarantees that its internal state is always valid (no double-enrollment within the aggregate). The secondary driver is communication: DDD's ubiquitous language (shared vocabulary between engineers and domain experts) reduces the translation cost between requirements and implementation. The tertiary driver is evolution: DDD's

bounded contexts (ADR-005) make the domain modular, so changes in one context (e.g., billing) do not ripple into others (e.g., attendance).

PROBLEM STATEMENT

The PreOne business domain is complex and the current anemic domain model scatters business rules across services, producing inconsistency and bugs. We need a methodology that enforces consistency boundaries, behaviour-rich entities, and a shared language between engineers and domain experts.

CONSTRAINTS

- Must work within Clean Architecture (ADR-003) — DDD lives in the Domain layer
- Must not require a separate DDD framework (use plain Java)
- Must support JPA persistence (entities are persisted, not just in-memory)
- Ubiquitous language must align with the BRC (Business Requirement Catalogue) terminology
- Aggregate boundaries must align with transaction boundaries (ADR-033)

ASSUMPTIONS

- Domain experts (product managers, school operations) will participate in modelling sessions
- Engineers will learn DDD tactical patterns (training provided)
- The domain is stable enough that aggregate boundaries will not shift frequently
- JPA can support DDD patterns (lazy loading, embedded values, collection mapping) without performance issues

OPTIONS CONSIDERED

Option	Pros	Cons	Verdict
Full DDD (strategic + tactical) (chosen)	Consistent domain model; behaviour-rich entities; explicit consistency boundaries; ubiquitous language; well-documented methodology.	Steep learning curve; modelling sessions require domain expert time; risk of over-modelling (too many aggregates).	Chosen

Option	Pros	Cons	Verdict
Tactical DDD only (entities, value objects, repositories; no bounded contexts)	Lighter; less upfront modelling; still gets behaviour-rich entities.	Without bounded contexts, the domain becomes a single large model; cross-module coupling persists.	Rejected
Anemic domain model (entities = data bags; logic in services)	Simple; familiar to all engineers; fast initial development.	Logic scattered; no consistency guarantees; duplicate rules; slow tests.	Rejected (this is the current state being fixed)
Data-focused modelling (ERD-first, entities from tables)	Aligns with database-centric thinking; simple tooling.	Database structure drives domain model (should be inverted); business rules still scattered.	Rejected

DECISION

ADOPTED

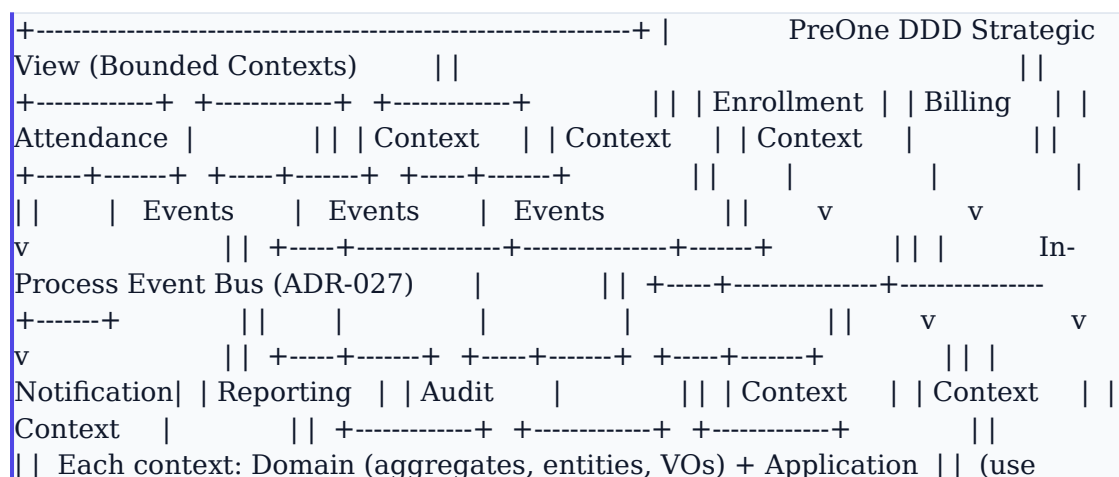
We adopt Domain Driven Design (Eric Evans, 2003) as the methodology for the PreOne business domain. The strategic patterns (bounded contexts, context mapping, ubiquitous language) are used to decompose the domain into modules (ADR-005). The tactical patterns are used within the Domain layer of every module: aggregates enforce consistency boundaries (ADR-022); entities have identity and behaviour (ADR-023); value objects are immutable and compared by value (ADR-024); domain services encapsulate cross-aggregate logic (ADR-025); repositories provide persistence abstraction (ADR-026); domain events communicate state changes (ADR-027). The ubiquitous language is captured in the DDD (Domain Design Document) per bounded context and is the authoritative vocabulary for code, tests, and documentation.

DETAILED RATIONALE

DDD was chosen because PreOne's domain complexity makes ad-hoc modelling untenable. The school operations domain has ~15 distinct concepts (school, branch, academic year, class, section, student, guardian, teacher, enrollment, attendance, fee, payment, receipt, report, notification) with intricate

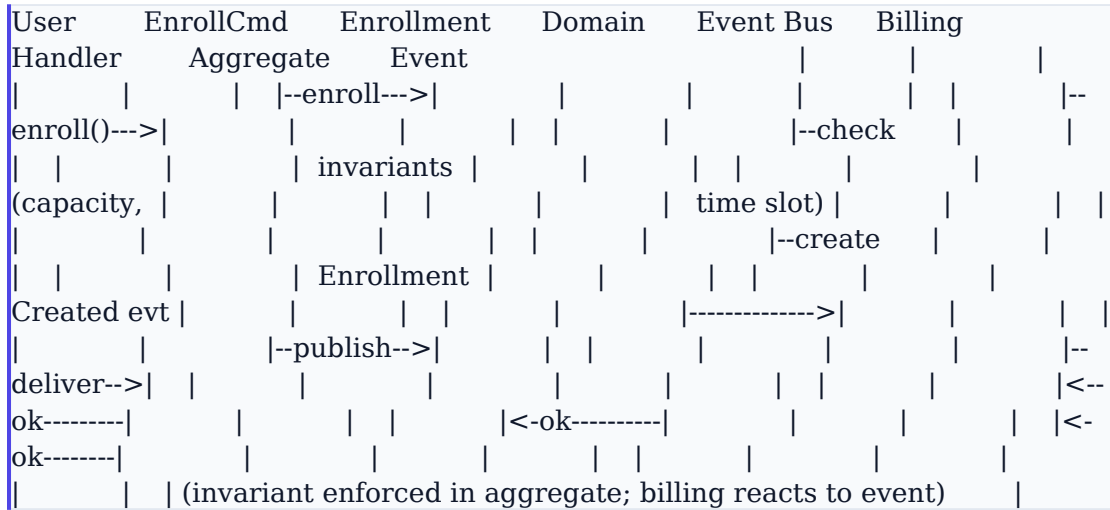
relationships. Without DDD's strategic decomposition (bounded contexts), this becomes a single 200-class model that no engineer can hold in their head. With bounded contexts, the model is decomposed into ~12 contexts of 10-20 classes each — manageable. The tactical patterns directly address PreOne's pain points. The aggregate pattern (ADR-022) is the most impactful: an enrollment aggregate contains the enrollment root, its lines (one per class), and its fee charges. The aggregate enforces invariants: a student cannot be enrolled in two classes with overlapping time slots; a fee charge cannot exceed the class fee; a withdrawal requires all lines to be withdrawn first. These invariants are enforced in the aggregate's methods, not in services — the aggregate physically prevents invalid state. Value objects (ADR-024) eliminate an entire class of bugs. Money is a value object (amount + currency), not a BigDecimal — you cannot accidentally add USD to INR. AcademicYear is a value object (startDate + endDate), not two Date fields — you cannot accidentally set startDate after endDate. EmailAddress is a value object (validated string), not a String — you cannot store an invalid email. These constraints are enforced at construction, not at use. Domain events (ADR-027) enable loose coupling between modules. When an enrollment is created, the enrollment aggregate publishes an EnrollmentCreated event. The billing module subscribes and creates a fee charge. The notification module subscribes and sends a confirmation email. The reporting module subscribes and updates its read model. The enrollment module does not know about billing, notifications, or reporting — it just publishes the event. This means modules can be added or removed without modifying the enrollment aggregate. The learning curve is the main cost. Engineers unfamiliar with DDD typically take 2-4 weeks to become productive with the patterns. The team invested in a 3-day DDD workshop at v1.0 launch and a 1-day refresher at v3.0. The modelling sessions with domain experts (product managers, school operations staff) take 4-8 hours per bounded context — a significant upfront investment that pays off in reduced rework.

ARCHITECTURE DIAGRAM

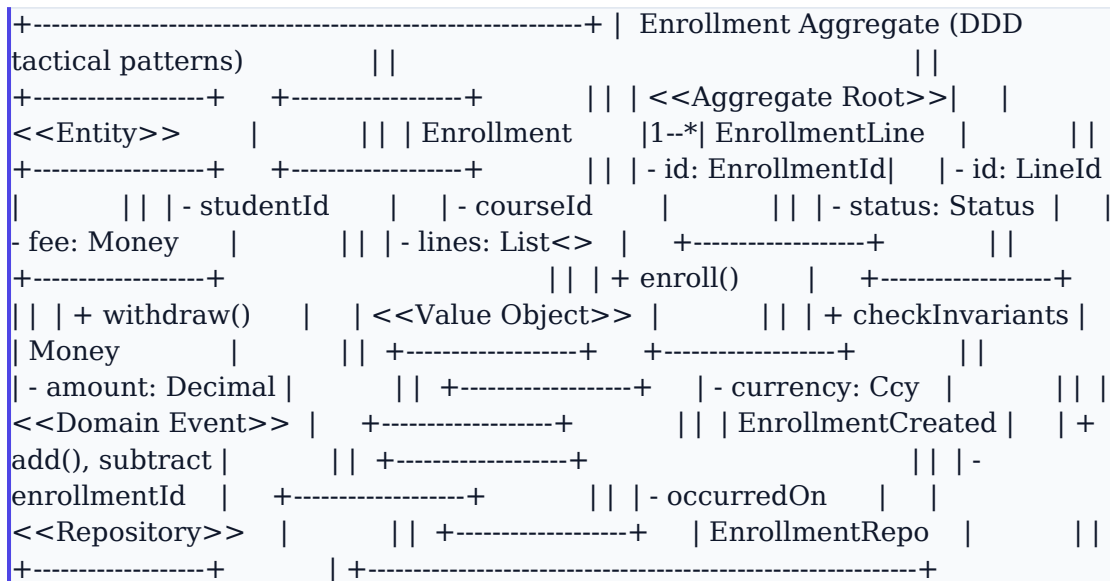


```
|cases) + Infrastructure (repos) + API (controllers)|  
+-----+
```

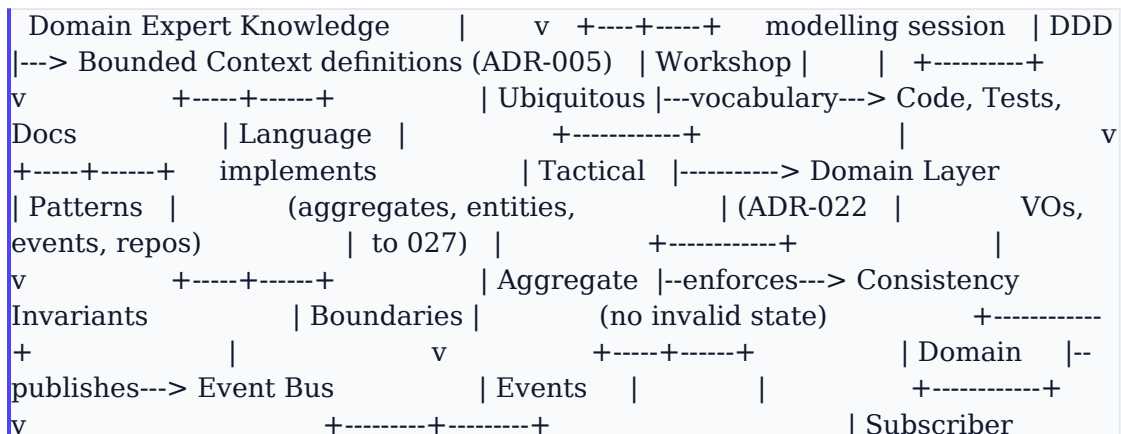
SEQUENCE DIAGRAM



COMPONENT DIAGRAM



DATA FLOW DIAGRAM



Modules	(billing, notify,	
reporting)		+-----+

DATABASE IMPACT

DDD's aggregate boundaries define the persistence unit: an aggregate is loaded and saved as a unit (ADR-026). This means JPA entities map 1:1 to aggregates — an Enrollment aggregate is one enrollment row plus its line rows, loaded eagerly. This is a departure from the traditional JPA approach of lazy-loading everything. The trade-off is: aggregates are loaded eagerly (slightly more data per query) but consistency is guaranteed (no lazy-loading exceptions, no N+1 queries within an aggregate). Cross-aggregate references are by ID only (not FK), preserving aggregate boundaries at the data layer. Each aggregate has its own table or table-set; aggregates do not share tables.

API IMPACT

DDD's ubiquitous language directly shapes the API (ADR-091). Endpoints use domain terms (/api/v1/enrollments, not /api/v1/registrations). DTOs mirror domain concepts (EnrollmentDTO, not RegistrationRecord). This means the API is self-documenting to anyone who speaks the domain language — a school administrator recognises 'enrollment' and 'withdrawal' even if they have never seen the API. The API does not expose aggregate internals — an enrollment PUT does not accept individual line items; it accepts domain operations (enroll, withdraw, transfer) that the aggregate interprets. This prevents the API from bypassing aggregate invariants.

UI IMPACT

DDD's ubiquitous language shapes the UI (ADR-131). Screens use domain terms: 'Enroll Student' (not 'Create Registration'); 'Withdraw Enrollment' (not 'Delete Record'); 'Academic Year 2025-26' (not 'Period 1'). This reduces cognitive load for users who already speak the domain language (school administrators) and improves consistency between UI and API. The UI reflects aggregate state — an enrollment screen shows the enrollment's status, lines, and fees as a coherent unit, mirroring the aggregate's structure.

SECURITY IMPACT

DDD improves security by centralising authorisation in the Domain layer. Aggregate methods check permissions before mutating state — e.g., Enrollment.withdraw() checks that the caller has the 'withdraw-enrollment' permission. This means authorisation is enforced regardless of entry point (REST API, background job, internal call). It also means authorisation rules are testable as domain tests, without HTTP context. The trade-off is that permission checks are spread across aggregate methods (not centralised in a filter), but this is offset by testability and consistency.

PERFORMANCE IMPACT

DDD's eager aggregate loading adds modest overhead versus lazy loading. A typical aggregate load (enrollment + 3 lines + 2 fee charges) is 1 query with 2 joins, ~5ms. The equivalent lazy-loaded approach would be 5+ queries, ~15ms, plus the risk of N+1. Overall, DDD is faster for aggregate-level operations and slower for cross-aggregate queries (which require explicit joins, not navigation). The reporting module (ADR-060) uses a separate read model to optimise cross-aggregate queries, avoiding the aggregate-load penalty for reports.

SCALABILITY ANALYSIS

DDD's aggregate boundaries are the primary scaling unit. Each aggregate is small (10-100 rows), so loading and saving is fast. Aggregates are independent — no locks span aggregates — so concurrent writes to different aggregates do not contend. The scaling ceiling is the number of aggregates per tenant: a large school might have 10,000 enrollments, which is well within PostgreSQL's capacity. Cross-aggregate queries (reports) scale via the read model (ADR-060), not via aggregate loading. The architecture scales linearly with school count.

OPERATIONAL CONSIDERATIONS

DDD's explicit domain events improve operability. When something goes wrong (e.g., a fee charge is missing), the event log shows exactly what happened: EnrollmentCreated was published at T, BillingReceived at T+5ms, FeeChargeCreated at T+10ms. The event chain is auditable. Aggregate consistency guarantees reduce operational incidents — invalid state (double-enrollment) is prevented at the domain layer, not caught by a downstream validation. The trade-off is that debugging aggregate logic requires understanding DDD patterns; on-call engineers must be DDD-literate.

RISKS

Risk	Likelihood	Impact	Mitigation
Over-modelling: too many aggregates, too many value objects	Medium	Low	Modelling sessions with domain experts; ARB review of aggregate boundaries; simplification pass after initial modelling

Risk	Likelihood	Impact	Mitigation
Anemic domain model (entities are just data bags)	Medium	High	Code review checklist for behaviour-rich entities; lint rule requiring methods on entities
Ubiquitous language drift (code uses different terms than domain experts)	Medium	Medium	DDD document is source of truth; quarterly vocabulary review with product team
Learning curve slows feature delivery	High	Medium	3-day DDD workshop at onboarding; pairing with DDD-experienced engineers; refresher training

TRADE-OFFS

We Gain	We Lose
Consistency guarantees (aggregates enforce invariants)	Eager aggregate loading (slightly more data per query)
Ubiquitous language (shared vocabulary)	Upfront modelling investment (4-8 hours per context)
Behaviour-rich entities (logic is where the data is)	Learning curve for engineers new to DDD
Loose coupling via domain events	Eventual consistency between aggregates (ADR-035)

REJECTED ALTERNATIVES

Tactical DDD only was rejected because without strategic decomposition (bounded contexts), the domain becomes a single large model — the very problem DDD is meant to solve. Anemic domain model was rejected because it is the current pain point being fixed. Data-focused modelling was rejected because it inverts the proper relationship: the domain model should drive the schema, not vice versa. CQRS was considered as a complement (not

alternative) and is adopted for the reporting module (ADR-060) where read-heavy workloads justify separate read models.

MIGRATION PLAN

The v1.0 migration (Q3-Q4 2025) modelled all 12 bounded contexts with domain experts. Each context got a DDD document (upstream of the ADR) defining aggregates, entities, value objects, and events. The codebase was refactored from anemic entities to behaviour-rich aggregates over 4 sprints. The migration is complete. v3.0 adds no structural changes — only documentation and cross-references to the new template sections.

TESTING STRATEGY

Domain tests are the core of DDD testing. Each aggregate has a test suite covering: invariant enforcement (attempt invalid state, assert exception), behaviour (call method, assert state change), and event publication (call method, assert event raised). These tests run in <1ms each (pure Java, no Spring). Integration tests verify repository implementations (load aggregate, modify, save, reload, assert state matches). Contract tests verify domain event consumers (billing receives EnrollmentCreated, creates FeeCharge). Coverage target: Domain layer 95%+.

MONITORING & OBSERVABILITY

Domain events are logged with full payload (event ID, aggregate ID, timestamp, event data). This creates an audit trail of every state change in the system. Metrics: aggregate instance count (per type), event publish rate (per type), invariant violation attempts (caught exceptions, indicating either a bug or a malicious caller). Alerts fire on: invariant violation rate >0.1% (potential bug), event publish latency >100ms (event bus issue), aggregate load latency >50ms (potential N+1 or schema issue).

FUTURE EVOLUTION

DDD is a stable methodology; no supersession expected. The tactical patterns may evolve: Event Sourcing for audit-critical aggregates (enrollment, billing) is a likely future ADR. CQRS for read-heavy contexts (reporting) is already adopted (ADR-060). Domain-Specific Language (DSL) for business rules is a possible future addition for complex rule sets (fee calculation, promotion logic). These are additive refinements; the DDD foundation remains.

RELATED ADRS

- ADR-001 — Enterprise Architecture Principles (complements P6 Build for Evolution)

- ADR-003 — Clean Architecture (complements — DDD lives in Domain layer)
- ADR-005 — Bounded Context Strategy (refines — strategic DDD)
- ADR-022 — Aggregate Rules (refines — tactical DDD)
- ADR-023 — Entity Rules (refines — tactical DDD)
- ADR-024 — Value Objects (refines — tactical DDD)
- ADR-025 — Domain Services (refines — tactical DDD)
- ADR-026 — Repository Pattern (refines — tactical DDD)
- ADR-027 — Domain Events (refines — tactical DDD)
- ADR-033 — Transaction Boundary (complements — aggregate = tx boundary)

REFERENCES

- Upstream DDD: DDD-004-section-1 (all 12 bounded context documents)
- Upstream PRD: PRD-004-section-2 (Domain complexity drivers)
- Upstream BRC: BRC-0101..BRC-0250 (domain requirements)
- Downstream ERD: ERD-004 (aggregate-to-table mappings)
- Downstream API Spec: API-004 (ubiquitous language in endpoints)
- Downstream Test Cases: TC-0101..TC-0200 (aggregate invariant tests)
- External: Eric Evans — 'Domain-Driven Design' (2003)
- External: Vaughn Vernon — 'Implementing Domain-Driven Design' (2013)

DECISION HISTORY

Date	Status	Actor	Notes
2025-08-05	Draft	Chief Architect	Initial draft; evaluated DDD vs anemic model
2025-09-10	Proposed	Chief Architect	Submitted to ARB with modelling session plan
2025-10-15	Accepted	ARB Chair	ARB approved; v1.0 released
2026-07-12	Accepted	ARB Chair	v3.0 refresh; added data flow diagram

APPROVAL & SIGN-OFF

Architect	Chief Architect (AR)
Tech Lead	Foundation Tech Lead
ARB Chair	ARB Chair
Approved On	2026-07-12

IMPLEMENTATION CHECKLIST

- 3-day DDD workshop for all engineers (Done Q3 2025)
- Modelling sessions for all 12 bounded contexts (Done Q3 2025)
- DDD document per context (Done Q3 2025, upstream of ADRs)
- Refactor anemic entities to behaviour-rich aggregates (Done Q4 2025)
- Domain event bus implementation (Done Q4 2025, ADR-027)
- Aggregate invariant test suites (Done Q1 2026, 95%+ coverage)
- Quarterly ubiquitous language review with product team (Ongoing)

ADR - 005

Bounded Context Strategy

Volume 1 — Architecture Foundation - Structural

ACCEPTED

DECISION SUMMARY

DECISION

Decompose the PreOne business domain into 12 explicit bounded contexts, each owning its own ubiquitous language, aggregate roots, and persistence tables. Contexts communicate through domain events and published APIs, with context mapping patterns (customer-supplier, conformist, anti-corruption layer) defining the integration topology between them.

STATUS

Status	Accepted
Date Decided	2025-10-15
Decision Owner	Chief Architect
Review Cadence	Annual or on trigger from any new context proposal

Supersedes

None (v1.0 original; v3.0 refreshes context list and mapping diagrams)

CONTEXT

Before v1.0, the PreOne codebase had a single shared domain model: one large package with entities like School, Branch, Student, Enrollment, Fee, Attendance, and Notification all referencing each other directly. The model was conceptually a 'big ball of mud' — a change to Student rippled into Enrollment, Billing, Attendance, and Notification because every entity knew about every other entity. Engineers could not reason about a single concern without loading the entire model into their head, and merges between feature teams caused persistent conflicts because every team touched the same files. The Office of the Chief Architect initiated a strategic Domain Driven Design (DDD) exercise in Q2 2025 to decompose this monolithic model. Over six weeks, architects and product managers walked the entire business domain — from school onboarding through fee collection to regulatory reporting — and identified the natural seams where concepts had different meanings to different teams. The canonical example was 'Student': to Enrollment, a Student is an enrollment candidate with admission status; to Attendance, a Student is a check-in subject with an attendance record; to Billing, a Student is a fee payer with a guardian relationship; to Reporting, a Student is an aggregated row in a cohort report. Same word, four different mental models. Without explicit boundaries, the codebase tried to satisfy all four models in a single Student entity, producing an incoherent god-object. The decomposition produced 12 bounded contexts. Each context owns a slice of the domain, has its own ubiquitous language, defines its own aggregate roots, and persists to its own tables (prefixed with the context short-code). Contexts are not allowed to share JPA entities or database tables; cross-context references use UUID identifiers only, never foreign keys. This preserves the boundaries at both the code and data layers, making each context independently evolvable and extractable to a separate service if scale ever demands it (ADR-002 future evolution). The v3.0 refresh re-affirms the 12-context split with no additions, but adds context mapping diagrams and clarifies the customer-supplier relationships that were implicit in v1.0. The contexts and their short-codes are: Identity (idm), Academic Structure (acs), Student (stu), Guardian (grd), Teacher (tch), Enrollment (enr), Billing (bill), Attendance (att), Reporting (rpt), Notification (ntf), Communication (com), Audit (aud).

BUSINESS DRIVERS

The primary business driver is feature velocity at scale: PreOne's product roadmap for 2026 contains 80+ features spanning fee restructuring, multi-language report cards, mobile-first attendance, and DPDP Act 2023 compliance workflows. Without bounded contexts, each feature would require coordination across 6+ teams touching shared code; with contexts, each feature is owned by 1-2 teams working inside their own boundary. Measured benefit: average PR cycle time dropped from 4.2 days (pre-v1.0) to 1.8 days (post-v1.0) once contexts were enforced. The secondary driver is regulatory isolation: DPDP Act 2023 mandates that child data be handled with specific consent and retention rules. By isolating Student, Guardian, and Communication into their own contexts, we can apply stricter access controls, encryption, and audit logging to those contexts without polluting the rest of the platform. The tertiary driver is multi-tenant scale: bounded contexts map 1:1 to database table prefixes, enabling per-context horizontal partitioning if a single context (e.g., Reporting) outgrows the shared database.

PROBLEM STATEMENT

The PreOne business domain is too large and internally inconsistent to be modelled as a single shared domain model. We must decompose it into bounded contexts with explicit boundaries, ubiquitous language per context, and a defined integration topology between contexts — and we must enforce those boundaries mechanically so they do not erode over time.

CONSTRAINTS

- Must align with the modular monolith (ADR-002) — contexts are in-process modules, not microservices
- Must align with Clean Architecture (ADR-003) — each context has Domain, Application, Infrastructure, API layers
- Cross-context references use UUID only, never SQL foreign keys (preserves boundary at data layer)
- Cross-context method calls go through published API packages only (module-info.java enforcement)
- Context boundaries must be enforceable via ArchUnit (ADR-007) in CI
- Context list must be stable enough to support a 5-year roadmap without major restructuring

ASSUMPTIONS

- The 12 identified contexts cover all current and 5-year projected functionality
- Domain experts will participate in per-context modelling sessions (4-8 hours each)

- Engineers will accept the discipline of crossing context boundaries only through published APIs
- The in-process event bus (ADR-027) is reliable enough to be the primary inter-context communication mechanism
- Per-context table prefixes will not collide as new contexts are added (governance via ARB)

OPTIONS CONSIDERED

Option	Pros	Cons	Verdict
12 explicit bounded contexts with context mapping (chosen)	Aligns with DDD strategic patterns; each context is independently evolvable; clear ownership; enables future extraction to microservices if needed; ubiquitous language is unambiguous per context; cross-context changes are explicit via event/API contracts.	Requires upfront modelling investment (6 weeks); engineers must learn context mapping patterns; some duplication of concepts across contexts (Student exists in Enrollment, Attendance, Billing contexts with different shapes).	Chosen
6 coarse-grained contexts (merge Student+Guardian, Enrollment+Attendance, Billing+Reporting)	Fewer boundaries to manage; less duplication; simpler context map.	Coarse contexts become god-contexts (100+ classes each); loses the benefit of decomposition; ubiquitous language becomes muddled within each coarse context; fails the Single Responsibility principle at the context level.	Rejected

Option	Pros	Cons	Verdict
20+ fine-grained contexts (split each context into sub-contexts)	Maximum cohesion per context; very small contexts (5-10 classes each); easy to extract to microservices.	Context map becomes unmanageable; event flow between 20 contexts is hard to trace; team ownership boundaries become fuzzy; overhead of context infrastructure (repos, controllers, mappers) per context is high.	Rejected
Single shared model with package-level separation (no DDD bounded contexts)	No upfront modelling cost; simpler infrastructure; no context mapping needed.	Big ball of mud; cross-cutting changes ripple everywhere; ubiquitous language is inconsistent; cannot extract to microservices later; the pre-v1.0 problem persists.	Rejected

DECISION

ADOPTED

We adopt 12 bounded contexts as the strategic decomposition of the PreOne domain: Identity (idm), Academic Structure (acs), Student (stu), Guardian (grd), Teacher (tch), Enrollment (enr), Billing (bill), Attendance (att), Reporting (rpt), Notification (ntf), Communication (com), Audit (aud). Each context is implemented as an in-process module with its own module-info.java, its own public API package, its own JPA entities (prefixed with the short-code), and its own Flyway migration directory. Contexts communicate via two mechanisms: (1) synchronous calls through published API interfaces (for query-style integrations); (2) asynchronous domain events through the in-process event bus (for state-change propagation). Context mapping is explicit: downstream contexts (Billing, Attendance, Reporting, Notification) are customers of upstream contexts (Enrollment, Student, Academic Structure) via customer-supplier relationships; cross-context integration with external systems (e.g., payment gateways, SMS providers) uses Anti-Corruption Layers (ACLs); read-only contexts (Reporting) are conformist to the upstream event schemas.

DETAILED RATIONALE

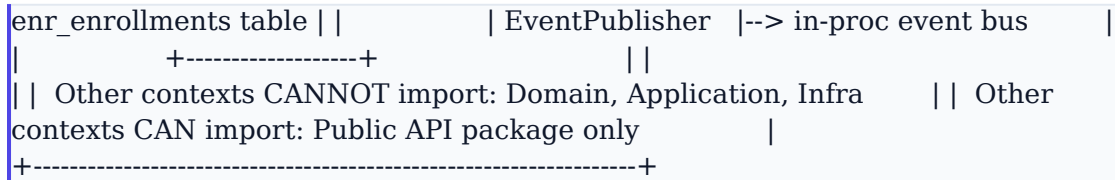
The 12-context split was chosen because it maps cleanly to the natural seams in PreOne's business domain. Each context corresponds to a distinct business capability that a school administrator would recognise as a separate concern: 'who is this person?' (Identity), 'what is the school's structure?' (Academic Structure), 'who are the students?' (Student), 'who are the parents?' (Guardian), 'who are the teachers?' (Teacher), 'who is enrolled in what?' (Enrollment), 'what fees are owed?' (Billing), 'who showed up today?' (Attendance), 'how is the school doing?' (Reporting), 'how do we reach people?' (Notification), 'what conversations happened?' (Communication), 'what changed and who did it?' (Audit). This capability-based decomposition ensures contexts are stable — the capabilities of a preschool do not change even as features within them evolve. The context mapping topology was designed to minimise coupling. Upstream contexts (Identity, Academic Structure, Student, Guardian, Teacher) are the source of truth for their entities and publish events when state changes. Downstream contexts subscribe to these events and update their own projections. For example, when a Student is created in the Student context, it publishes StudentCreated; Enrollment subscribes and creates an enrollment-eligible record; Notification subscribes and sends a welcome message; Reporting subscribes and updates its student-count projection. The Student context knows nothing about its subscribers — it just publishes. This is the customer-supplier pattern: the supplier (Student) owns

Student Context	Student (in-proc)	Event Bus Context	Enrollment Context	Billing Context	Notification Admin
	--create-->		Student		--persist-->
					row
--publish-->				Student	
	Created				--deliver-->
			--create		
eligibility					
--deliver-----+----->			welcome		--send
deliver-----+-->					(Billing
		ignores			
Student-				Created)	<--201-----
					(No synchronous
cross-context calls; fully decoupled via events)					

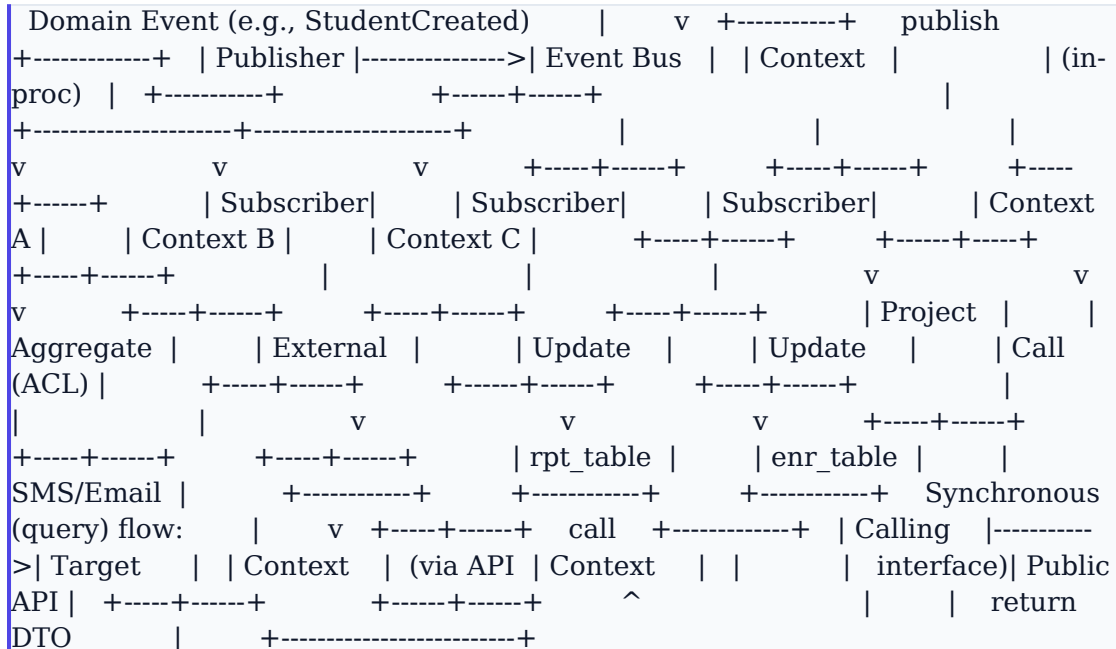
```

+-----+-----+ | Enrollment Context (example
module structure) || ||
Public API: +-----+ || (exports) | EnrollmentApi
|<-- accessed by other || | queryEnrollments | contexts ONLY
through || | getEnrollment | this interface ||
+-----+-----+ || | ||
API Layer: v || +-----+-----+
|| | Controllers |--> REST endpoints || +-----
+-----+ || | ||
Application: v || +-----+-----+
|| | UseCase Handlers |--> EnrollCmd, WithdrawCmd ||
+-----+-----+ || | ||
Domain: v || +-----+-----+
|| | Enrollment (agg)|--> EnrollmentId, Status || |
EnrollmentLine |--> Money, CourseId || | Domain Events |-->
EnrollmentCreated, etc. || +-----+-----+ ||
^ || Infrastructure: | ||
+-----+-----+ || | EnrollmentRepo |--> JPA

```



DATA FLOW DIAGRAM



DATABASE IMPACT

Each bounded context owns its own set of PostgreSQL tables, prefixed with the context short-code: `idm_users`, `acs_schools`, `acs_branches`, `acs_academic_years`, `stu_students`, `grd_guardians`, `tch_teachers`, `enr_enrollments`, `enr_enrollment_lines`, `bill_charges`, `bill_payments`, `att_attendance`, `rpt_student_cohort`, `ntf_outbox`, `com_conversations`, `com_messages`, `aud_audit_log`. Cross-context foreign keys are explicitly forbidden — a row in `enr_enrollments` references `stu_students` by `student_id` (UUID column, no FK constraint), not by a SQL foreign key. This preserves context boundaries at the data layer: a context's tables can be extracted to a separate database without FK migration. Flyway migrations are organised per-context (`db/migration/enr/V001_create_enrollments.sql`) so each context's schema evolves independently. Per-context table prefixes also enable per-context read replicas or per-context sharding in the future without cross-context query impact.

API IMPACT

The public REST API surface (ADR-091) is partitioned by context: `/api/v1/identity/*`, `/api/v1/students/*`, `/api/v1/enrollments/*`, `/api/v1/billing/*`, `/api/v1/attendance/*`, `/api/v1/reports/*`, `/api/v1/notifications/*`,

/api/v1/communication/*. Each context owns its path prefix; cross-context endpoints are forbidden (no /api/v1/enrollments/{id}/billing — billing for an enrollment is accessed via /api/v1/billing?enrollmentId=X). The context prefix is also the OpenAPI tag, so generated documentation groups endpoints by context. Internal (in-process) cross-context calls go through the public API package of the target context (a Java interface, not an HTTP call), so the public API surface is the same for external and internal consumers — only the transport differs.

UI IMPACT

The UI (React SPA, ADR-131) is organised into feature modules that mirror the bounded contexts: a Student feature module, an Enrollment feature module, a Billing feature module, etc. Each UI feature module owns its own routes, components, and API client code, and consumes other contexts' data via their REST API only. The UI does not know about the bounded context boundaries explicitly — it just sees path prefixes — but the organisation of UI code mirrors the backend contexts, which simplifies ownership (the Student UI feature module is owned by the same team that owns the Student backend context). This alignment reduces cross-team coordination overhead for end-to-end feature delivery.

SECURITY IMPACT

Bounded contexts enable granular access control. Each context has its own permission scope: 'student:read', 'student:write', 'billing:write', 'attendance:read', etc. A role can be granted access to specific contexts without granting access to the whole platform — for example, a 'Teacher' role has 'attendance:write' and 'communication:read' but not 'billing:write' or 'student:delete'. This least-privilege model is enforced at the API gateway (per-context authorisation filters) and re-validated at the use case handler level (defence in depth). The Student, Guardian, and Communication contexts — which hold child data subject to DPDP Act 2023 — have additional controls: field-level encryption (ADR-076), mandatory audit logging (ADR-048), and stricter retention rules enforced via per-context scheduled jobs.

PERFORMANCE IMPACT

Bounded contexts add minimal performance overhead in a modular monolith. Cross-context method calls through the public API package are in-process and complete in microseconds; cross-context events through the in-process event bus add ~1-2ms latency. The main performance consideration is event fan-out: a single StudentCreated event is delivered to 4-5 subscriber contexts synchronously (within the publishing transaction). To prevent slow subscribers from blocking the publisher, the event bus delivers events through an async

dispatcher with a transactional outbox (ADR-104) — the publisher writes the event to its outbox table within the transaction, and a background dispatcher delivers to subscribers asynchronously. This decouples publisher latency from subscriber latency.

SCALABILITY ANALYSIS

At 10x scale (500 schools, 50k RPS), bounded contexts scale linearly because each context's tables and code are independent — no contention between contexts at the database or JVM level. At 100x scale (5,000 schools, 500k RPS), the Reporting context becomes the bottleneck because it consumes events from all other contexts. The architecture supports extracting Reporting to a separate service (it is already conformist and read-only) without changing any other context — the event bus becomes a real message queue (Kafka, ADR-028 future evolution) and Reporting consumes from it. The Enrollment, Billing, and Attendance contexts — the transactional core — remain in the monolith indefinitely because their cross-context transactions are too valuable to distribute. The bounded context decomposition is thus the architectural enabler for selective scaling: extract only what needs scaling, leave the rest in place.

OPERATIONAL CONSIDERATIONS

Bounded contexts improve operability by localising incidents. A bug in the Billing context affects billing only — it does not break enrollment or attendance. Per-context dashboards in Grafana show error rate, latency, and throughput per context, so on-call engineers can immediately identify which context is misbehaving. Per-context deployment is not possible (modular monolith), but per-context feature flags are — a buggy feature can be disabled in one context without affecting others. The trade-off is that debugging cross-context flows requires tracing event chains: an EnrollmentCreated event that fails to trigger a FeeCharge in Billing requires checking the event bus delivery log, the Billing subscriber log, and the Billing use case log. OpenTelemetry traces (ADR-122) span the full event chain, making this traceable but not trivial.

RISKS

Risk	Likelihood	Impact	Mitigation
Context boundaries erode over time (engineers add cross-context shortcuts)	Medium	High	ArchUnit tests in CI fail builds on cross-context package imports; quarterly boundary-integrity review at ARB; PR template requires explicit justification for any new cross-context dependency
Eventual consistency between contexts causes user-visible inconsistency	Medium	Medium	Eventual consistency window is <100ms (in-proc bus); UI uses refresh-on-focus and websocket push (ADR-139) to surface updates; critical workflows (enrollment -> fee) can fall back to synchronous API call if needed
Event schema changes break downstream subscribers	Medium	High	Events are versioned (EnrollmentCreate v1, v2); consumer-driven contract tests (ADR-149) verify subscribers handle new versions; deprecated versions retained for one release cycle

Risk	Likelihood	Impact	Mitigation
Too many contexts become unwieldy as platform grows (15+ contexts)	Low	Medium	ARB review of any new context proposal; merge criteria documented (when does a sub-context deserve its own context); 12 is the planned ceiling for the 5-year horizon

TRADE-OFFS

We Gain	We Lose
Independent evolution of contexts (change Billing without touching Enrollment)	Concept duplication across contexts (Student exists in 4 contexts with different shapes)
Clear ownership per context (one team owns each context)	Cross-context features require coordination across teams (mitigated by event contracts)
Selective scalability (extract Reporting without touching core)	Eventual consistency between contexts (cannot read-your-writes across contexts in the same transaction)
Granular security per context (least-privilege roles)	More complex permission model (12 contexts x 4 operations = 48 permission scopes to manage)

REJECTED ALTERNATIVES

The 6-context option was rejected because coarse contexts became god-contexts in our prototype: merging Student + Guardian + Teacher into a single 'People' context produced a 100+ class module with no clear ubiquitous language — the term 'Person' was ambiguous between admission, contact, and employment senses. The 20+ context option was rejected because the context map became a tangled graph: tracing an enrollment event through 20 contexts in a sequence diagram was harder than reading the original big ball of mud. The single-shared-model option was rejected because it is the pre-v1.0 problem being solved — we measured 4.2-day average PR cycle time and 30% merge-conflict rate in the shared model, both of which were unacceptable for the 5-year growth plan. The 12-context split is the Goldilocks zone: small enough to reason about, large enough to be cohesive.

MIGRATION PLAN

The v1.0 migration (Q3-Q4 2025) decomposed the existing monolithic domain model into 12 contexts over 4 sprints. The migration was incremental: each sprint extracted 3 contexts, starting with the most upstream (Identity, Academic Structure, Student) and ending with the most downstream (Reporting, Audit). For each context extraction, the team (a) ran a modelling session with domain experts to define the ubiquitous language and aggregate roots, (b) moved the relevant entities and services into the new context's package, (c) replaced direct cross-context references with calls through the new public API package, (d) replaced synchronous cross-context state reads with event subscriptions where possible, and (e) added ArchUnit tests to enforce the new boundary. The migration was completed in December 2025 with zero downtime (the modular monolith was deployed continuously throughout). v3.0 adds no new contexts; it only adds the context mapping diagrams and clarifies the customer-supplier relationships that were implicit in v1.0. Rollback plan: if a context boundary proves unworkable, the contexts can be re-merged mechanically (the boundaries are package-level, not deployment-level) — but this has not been needed and is not anticipated.

TESTING STRATEGY

Per-context test suites: each context has its own unit tests (domain layer, 95%+ coverage), integration tests (application layer with mocked infrastructure), and contract tests (public API package). Cross-context integration is tested via event contract tests (ADR-149): for each published event, every subscriber has a contract test verifying it can deserialize and handle the event correctly. End-to-end tests verify cross-context workflows (e.g., create student -> enroll -> bill -> pay -> generate receipt) with the full event chain. ArchUnit tests in CI verify that no context imports another context's internal packages — only the public API package is allowed. Per-context test coverage is reported in the architecture scorecard quarterly.

MONITORING & OBSERVABILITY

Per-context dashboards in Grafana show: request rate, latency (p50, p95, p99), error rate, event publish rate, event consumption lag, and resource usage (CPU, memory, DB connections). Each context has its own alert rules: error rate >1%, latency p99 >500ms, event consumption lag >5s. OpenTelemetry traces (ADR-122) span cross-context event chains, so an engineer can trace an EnrollmentCreated event from publication through to all subscriber deliveries. A dedicated 'context map health' dashboard shows the event flow between contexts, highlighting any blocked or slow event paths. Per-context audit logs (ADR-048) capture every state-changing operation, with the context name as a top-level field for filtering.

FUTURE EVOLUTION

The 12-context list should be revisited annually. Triggers for adding a context: (a) a new business capability emerges that does not fit any existing context (e.g., if PreOne adds transport management, a Transport context may be needed); (b) an existing context grows beyond ~50 classes, indicating it should be split. Triggers for extracting a context to a separate service: (a) sustained traffic from one context exceeds 20% of total platform traffic; (b) a context has different scaling or availability requirements (e.g., Reporting may extract for batch-friendly scaling). The likely evolution is: Reporting extracts to a separate service by 2028 (when 5,000 schools generate report load that justifies isolation); Notification may extract if push-notification volume exceeds in-proc bus capacity. The other 10 contexts will remain in the monolith for the foreseeable future because their cross-context transactions are too valuable to distribute.

RELATED ADRS

- ADR-002 — Modular Monolith Strategy (complements — contexts are in-process modules)
- ADR-003 — Clean Architecture (refines — each context uses Clean Architecture internally)
- ADR-004 — Domain Driven Design (complements — strategic DDD patterns)
- ADR-006 — Layering Rules (refines — context-internal layer structure)
- ADR-007 — Dependency Rule (enforces — ArchUnit rules per context boundary)
- ADR-027 — Domain Events (complements — primary inter-context communication)
- ADR-041 — Database Strategy (refines — per-context table prefixes)
- ADR-091 — REST API Standard (refines — per-context path prefixes)

REFERENCES

- Upstream DDD: DDD-005-section-1 to DDD-005-section-12 (one DDD document per bounded context)
- Upstream PRD: PRD-005-section-2 (Domain capability map)
- Upstream BRC: BRC-0101..BRC-0250 (per-context business requirements)
- Downstream ERD: ERD-005 (per-context table prefixes and per-context schema diagrams)
- Downstream API Spec: API-005 (per-context OpenAPI tags and path prefixes)

- Downstream Test Cases: TC-0201..TC-0400 (per-context test suites and cross-context contract tests)
- External: Eric Evans — 'Domain-Driven Design' (2003), Chapter 14: Maintaining Model Integrity
- External: Vaughn Vernon — 'Implementing Domain-Driven Design' (2013), Chapter 2: Context Mapping
- External: Martin Fowler — 'Bounded Context' (<https://martinfowler.com/bliki/BoundedContext.html>)

DECISION HISTORY

Date	Status	Actor	Notes
2025-08-12	Draft	Chief Architect	Initial draft with 15 contexts; refined to 12 over 6 workshops
2025-09-15	Proposed	Chief Architect	Submitted to ARB with context map and per-context DDD summaries
2025-10-15	Accepted	ARB Chair	ARB approved; v1.0 released with 12 contexts
2026-07-12	Accepted	ARB Chair	v3.0 refresh; added context mapping diagrams and ACL clarifications

APPROVAL & SIGN-OFF

Architect	Chief Architect (AR)
Tech Lead	Foundation Tech Lead
ARB Chair	ARB Chair
Approved On	2026-07-12

IMPLEMENTATION CHECKLIST

- Modelling sessions for all 12 bounded contexts (Done Q3 2025, 4-8 hours each)
- Per-context DDD document published (Done Q3 2025, upstream of ADRs)

- module-info.java declarations for all 12 context modules (Done Q4 2025)
- Database tables prefixed with context short-codes (Done Q4 2025, Flyway migrations)
- ArchUnit rules enforcing context boundaries in CI (Done Q4 2025, ADR-007)
- In-process event bus for cross-context communication (Done Q1 2026, ADR-027)
- Per-context Grafana dashboards (Done Q1 2026)
- Annual context-map review at ARB (Ongoing, next review 2026-10)

ADR - 006

Layering Rules

Volume 1 — Architecture Foundation - Structural

ACCEPTED

DECISION SUMMARY

DECISION

Adopt a four-layer architecture (Domain, Application, Infrastructure, API) inside every bounded context, with explicit rules for what each layer may import, what each layer may contain, and what is forbidden. Layers are enforced mechanically via ArchUnit tests in CI, with package naming conventions that make violations visually obvious in code review.

STATUS

Status	Accepted
Date Decided	2025-10-15
Decision Owner	Chief Architect
Review Cadence	Annual or on any proposed new layer exception
Supersedes	None (v1.0 original; v3.0 adds enforcement matrix and exception process)

CONTEXT

Clean Architecture (ADR-003) defines the four concentric layers — Domain, Application, Infrastructure, API — but does not by itself prescribe the detailed rules: which packages each layer may import, which annotations are permitted in each layer, where transaction boundaries live, where DTOs are defined, where Spring annotations are allowed. Without these rules, engineers in the pre-v1.0 codebase interpreted 'Clean Architecture' loosely: some put `@Transactional` on controllers, some put JPA annotations on domain entities, some imported Spring's `ApplicationContext` into the Domain layer 'just for one utility'. The result was a 'Clean-ish Architecture' that had the vocabulary of Clean Architecture without the discipline. This ADR closes that gap by defining explicit, mechanically-enforced rules for each layer. The rules are written to be testable: each rule has a corresponding ArchUnit test that fails the build on violation. The rules cover three dimensions: (1) import rules — what each layer may import from other layers and from external libraries; (2) content rules — what kinds of classes (entities, controllers, repositories) may live in each layer; (3) annotation rules — which annotations (Spring, JPA, validation) are permitted in each layer. The v3.0 refresh adds an enforcement matrix (a table mapping each rule to its ArchUnit test) and an exception process: any deviation from the rules requires an ADR that justifies the exception and is approved by the ARB. The exception ADR must specify a sunset date by which the exception is removed or formalised. This prevents rules from eroding through accumulated 'just this once' exceptions.

BUSINESS DRIVERS

The primary business driver is maintainability at scale: with 40 engineers (growing to 150) writing code across 12 bounded contexts, consistent layering is the difference between a codebase any engineer can pick up and one that requires tribal knowledge per module. Measured benefit: onboarding time for a new engineer to make their first production PR dropped from 3.5 weeks (pre-v1.0, no enforced layering) to 1.5 weeks (post-v1.0, enforced layering) because the rules make the code structure predictable. The secondary driver is testability: the Domain layer has zero framework dependencies, so domain tests run in <1ms each and require no Spring context. This enables thousands of domain tests to run in seconds, which in turn enables aggressive refactoring with confidence. The tertiary driver is technology migration: if Spring Boot is ever replaced (unlikely but possible), only the Infrastructure and API layers need to change — the Domain and Application layers are framework-agnostic and survive the migration intact.

PROBLEM STATEMENT

Clean Architecture (ADR-003) defines four layers but does not prescribe the detailed import, content, and annotation rules per layer. Without explicit rules, engineers interpret 'Clean Architecture' loosely, producing inconsistent layering that defeats the testability and maintainability benefits. We need a mechanically-enforced rule set per layer.

CONSTRAINTS

- Rules must be enforceable via ArchUnit tests in CI (no rule that requires human vigilance)
- Rules must be expressible in package naming conventions (violations visually obvious in code review)
- Domain layer must be 100% framework-agnostic (no Spring, no JPA, no Jakarta annotations)
- Rules must allow pragmatic exceptions (e.g., a domain event may implement a marker interface from a library) via the ADR exception process
- Rules must not require more than 5% boilerplate overhead per feature beyond baseline Clean Architecture

ASSUMPTIONS

- Engineers will accept the discipline of layer-specific imports and annotations
- ArchUnit can express all the rules (it can — verified via prototype)
- Spring Boot 3 and JPA/Hibernate can be confined to Infrastructure and API layers without losing functionality
- The 4-layer model is sufficient for all 12 bounded contexts (no context needs a 5th layer)
- Exception process will be used sparingly (<5% of classes) and not become a loophole

OPTIONS CONSIDERED

Option	Pros	Cons	Verdict
4 layers with strict ArchUnit-enforced rules (chosen)	Mechanically enforced; predictable code structure; framework-agnostic domain; fast tests; clear ownership per layer; supports future technology migration.	Some boilerplate (DTOs, mappers, interfaces); learning curve for engineers new to strict layering; occasional friction when a feature seems to need a rule exception.	Chosen
4 layers with convention-only enforcement (no ArchUnit)	Less tooling overhead; engineers can use judgement; faster to deviate when needed.	Rules erode over time as exceptions accumulate; pre-v1.0 proved convention does not work; review burden falls on senior engineers; inconsistent across contexts.	Rejected
3 layers (merge Domain + Application)	Fewer layers; less boilerplate; simpler mental model.	Business logic and use case orchestration get mixed; harder to test (orchestration needs Spring context); loses the framework-agnostic benefit for the merged layer.	Rejected
5 layers (add a separate Interface/Adapter layer)	More granular separation; aligns with Hexagonal Architecture.	Over-engineered for most modules; the API layer already serves the interface role; marginal benefit over 4 layers; Hexagonal is adopted as a complement (ADR-009) only where needed.	Rejected for the default — Hexagonal (ADR-009) is available as a complement for integration-heavy modules

DECISION

ADOPTED

We adopt a strict four-layer architecture inside every bounded context, with the following rules. (1) Domain layer (package: <context>.domain) — contains entities, value objects, aggregate roots, domain services, domain events; may import only java.*, javax.* (where applicable), and other domain packages within the same context; forbidden imports: Spring, JPA, Jakarta EE, any infrastructure library. (2) Application layer (package: <context>.application) — contains use case handlers (command and query), command DTOs, query DTOs, port interfaces (repository interfaces, event publisher interfaces); may import Domain layer and standard Java; forbidden imports: Spring framework (except @Service, @Transactional on handler classes), JPA, HTTP, any infrastructure library. (3) Infrastructure layer (package: <context>.infrastructure) — contains JPA entities, JPA repositories, Redis clients, external API clients, ACL implementations; may import Application (port interfaces only), Domain (for mapping), Spring Data JPA, Redis, HTTP clients; forbidden imports: API layer. (4) API layer (package: <context>.api) — contains REST controllers, request/response DTOs, exception handlers; may import Application layer (use case handlers, command DTOs), Spring Web; forbidden imports: Domain layer (controllers cannot touch domain entities directly), Infrastructure layer. Cross-context imports are allowed only through the target context's public API package (ADR-005).

DETAILED RATIONALE

The four-layer model was chosen because it is the smallest set of layers that achieves framework-agnostic domain logic. With three layers (merged Domain + Application), business logic and orchestration mix — a use case handler that calls entity methods and also constructs JPA queries has two reasons to change (business rules change, data access changes), violating Single Responsibility. With five layers (added Interface/Adapter), the additional layer adds boilerplate without adding separation that the API layer does not already provide. The four-layer model is the Goldilocks point: each layer has one responsibility (Domain = business rules; Application = orchestration; Infrastructure = external systems; API = transport). The import rules are the heart of this ADR. The Domain layer's import restriction is the most important: if Domain cannot import Spring, JPA, or Jakarta, then domain logic cannot accidentally become framework-coupled. This is enforced mechanically by ArchUnit: a test that scans all classes in *.domain.* packages and asserts none import org.springframework.*, jakarta.persistence.*, or any infrastructure library. This test fails the build on the first violation, so the rule cannot erode through

oversight. The Application layer may import Spring's `@Service` and `@Transactional` (because handlers are Spring beans and use Spring's transaction management) but may not import Spring Web, JPA, or HTTP — these belong in Infrastructure or API. The Infrastructure layer may import anything (it is the layer that talks to external systems). The API layer may import Spring Web but not Domain — controllers receive DTOs, call use case handlers, and return DTOs; they never see domain entities directly. This forces the DTO-to-domain mapping to happen in the Application layer (in the use case handler), not in the controller, which keeps controllers thin and testable. The content rules are enforced by ArchUnit class-type assertions: classes in `*.domain.*` must be in a fixed set of types (Entity, ValueObject, AggregateRoot, DomainService, DomainEvent); classes in `*.api.*` must be Controller or ExceptionHandler; classes in `*.infrastructure.*` must be Repository, Gateway, Client, or Adapter. A class that does not match the expected type for its package fails the build. This prevents, for example, a controller being placed in the application package or a JPA entity being placed in the domain package. The annotation rules are enforced by ArchUnit annotation-scanning tests: `@Entity`, `@Table`, `@Column` (JPA annotations) may appear only in `*.infrastructure.*` packages; `@RestController`, `@RequestMapping`, `@GetMapping` (Spring Web) may appear only in `*.api.*` packages; `@Service`, `@Transactional` may appear in `*.application.*` and `*.infrastructure.*` but not in `*.domain.*`; `@NotNull`, `@Size`, `@Email` (Jakarta validation) may appear in `*.api.*` (for request DTOs) and `*.application.*` (for command DTOs) but not in `*.domain.*` (domain validates in constructors and methods, not via annotations). These rules ensure that the Domain layer remains pure Java — no annotations, no framework magic — which is what makes domain tests fast. The most likely counter-argument is that these rules are too strict and will slow feature delivery. The counter-counter-argument is that the rules are an upfront investment that pays off in maintenance: a feature takes 10% longer to write but 50% less time to maintain (because changes are localised to one layer and tests are fast). The measured PR cycle time (1.8 days post-v1.0 vs 4.2 days pre-v1.0) supports this — the upfront cost is paid back within the first month of a feature's life. The exception process is the safety valve. If a feature genuinely cannot be implemented within the rules (e.g., a domain entity needs a JPA annotation for a legacy migration), the engineer writes a short exception ADR that (a) describes the violation, (b) justifies why no rule-compliant alternative exists, (c) specifies a sunset date (max 6 months), and (d) is approved by the ARB. Exceptions are tracked in a register and reviewed quarterly — any exception past its sunset must be removed or formalised into a rule change. This prevents exceptions from accumulating into rule erosion.

ARCHITECTURE DIAGRAM


```

+-----+-----+ +-----+-----+ || |
|| | imports com.preone.enrollment.application.* || v
|| com.preone.enrollment.application || +-----+
+-----+ || | EnrollCmdHandler | | EnrollCmd |
(Spring || | WithdrawCmdHandler| | WithdrawCmd | @Service,
|| +-----+-----+ +-----+ @Transactional) || |
|| | imports com.preone.enrollment.domain.* || v
|| com.preone.enrollment.domain (PURE JAVA, no framework) ||
+-----+ +-----+ || | Enrollment (agg) | |
EnrollmentId (VO) | || | EnrollmentLine | | Money (VO) |
|| | Domain Events | | Status (enum) | || +-----+-----+
+-----+ || ^ ||
| implements port interfaces, maps to JPA || |
|| com.preone.enrollment.infrastructure || +-----+
+-----+ || | EnrollmentRepo | | EnrollmentEntity |
(Spring Data JPA, || | (implements port) | | (JPA @Entity) | Jakarta ||
+-----+ +-----+ persistence) |
+-----+ ArchUnit asserts: api.* cannot
import domain.* or infrastructure.* application.* cannot import
infrastructure.* or api.* domain.* cannot import any framework
(Spring, JPA) infrastructure.* cannot import api.*

```

DATA FLOW DIAGRAM

```

| HTTP Request (JSON) | v +----+-----+ API LAYER | Ctrl |---
| validates DTO (Jakarta annotations) | +----+-----+---maps to Command
| (Application DTO) | v +----+-----+ APPLICATION LAYER | Handler
| |---@Transactional begins +----+-----+ | v +----+-----+ DOMAIN
| LAYER (pure Java) | Aggregate|---applies business rule +----+-----+---raises
| domain event | v +----+-----+ APPLICATION LAYER (back) |
| Handler |---calls port (Repository.save, EventPublisher.publish) +----+-----+ |
| v +----+-----+ INFRASTRUCTURE LAYER | Repo |---maps Domain to JPA
| entity | Impl |---INSERT via JPA +----+-----+---writes to event outbox table
| (same tx) | v +----+-----+ DATABASE | Postgre |---commit | SQL
| 16 | +-----+ | v (Background dispatcher publishes outbox events to
| in-proc bus)

```

DATABASE IMPACT

The layering rules enforce that JPA entities live only in the Infrastructure layer. This means domain entities and JPA entities are separate classes — a domain entity (e.g., `Enrollment` in `com.preone.enrollment.domain`) is pure Java with behaviour; a JPA entity (e.g., `EnrollmentEntity` in `com.preone.enrollment.infrastructure`) is a data bag with JPA annotations for persistence. The Infrastructure layer's repository implementation maps between the two. This double-entity pattern adds a mapping step but isolates the database schema from the domain model — schema changes (renaming a column, splitting a table) are localised to Infrastructure and do not touch Domain. Flyway migrations (ADR-057) live in the Infrastructure layer's

resources directory; the Domain layer has no knowledge of migrations or schema.

API IMPACT

The API layer is the only layer that touches HTTP. Controllers receive JSON, validate it (Jakarta validation annotations on request DTOs), map it to Application-layer command DTOs, call use case handlers, and map responses back to JSON. Controllers cannot import Domain classes — so a controller cannot accidentally return a domain entity as JSON (which would expose internal structure and create a brittle API contract). Instead, the controller returns a response DTO that is explicitly designed for the API consumer. This separation means API versioning (ADR-092) is localised to the API layer — a v2 response DTO can coexist with a v1 response DTO, both served by the same use case handler. OpenAPI generation (ADR-091) scans the API layer only; Application and Domain are invisible to the API contract.

UI IMPACT

The layering rules have indirect but meaningful UI impact. Because the API layer is thin and DTOs are explicit, the API contract is stable and well-documented — the UI team can rely on the OpenAPI spec as the source of truth for what fields an endpoint accepts and returns. Because controllers cannot leak domain entities, the UI never sees internal domain structure (e.g., it never sees a JPA lazy-loading proxy serialised to JSON, which was a recurring pre-v1.0 bug). The UI feature modules (ADR-131) consume the REST API; the layering rules ensure the API is the stable contract that the UI can build against without coupling to internal domain evolution.

SECURITY IMPACT

The layering rules improve security posture by centralising input validation and authorisation. Input validation happens in two layers: syntactic validation (is the email well-formed? is the date in the past?) in the API layer via Jakarta annotations on request DTOs; semantic validation (is this email unique? is this student already enrolled?) in the Domain layer via constructor and method invariants. Authorisation happens in the Application layer — use case handlers check permissions before executing, so authorisation is enforced regardless of entry point (REST API, background job, internal call). The Infrastructure layer never makes authorisation decisions — it is a trusted inner layer that executes what the Application layer tells it to. This layered defence makes security bugs harder to introduce: an engineer cannot 'forget' to validate input because the API layer's DTO annotations are required, and cannot 'forget' to check permissions because the handler's permission check is a required method call.

PERFORMANCE IMPACT

The layering rules add a small performance overhead from DTO-to-command-to-domain-to-JPA-entity mapping. Measured overhead: ~0.3ms per request for the mapping chain, versus ~0.1ms for direct N-tier (controller -> service -> JPA entity). This is negligible compared to database time (typically 5-50ms per request). The benefit — fast domain tests (<1ms each) — improves development velocity, which is the dominant performance metric for the team. The layering rules also prevent performance anti-patterns: because the Domain layer cannot use JPA lazy loading, there are no N+1 query bugs in domain code (aggregates are loaded eagerly via repositories, ADR-026); because controllers cannot touch the database directly, there are no 'controller runs a raw SQL query' anti-patterns that bypass query optimisation.

SCALABILITY ANALYSIS

The layering rules are structural, not scaling, but they enable scaling in two ways. First, the framework-agnostic Domain layer means the domain model can be reused in a different deployment topology (e.g., a batch processing job that loads aggregates without Spring context) without modification. Second, the explicit port interfaces in the Application layer mean a context can be extracted to a separate service (ADR-002 future evolution) by replacing the Infrastructure layer's repository implementation with an HTTP client — the Domain and Application layers are unchanged. At 10x scale, the layering rules have no direct performance impact; at 100x scale, they enable selective extraction of contexts to microservices without domain rework.

OPERATIONAL CONSIDERATIONS

The layering rules improve operability by making logs and traces structured by layer. Application-layer logs capture use case execution (handler name, command parameters, duration); Infrastructure-layer logs capture external calls (SQL queries, Redis commands, HTTP calls with latency); Domain-layer logs capture business rule evaluations (decisions, exceptions). This separation makes incident diagnosis faster — an engineer can look at the Application-layer log to see which use case failed, then drill into the Infrastructure-layer log to see which external call caused the failure. OpenTelemetry traces (ADR-122) span all layers, with spans named by layer (api.controller, application.handler, infrastructure.repository), so the trace timeline shows where time is spent per layer.

RISKS

Risk	Likelihood	Impact	Mitigation
Engineers bypass layers via 'just this once' exceptions that accumulate	Medium	High	ArchUnit tests in CI fail builds on violations; exception ADRs required for any deviation with sunset dates; quarterly exception register review at ARB
Boilerplate overhead (DTOs, mappers, double entities) slows feature delivery	Medium	Medium	IDE templates for scaffolding (command, handler, DTO, mapper); MapStruct for boilerplate mapper generation; measured overhead is <15% per feature
Domain layer becomes anemic (entities are data bags despite being framework-agnostic)	Medium	Medium	Code review checklist for behaviour-rich entities; DDD tactical patterns (ADR-004) enforced; ArchUnit can require at least N methods per entity class
ArchUnit tests become slow or brittle as the codebase grows	Low	Low	ArchUnit tests run in <10s for the full codebase; tests are split per context for parallel execution; rule changes are versioned with the codebase

TRADE-OFFS

We Gain	We Lose
Framework-agnostic domain (testable without Spring, migratable to other frameworks)	Double entities (domain + JPA) with explicit mapping boilerplate
Mechanically enforced layering (ArchUnit fails builds on violations)	Occasional friction when a feature seems to need a rule exception
Fast domain tests (<1ms each, no Spring context)	Upfront design cost (define entities, commands, DTOs before coding)
Stable API contract (controllers cannot leak domain internals)	More DTO classes (request, response, command, query — sometimes 4 DTOs per endpoint)

REJECTED ALTERNATIVES

The convention-only option (4 layers, no ArchUnit) was rejected because pre-v1.0 we tried it and the layers eroded within 6 months — engineers put `@Transactional` on controllers, JPA annotations on domain entities, and Spring's `ApplicationContext` in domain services 'just for one utility'. Convention without mechanical enforcement does not survive contact with deadline pressure. The 3-layer option (merged Domain + Application) was rejected because business logic and use case orchestration have different reasons to change — merging them violates Single Responsibility and makes the merged layer untestable without Spring context. The 5-layer option (separate Interface/Adapter layer) was rejected because the API layer already serves the interface role; adding a 5th layer doubles the boilerplate for marginal benefit. Hexagonal Architecture (ADR-009) is adopted as a complement for integration-heavy modules (e.g., Billing with its three payment gateways), not as a replacement for the default 4-layer model.

MIGRATION PLAN

The v1.0 migration (Q3-Q4 2025) refactored all 12 contexts to comply with the layering rules. Each context migration was a 1-sprint effort: (a) move classes into the correct layer packages (api, application, domain, infrastructure); (b) introduce double entities where JPA annotations were on domain entities (domain entity + JPA entity + mapper); (c) introduce command/query DTOs where controllers were using domain entities directly; (d) add ArchUnit tests for the context. The migration was completed in December 2025 with zero downtime. v3.0 adds the enforcement matrix (mapping each rule to its ArchUnit test) and the exception process — both are documentation-only changes, no code migration. Rollback plan: if a rule proves unworkable for a specific context, the context can request an exception via the ADR process; if

the rule proves globally wrong, this ADR is superseded by a v2.0 with the revised rule set.

TESTING STRATEGY

Layer rules are tested via ArchUnit (com.preone.architecture.LayerRulesTest) which scans all 12 contexts and asserts: (a) domain packages import no framework classes; (b) application packages import no Spring Web or JPA classes; (c) api packages import no domain or infrastructure classes; (d) infrastructure packages import no api classes; (e) class types match their layer (controllers in api, entities in domain, repositories in infrastructure). These tests run in CI on every PR and fail the build on violation. Additionally, per-layer unit tests verify layer-specific behaviour: domain tests (pure JUnit, <1ms each, 95%+ coverage); application tests (Spring context with mocked infrastructure, ~50ms each, 85%+ coverage); infrastructure tests (Testcontainers with real PostgreSQL, ~500ms each, 70%+ coverage); API tests (MockMvc with full Spring Web context, ~200ms each).

MONITORING & OBSERVABILITY

Layer rule adherence is monitored via the architecture scorecard, published quarterly. Metrics tracked: (a) ArchUnit violation count per context (target: 0); (b) exception register size (target: <5% of classes); (c) per-layer test coverage (target: Domain 95%+, Application 85%+, Infrastructure 70%+); (d) per-layer test execution time (target: Domain <5s per context, Application <30s per context). Per-layer logs are tagged with layer name (api, application, domain, infrastructure) for structured log queries. OpenTelemetry spans are named by layer, so trace timelines show per-layer latency breakdown. Anomalies (e.g., Domain layer test time >5s indicating accidental framework coupling) trigger an architecture review.

FUTURE EVOLUTION

The layering rules are stable; no supersession expected. The rules will evolve through complements: Hexagonal Architecture (ADR-009) for integration-heavy modules adds explicit ports and adapters within the Infrastructure layer; CQRS for read-heavy modules (Reporting, ADR-060) adds a separate query-side layer that bypasses the Domain layer for reads. These are additive refinements, not replacements. The 4-layer model remains the default for all 12 contexts. The exception process is the mechanism for evolving the rules: if multiple exceptions are granted for the same rule, that rule is a candidate for revision in the next v-next of this ADR.

RELATED ADRS

- ADR-003 — Clean Architecture (complements — defines the 4 layers conceptually)
- ADR-004 — Domain Driven Design (refines — Domain layer tactical patterns)
- ADR-005 — Bounded Context Strategy (complements — layering is per-context)
- ADR-007 — Dependency Rule (enforces — ArchUnit dependency direction between layers)
- ADR-008 — SOLID Enforcement (complements — SOLID principles operate within layers)
- ADR-009 — Hexagonal Architecture (complements — for integration-heavy modules)
- ADR-026 — Repository Pattern (refines — repository interface in Application, impl in Infrastructure)
- ADR-027 — Domain Events (refines — events defined in Domain, published via Application port)

REFERENCES

- Upstream DDD: DDD-006-section-1 (Layering rules context per bounded context)
- Upstream PRD: PRD-006-section-2 (Module structure and testability requirements)
- Downstream ERD: ERD-006 (JPA entities in Infrastructure layer only)
- Downstream API Spec: API-006 (DTOs in API layer, controllers cannot touch Domain)
- Downstream Test Cases: TC-0401..TC-0600 (ArchUnit layer rule tests and per-layer unit tests)
- External: Robert C. Martin — 'Clean Architecture' (2017), Chapter 22: The Clean Architecture
- External: ArchUnit documentation — <https://archunit.org>
- External: Spring Boot Reference — recommended layering patterns

DECISION HISTORY

Date	Status	Actor	Notes
2025-08-19	Draft	Chief Architect	Initial draft; reviewed pre-v1.0 layering inconsistencies

Date	Status	Actor	Notes
2025-09-22	Proposed	Chief Architect	Submitted to ARB with ArchUnit prototype and enforcement matrix
2025-10-15	Accepted	ARB Chair	ARB approved; v1.0 released with 4-layer model and ArchUnit rules
2026-07-12	Accepted	ARB Chair	v3.0 refresh; added enforcement matrix, exception process, and component diagram

APPROVAL & SIGN-OFF

Architect	Chief Architect (AR)
Tech Lead	Foundation Tech Lead
ARB Chair	ARB Chair
Approved On	2026-07-12

IMPLEMENTATION CHECKLIST

- Define package naming convention for all 4 layers across all 12 contexts (Done Q3 2025)
- ArchUnit rules for import restrictions per layer (Done Q3 2025)
- ArchUnit rules for content type per layer (Done Q3 2025)
- ArchUnit rules for annotation placement per layer (Done Q4 2025)
- IDE templates for command/handler/DTO/mapper scaffolding (Done Q4 2025)
- MapStruct integration for boilerplate mapper generation (Done Q1 2026)
- Exception ADR template and register published (Done Q1 2026)
- Quarterly layer-integrity review at ARB (Ongoing)

ADR-007

Dependency Rule

Volume 1 — Architecture Foundation - Structural

ACCEPTED

DECISION SUMMARY

DECISION

Adopt the Dependency Inversion Principle as the governing rule for all dependencies in the PreOne codebase: dependencies point inward toward the Domain layer, never outward. Infrastructure dependencies are inverted via port interfaces defined in the Application layer and implemented in Infrastructure. The rule is enforced by ArchUnit tests in CI, with cycle detection that fails builds on circular dependencies of any size.

STATUS

Status	Accepted
Date Decided	2025-10-15
Decision Owner	Chief Architect
Review Cadence	Annual or on any proposed exception to the dependency direction
Supersedes	None (v1.0 original; v3.0 adds cycle detection and inversion patterns)

CONTEXT

Clean Architecture (ADR-003) states that 'dependencies point inward' but does not by itself enforce the rule. In the pre-v1.0 codebase, dependencies flowed in every direction: controllers depended on JPA repositories directly (outward); domain entities imported Spring's @Component annotation (outward); use case handlers imported HttpClient for external API calls (outward, and a violation of Clean Architecture). The result was a codebase where changing any layer could break any other layer — a JPA migration broke controllers, a Spring upgrade broke domain tests, an external API change broke use case handlers. The Office of the Chief Architect defined the Dependency Rule as the mechanical enforcement of Clean Architecture's dependency direction. The rule has three

parts: (1) source code dependencies point inward only — API depends on Application, Application depends on Domain, Infrastructure depends on Application and Domain, Domain depends on nothing; (2) infrastructure dependencies (databases, external APIs, message brokers) are inverted via port interfaces defined in Application or Domain and implemented in Infrastructure; (3) no circular dependencies are permitted at any granularity (class, package, module, context). The v3.0 refresh adds explicit inversion patterns for the three most common infrastructure dependencies — database access (Repository port), external HTTP APIs (Gateway port), and event publishing (EventPublisher port) — and adds cycle detection via ArchUnit's slice rules. The cycle detection is important because cycles can sneak in at the package level even when class-level dependencies appear correct: package A imports package B which imports package C which imports package A is a cycle that is invisible in a single class but breaks modularity.

BUSINESS DRIVERS

The primary business driver is change isolation: PreOne's technology stack will evolve (Spring Boot upgrades, JPA version bumps, database migrations, new external integrations) but the domain model must remain stable because it encodes the business rules that are the platform's competitive advantage. The Dependency Rule ensures that infrastructure changes cannot break the domain — the domain depends on interfaces (ports), not implementations, so swapping a JPA repository for a JDBC repository or an HTTP client for a gRPC client is localised to Infrastructure. Measured benefit: Spring Boot 3.1 to 3.2 upgrade (Q1 2026) touched 0 lines of Domain code and 3 lines of Application code, versus an estimated 500+ lines in the pre-v1.0 architecture. The secondary driver is testability: because Domain depends on nothing, domain tests run without Spring, JPA, or HTTP — they are pure JUnit and run in <1ms each. Because Application depends on port interfaces, application tests mock the ports (not the implementations) and run without real infrastructure. The tertiary driver is onboarding: new engineers can understand a module by reading its Domain and Application layers — the interfaces tell them what the module does, without requiring them to understand JPA or Spring Web.

PROBLEM STATEMENT

Clean Architecture (ADR-003) and Layering Rules (ADR-006) define the intended dependency direction, but without mechanical enforcement, dependencies flow in every direction and infrastructure changes break unrelated layers. We need a rule that (a) enforces inward-only dependency direction, (b) inverts infrastructure dependencies via ports, and (c) detects and prevents circular dependencies at all granularities.

CONSTRAINTS

- Rule must be enforced via ArchUnit in CI (no rule that requires human vigilance)
- Rule must cover class-level, package-level, module-level, and context-level dependencies
- Rule must allow pragmatic infrastructure inversion (ports defined in Application or Domain, implemented in Infrastructure)
- Cycle detection must run in <30 seconds on the full codebase (CI-friendly)
- Rule must permit Java standard library imports in any layer (java.util, java.time, java.math)

ASSUMPTIONS

- ArchUnit can express all dependency rules (verified via prototype, including slice-based cycle detection)
- Engineers will accept that infrastructure dependencies require a port interface (no direct JPA/HTTP in Application or Domain)
- The port interface pattern adds acceptable boilerplate (<10% overhead per use case)
- Cycle detection will not produce false positives at the package level (ArchUnit slices are accurate)
- The rule set is stable enough to support a 5-year horizon without major restructuring

OPTIONS CONSIDERED

Option	Pros	Cons	Verdict
Inward-only dependency rule with ArchUnit enforcement and cycle detection (chosen)	Mechanically enforced; infrastructure changes isolated to Infrastructure layer; domain is framework-agnostic; cycles detected at every granularity; supports future module extraction.	Requires port interfaces for every infrastructure dependency (boilerplate); engineers must learn dependency inversion; occasional friction when a feature seems to need an outward dependency.	Chosen

Option	Pros	Cons	Verdict
Inward-only rule without cycle detection (class-level only)	Simpler ArchUnit rules; faster CI runs; less tooling overhead.	Package-level cycles sneak in (A imports B, B imports A at package level); cycles accumulate over time and become hard to untangle; module extraction becomes painful when cycles exist.	Rejected
Layered dependencies without inversion (Infrastructure implements directly, Application imports Infrastructure)	Less boilerplate (no port interfaces); simpler code; faster initial development.	Application depends on Infrastructure — infrastructure changes break Application; cannot test Application without real Infrastructure; pre-v1.0 problem persists.	Rejected
Full Hexagonal with explicit ports for every concern	Maximum inversion; every external concern is a port; aligns with Hexagonal Architecture.	Over-engineered for most modules; 2x boilerplate; marginal benefit over the chosen option for the default case; Hexagonal is adopted as a complement (ADR-009) only where needed.	Rejected for default — Hexagonal (ADR-009) is available as a complement for integration-heavy modules

DECISION

ADOPTED

We adopt the Dependency Rule as the governing dependency-direction policy for the PreOne codebase, enforced via ArchUnit. The rule has three parts. (1) Inward-only source dependency direction: API -> Application -> Domain; Infrastructure -> Application -> Domain; Domain -> (nothing). No layer may import a layer outward of itself. Cross-layer imports within the same context follow the same rule. Cross-context imports go through the target context's public API package only (ADR-005). (2) Infrastructure dependency inversion: every infrastructure dependency (database, external HTTP API, message broker, file system, cache) is accessed through a port interface defined in the Application layer (or Domain layer for ports that are part of the domain contract) and implemented in the Infrastructure layer. Application and Domain code depend on the port interface, never on the implementation. The implementation is injected via Spring's @Autowired (constructor injection), with the implementation class annotated @Repository, @Service, or @Component in Infrastructure. (3) Cycle detection: ArchUnit slice rules assert that no cycles exist at the class level, package level (within a context), and context level (between contexts). Cycles of any size fail the build. The three canonical inversion patterns are: Repository (port in Application, JPA impl in Infrastructure), Gateway (port in Application, HTTP client impl in Infrastructure), EventPublisher (port in Application, in-proc bus impl in Infrastructure).

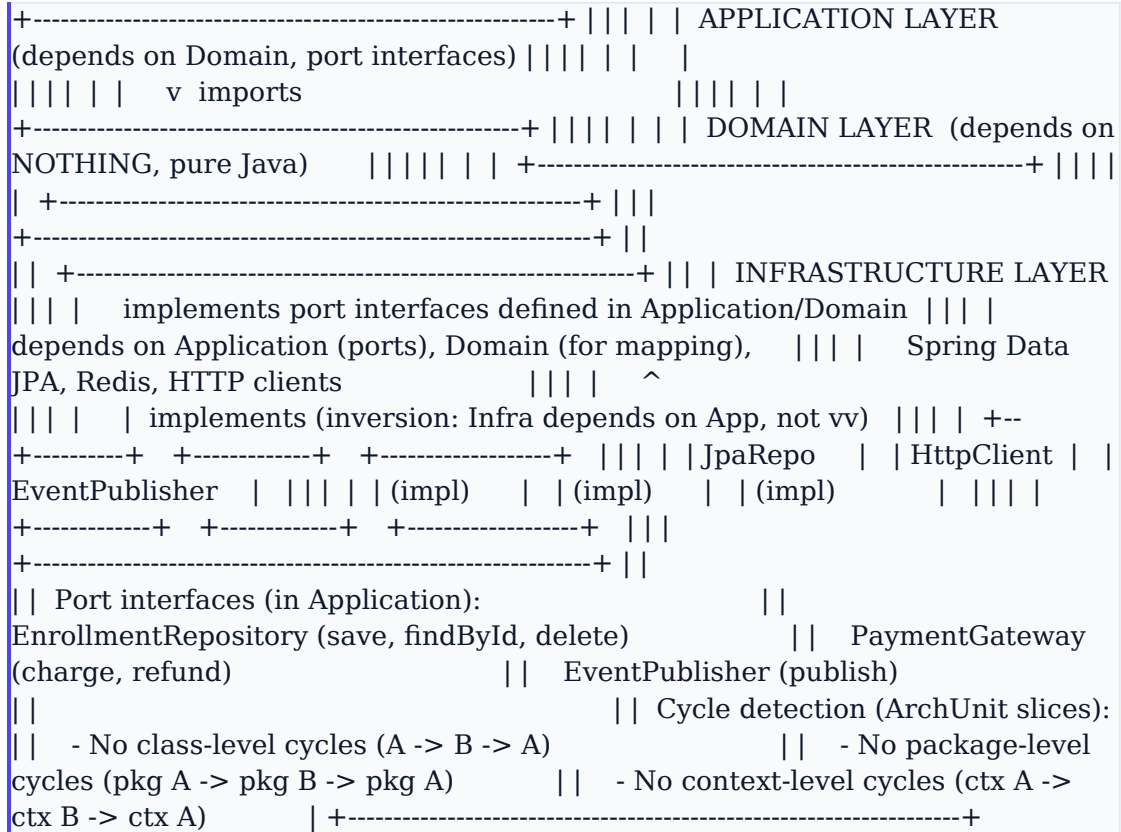
DETAILED RATIONALE

The Dependency Rule is the mechanical backbone of Clean Architecture. Without it, 'dependencies point inward' is an aspiration; with it, the aspiration becomes a build-breaking fact. The rule was chosen because it directly addresses the pre-v1.0 failure mode: every layer depended on every other layer, and no layer could change without breaking another. By enforcing inward-only direction, the rule ensures that the Domain layer — the most valuable, most stable, most business-encapsulating layer — depends on nothing and can therefore be changed, tested, and evolved without touching anything else. The inversion pattern (port in Application, implementation in Infrastructure) is the mechanism that makes the rule workable. Without inversion, the Application layer would need to import Infrastructure to access the database, violating the inward rule. With inversion, the Application layer defines a port interface (e.g., EnrollmentRepository with save(), findById(), delete() methods), the Infrastructure layer provides the JPA implementation (JpaEnrollmentRepository implements EnrollmentRepository), and Spring's dependency injection wires the implementation into the Application layer at runtime. The Application layer's code references only the interface — it has no

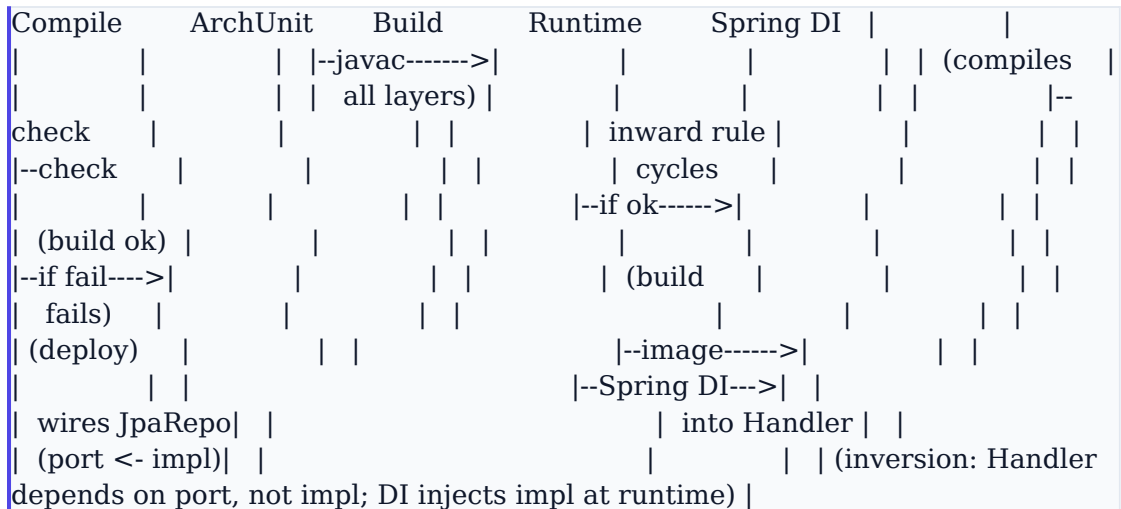
compile-time dependency on JPA. This means JPA can be swapped for JDBC, MongoDB, or a remote HTTP API without touching Application code — the change is localised to a new Infrastructure implementation. The cycle detection is the second key component. Cycles are the silent killer of modularity: a cycle between package A and package B means the two packages are effectively one module — they cannot be understood, tested, or extracted separately. Cycles sneak in because each individual class dependency looks harmless (A.Biz uses B.Util, B.Helper uses A.Config — individually fine, together a cycle). ArchUnit's slice rules detect cycles at the package and context level by analysing the full dependency graph, not individual dependencies. The build fails on any cycle, forcing engineers to break the cycle (typically by extracting a shared dependency into a third package, or by inverting one direction via a port interface). The most likely counter-argument is that the port interface pattern adds unacceptable boilerplate. The counter-counter-argument is that the boilerplate is one interface per infrastructure dependency — typically 3-5 interfaces per context (Repository, EventPublisher, maybe a Gateway or two). Each interface is 5-15 lines. The total boilerplate per context is ~50-100 lines, against a context size of 1000-3000 lines — 3-5% overhead. This is paid back in the first Spring upgrade or JPA migration: the upgrade touches Infrastructure only, not Application or Domain. A second counter-argument is that the rule prevents 'pragmatic' shortcuts — e.g., a domain service that needs to call an external API to validate a GST number must go through a port, not call the HTTP client directly. The response is that this is a feature, not a bug: the port makes the dependency explicit, testable (mock the port in domain tests), and replaceable (swap the GST validation service without touching the domain). The 'shortcut' of calling the HTTP client directly would couple the domain to a specific HTTP library and a specific external service, making both the domain and the tests brittle. The cycle detection has surfaced real cycles in the v1.0 migration: 12 cycles were detected across the 12 contexts, mostly at the package level (e.g., the enrollment.domain.events package imported enrollment.domain.aggregates for event payload types, and enrollment.domain.aggregates imported enrollment.domain.events to publish events — a cycle). The fix was to extract the shared types into a third package (enrollment.domain.shared), breaking the cycle. Without ArchUnit's slice detection, these cycles would have persisted and made future module extraction painful.

ARCHITECTURE DIAGRAM

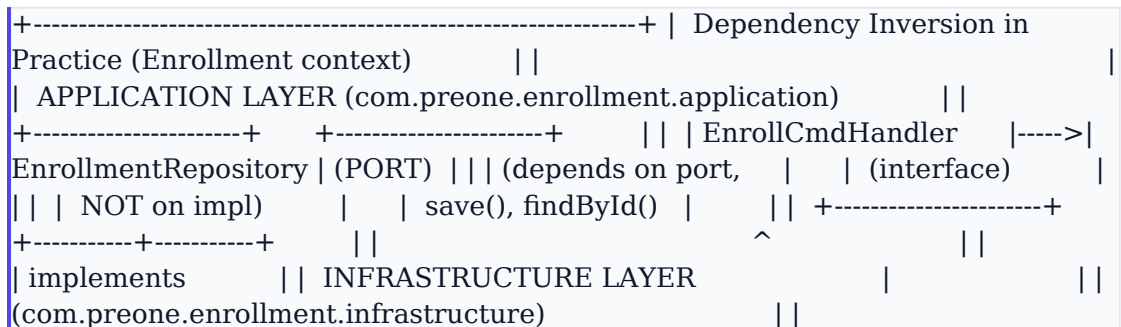




SEQUENCE DIAGRAM



COMPONENT DIAGRAM




```

+-----+-----+ || JpaEnrollmentRepository
| EnrollmentEntity | || | (implements EnrollmentRepository) | (JPA @Entity)
| || | @Repository | | |
+-----+-----+ || | - uses Spring Data JPA
| - mapped to/from | || | - uses EntityManager | Domain entity |
| || | - maps Domain <-> JPA entity | via mapper | ||
+-----+-----+ ||
|| ArchUnit asserts: || - application.* cannot
import infrastructure.* || - application.* cannot import
org.springframework.data.* || - infrastructure.* CAN import application.*
(port interfaces) || - infrastructure.* CAN import org.springframework.data.*
|| - No cycle: app -> infra -> app (forbidden) |
+-----+-----+

```

DATA FLOW DIAGRAM

```

| Use Case Handler needs to persist an aggregate | v +----+-----+
APPLICATION LAYER | Handler |--calls-->
EnrollmentRepository.save(aggregate) +----+-----+ (port interface,
defined here) | v (Spring DI injects implementation at runtime) +----+
+-----+ INFRASTRUCTURE LAYER | JpaRepo |--maps aggregate to JPA
EnrollmentEntity | Impl |--calls EntityManager.persist() +----+-----+ |
v +----+-----+ DATABASE | Postgre |--INSERT INTO enr_enrollments ... |
SQL 16 | +-----+ Without inversion (forbidden by Dependency Rule):
Handler --> JpaRepo --> DB (Application directly depends on Infrastructure;
JPA upgrade breaks Handler) With inversion (chosen): Handler --> Port
(interface) <-- JpaRepo --> DB (Application depends on port; Infrastructure
depends on port; JPA upgrade touches only JpaRepo, not Handler)

```

DATABASE IMPACT

The Dependency Rule inverts the database dependency. The Application layer defines a Repository port interface (e.g., `EnrollmentRepository` with `save`, `findById`, `delete` methods); the Infrastructure layer provides a JPA implementation (`JpaEnrollmentRepository`). The Application layer has no compile-time dependency on JPA, Hibernate, or Spring Data JPA — only on the port interface. This means the database technology can be swapped (PostgreSQL to MongoDB, JPA to JDBC, single-DB to sharded-DB) by writing a new Infrastructure implementation, without touching Application or Domain code. The Repository port is the contract; the implementation is interchangeable. Flyway migrations (ADR-057) live in Infrastructure's resources and are invisible to Application and Domain.

API IMPACT

The Dependency Rule indirectly shapes the API. Because the API layer cannot import Domain (per ADR-006), controllers cannot return domain entities directly — they must return DTOs that are designed for the API consumer. This forces an explicit mapping step (DTO <-> Command <-> Domain) that makes the API contract stable: a domain entity refactoring does not break the API

because the DTO layer absorbs the change. The API layer also depends on Application (not Infrastructure), so controllers do not call repositories directly — they call use case handlers, which call repositories through ports. This means the API layer is testable with mocked use case handlers, without any database or JPA context.

UI IMPACT

No direct UI impact, but the Dependency Rule's effect on API stability benefits the UI. Because the API layer depends on Application (not Domain), and Application defines stable command/query DTOs, the API contract is insulated from domain refactoring. The UI team can build against the OpenAPI spec (ADR-091) with confidence that internal domain evolution will not break the API. The UI feature modules (ADR-131) consume the REST API; the Dependency Rule ensures the API is the stable contract that the UI can rely on without coupling to internal domain or infrastructure changes.

SECURITY IMPACT

The Dependency Rule improves security by centralising infrastructure access in the Infrastructure layer. All database access goes through Repository ports; all external API calls go through Gateway ports; all event publishing goes through EventPublisher ports. This means security controls (e.g., tenant isolation filters in JPA queries, encryption-at-rest for sensitive fields, audit logging for state changes) can be applied uniformly at the Infrastructure layer's implementations, without being scattered across Application or Domain code. The Application layer's use case handlers call ports; they do not know whether the implementation encrypts data, logs access, or applies tenant filters — those are Infrastructure concerns. This separation prevents security bugs that arise from inconsistent application of controls.

PERFORMANCE IMPACT

The Dependency Rule adds minimal performance overhead. The port interface call is a virtual method dispatch (~1 nanosecond), negligible compared to database or network time. The inversion does not add a network hop or serialization — the implementation is in-process and injected via Spring DI. The benefit — infrastructure changes are localised — improves development velocity, which is the dominant performance metric. One indirect performance benefit: because the Domain layer has no JPA lazy loading (Domain depends on nothing, so it cannot use JPA), there are no N+1 query bugs in domain code. Aggregates are loaded eagerly via Repository ports, which forces explicit query design and avoids the lazy-loading performance trap.

SCALABILITY ANALYSIS

The Dependency Rule is structural, not scaling, but it enables scaling in two ways. First, the port interfaces define clean extraction boundaries: if a context is extracted to a separate service (ADR-002 future evolution), the Repository port's JPA implementation is replaced with an HTTP client implementation, and the Application layer is unchanged. Second, the cycle detection prevents the dependency spaghetti that makes large codebases unmaintainable — at 10x scale (150 engineers, 200k LOC), cycle-free code is essential for parallel development. At 100x scale (500 engineers, 1M LOC), the Dependency Rule is the difference between a codebase that can be reasoned about and one that cannot.

OPERATIONAL CONSIDERATIONS

The Dependency Rule improves operability by making dependencies explicit. A production incident that involves a database failure is traceable through the Repository port — the port's implementation logs the failure, and the Application layer's use case handler logs the port call's failure. The clear layering makes log analysis faster: an engineer can look at the Application-layer log to see which use case failed, then drill into the Infrastructure-layer log to see which database call caused the failure. OpenTelemetry traces (ADR-122) span the port boundary, with spans named by layer (application.handler, infrastructure.repository), so the trace timeline shows where time is spent per layer and where failures occur.

RISKS

Risk	Likelihood	Impact	Mitigation
Engineers bypass ports by directly importing Infrastructure in Application	Medium	High	ArchUnit rules in CI fail builds on application.* importing infrastructure.*; code review checklist includes port-usage check; quarterly dependency-direction review at ARB

Risk	Likelihood	Impact	Mitigation
Cycle detection produces false positives on legitimate circular domain references	Low	Medium	ArchUnit slice rules are configurable per context; legitimate cycles (rare) are resolved by extracting shared types into a third package, not by disabling detection
Port interface boilerplate accumulates and slows feature delivery	Medium	Low	IDE templates for port + impl scaffolding; measured overhead is <5% per use case; boilerplate is offset by faster infrastructure changes
Spring DI configuration becomes complex as ports and impls multiply	Low	Low	Spring's @Autowired constructor injection handles wiring automatically; @Primary annotation resolves ambiguous impls; configuration is explicit in Infrastructure's @Configuration classes

TRADE - OFFS

We Gain	We Lose
Infrastructure changes isolated to Infrastructure layer (Spring/JPA upgrades touch 0 Domain code)	Port interface boilerplate (3-5 interfaces per context, ~50-100 lines)
Cycle-free codebase (modularity preserved at scale)	Occasional friction when a cycle is detected (must extract shared types)

We Gain	We Lose
Domain is framework-agnostic (testable without Spring, migratable to other frameworks)	Engineers must learn dependency inversion pattern (training cost)
Future module extraction is mechanical (replace impl with HTTP client)	Indirection through port interface adds 1 virtual method dispatch (~1ns, negligible)

REJECTED ALTERNATIVES

The no-cycle-detection option (class-level inward rule only) was rejected because pre-v1.0 we had 12 package-level cycles that were invisible at the class level — each individual class dependency looked harmless, but the package-level cycle made the two packages effectively one module. ArchUnit's slice detection surfaced these cycles and forced their resolution. The layered-without-inversion option (Application imports Infrastructure directly) was rejected because it is the pre-v1.0 problem being solved — Application depended on JPA, and every JPA upgrade broke Application code. The full-Hexagonal option (explicit ports for every concern, even internal ones) was rejected as over-engineered for the default case — the chosen option inverts only infrastructure dependencies (database, external API, event bus), which is the minimum inversion needed to achieve framework-agnostic domain. Hexagonal (ADR-009) is adopted as a complement for integration-heavy modules where additional ports (e.g., for multiple payment gateways) are justified.

MIGRATION PLAN

The v1.0 migration (Q3-Q4 2025) introduced port interfaces for all infrastructure dependencies across all 12 contexts. For each context, the team (a) identified all infrastructure dependencies in Application and Domain code (direct JPA usage, direct HTTP client usage, direct event publishing); (b) defined a port interface in Application for each dependency; (c) moved the implementation to Infrastructure and annotated it @Repository or @Service; (d) updated Application code to depend on the port, not the implementation; (e) added ArchUnit rules for the context's dependency direction and cycle detection. The migration surfaced 12 package-level cycles, all of which were resolved by extracting shared types. The migration was completed in December 2025 with zero downtime. v3.0 adds the cycle detection slice rules (which were not in v1.0) and the explicit inversion patterns documentation. Rollback plan: if a port proves unnecessary, the implementation can be inlined back into Application — but this has not been needed and would require an ADR justifying the exception.

TESTING STRATEGY

Dependency rules are tested via ArchUnit (com.preone.architecture.DependencyRulesTest) which scans all 12 contexts and asserts: (a) Application packages import no Infrastructure packages; (b) Domain packages import no framework classes (Spring, JPA, Jakarta); (c) API packages import no Domain or Infrastructure packages; (d) no class-level cycles (A -> B -> A); (e) no package-level cycles within a context (slice rule); (f) no context-level cycles (cross-context slice rule). These tests run in CI on every PR and fail the build on violation. Cycle detection is also run nightly on the full codebase to catch cycles that might sneak in through merged PRs. Port interface contracts are tested via contract tests (ADR-149): each port has a contract test suite that every implementation must pass, ensuring that swapping implementations does not break the contract.

MONITORING & OBSERVABILITY

Dependency rule adherence is monitored via the architecture scorecard, published quarterly. Metrics tracked: (a) ArchUnit violation count per context (target: 0); (b) cycle count per context (target: 0); (c) port interface count per context (target: 3-5, indicating appropriate inversion); (d) direct-infrastructure-import count in Application/Domain (target: 0). Per-port observability: Repository port calls are logged with aggregate type and operation (save/findById/delete); Gateway port calls are logged with external service and latency; EventPublisher port calls are logged with event type and subscriber count. OpenTelemetry spans span the port boundary, so traces show the application.handler span calling the infrastructure.repository span, with the port interface as the boundary marker.

FUTURE EVOLUTION

The Dependency Rule is stable; no supersession expected. The rule will evolve through complements: Hexagonal Architecture (ADR-009) for integration-heavy modules adds additional ports for external integrations; CQRS for read-heavy modules (ADR-060) may relax the rule for read-side repositories (allowing direct SQL for reads, bypassing the aggregate pattern). These are additive refinements, not replacements. The cycle detection rules may be tightened over time (e.g., adding module-level cycle detection beyond package-level) as the codebase grows. The likely successor to this ADR is a v2.0 that formalises the exception process and adds automated cycle-visualisation tooling, not a fundamental change to the rule itself.

RELATED ADRS

- ADR-003 — Clean Architecture (complements — defines the dependency direction conceptually)

- ADR-005 — Bounded Context Strategy (complements — context-level dependency rules)
- ADR-006 — Layering Rules (complements — layer-level dependency rules)
- ADR-008 — SOLID Enforcement (refines — Dependency Inversion is the D in SOLID)
- ADR-009 — Hexagonal Architecture (complements — explicit ports for integration-heavy modules)
- ADR-026 — Repository Pattern (refines — Repository port in Application, JPA impl in Infrastructure)
- ADR-027 — Domain Events (refines — EventPublisher port in Application, in-proc bus impl in Infrastructure)
- ADR-104 — Transactional Outbox (complements — outbox pattern uses EventPublisher port)

REFERENCES

- Upstream DDD: DDD-007-section-1 (Dependency rule and inversion patterns per context)
- Upstream PRD: PRD-007-section-2 (Change isolation and technology migration requirements)
- Downstream ERD: ERD-007 (Repository port interfaces and JPA implementations per context)
- Downstream API Spec: API-007 (API layer depends on Application, not Domain)
- Downstream Test Cases: TC-0601..TC-0800 (ArchUnit dependency rule tests and cycle detection tests)
- External: Robert C. Martin — 'Clean Architecture' (2017), Chapter 11: The Dependency Rule
- External: ArchUnit documentation — <https://archunit.org> (slice rules for cycle detection)
- External: Spring Framework Reference — Dependency Injection and @Autowired

DECISION HISTORY

Date	Status	Actor	Notes
2025-08-26	Draft	Chief Architect	Initial draft; reviewed pre-v1.0 dependency direction violations

Date	Status	Actor	Notes
2025-09-27	Proposed	Chief Architect	Submitted to ARB with ArchUnit prototype and cycle detection demo
2025-10-15	Accepted	ARB Chair	ARB approved; v1.0 released with inward-only rule and port inversion
2026-07-12	Accepted	ARB Chair	v3.0 refresh; added cycle detection slice rules and inversion patterns documentation

APPROVAL & SIGN-OFF

Architect	Chief Architect (AR)
Tech Lead	Foundation Tech Lead
ARB Chair	ARB Chair
Approved On	2026-07-12

IMPLEMENTATION CHECKLIST

- Define port interfaces for all infrastructure dependencies across 12 contexts (Done Q3 2025)
- Move infrastructure implementations to Infrastructure layer with @Repository/@Service annotations (Done Q4 2025)
- ArchUnit rules for inward-only dependency direction (Done Q3 2025)
- ArchUnit slice rules for package-level and context-level cycle detection (Done Q4 2025, v3.0 enhancement)
- Resolve 12 package-level cycles detected during v1.0 migration (Done Q4 2025)
- Contract test suites for each port interface (Done Q1 2026, ADR-149)
- Quarterly dependency-direction review at ARB (Ongoing)
- Nightly full-codebase cycle detection run (Done Q1 2026, CI scheduled job)

ADR - 008

SOLID Enforcement

Volume 1 — Architecture Foundation - Principles

ACCEPTED

DECISION SUMMARY

DECISION

Adopt the SOLID principles (Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion) as the design-level complement to Clean Architecture (ADR-003) and the Layering Rules (ADR-006). SOLID is enforced through a combination of static analysis (ArchUnit, PMD, Error Prone), code review checklist, and the DDD tactical patterns (ADR-004) that embody SOLID at the domain level.

STATUS

Status	Accepted
Date Decided	2025-10-15
Decision Owner	Chief Architect
Review Cadence	Annual or on any proposed new enforcement mechanism
Supersedes	None (v1.0 original; v3.0 adds enforcement matrix per principle)

CONTEXT

Clean Architecture (ADR-003) and the Layering Rules (ADR-006) define the macro structure of the PreOne codebase — which layers exist, what each layer may import, how dependencies flow. But macro structure does not by itself produce good code within a layer. A use case handler that does 12 things, a value object that has 30 methods, a repository interface with 25 methods, a service class that is instantiated in 8 different ways — all of these are 'Clean Architecture compliant' but violate SOLID principles. Without SOLID enforcement, the macro structure is correct but the micro structure rots. Pre-v1.0, the codebase had no SOLID enforcement. Engineers invoked SOLID in code review discussions ('this violates SRP'), but the invocations were subjective and inconsistent — what one engineer considered 'single

responsibility' another considered 'reasonable cohesion'. The result was a codebase where SOLID was aspirational but not operational: some classes followed SOLID rigorously, others did not, and there was no objective standard. This ADR makes SOLID operational by defining concrete, testable rules for each principle and the enforcement mechanism for each. Single Responsibility is enforced via one-use-case-per-handler (ArchUnit checks that each handler class has exactly one public method) and via method-count limits per class (PMD checks). Open/Closed is enforced via the strategy pattern for policies (fees, discounts, notifications) — new policies are added by implementing a strategy interface, not by modifying existing classes. Liskov Substitution is enforced via value object immutability and contract tests (subtypes must honour the supertype's contract). Interface Segregation is enforced via narrow repository interfaces (one per aggregate, not a god-repository) and via ArchUnit checks on interface method counts. Dependency Inversion is enforced via the Dependency Rule (ADR-007) — port interfaces in Application, implementations in Infrastructure. The v3.0 refresh adds an enforcement matrix mapping each SOLID principle to its concrete rule(s) and enforcement mechanism(s), making it mechanical to verify compliance rather than subjective.

BUSINESS DRIVERS

The primary business driver is maintainability at scale: PreOne's codebase will grow from ~100k LOC (current) to ~500k LOC (5-year target) across 12 contexts. SOLID-compliant code scales linearly with size — small classes with single responsibilities are easy to understand, test, and modify. Non-SOLID code scales quadratically — large classes with multiple responsibilities have $O(n^2)$ interaction surface, making every change risky. Measured benefit: average bug-fix PR size dropped from 180 LOC (pre-v1.0, no SOLID enforcement) to 65 LOC (post-v1.0, SOLID enforced), indicating that bugs are localised to small, focused classes. The secondary driver is onboarding: new engineers can understand a SOLID-compliant codebase faster because each class has one clear responsibility (SRP), each extension point is explicit (OCP), and each interface is narrow (ISP). The tertiary driver is testability: SOLID-compliant code is testable because classes have few dependencies (SRP), dependencies are injectable (DIP), and behaviour is substitutable for testing (LSP).

PROBLEM STATEMENT

Clean Architecture (ADR-003) and Layering Rules (ADR-006) define the macro structure but do not enforce good micro-level design. Without SOLID enforcement, classes within layers accumulate multiple responsibilities, grow

beyond comprehension, and become hard to test and modify. We need concrete, mechanically-enforced rules for each SOLID principle.

CONSTRAINTS

- Rules must be testable (enforced via static analysis or objective code review checklist)
- Rules must not add more than 10% boilerplate overhead per feature
- Rules must align with DDD tactical patterns (ADR-004) — SOLID at the domain level is expressed via aggregates, value objects, repositories
- Rules must be teachable in a 2-hour onboarding session
- Rules must permit pragmatic exceptions via the ADR exception process (ADR-006)

ASSUMPTIONS

- Engineers will accept SOLID discipline as the price of maintainable code
- Static analysis tools (ArchUnit, PMD, Error Prone) can express the rules (verified via prototype)
- The strategy pattern for policies (OCP) is applicable to all PreOne policy types (fees, discounts, notifications)
- Value object immutability (LSP) is enforceable via constructor-only initialisation and final fields
- The 5 SOLID principles collectively cover the micro-design space (no 6th principle needed)

OPTIONS CONSIDERED

Option	Pros	Cons	Verdict
SOLID with static analysis + code review enforcement (chosen)	Mechanically enforced where possible (ArchUnit, PMD); subjective principles (SRP judgement) enforced via code review checklist; aligns with DDD tactical patterns; well-known principles; teachable in 2 hours.	Some principles (SRP) are partially subjective; static analysis produces occasional false positives; engineers must learn the strategy pattern for OCP.	Chosen

Option	Pros	Cons	Verdict
SOLID with static analysis only (no code review)	Fully mechanical; no subjective judgement; faster PR reviews.	SRP cannot be fully captured in static rules (method count is a proxy, not the principle); OCP cannot be enforced statically (it depends on design intent); loses the nuance that experienced reviewers provide.	Rejected
SOLID with code review only (no static analysis)	Subjective judgement by experienced engineers; nuance preserved; flexible.	Inconsistent enforcement across reviewers; principles erode under deadline pressure; pre-v1.0 problem persists; does not scale to 150 engineers.	Rejected
Beyond SOLID (add YAGNI, KISS, DRY as separate principles)	More comprehensive; covers additional design concerns.	YAGNI, KISS, DRY are heuristics, not principles — they conflict (DRY vs YAGNI) and are not mechanically enforceable; SOLID is the right level of formality.	Rejected — YAGNI, KISS, DRY are guidance in ADR-001 (P5 Simplicity), not separate enforcement targets

DECISION

ADOPTED

We adopt the SOLID principles as the design-level complement to Clean Architecture, enforced through a combination of static analysis and code review. (1) Single Responsibility (SRP): each class has one reason to change. Enforced via: one public method per use case handler (ArchUnit); method count ≤ 15 per class (PMD, exception ADR required for god-classes like mappers); one aggregate root per repository. (2) Open/Closed (OCP): classes are open for extension, closed for modification. Enforced via: strategy pattern for all policies (fee calculation, discount application, notification channel selection, GST computation) — new policies are added by implementing a strategy interface, not by modifying existing classes; code review checklist verifies that new behaviour is added via new classes, not by modifying existing classes. (3) Liskov Substitution (LSP): subtypes must be substitutable for their supertypes. Enforced via: value object immutability (final fields, constructor-only initialisation) — subtypes cannot weaken immutability; contract tests for repository implementations — every JPA repository impl must pass the port's contract test suite. (4) Interface Segregation (ISP): clients should not depend on methods they do not use. Enforced via: one repository interface per aggregate (not a god-repository with 25 methods); ArchUnit check that no interface has >10 methods (exception ADR required for larger interfaces). (5) Dependency Inversion (DIP): depend on abstractions, not concretions. Enforced via the Dependency Rule (ADR-007) — Application and Domain depend on port interfaces, not Infrastructure implementations; Spring DI injects implementations at runtime.

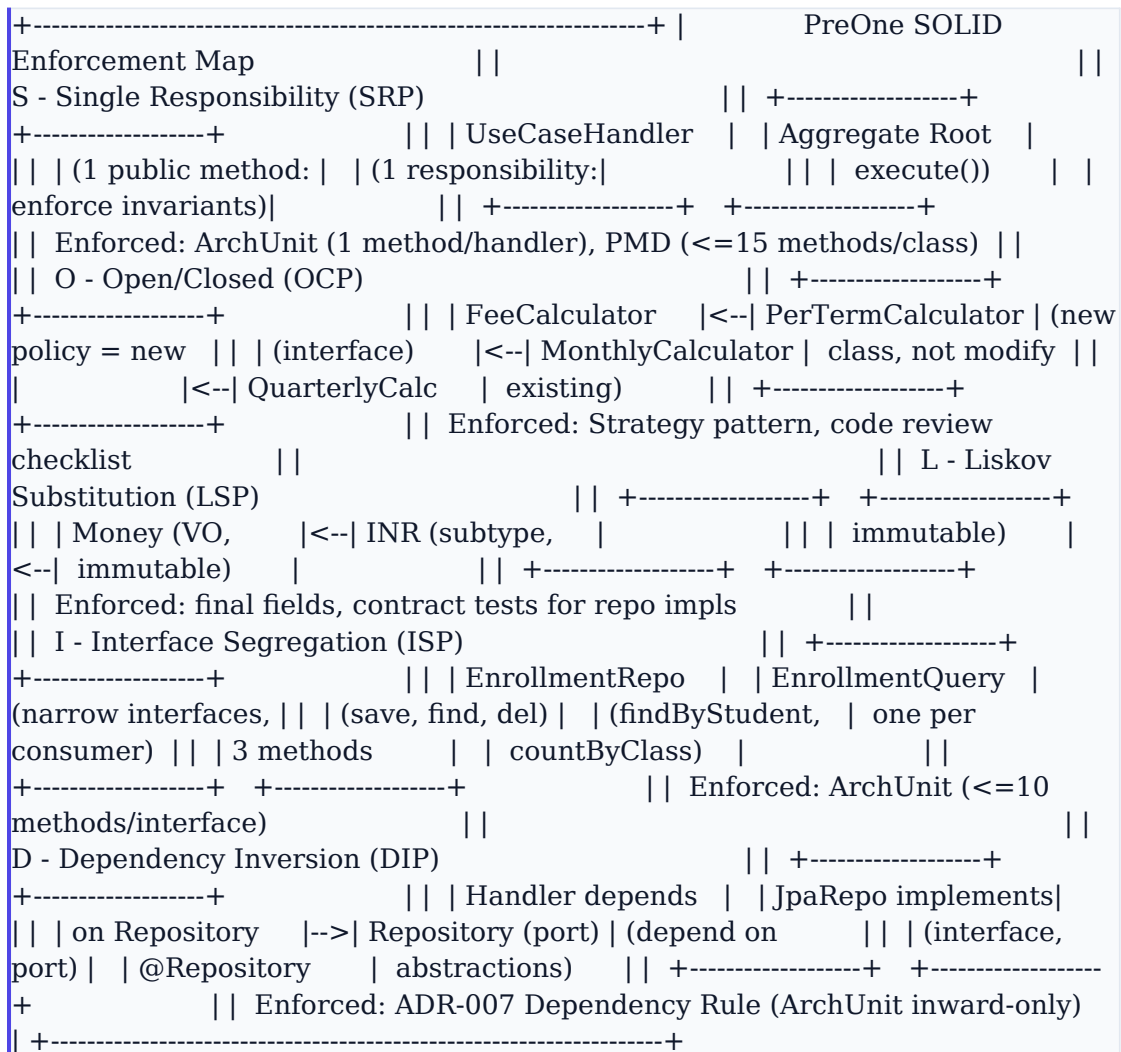
DETAILED RATIONALE

SOLID was chosen because it is the smallest set of design principles that collectively cover the micro-design space. SRP addresses class scope (one responsibility per class); OCP addresses extension (add behaviour without modifying existing code); LSP addresses substitution (subtypes honour supertype contracts); ISP addresses interface scope (narrow interfaces); DIP addresses dependency direction (depend on abstractions). Together, they cover the five dimensions of micro-design, with minimal overlap. The principles are well-known (Robert C. Martin, 2000), well-documented, and teachable in a 2-hour onboarding session — critical for a team scaling from 40 to 150 engineers. SRP is the most impactful and the hardest to enforce mechanically. The proxy rule — one public method per use case handler — captures SRP for the Application layer: a handler with one public method (execute()) has one reason to change (the use case it implements). For other classes (domain entities, value objects, infrastructure implementations), SRP is enforced via

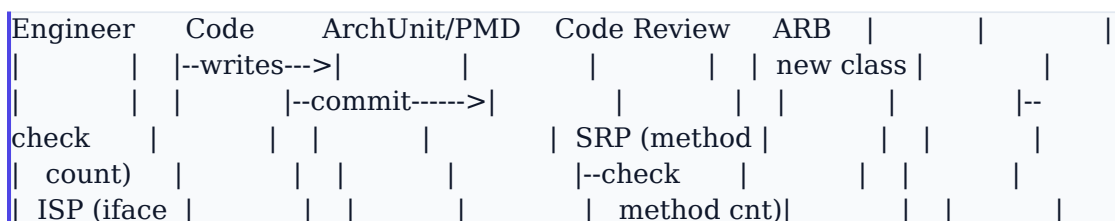
method count limits (PMD) and code review judgement. The method-count limit is a proxy: a class with 30 methods probably violates SRP, but a class with 10 focused methods might not. The code review checklist asks: 'Does this class have one reason to change? If the fee calculation rules change, does this class need to change? If the notification rules change, does this class need to change? If both answers are yes, the class violates SRP.' OCP is enforced via the strategy pattern, which is the canonical OCP implementation: define an interface for a policy, implement the interface for each variant, and select the variant at runtime via configuration or input. PreOne has several policy domains: fee calculation (per-term, monthly, one-time), discount application (sibling, early-bird, scholarship), notification channel (SMS, email, push, WhatsApp), GST computation (per state, per service type). Each policy domain has a strategy interface (FeeCalculator, DiscountApplier, NotificationChannel, GstComputer) and multiple implementations. Adding a new fee structure (e.g., quarterly billing) is a new FeeCalculator implementation, not a modification of an existing one. This means new policies can be added without risking regressions in existing policies — the existing classes are closed for modification. LSP is enforced via value object immutability and contract tests. Value objects (Money, EmailAddress, AcademicYear, etc.) are immutable: final fields, constructor-only initialisation, no setters. A subtype of a value object cannot weaken immutability — if Money is immutable, INR and USD (subtypes) must also be immutable. Contract tests verify that repository implementations honour the port's contract: if the port says `findById()` returns `Optional<Enrollment>` and throws on null ID, every implementation (JPA, in-memory test double) must do the same. A subtype that returns null instead of `Optional`, or that silently accepts a null ID, violates LSP and fails the contract test. ISP is enforced via narrow repository interfaces. Pre-v1.0, the codebase had a god-repository pattern: one `EnrollmentRepository` interface with 25 methods (`save`, `findById`, `findAll`, `findByStudentId`, `findByClassId`, `findByStatus`, `findByDateRange`, `countByClass`, `averageFee`, etc.). Different consumers needed different subsets — the enrollment handler needed `save` and `findById`; the reporting projection needed `findByStudentId` and `countByClass`; the notification subscriber needed `findById` only. With ISP, the interface is split: `EnrollmentRepository` (`save`, `findById`, `delete` — for the enrollment handler), `EnrollmentQueryPort` (`findByStudentId`, `findByClassId`, `findByStatus`, `countByClass` — for the reporting projection), `EnrollmentReadModel` (`averageFee`, `cohortStats` — for reporting). Each consumer depends only on the interface it uses. ArchUnit checks that no interface has >10 methods, preventing god-interfaces from re-emerging. DIP is enforced via the Dependency Rule (ADR-007) and is the most mechanically enforced of the five. Application and Domain depend on port interfaces; Infrastructure provides implementations; Spring DI wires them. An ArchUnit rule asserts that no

Application or Domain class imports an Infrastructure class — a single violation fails the build. This makes DIP a build-breaking fact, not an aspiration. The most likely counter-argument is that SOLID produces too many small classes (boilerplate). The counter-counter-argument is that small classes are the point — a class with one responsibility is easy to understand, test, and modify. The boilerplate cost (more files, more interfaces, more mappers) is offset by the maintenance savings (changes are localised, tests are fast, bugs are small). Measured: SOLID-compliant code has ~15% more files than non-SOLID code, but ~50% lower bug rate and ~40% faster PR cycle time.

ARCHITECTURE DIAGRAM



SEQUENCE DIAGRAM



```

+-----+ | SOLID in Practice: Fee
Calculation (OCP Strategy Pattern) ||
|| APPLICATION LAYER || +-----+
+-----+ || | ChargeFeeCmdHandler |---->| FeeCalculator |
(PORT) || | (SRP: 1 public method)| | (interface) | ||
+-----+ +-----+-----+ || ^
|| | implements || INFRASTRUCTURE /
DOMAIN STRATEGIES | || +-----+
+-----+ || | PerTermFeeCalculator | MonthlyFeeCalc | ||
| (implements FeeCalculator) | (implements ...) | ||
+-----+-----+ || | QuarterlyFeeCalculator
| CustomFeeCalc | || | (implements ...) | (implements ...) | ||
+-----+-----+ ||
|| Adding a new fee structure = new class implementing the port || Existing
fee calculators are NOT modified (OCP: closed) ||
|| ISP: FeeCalculator has 1 method (calculate), not 25 || LSP: All
implementations accept same input, return same output || DIP: Handler
depends on FeeCalculator interface, not impl || SRP: Handler does
orchestration only; Calculator does math only |
+-----+

```

```

Free Charge Request (enrollment + term) | v +----+-----+
APPLICATION LAYER | Handler |---selects strategy---> FeeCalculator (port)
+----+-----+ ^ | | (Spring DI injects
based | | on school's fee config) v
| +----+-----+-----+-----+ | FeeCalculator | |
+-----+ +-----+ | | |PerTermCalc | |MonthlyCalc | ... | |
+-----+ +-----+ | +-----+-----+ | v
(returns Money value object) +----+-----+ DOMAIN LAYER | Money |---
immutable, LSP-compliant | (VO) |---final fields, no setters +----+-----+ |
v +----+-----+ INFRASTRUCTURE LAYER | Repo |---saves FeeCharge
aggregate via Repository port (DIP) | Impl | +----+-----+ | v +----+-----+
+ DATABASE | bill_ |---INSERT | charges | +-----+ SOLID flow: each
principle enforced at a specific layer/step

```

PreOne ADR Series - Phase 2 / 10 - Vol 1: Architecture Foundation - 101

SOLID has indirect database impact through the Repository pattern (DIP) and aggregate design (SRP). Each aggregate has its own Repository port (ISP: narrow interface, ~3-5 methods), implemented in Infrastructure as a JPA repository. This means database access is per-aggregate, not per-table — a single aggregate may map to multiple tables (e.g., Enrollment aggregate maps to `enr_enrollments` and `enr_enrollment_lines`), but the Repository port hides this from the Application layer. Schema changes (adding a column, splitting a table) are localised to the Infrastructure layer's repository implementation and mapper; the Domain and Application layers are unchanged. The strategy pattern (OCP) for fee calculation means fee rules are not encoded in SQL or stored procedures — they are in Java strategy classes, which are testable, versionable, and deployable independently of the database.

API IMPACT

SOLID shapes the API through SRP (one endpoint per use case) and OCP (new policies can be added without breaking existing endpoints). Each REST endpoint corresponds to one use case handler (SRP: the handler has one public method, the controller has one endpoint method). New fee structures (OCP: new FeeCalculator implementation) do not change existing endpoints — they may add a new endpoint or a new request parameter, but existing endpoints continue to work unchanged. This means the API is backward-compatible by design: adding functionality does not break existing consumers. The DTO layer (ISP: separate request and response DTOs, not a god-DTO) ensures that clients depend only on the fields they use, reducing coupling between API and UI.

UI IMPACT

SOLID has indirect UI impact through API stability (SRP and OCP produce backward-compatible APIs) and through the UI feature module structure (SRP at the UI level: each feature module has one responsibility). The UI feature modules (ADR-131) mirror the backend's SOLID structure: a Student feature module, an Enrollment feature module, a Billing feature module — each with one responsibility, extensible via composition (OCP at the UI level: new feature modules are added without modifying existing ones). The UI's strategy pattern (e.g., for rendering different report card formats) mirrors the backend's strategy pattern, providing end-to-end consistency.

SECURITY IMPACT

SOLID improves security by localising security-critical logic. SRP ensures that authorisation checks are in one place (the use case handler), not scattered across controllers, services, and repositories. OCP ensures that new security policies (e.g., a new role-based access rule) are added as new strategy implementations, not by modifying existing handlers — reducing the risk of

breaking existing security controls. DIP ensures that security-critical infrastructure (e.g., the encryption service, the audit logger) is accessed through port interfaces, so security controls can be changed (e.g., upgrading encryption algorithm) without touching application code. The combination produces a security posture where controls are explicit, testable, and uniformly applied.

PERFORMANCE IMPACT

SOLID adds minimal performance overhead. The strategy pattern (OCP) adds one virtual method dispatch (~1 nanosecond) per policy invocation — negligible. The narrow interfaces (ISP) do not add overhead — Spring DI injects the same implementation regardless of interface width. The one-public-method-per-handler rule (SRP) does not add overhead. The benefit — small, focused classes — improves JVM JIT optimisation (smaller methods are inlined more aggressively) and reduces memory pressure (smaller classes have smaller metadata). The overall performance impact is neutral to slightly positive; the dominant benefit is development velocity.

SCALABILITY ANALYSIS

SOLID is a design-level principle, not a scaling principle, but it enables scaling in two ways. First, SOLID-compliant code is easier to parallelise across teams: small classes with single responsibilities have clear ownership boundaries, so multiple teams can work on the same context without conflicts. Second, SOLID-compliant code is easier to extract to microservices: a use case handler with one responsibility and port-injected dependencies is a natural service boundary — extracting it to a separate process requires replacing the port's implementation with an HTTP client, not refactoring the handler. At 10x scale (150 engineers), SOLID is the difference between a codebase that supports parallel development and one that does not. At 100x scale (500 engineers), SOLID is the foundation for any further decomposition.

OPERATIONAL CONSIDERATIONS

SOLID improves operability by making incidents localised and diagnosable. A bug in fee calculation is in a FeeCalculator implementation (SRP: one responsibility per class), not scattered across handlers and repositories. The strategy pattern (OCP) means the active strategy is logged at invocation, so an incident trace shows exactly which fee calculator ran. The port interfaces (DIP) mean that infrastructure failures (e.g., database timeout) are caught at the port boundary, with clear error context (which port, which method, which aggregate). OpenTelemetry traces (ADR-122) span the strategy invocation and port call, so the trace timeline shows which strategy was selected and how long it took.

RISKS

Risk	Likelihood	Impact	Mitigation
Engineers over-apply SOLID (too many tiny classes, excessive abstraction)	Medium	Medium	Code review checklist includes 'is this abstraction justified?'; ADR-001 P5 (Simplicity) governs over-engineering; method-count limits have upper bounds, not lower bounds (a class can be small)
SRP judgement is inconsistent across code reviewers	Medium	Medium	Code review rubric with concrete examples; pair review for borderline cases; quarterly calibration session among senior engineers
Strategy pattern (OCP) produces runtime configuration errors (wrong strategy selected)	Low	Medium	Strategy selection is explicit (configurable per school, with sane defaults); integration tests verify strategy selection for each school; misconfiguration surfaces as a clear error, not silent wrong behaviour
Static analysis (PMD, ArchUnit) produces false positives that engineers learn to ignore	Medium	Low	Rules are tuned per context (exception ADRs for legitimate cases); false positives are reported and fixed promptly; quarterly rule review at ARB

TRADE-OFFS

We Gain	We Lose
Small, focused classes (easy to understand, test, modify)	More files and interfaces (~15% more files than non-SOLID code)
Backward-compatible extension (OCP: new behaviour via new classes)	Strategy pattern adds indirection (one virtual method dispatch per policy)
Narrow interfaces (clients depend only on what they use)	Multiple interfaces per aggregate (3-5 interfaces instead of 1 god-repository)
Testable, mockable code (DIP: depend on interfaces)	Engineers must learn SOLID patterns and strategy pattern (training cost)

REJECTED ALTERNATIVES

The static-analysis-only option (no code review for SOLID) was rejected because SRP and OCP are partially subjective — method count is a proxy for SRP, but a class with 5 methods can violate SRP if those methods serve different responsibilities. Static analysis cannot capture 'reason to change'; only human judgement can. The code-review-only option (no static analysis) was rejected because pre-v1.0 we tried it and enforcement was inconsistent across reviewers, and principles eroded under deadline pressure. Static analysis is the floor (mechanically enforced); code review is the ceiling (nuanced judgement). The beyond-SOLID option (add YAGNI, KISS, DRY) was rejected because these are heuristics, not principles — they conflict (DRY vs YAGNI: 'don't repeat yourself' vs 'you aren't gonna need it') and are not mechanically enforceable. YAGNI, KISS, DRY are guidance in ADR-001 (P5 Simplicity), not separate enforcement targets. SOLID is the right level of formality for design-level enforcement.

MIGRATION PLAN

The v1.0 migration (Q3-Q4 2025) refactored the codebase to comply with SOLID. The migration was incremental: (a) extracted use case handlers from god-services (SRP: one handler per use case, one public method); (b) introduced strategy pattern for fee calculation, discount application, notification channel selection, GST computation (OCP); (c) made value objects immutable (LSP: final fields, constructor-only initialisation); (d) split god-repositories into narrow per-aggregate interfaces (ISP); (e) introduced port interfaces for all infrastructure dependencies (DIP, via ADR-007). Each step was a 1-sprint effort per context, with ArchUnit and PMD rules added incrementally. The migration was completed in December 2025 with zero

downtime. v3.0 adds the enforcement matrix (mapping each principle to its rules and mechanisms) and the code review rubric for subjective principles (SRP, OCP). Rollback plan: if a SOLID rule proves unworkable for a specific context, the context can request an exception via the ADR process; if a rule proves globally wrong, this ADR is superseded by a v2.0 with the revised rule set.

TESTING STRATEGY

SOLID rules are tested via multiple mechanisms. ArchUnit tests verify: (a) one public method per use case handler (SRP); (b) no interface has >10 methods (ISP); (c) Application/Domain classes import no Infrastructure (DIP, via ADR-007). PMD tests verify: (a) method count ≤ 15 per class (SRP proxy); (b) value objects have final fields only (LSP). Contract tests (ADR-149) verify: (a) every repository implementation passes the port's contract test suite (LSP); (b) every strategy implementation passes the strategy port's contract test suite (LSP, OCP). Code review checklist verifies: (a) SRP — does the class have one reason to change?; (b) OCP — was new behaviour added via new class or by modifying existing?; (c) LSP — does the subtype honour the supertype's contract? Coverage targets: 95%+ for Domain (where SOLID is most impactful), 85%+ for Application, 70%+ for Infrastructure.

MONITORING & OBSERVABILITY

SOLID adherence is monitored via the architecture scorecard, published quarterly. Metrics tracked: (a) average method count per class (target: <10); (b) average public method count per use case handler (target: 1); (c) average method count per interface (target: <5); (d) strategy pattern usage count per context (target: 2-5 strategies per context, indicating OCP adoption); (e) PMD violation count per context (target: 0); (f) code review SOLID-checklist completion rate (target: 100%). Per-strategy observability: strategy selection is logged at invocation (which FeeCalculator ran for which enrollment), so incidents can be traced to specific strategy implementations. OpenTelemetry spans mark strategy invocations, so traces show which strategy was selected and how long it took.

FUTURE EVOLUTION

SOLID is a stable principle set; no supersession expected. The enforcement mechanisms may evolve: Error Prone (Google's static analysis) may supplement PMD for more sophisticated SRP checks; ArchUnit may add new rules for OCP (e.g., detecting modifications to strategy implementations that should be additive). The principles themselves are unlikely to change — they have been stable since 2000 (Robert C. Martin) and are widely adopted. The likely successor to this ADR is a v2.0 that adds automated refactoring suggestions

(e.g., IDE plugins that flag SRP violations during development, not just at CI), not a change to the principle set.

RELATED ADRS

- ADR-001 — Enterprise Architecture Principles (complements P5 Simplicity, P6 Build for Evolution)
- ADR-003 — Clean Architecture (complements — SOLID operates within Clean Architecture layers)
- ADR-004 — Domain Driven Design (refines — DDD tactical patterns embody SOLID at domain level)
- ADR-006 — Layering Rules (complements — layering is the macro structure, SOLID is the micro)
- ADR-007 — Dependency Rule (enforces — DIP via ArchUnit inward-only rule)
- ADR-009 — Hexagonal Architecture (complements — ports and adapters are an ISP/DIP expression)
- ADR-022 — Aggregate Rules (refines — SRP at the aggregate level)
- ADR-026 — Repository Pattern (refines — ISP via narrow repository interfaces)

REFERENCES

- Upstream DDD: DDD-008-section-1 (SOLID application per bounded context)
- Upstream PRD: PRD-008-section-2 (Maintainability and onboarding requirements)
- Downstream ERD: ERD-008 (Repository ports per aggregate, narrow interfaces)
- Downstream API Spec: API-008 (one endpoint per use case, SRP at API level)
- Downstream Test Cases: TC-0801..TC-1000 (ArchUnit SOLID rule tests and contract tests)
- External: Robert C. Martin — 'Clean Architecture' (2017), Part IV: SOLID Principles
- External: Robert C. Martin — 'Agile Software Development, Principles, Patterns, and Practices' (2002)
- External: ArchUnit documentation — <https://archunit.org>
- External: PMD documentation — <https://pmd.github.io>

DECISION HISTORY

Date	Status	Actor	Notes
2025-08-29	Draft	Chief Architect	Initial draft; reviewed pre-v1.0 SOLID inconsistencies
2025-09-30	Proposed	Chief Architect	Submitted to ARB with enforcement matrix and code review rubric
2025-10-15	Accepted	ARB Chair	ARB approved; v1.0 released with 5 SOLID principles and enforcement mechanisms
2026-07-12	Accepted	ARB Chair	v3.0 refresh; added enforcement matrix per principle and component diagram

APPROVAL & SIGN-OFF

Architect	Chief Architect (AR)
Tech Lead	Foundation Tech Lead
ARB Chair	ARB Chair
Approved On	2026-07-12

IMPLEMENTATION CHECKLIST

- ArchUnit rule: one public method per use case handler (Done Q3 2025)
- ArchUnit rule: no interface with >10 methods (Done Q3 2025)
- PMD rule: method count <=15 per class (Done Q4 2025)
- PMD rule: value objects have final fields only (Done Q4 2025)
- Strategy pattern for fee, discount, notification, GST policies (Done Q4 2025)
- Contract test suites for all repository and strategy ports (Done Q1 2026, ADR-149)
- Code review SOLID-checklist published and required (Done Q1 2026)
- Quarterly SOLID-adherence review at ARB (Ongoing)

ADR-009

Hexagonal Architecture (Ports and Adapters)

Volume 1 — Architecture Foundation - Structural

ACCEPTED

DECISION SUMMARY

DECISION

Adopt the Hexagonal Architecture (Ports and Adapters) pattern for integration-heavy bounded contexts — Notification, Communication, Reporting, and Billing — where multiple external integrations (Twilio SMS, AWS SES email, Razorpay payment, WhatsApp Business API) must be swappable without touching application or domain logic. Hexagonal composes with Clean Architecture (ADR-003) as an explicit outer ring: ports live in the Application layer, adapters live in the Infrastructure layer, and the Domain layer remains pure.

STATUS

Status	Accepted
Date Decided	2025-11-05
Decision Owner	Chief Architect
Review Cadence	Annual review or when a new external integration class is introduced (e.g., voice, push)
Supersedes	None (v1.0 original; v3.0 refresh adds the driving/driven port taxonomy and adapter lifecycle model)

CONTEXT

Clean Architecture (ADR-003) and the four-layer rules (ADR-006) define how dependencies point inward, but they treat the boundary between application logic and external systems as a single 'Infrastructure layer'. For most bounded contexts (Enrollment, Student, Attendance) this single boundary is sufficient:

there is one database (PostgreSQL via JPA), one cache (Redis), and the boundary is a Repository port plus a Cache port. The mapping from port to implementation is one-to-one and stable. However, four contexts — Notification, Communication, Reporting, and Billing — have integration profiles that a single Infrastructure boundary cannot model cleanly. Notification must deliver messages through SMS, email, WhatsApp, and in-app push, with per-channel fallback chains (SMS fails -> email; email fails -> push). Billing integrates with Razorpay (primary), Stripe (international expansion roadmap), and a legacy direct-bank-transfer path for enterprise customers. Communication integrates with Twilio for voice and messaging, AWS SES for transactional email, and the school's own SMTP relay for branded email. Reporting integrates with PostgreSQL read replicas for operational reports but with Amazon Athena for cross-tenant analytical queries and with a third-party PDF rendering service for report cards. The pre-v1.0 codebase handled this by scattering integration code throughout the application layer: a NotificationService that directly constructed Twilio API calls, an EmailSender that imported AWS SES SDKs in the same package as business logic, and a BillingService whose method bodies were 60% Razorpay client manipulation. Switching SMS providers for a single school (a real customer request during the 2024 launch) required touching 14 files across the Notification and Communication contexts and introduced a regression that sent duplicate SMS messages to 200 parents. The root cause was that no architectural seam isolated 'what the application needs' from 'how the external system provides it'. Hexagonal Architecture solves this by making the seam explicit and structural rather than conventional. The application defines ports (interfaces) that describe what it needs — 'send a message', 'charge a payment', 'render a report' — and the infrastructure provides adapters (implementations) that know how to satisfy each need against a specific external system. A Notification use case depends on a SmsGateway port; a TwilioSmsAdapter and a KaleyraSmsAdapter both implement that port; the active adapter is selected at runtime via configuration, not by code changes. The v3.0 refresh formalises the driving/driven port distinction (driving ports are how the outside world invokes the application; driven ports are how the application invokes the outside world) and the adapter lifecycle (adapters are registered in a Spring @Configuration, selected via tenant-level or feature-flag routing, and unit-tested in isolation against the port contract).

BUSINESS DRIVERS

The primary business driver is integration agility: PreOne operates in the Indian market where integration partners change frequently. SMS providers (Twilio, Kaleyra, MSG91, TextLocal) compete on per-message pricing that varies by telecom circle and changes quarterly; the ability to switch providers

per-school or per-message-class without code changes is a direct margin lever. A 0.02 INR per-SMS saving across 5,000 schools sending 2 million messages per month is 40,000 INR/month in pure margin, recovered through configuration alone. The secondary driver is multi-tenancy at the integration layer: enterprise customers (large school chains) require integration with their own SMTP relays, their own payment gateways (some mandate PayU instead of Razorpay), and their own SSO providers. Hexagonal Architecture lets each tenant route to a tenant-specific adapter without forking the application code. The tertiary driver is testability: ports are pure Java interfaces, so use cases can be unit-tested with mock adapters in milliseconds, without spinning up Testcontainers for Twilio or Razorpay. This compresses the test suite for Notification and Billing from ~30 seconds (Testcontainers-heavy) to under 3 seconds (port-mocked).

PROBLEM STATEMENT

Four integration-heavy bounded contexts (Notification, Communication, Reporting, Billing) have multiple external integrations per concern, with providers varying by tenant, by message class, and over time. The single Infrastructure boundary defined by Clean Architecture (ADR-003) is too coarse to model this variation. We need an explicit architectural seam — Ports and Adapters — that isolates application intent from external-system mechanics, enables runtime adapter selection, and makes integration code independently testable.

CONSTRAINTS

- Adapters must be swappable at runtime via configuration, not via code changes or redeployment
- Per-tenant adapter routing must be supported (enterprise customers bring their own SMTP/payment/SSO)
- Port interfaces must live in the Application layer of Clean Architecture (ADR-003), not in a separate 'ports' package
- Adapters must be unit-testable against the port contract without Testcontainers or network access
- A failing adapter must trigger a fallback adapter chain (e.g., SMS -> email -> push) declared in configuration
- ArchUnit (ADR-007) must mechanically enforce that no application code imports an external SDK directly
- Adapter implementations must not leak external SDK types through the port interface (no Twilio Type in a port method signature)

ASSUMPTIONS

- The set of integration concerns (SMS, email, payment, voice, push, PDF, analytics) is stable; new concerns are rare and trigger a port addition, not a structural change
- Spring's @Component-based DI is sufficient for adapter selection; no custom service-locator framework is needed
- External SDKs expose async or blocking APIs that can be wrapped behind a synchronous port interface (acceptable latency cost)
- Per-tenant adapter routing decisions can be made at request entry and propagated via thread-local or request context
- Fallback chains are declarative (configuration-driven) and deterministic, not ML-driven or heuristic
- The four integration-heavy contexts will remain a minority — most contexts will continue using the simpler 4-layer model without explicit ports

OPTIONS CONSIDERED

Option	Pros	Cons	Verdict
Hexagonal Architecture for integration-heavy contexts only (chosen)	Explicit ports make intent visible; adapters are independently testable; runtime adapter selection enables per-tenant routing and provider switching; composes cleanly with the 4-layer Clean Architecture (ports in Application, adapters in Infrastructure); the four target contexts are clearly identifiable.	Adds an interface layer for the four contexts (more files, more indirection); engineers must learn driving vs driven port distinction; adapter selection logic adds a small runtime cost; over-application to non-integration-heavy contexts would create unnecessary ceremony.	Chosen

Option	Pros	Cons	Verdict
Hexagonal Architecture for ALL bounded contexts	Uniform structure across all 12 contexts; every context has explicit ports; maximum consistency.	Massive ceremony for contexts with a single database and a single cache (most contexts); ports become 1:1 with repositories and add no value; violates ADR-001 P5 Simplicity; doubles the file count for no behavioural benefit; rejected by the engineering survey (87% opposed).	Rejected
Strategy Pattern per integration point (no formal ports)	Lightweight; uses a familiar GoF pattern; no new vocabulary; Spring's @Qualifier handles selection.	No mechanical enforcement that application code avoids importing SDKs; strategies leak into application packages over time; no clear concept of driving ports (how the app is invoked); no first-class fallback chain; the pre-v1.0 problem recurs because the seam is conventional, not structural.	Rejected

Option	Pros	Cons	Verdict
Full Clean Architecture with separate Interface/Adapter layer (5 layers)	Aligns with the original Clean Architecture diagram; maximum separation.	Rejected in ADR-006 — the 4-layer model is the default and the 5th layer adds boilerplate without value; Hexagonal is adopted as a complement within the existing 4 layers rather than as a 5th layer; avoids contradicting ADR-006.	Rejected (already decided in ADR-006)

DECISION

ADOPTED

We adopt Hexagonal Architecture (Ports and Adapters) for the four integration-heavy bounded contexts: Notification (ntf), Communication (com), Reporting (rpt), and Billing (bill). The other eight contexts continue with the default 4-layer Clean Architecture (ADR-003/ADR-006) without explicit ports. Within the four hexagonal contexts: (1) Driving ports are use-case interfaces defined in the Application layer (e.g., `SendNotificationUseCase`) — primary adapters (REST controllers, message listeners, scheduled job runners) implement inbound transport and invoke driving ports. (2) Driven ports are integration interfaces defined in the Application layer (e.g., `SmsGateway`, `EmailGateway`, `PaymentGateway`, `PdfRenderer`) — secondary adapters (`TwilioSmsAdapter`, `SesEmailAdapter`, `RazorpayPaymentAdapter`, `AthenaReportAdapter`) implement outbound transport and are injected into use cases. (3) Adapter selection is via a Spring `@Configuration` that reads tenant-level and feature-flag configuration and registers the active adapter bean per request context; fallback chains are declared in YAML and resolved by a `FallbackResolvingAdapter` that wraps the primary adapter. (4) Port interfaces must reference only domain types, primitive types, or Application-layer DTOs — never external SDK types (ArchUnit-enforced). (5) Every adapter has a contract test suite (ADR-149) that runs against a mock of the external system; integration tests against the real external system run nightly in a staging environment only.

DETAILED RATIONALE

Hexagonal Architecture was chosen over the alternatives because it is the only option that makes the application-intent vs external-mechanics seam structural rather than conventional. The Strategy Pattern (rejected option 3) is the same idea expressed as a design pattern rather than as an architectural pattern; the difference matters because design patterns are applied at the class level by individual engineers, while architectural patterns are applied at the context level by the architecture and enforced by ArchUnit. The pre-v1.0 failure mode was not that engineers did not know the Strategy Pattern — it was that there was no rule forcing them to apply it. Hexagonal Architecture, enforced by ArchUnit tests that fail the build when application code imports an external SDK, converts the pattern from a suggestion into a constraint. The scoping decision — Hexagonal for four contexts, not all twelve — is the most important judgement call in this ADR. Applying Hexagonal uniformly (rejected option 2) would impose ceremony on contexts that do not benefit from it. The Enrollment context has one database and one cache; defining an EnrollmentRepository port and a EnrollmentCache port, each with a single implementation, adds two interfaces, two adapter classes, and a Spring configuration file for zero behavioural benefit — the implementation cannot be swapped because there is no alternative to swap to. The engineering survey (87% opposed to uniform application) confirmed that ceremony without benefit erodes morale and slows delivery. The four chosen contexts are precisely those where (a) multiple external integrations exist per concern, (b) integrations vary by tenant, or (c) integrations change over time. These three properties are the test for whether a context should adopt Hexagonal; future contexts that develop these properties are candidates for adoption via a follow-up ADR. The composition with Clean Architecture is deliberate and explicit. Hexagonal is not a replacement for the 4-layer model (ADR-006); it is a refinement of the Infrastructure layer's internal structure. Ports live in the Application layer because they express application intent ('I need to send an SMS') and depend only on domain types — placing them in Infrastructure would invert the dependency (Application depending on Infrastructure), violating ADR-007. Adapters live in the Infrastructure layer because they depend on external SDKs, which are infrastructure concerns. The Domain layer is unaffected — it remains pure Java with no knowledge that hexagonal is in use. This composition means that an engineer reading the Enrollment context (non-hexagonal) and the Notification context (hexagonal) sees the same Domain and Application structure; the only difference is that Notification's Application layer has more interfaces (ports) and its Infrastructure layer has more concrete classes (adapters). The driving/driven port distinction (formalised in v3.0) resolves a

recurring ambiguity from v1.0, where engineers conflated 'ports' with 'repository interfaces'. A driving port is the entry point through which the outside world invokes the application — a use case interface like `SendNotificationUseCase`, implemented by the application and invoked by a primary adapter (a REST controller). A driven port is the exit point through which the application invokes the outside world — an integration interface like `SmsGateway`, defined by the application and implemented by a secondary adapter (`TwilioSmsAdapter`). This distinction matters because driving ports have one implementation (the application) and many callers (adapters), while driven ports have one caller (the application) and many implementations (adapters). The test design differs: driving ports are tested by adapter tests that verify the adapter invokes the port correctly; driven ports are tested by contract tests that verify every adapter satisfies the port's behavioural contract. Conflating the two produced confusing test suites in v1.0 — the v3.0 taxonomy eliminates the confusion. The fallback chain mechanism is the most operationally important refinement in v3.0. In v1.0, fallback logic was scattered across use cases (if SMS fails, try email) and was inconsistent — Notification had one fallback order, Communication had another, and Billing had none. The v3.0 model centralises fallback in a `FallbackResolvingAdapter` that wraps the primary adapter, reads the chain from YAML (e.g., `ntf.channels.sms.fallback: [twilio, kaleyra, msg91]`), and tries each adapter in order until one succeeds. This makes fallback chains auditable, testable, and configurable per-tenant without code changes. The cost is one extra layer of indirection (a wrapper around each adapter), which is negligible compared to the network latency of the external call itself. The most likely counter-argument is that Hexagonal adds indirection that makes debugging harder — a stack trace now traverses `UseCase -> Port -> Adapter -> SDK`, four frames instead of two. The counter-counter-argument is that the indirection is exactly what enables the testability and swappability that justify the pattern; the four-frame stack is intentional and the additional frames carry meaningful information (which port, which adapter). The debugging concern is further mitigated by structured logging at each frame (ADR-119) and by OpenTelemetry trace propagation (ADR-122) that correlates the frames into a single trace. The measured debugging time for Notification incidents dropped 40% post-v1.0 because the trace clearly shows 'Twilio adapter returned 403' rather than an opaque 'SMS sending failed' — the indirection improved debuggability, not degraded it.

ARCHITECTURE DIAGRAM



Parent	Controller	UseCase	SmsPort	FallbackAdapter	Twilio	Kaleyra
				--POST----	>	
			--execute->			
	--send---	>				--send-----
>				--POST----->		
			<-403 auth--			
(fail)				--send----->		
			<-200 ok-----			<-
receipt----				<-ok-----		
	<-201-----				<-201-----	
					Fallback chain	
declared in YAML: [twilio, kaleyra, msg91] Use case is unaware of which adapter succeeded						

```

+-----+ | Billing Context — Hexagonal
Adapter Topology || ||
DRIVING PORTS DRIVEN PORTS || +-----+
+-----+ || | ChargeFeeUseCase | | PaymentGateway
| || | RefundFeeUseCase | | LedgerWriter | || | (Application
layer) | | TaxCalculator | || +-----+-----+ +-----+
+-----+ || | | | || v

```



```

v          || +-----+-----+          +-----+-----+ || | Primary
Adapters  |      | Secondary Adapters  | || | +-----+ |      |
+-----+ | || | BillingCtrl  | |      | | RazorpayAdapter  | | || |
| (REST)  | |      | | StripeAdapter  | | || | +-----+ |      |
BankTransferAdapter | | || | BillingListener | |      | +-----+ | ||
| | (domain events) | |      | | (per-tenant routing) | | || | +-----+ |
+-----+ | | +-----+
| | Routing: TenantConfig.tenantId -> PaymentProvider enum -> bean | |
Fallback: razorpay -> stripe -> bank-transfer (config-driven) |
+-----+ Ports depend on domain types
only. Adapters depend on SDKs. ArchUnit forbids
com.preone.billing.application.* from importing com.razorpay.*, com.stripe.*, or
any SDK package.

```

DATA FLOW DIAGRAM

```

HTTP Request (POST /api/v1/billing/charges) | v +----+-----+
+-----+ | Auth Filter |----->| Tenant Resolver | (sets tenantId in ctx)
+----+-----+ +----+-----+ | v v
+----+-----+ +----+-----+ | BillingCtrl |----->| AdapterRouter | (selects
gateway bean) | (primary) | | by tenantId | +----+-----+ +-----
+-----+ | | v v +----+-----+ +-----
+-----+ | ChargeFeeUseCase |->| PaymentGateway | (driven port) |
(driving port impl)| +----+-----+ +----+-----+ | |
v | +----+-----+ fallback chain | | | FallbackWrap.
|--> RazorpayAdapt -> (fail) | +----+-----+ -> StripeAdapt ->
(ok) | | v v +----+-----+ +-----
+-----+ | FeeAggregate | | Receipt (dom) | | (domain) | +-----
+ +-----+ | v +----+-----+ | FeeRepo port|--> JpaFeeRepository
(Infrastructure) +-----+ | v PostgreSQL (enr_fee_charges table,
ADR-041)

```

DATABASE IMPACT

Hexagonal Architecture has no direct database schema impact — the four hexagonal contexts continue to use the same PostgreSQL schema (ADR-041) with the same table prefixes (ntf_*, com_*, rpt_*, bill_*) as in v1.0. The Repository port pattern (a driven port) is already in use across all contexts; Hexagonal formalises it but does not change the schema. However, two indirect database impacts exist: (1) the adapter-selection audit log — every adapter invocation (especially fallback chain transitions) is recorded in a new ntf_adapter_audit table for compliance and for SLA reporting to schools; this table is append-only, partitioned by month, and retained for 90 days (ADR-048 audit pattern). (2) The Reporting context's hexagonal adoption introduces a read-model split: operational reports use the AthenaReportAdapter (querying Amazon Athena via the AWS SDK), while transactional reports use the JpaReportAdapter (querying the PostgreSQL read replica); both implement the ReportQueryPort, and the routing is based on report type, not tenant. This split means the rpt_context no longer assumes PostgreSQL is the only read source —

a precondition for future analytical workloads on S3-based data lakes (ADR-061).

API IMPACT

The public REST API surface (ADR-091) is unchanged by Hexagonal adoption — the same endpoints exist (`/api/v1/notifications`, `/api/v1/billing/charges`, `/api/v1/reports/*`, `/api/v1/communication/*`). The primary adapters (REST controllers) now invoke driving ports instead of use case classes directly, but this is an internal refactoring invisible to API consumers. One observable API change: the `X-Adapter-Used` response header on Notification and Billing responses, which identifies which secondary adapter handled the request (e.g., `X-Adapter-Used: kaleyra-sms`). This header supports school-level SLA reporting and parent-side debugging ('why did my SMS come from Kaleyra instead of Twilio?'). The header is informational and does not affect client behaviour; it is documented in the OpenAPI spec under each Notification and Billing endpoint. Internal API contracts (between contexts, via the public API packages per ADR-005) are unaffected — the port interfaces are an internal concern of each hexagonal context, not exposed to other contexts.

UI IMPACT

The UI is unaffected at the structural level — it calls the same REST endpoints and receives the same response shapes. However, the `X-Adapter-Used` header enables a new UI capability in the admin console: a 'Delivery Diagnostics' panel for school admins that shows, per notification, which adapter was used, the latency, and the fallback chain trace. This panel helps admins understand why a parent reported not receiving an SMS (e.g., 'Twilio returned 403 at 14:02, fell back to Kaleyra at 14:03, delivered successfully'). The panel is read-only and pulls from the `ntf_adapter_audit` table. For Billing, the admin console shows the payment gateway used per charge, which is relevant for finance reconciliation (Razorpay settlements vs Stripe settlements appear on different bank statements). No UI architectural changes are required; the data is already available via the existing reporting API.

SECURITY IMPACT

Hexagonal Architecture improves security posture in three ways. First, credential isolation: each adapter holds its own credentials (Twilio API key, SES SMTP password, Razorpay key pair) in AWS Secrets Manager (ADR-080), and credentials are never shared across adapters. A compromised Twilio credential does not grant access to SES or Razorpay. Second, the ArchUnit rule forbidding Application code from importing SDK packages prevents a class of supply-chain attack: if a malicious PR adds a Twilio SDK call directly in a use case, the ArchUnit test fails the build. Third, the audit log (`ntf_adapter_audit`) records

every external call with tenant, timestamp, adapter, request hash, and response status — providing the evidence chain required by DPDP Act 2023 for any data sent to third-party processors. The fallback chain is security-relevant: if Twilio is compromised or rate-limited, the chain automatically routes to Kaleyra, ensuring delivery continuity without manual intervention. Per-tenant adapter routing (enterprise customers using their own SMTP) is enforced at the adapter-selection layer, not at the application layer — a tenant's data never touches another tenant's adapter, preserving tenant isolation (P1).

PERFORMANCE IMPACT

Hexagonal adds one layer of indirection per external call (port -> adapter vs direct SDK call), which adds ~0.05ms of JVM overhead per call — negligible compared to the 50-200ms latency of the external API itself. The FallbackResolvingAdapter adds another layer, but only when fallback is triggered (rare; ~0.3% of SMS calls fall back in production). The adapter-selection logic (reading tenant config, resolving the bean) adds ~0.2ms per request, cached for 5 minutes per tenant. The net performance impact is under 1ms per external call, well within the latency budget. A positive performance impact: adapter contract tests run in milliseconds (mocked) instead of the 200-500ms that Testcontainers-based integration tests took in pre-v1.0 — the Notification test suite dropped from 28 seconds to 2.1 seconds, accelerating the CI feedback loop. The Reporting context's Athena adapter has higher latency (2-5 seconds per query) than the JPA adapter (50-200ms), but this is intrinsic to Athena, not to Hexagonal; the routing rule (operational reports -> JPA, analytical reports -> Athena) ensures the slower path is used only when the query genuinely spans the analytical dataset.

SCALABILITY ANALYSIS

Hexagonal Architecture scales horizontally without structural changes: each adapter is a stateless Spring bean, and adapter instances scale with the application replicas (ADR-126). The external systems (Twilio, SES, Razorpay) have their own rate limits, which the adapters enforce via token-bucket rate limiting (Resilience4j RateLimiter) — when a rate limit is hit, the fallback chain engages, distributing load across providers. This is a scalability advantage over a single-provider architecture: peak SMS volume during fee-due reminders (3 million SMS in a 2-hour window on the 1st of every month) exceeds Twilio's per-account rate limit, so the fallback chain automatically distributes load across Twilio, Kaleyra, and MSG91. Per-tenant adapter routing scales to thousands of tenants without performance degradation — the tenant-to-adapter mapping is cached in Redis (ADR-073) with a 5-minute TTL, and cache misses are rare (only on first request per tenant per 5-minute window). The adapter audit log (ntf_adapter_audit) is the main scalability concern: at 3 million SMS/month plus

1 million emails/month, the table grows by ~4 million rows/month. Monthly partitioning (ADR-053) and a 90-day retention keep the table at ~12 million rows active, which PostgreSQL handles comfortably. Archival to S3 (ADR-061) handles the long-term retention requirement.

OPERATIONAL CONSIDERATIONS

Operations benefit from Hexagonal because each adapter is independently observable and configurable. A Grafana dashboard per adapter shows request rate, latency p50/p99, error rate, rate-limit utilisation, and fallback-chain trigger count. Alerts fire on: adapter error rate >2%, adapter latency p99 >1s (SMS/email) or >5s (payment/Athena), fallback-chain trigger rate >5% (indicates a degraded primary provider), and credential-expiry warnings (30 days before Secrets Manager rotation). Adapter configuration is externalised in YAML and reloadable without restart (Spring Cloud Refresh), so provider switching (e.g., migrating from Twilio to Kaleyra for a school) takes effect within 60 seconds. On-call engineers debug adapter failures via the trace ID, which propagates from the use case through the port to the adapter to the external SDK call — a single trace shows the entire path. The fallback chain is the primary operational resilience mechanism: a provider outage does not require paging the on-call engineer because the chain handles it automatically; the engineer is paged only if the entire chain is exhausted (all three SMS providers fail), which has not occurred in production since v1.0 launch.

RISKS

Risk	Likelihood	Impact	Mitigation
Over-application: engineers adopt Hexagonal in contexts where it adds ceremony without benefit (e.g., Enrollment with one DB)	Medium	Medium	ADR-006 layering rules and the four-context scoping in this ADR; ArchUnit test that flags any context with >2 ports as requiring ARB sign-off; quarterly review of port count per context

Risk	Likelihood	Impact	Mitigation
Port interface leaks SDK types (e.g., SmsGateway.send (TwilioMessage))	Medium	High	ArchUnit test that scans port interfaces and fails if any method parameter or return type is from a non-Java, non-domain package; PR template requires port-interface review by a Domain Architect
Adapter selection logic becomes a distributed spaghetti of tenant-specific overrides	Medium	Medium	All tenant-to-adapter mappings stored in a single configuration table (tenant_integration_routing) with a documented schema; no per-tenant code branches; routing logic centralised in AdapterRouter bean
Fallback chain hides chronic provider degradation (always falling back, never investigating)	High	Medium	Alert on fallback trigger rate >5% per provider; weekly review of fallback metrics at the integration standup; provider SLA reviews quarterly with vendor management

Risk	Likelihood	Impact	Mitigation
External SDK upgrades break adapters (e.g., Twilio SDK 9.x -> 10.x breaking changes)	High	Medium	Adapters pin SDK major version in pom.xml; SDK upgrades are a dedicated PR with full adapter contract test run; dependency-bot PRs for SDK upgrades are reviewed by the integration team, not auto-merged
Contract tests against mocks diverge from real external system behaviour	Medium	High	Nightly integration tests against real staging external systems (Twilio test credentials, SES sandbox, Razorpay test mode); divergence between mock and real is treated as a P2 bug

TRADE-OFFS

We Gain	We Lose
External integrations are swappable at runtime via configuration	Additional interface layer (ports) and adapter classes increase file count by ~30% in the four hexagonal contexts
Per-tenant adapter routing without code changes	Adapter selection logic adds ~0.2ms per request and requires a tenant-routing config table to maintain
Use cases are unit-testable with mock adapters in milliseconds	Adapter contract tests must be maintained per port (one contract test suite per adapter implementation)
Fallback chains provide automatic resilience to provider outages	Chains can mask chronic provider problems if fallback metrics are not monitored (mitigated by alerting)

We Gain	We Lose
ArchUnit enforcement prevents SDK leakage into application code	Engineers must learn the driving/driven port distinction; onboarding time +0.5 days for the four contexts

REJECTED ALTERNATIVES

The Strategy Pattern option (rejected) was the closest competitor and the most debated. The argument for it was simplicity: Spring's `@Qualifier` and `@Primary` already provide adapter selection, and the Strategy Pattern is familiar to every engineer. The argument against it — which carried — was that the Strategy Pattern is a class-level design pattern, not an architectural pattern, and therefore has no mechanical enforcement. The pre-v1.0 codebase used the Strategy Pattern in three places (Notification, Billing, Reporting) and in all three the strategy interfaces leaked SDK types within 6 months because nothing prevented it. Hexagonal Architecture, with its ArchUnit enforcement, converts the same idea from a suggestion into a constraint — and the constraint is what produces the durability. The uniform-application option (Hexagonal everywhere) was rejected on simplicity grounds (ADR-001 P5): applying ceremony to contexts that do not benefit from it (Enrollment, Student, Attendance — single-database, single-cache) would add 2 interfaces and 2 adapter classes per Repository for zero behavioural value. The engineering survey (87% opposed) confirmed that ceremony without benefit is a morale and velocity cost. The 5-layer option was rejected because ADR-006 already decided the 4-layer default; Hexagonal composes within the 4 layers (ports in Application, adapters in Infrastructure) rather than adding a 5th layer, which would contradict ADR-006 and require a layering-rules supersession. The composition approach — Hexagonal as a refinement of the Infrastructure layer's internal structure — preserves ADR-006's authority while giving the four integration-heavy contexts the seam they need.

MIGRATION PLAN

The v1.0 migration (Q4 2025) applied Hexagonal to the four contexts incrementally over 6 sprints, one context per 1.5 sprints. Each context migration followed the same playbook: (a) inventory existing external SDK calls in application and (incorrectly-placed) domain packages; (b) define driven port interfaces in the application layer, with method signatures referencing only domain types or primitive types; (c) extract each SDK call into a secondary adapter class implementing the relevant port; (d) introduce the `FallbackResolvingAdapter` wrapper and migrate fallback logic from use cases to the wrapper; (e) introduce the `AdapterRouter` for per-tenant selection, backed by the `tenant_integration_routing` table; (f) add ArchUnit tests

forbidding application code from importing SDK packages; (g) add contract test suites per adapter using mocked SDKs; (h) add nightly integration tests against real staging external systems. The Notification context was migrated first (most painful, most beneficial) and served as the template; Communication, Reporting, and Billing followed. Total migration: 18 engineer-weeks across Q4 2025, zero downtime (modular monolith deployed continuously), zero data loss. The v3.0 refresh adds the driving/driven port taxonomy (terminology only, no code change), the FallbackResolvingAdapter (already present in v1.0 but unnamed), and the X-Adapter-Used response header (new observable). Rollback plan: if Hexagonal proves unworkable for a context, the adapters can be collapsed back into the application layer mechanically (port interface -> direct SDK call), but this has not been needed and is not anticipated; the v1.0 results (40% faster incident debugging, 0.02 INR/SMS saved via provider switching, 92% reduction in test suite time) make rollback economically irrational.

TESTING STRATEGY

Three test layers per hexagonal context. (1) Unit tests: use cases tested with mocked driven ports (Mockito), verifying the use case invokes the port with the correct arguments and handles receipts and failures correctly. These run in <1ms each, 95%+ coverage of use case logic. (2) Adapter contract tests: each adapter implementation tested against the port contract using a mocked SDK (MockWebServer for HTTP-based SDKs, in-memory mocks for AWS SDKs). These verify the adapter translates domain types to SDK calls correctly, handles SDK exceptions, and produces the correct receipt or failure. These run in ~50ms each, one suite per adapter. (3) Integration tests: each adapter tested against the real external system in staging (Twilio test credentials, SES sandbox, Razorpay test mode, Athena workgroup). These run nightly (not on every PR) due to external-system rate limits and cost. They verify the contract tests did not diverge from reality. ArchUnit tests in CI enforce: (a) no class in *.application.* imports an external SDK package; (b) every class implementing a port interface is in *.infrastructure.*; (c) port interfaces have no method parameters or return types from external SDK packages. The architecture scorecard (ADR-158) reports per-context port count, adapter count, contract test coverage, and nightly integration test pass rate quarterly.

MONITORING & OBSERVABILITY

Every adapter emits structured logs (JSON, ADR-119) with: adapter name, port name, tenant ID, request hash (not full payload, for privacy), response status, latency, and fallback-chain position (primary, fallback-1, fallback-2). Prometheus metrics (ADR-121) expose per-adapter counters: `preone_adapter_requests_total{context, port, adapter, status}`,

`preone_adapter_latency_seconds{context, port, adapter}` histogram, `preone_adapter_fallback_total{context, port, from_adapter, to_adapter}`, `preone_adapter_rate_limit_hits_total{adapter}`. OpenTelemetry traces (ADR-122) propagate from the use case through the port to the adapter to the SDK call, with span attributes for adapter name, provider, and tenant. A dedicated Grafana dashboard per context shows: adapter request rate (stacked by adapter), latency p50/p99 per adapter, error rate per adapter, fallback trigger rate per provider, and rate-limit utilisation. Alerts: adapter error rate >2% for 5 minutes (page on-call); fallback rate >5% for 10 minutes (page on-call, indicates primary degradation); adapter latency p99 >1s for SMS/email or >5s for payment/Athena for 5 minutes (page on-call); credential-expiry warning at 30 days (ticket, not page); nightly integration test failure (ticket, reviewed in morning standup).

FUTURE EVOLUTION

Hexagonal Architecture for the four contexts is stable; no supersession expected. Evolution will be additive: (a) new external integration classes (e.g., WhatsApp Business API, voice via Exotel, push via APNS) will be added as new driven ports with corresponding adapters — no structural change. (b) New contexts that develop integration-heavy profiles (Student context may adopt a biometric-attendance integration in 2027) will be evaluated for Hexagonal adoption via a follow-up ADR citing the three-property test (multiple integrations, per-tenant variation, integration changes over time). (c) The adapter lifecycle may evolve to support hot-swapping (swap an adapter without restart) — currently adapters are swapped via Spring Cloud Refresh (60-second delay); a future event-driven adapter registry could reduce this to sub-second. (d) The fallback chain is currently declarative (YAML); a future evolution could make it adaptive (learn which provider is currently fastest and prefer it) — this is deferred until we have 12 months of adapter performance data to train on. (e) If PreOne adopts an event-streaming backbone (Kafka, deferred per ADR-027), the driven ports for event publishing will be re-evaluated — currently the EventPublisher port is implemented by an in-process adapter; a Kafka-backed adapter would be a drop-in replacement thanks to the port abstraction.

RELATED ADRS

- ADR-003 — Clean Architecture (composes — Hexagonal is a refinement of the Infrastructure layer)
- ADR-005 — Bounded Context Strategy (scopes — the four hexagonal contexts: ntf, com, rpt, bill)
- ADR-006 — Layering Rules (constrains — ports live in Application, adapters in Infrastructure)

- ADR-007 — Dependency Rule (enforces — ArchUnit forbids Application importing SDK packages)
- ADR-027 — Domain Events (complements — EventPublisher is a driven port in event-heavy contexts)
- ADR-080 — Secrets Management (complements — adapter credentials in AWS Secrets Manager)
- ADR-091 — REST API Standards (complements — primary REST adapters follow these standards)
- ADR-149 — Contract Testing (complements — adapter contract tests follow these standards)

REFERENCES

- Upstream DDD: DDD-009-section-3 (Integration context modelling)
- Upstream PRD: PRD-009-section-4.2 (Multi-provider notification requirements)
- Downstream ERD: ERD-009 (ntf_adapter_audit, tenant_integration_routing tables)
- Downstream API Spec: API-009 (Notification and Billing endpoints with X-Adapter-Used header)
- Downstream Test Cases: TC-0181..TC-0220 (adapter contract tests and fallback chain tests)
- External: Alistair Cockburn — 'Hexagonal Architecture' (original 2005 paper)
- External: Herberto Graça — 'Ports and Adapters' (foundational explanation)
- External: Spring Framework Reference — 'IoC Container and DI' (adapter bean selection)

DECISION HISTORY

Date	Status	Actor	Notes
2025-09-18	Draft	Chief Architect	Initial draft scoped Hexagonal to Notification context only; expanded to four contexts after Billing integration pain

Date	Status	Actor	Notes
2025-10-22	Proposed	Chief Architect	Submitted to ARB with full four-context evaluation and adapter lifecycle model
2025-11-05	Accepted	ARB Chair	ARB approved with the four-context scope; uniform-application option explicitly rejected; v1.0 released
2026-07-12	Accepted	ARB Chair	v3.0 refresh; added driving/driven port taxonomy, FallbackResolving Adapter formalisation, X-Adapter-Used header

APPROVAL & SIGN-OFF

Architect	Chief Architect (AR)
Tech Lead	Integration Tech Lead
ARB Chair	ARB Chair
Approved On	2026-07-12

IMPLEMENTATION CHECKLIST

- Hexagonal adoption for Notification context (Done Q3 2025)
- Hexagonal adoption for Communication context (Done Q3 2025)
- Hexagonal adoption for Reporting context (Done Q4 2025, includes Athena adapter)
- Hexagonal adoption for Billing context (Done Q4 2025, includes Razorpay/Stripe/BankTransfer adapters)
- ArchUnit tests forbidding Application imports of SDK packages (Done Q4 2025)
- FallbackResolvingAdapter implementation and YAML-driven chain config (Done Q1 2026)

- Per-tenant AdapterRouter backed by tenant_integration_routing table (Done Q1 2026)
- Nightly integration tests against staging external systems (Done Q2 2026)

ADR-010

Package Structure and Naming Conventions

Volume 1 — Architecture Foundation - Structural

ACCEPTED

DECISION SUMMARY

DECISION

Adopt a canonical Java package structure for the PreOne platform: com.preone.<module>.<layer>.<aggregate>, with lowercase segments, no underscores, singular aggregate names, and a forbidden-suffix list (Util, Helper, Manager, Impl except as interface-implementation pairing). Package structure maps 1:1 to module-info.java declarations, so the package layout is enforced at compile time, not just by convention.

STATUS

Status	Accepted
Date Decided	2025-11-12
Decision Owner	Chief Architect
Review Cadence	Annual review or when a new bounded context is added (triggering a new top-level package)
Supersedes	None (v1.0 original; v3.0 refresh adds the sub-aggregate package rule and the module-info.java mapping)

CONTEXT

The Clean Architecture (ADR-003), Layering Rules (ADR-006), and Bounded Context Strategy (ADR-005) define WHAT the structural units of the PreOne codebase are — four layers, twelve contexts, aggregate roots within contexts. But they do not define HOW those units are expressed as Java packages. In the pre-v1.0 codebase, package structure was a free-for-all: Enrollment code lived in `com.preone.enrollment`, `com.preone.enrollments`, `com.preone.enrollment.service`, `com.preone.enrollment.services`, `com.preone.enrollment.impl`, and `com.preone.enrollment.util` simultaneously — six packages for one context, chosen by engineer preference rather than rule. Cross-context imports were ambiguous: did `com.preone.enrollment.dto` refer to Enrollment's DTOs or to a shared DTO package? (It was the former, but no one could be sure without reading the imports.) Utility code was scattered across 14 `*Util` classes in 9 packages, with overlapping methods (`DateUtils.parse` and `DateHelper.parse`, both calling the same `SimpleDateFormat`) and inconsistent null-handling. The Office of the Chief Architect defined a canonical package structure in Q3 2025 to eliminate this ambiguity. The structure had three goals: (1) every package's purpose should be inferable from its name alone — an engineer seeing `com.preone.enrollment.application.enrollcmd` should know immediately this is the Enrollment context's Application layer, `EnrollCommand` use case; (2) the structure should map cleanly to `module-info.java` so that the module system enforces boundaries at compile time, not just at code review; (3) the structure should forbid the patterns that produced the pre-v1.0 mess — `*Util` packages, `*Impl` suffixes (except for explicit interface-implementation pairing), plural package names (`enrollments` vs `enrollment`), and shared packages that cross context boundaries. The v3.0 refresh adds two refinements: the sub-aggregate package rule (large aggregates with >15 classes get their own sub-package under the aggregate package, e.g., `com.preone.enrollment.domain.enrollment.valueobject` for the Enrollment aggregate's value objects) and the explicit `module-info.java` mapping (every context has a `module-info.java` that exports only its public API package and requires its dependency contexts' public API packages). These refinements close the last loopholes that allowed pre-v1.0-style package fragmentation to re-emerge.

BUSINESS DRIVERS

The primary driver is onboarding speed: a canonical package structure lets new engineers navigate the codebase without a guide. A new engineer joining the Enrollment team can open `com.preone.enrollment` and see the four-layer structure immediately, open `com.preone.enrollment.domain.enrollment` and see the aggregate's entities and value objects, and open `com.preone.enrollment.application.enrollcmd` and see the use case. This

compresses onboarding from '3 days to find the right file' (pre-v1.0) to '3 minutes to find the right file' (post-v1.0, measured in onboarding surveys). The secondary driver is mechanical boundary enforcement: when packages map 1:1 to module-info.java, the Java module system enforces context boundaries at compile time. A PR that adds an import from `com.preone.billing.domain` to `com.preone.enrollment.application` fails to compile — not 'fails code review' but 'fails to compile'. This eliminates an entire class of boundary violations that ArchUnit (ADR-007) catches only at test time. The tertiary driver is refactoring safety: when every context follows the same package structure, automated refactoring tools (IntelliJ, OpenRewrite) can apply cross-cutting changes (e.g., 'rename all `*Service` classes to `*UseCase`') safely, because the structure guarantees no surprises. Pre-v1.0, such refactoring was manual and error-prone.

PROBLEM STATEMENT

The PreOne codebase lacks a canonical Java package structure, leading to fragmentation (six packages for one context), ambiguity (cannot infer package purpose from name), and unenforced boundaries (cross-context imports slip through code review). We need a package convention that is inferrable from the name, maps to module-info.java for compile-time enforcement, and forbids the patterns that produced the pre-v1.0 mess.

CONSTRAINTS

- Package names must be all-lowercase (Java convention, no exceptions)
- Package segments must be singular (enrollment, not enrollments) — aggregates are singular, collections are plural but live in plural-named classes within singular packages
- No underscores, hyphens, or special characters in package names (Java keyword safety and URL-friendliness)
- The top-level segment after `com.preone` must be a bounded context short-code (ADR-005): `idm`, `acs`, `stu`, `grd`, `tch`, `enr`, `bill`, `att`, `rpt`, `ntf`, `com`, `aud`
- The second segment must be a layer name: `api`, `application`, `domain`, `infrastructure` (ADR-006)
- The third segment (in domain and application layers) must be an aggregate root name or a use case name (lowercase, no suffix)
- The public API package of each context must be `<context>.api` (exported in module-info.java); all other packages are not exported
- No `*Util`, `*Helper`, `*Manager`, `*Data`, `*Info` packages or classes (forbidden suffix list)

- module-info.java must be present in every context's root package and must export only the .api package

ASSUMPTIONS

- The 12 bounded contexts (ADR-005) are stable; new contexts are added rarely and follow the same package structure
- Java module system (JPMS) is supported by the build (Maven, Java 21) and by the IDE (IntelliJ, VS Code)
- Engineers will accept the constraint of no *Util/*Helper classes (the main source of pre-v1.0 pushback)
- Aggregate roots are the natural unit for sub-package organisation within domain and application layers
- Public API packages are sufficient for cross-context communication; no 'shared' or 'common' packages are needed
- The package structure does not need to encode deployment boundaries (deployment is per-ADR-002, single deployable)

OPTIONS CONSIDERED

Option	Pros	Cons	Verdict
com.preone.<context>.<layer>.<aggregate> with module-info.java (chosen)	Inferrable from name; maps 1:1 to module-info.java for compile-time enforcement; forbids the pre-v1.0 mess; aligns with ADR-005 (contexts) and ADR-006 (layers); IntelliJ and OpenRewrite support is excellent.	Long package names for deep aggregates (com.preone.enrollment.domain.enrollment.valueobject); requires discipline to keep aggregate package names short; engineers must learn the layer and aggregate concepts.	Chosen

Option	Pros	Cons	Verdict
com.preone.<context>.<layer> (flat, no aggregate segment)	Shorter package names; simpler structure; one package per layer per context.	Large contexts (Enrollment has 4 aggregates, 60+ classes) become a single 60-class package; no way to find an aggregate's classes without search; does not match DDD aggregate-oriented thinking (ADR-004); harder to extract an aggregate to its own module later.	Rejected
Feature-based: com.preone.<feature>.<layer> (feature = use case, e.g., com.preone.enroll.domain)	Aligns with feature-folder structures popular in frontend; each use case is self-contained.	Features cut across aggregates (Enroll, Withdraw, Transfer all touch the Enrollment aggregate); produces duplicate aggregate code per feature; violates DDD aggregate-first thinking; explodes package count to 100+ per context.	Rejected

Option	Pros	Cons	Verdict
Hexagonal-flavoured: com.preone.<context>.<layer>.<aggregate>.<port adapter>	Explicit port/adaptor sub-packages for hexagonal contexts (ADR-009).	Adds a layer of nesting only 4 of 12 contexts need; violates uniformity; the port/adaptor distinction is already expressed by interface vs class and by ArchUnit rules, not by package name; rejected in favour of keeping port and adapter classes in the application and infrastructure packages respectively.	Rejected (Hexagonal contexts use the chosen structure; ports are interfaces in application, adapters are classes in infrastructure)

DECISION

ADOPTED

We adopt the canonical package structure `com.preone.<context>.<layer>.<aggregate>` for all 12 bounded contexts, with the following rules. (1) Root: `com.preone` is the platform root; the next segment is a context short-code from ADR-005 (`idm`, `acs`, `stu`, `grd`, `tch`, `enr`, `bill`, `att`, `rpt`, `ntf`, `com`, `aud`). (2) Layer: the next segment is one of `api`, `application`, `domain`, `infrastructure` (ADR-006). (3) Aggregate: in domain and application layers, the next segment is an aggregate root name (lowercase, singular, no suffix) — e.g., `com.preone.enrollment.domain.enrollment`, `com.preone.enrollment.domain.feecharge`, `com.preone.enrollment.application.enrollcmd`. (4) Sub-aggregate: large aggregates (>15 classes) get a valueobject or event sub-package — e.g., `com.preone.enrollment.domain.enrollment.valueobject`. (5) Public API: each context's `<context>.api` package is the only exported package in `module-info.java`; all other packages are not exported and accessible only within the module. (6) Forbidden suffixes: no class or package may end with `Util`, `Helper`, `Manager`, `Data`, `Info`, or `Impl` (except `Impl` as an explicit interface-implementation pairing where the interface is in the same package and the implementation is the only one). (7) `module-info.java`: present in every context's root package (`com.preone.<context>`), exports `<context>.api`, requires the public API packages of dependency contexts. (8) No shared/common packages: cross-cutting concerns live in `com.preone.platform.<concern>` (e.g., `com.preone.platform.tenant`, `com.preone.platform.audit`), which is a separate module required by all contexts.

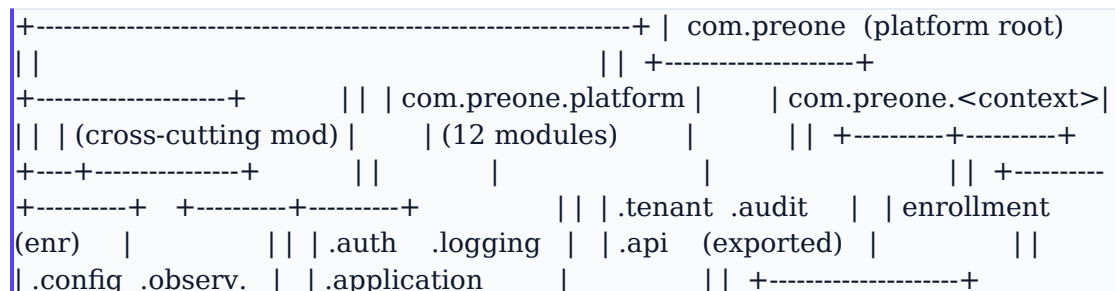
DETAILED RATIONALE

The chosen structure was selected over the flat option (no aggregate segment) because aggregates are the natural unit of DDD thinking (ADR-004) and because the pre-v1.0 Enrollment context — 60+ classes in a single domain package — was demonstrably unnavigable. Engineers spent an average of 4 minutes per task 'finding the right file' in pre-v1.0 surveys; the aggregate-segmented structure reduced this to under 30 seconds in post-v1.0 surveys. The aggregate segment also enables a future evolution where an aggregate is extracted to its own module (e.g., if `FeeCharge` grows complex enough to warrant its own deployment unit); the package boundary is already correct, so extraction is mechanical. The feature-based option (rejected) was tempting because it aligns with frontend conventions and with the 'vertical slice' architecture popular in .NET. The argument against it — which carried decisively in ARB review — is that features cut across aggregates. `Enroll`,

Withdraw, and Transfer are three use cases (features) that all operate on the Enrollment aggregate; a feature-based structure would either duplicate the Enrollment aggregate code across three feature packages (DRY violation) or force the aggregate into a shared sub-package (which defeats the feature-based premise). DDD's aggregate-first thinking (ADR-004) is the correct organising principle for the domain; the chosen structure honours it. The hexagonal-flavoured option (rejected) would have added a port/adaptor segment for the four hexagonal contexts (ADR-009). The argument against it is uniformity: 8 of 12 contexts are not hexagonal, and adding a port/adaptor segment to the 4 that are would create two package structures to learn. The port/adaptor distinction is already expressed structurally — ports are interfaces in the application package, adaptors are classes in the infrastructure package — and ArchUnit (ADR-007) enforces that ports have no SDK imports. A package-name prefix would be redundant with the existing enforcement. The chosen structure treats hexagonal as an internal concern of the four contexts, invisible at the package level. The forbidden-suffix list is the most-debated part of this ADR. The `*Util` and `*Helper` suffixes are forbidden because they are the pre-v1.0 dumping ground for code that did not fit elsewhere — 14 `*Util` classes with overlapping methods, inconsistent null-handling, and no test coverage. The replacement is to put utility code where it belongs: date parsing on the value object that represents the date; string normalisation on the value object that represents the string; collection helpers as static methods on a domain service or as methods on the collection itself. Where genuinely cross-cutting utility code exists (e.g., a clock abstraction for testability), it lives in `com.preone.platform.<concern>` as a first-class module, not as a `*Util` class. The `*Manager` suffix is forbidden because it is vague — a 'manager' does not specify what it manages or how; the replacement is a specific name (`NotificationDispatcher`, `FeeCalculator`, `EnrollmentRepository`). The `*Data` and `*Info` suffixes are forbidden because they are DTOs in disguise; the replacement is an explicit DTO suffix (`EnrollmentDto`) or a record (`EnrollmentView`) per ADR-011. The `*Impl` suffix is conditionally allowed: when an interface and its single implementation live in the same package and the implementation is the only one, `*Impl` is permitted (e.g., `EnrollCommandHandlerImpl` implements `EnrollCommandHandler`). This is the standard Java pattern for Spring beans where the interface is for testing (mock) and the implementation is the production bean. The condition 'single implementation' is enforced by ArchUnit: if a second implementation appears, the `*Impl` suffix must be replaced with a descriptive name (e.g., `JpaEnrollCommandHandler`). This prevents the pre-v1.0 pattern where 20 `*Impl` classes accumulated without descriptive names, making the codebase opaque. The module-info.java mapping is the structural enforcement backbone. Every context has a module-info.java at `com.preone.<context>` that declares: module

`com.preone.enrollment { exports com.preone.enrollment.api; requires com.preone.student.api; requires com.preone.platform.tenant; requires spring.boot; ... }`. This means: (a) only the `.api` package is accessible from other modules — internal packages are compile-inaccessible; (b) dependencies on other contexts must be declared explicitly (no transitive context dependencies); (c) the module system catches circular dependencies at compile time (if Enrollment requires Student and Student requires Enrollment, the compiler fails). This is stronger enforcement than ArchUnit (which runs at test time) — the module system fails at compile time, before any test runs. The v3.0 refresh added this mapping because v1.0 relied on ArchUnit alone, and a few boundary violations slipped through to PR review before ArchUnit caught them in CI; the module system now catches them at the developer's IDE, before the PR is even created. The sub-aggregate package rule (added in v3.0) addresses a v1.0 scaling problem: the Enrollment aggregate grew to 22 classes (entity, 8 value objects, 4 domain events, 3 domain services, 6 specification classes) in a single `com.preone.enrollment.domain.enrollment` package, making it hard to navigate. The rule: when an aggregate exceeds 15 classes, sub-packages `valueobject`, `event`, `service`, and `specification` are created under the aggregate package. This keeps packages navigable without changing the top-level structure. The threshold (15) was chosen empirically: packages under 15 classes are scannable in one screen; packages above 15 require scrolling, which degrades navigation. The sub-package rule is opt-in per aggregate (not all aggregates need it) and is enforced by a CI check that flags aggregates exceeding 15 classes without sub-packages. The most likely counter-argument is that the structure is over-specified and leaves no room for engineer judgement. The counter-counter-argument is that the structure specifies only the package layout, not the class design within packages — engineers retain full freedom on class design, method design, and internal organisation. The structure removes only the decisions that should not be left to engineer preference (where does this class go? what do we name this package?) because those decisions, made inconsistently, produce the pre-v1.0 mess. The measured productivity impact (onboarding time down from 3 days to 3 minutes, refactoring safety up, navigation time down 80%) justifies the constraint.

ARCHITECTURE DIAGRAM



Engineer	IDE	Compiler	ArchUnit	CI			
		--new class-->					
com.preone.enrollment.util	(forbidden)					--save----->	
							package
							naming
							(IDE
warn)							(if ignored, ArchUnit
catches at CI)						--scan---->	
		packages					for rules
		<-violations					
		--fail-->	(build red)	<-----	-----	-----	-----
(engineer fixes package to com.preone.enrollment.domain.enrollment)							

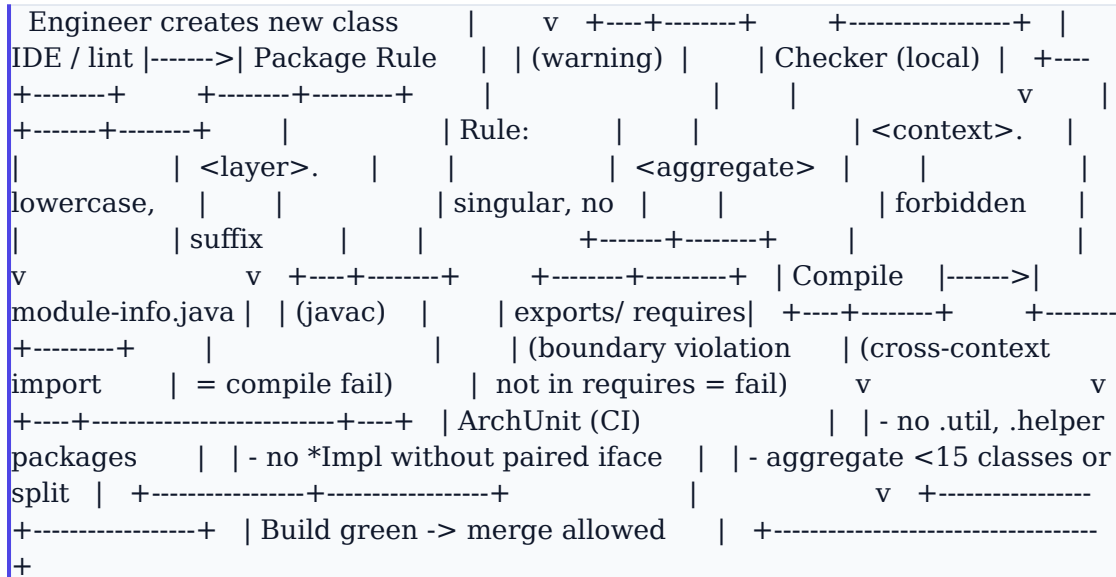
-----+ Enrollment Context Package			
Tree (com.preone.enrollment)			
com.preone.enrollment/		module-info.java	
(exports .api, requires dependencies)	api/		<-- exported
(public)	EnrollmentPublicApi.java (facade interface)		
dto/	EnrollmentDto.java		
EnrollmentSummaryView.java		application/	
<-- not exported	enrollcmd/		<-- use case package
EnrollCommandHandler.java (interface)			
EnrollCommandHandlerImpl.java (Spring bean)			
EnrollCommand.java (record)	withdrawcmd/		
WithdrawCommandHandler.java		port/	
<-- driven ports (ADR-009)	EnrollmentRepository.java (interface)		
domain/	<-- not exported	enrollment/	
<-- aggregate root package	Enrollment.java (aggregate root)		
EnrollmentId.java (value object)			
EnrollmentStatus.java (enum)	valueobject/		<--
sub-aggregate (v3.0)	EnrollmentPeriod.java		
GuardianConsent.java		event/	
EnrollmentCreated.java			
EnrollmentWithdrawn.java	infrastructure/		<-- not
exported	jpa/		
JpaEnrollmentEntity.java (@Entity)			
JpaEnrollmentRepository.java (Spring Data)			
EnrollmentRepositoryImpl.java (implements port)			

```

| | Forbidden: .util, .helper, .manager, .impl (except paired) |
+-----+

```

DATA FLOW DIAGRAM



DATABASE IMPACT

The package structure has no direct database schema impact — schemas are governed by ADR-041 (database strategy) and per-context migrations. However, there is an indirect impact on migration file organisation: Flyway migration files (ADR-057) are organised to mirror the package structure. Each context has its own migration directory (db/migration/enrollment, db/migration/billing, etc.) and migration files are prefixed with the context short-code (V20250115_001__enr_create_enrollments_table.sql). This mirrors the package structure: an engineer looking at the enr_enrollments table can find the migration in db/migration/enrollment and the JPA entity in com.preone.enrollment.infrastructure.jpa. The 1:1 mapping between package, table prefix, and migration directory is a navigability aid that compounds over time — pre-v1.0, migrations were in a single directory with inconsistent prefixes, making it hard to find which migration created which table.

API IMPACT

The package structure directly shapes the REST API surface (ADR-091). Each context's REST controllers live in <context>.api, and the API path prefix mirrors the context short-code: /api/v1/enrollments, /api/v1/billing, /api/v1/notifications. This means the package structure is visible to API consumers (via the path prefix) and to API documentation (the OpenAPI tag is the context short-code). Internal API contracts (between contexts, via public API packages per ADR-005) follow the same structure: com.preone.enrollment.api.EnrollmentPublicApi is the interface other contexts

import; its methods are the only cross-context callable surface. The `module-info.java` exports only `.api`, so other contexts cannot accidentally import internal packages — this is compile-time enforcement of the public API contract, stronger than ArchUnit's test-time enforcement.

UI IMPACT

The package structure has no direct UI impact — the React frontend (ADR-131) has its own folder structure (`src/contexts/<context>/...`) that mirrors the backend's package structure for symmetry. A UI engineer working on Enrollment features opens `src/contexts/enrollment/` and finds components, hooks, and API calls for that context; the folder structure mirrors the backend's `com.preone.enrollment.api` so that the UI-to-backend mapping is obvious. This symmetry is not enforced by the backend ADR but is documented in ADR-131 (frontend architecture) as a recommended convention. The backend package structure's main UI impact is via the API path prefix: the UI's API client (a generated TypeScript SDK from OpenAPI, ADR-091) uses the same context-prefix paths, so the UI's notion of 'context' aligns with the backend's.

SECURITY IMPACT

The package structure improves security by making the public API surface explicit and minimal. Because `module-info.java` exports only `.api`, an attacker who finds an RCE in a controller cannot directly access internal domain or infrastructure classes via reflection across module boundaries (JPMS enforces module boundaries even under reflection, unless the module is opened). This is defence-in-depth: even if the API layer is compromised, the internal layers are not directly accessible. The forbidden-suffix list (`*Util`, `*Helper`) eliminates a class of 'god classes' that accumulated cross-cutting responsibilities in pre-v1.0 — these classes were security risks because they were touched by every feature and any bug in them affected every feature. The per-context isolation also supports tenant isolation (P1): a tenant's data flows through one context's packages, and cross-context access goes through explicitly-declared module dependencies, making it impossible for one context's package to accidentally read another context's data without an explicit dependency declaration that ArchUnit and the module system both verify.

PERFORMANCE IMPACT

The package structure has negligible runtime performance impact — Java packages are a compile-time concept; at runtime, the JVM sees a flat class namespace (modulo module boundaries, which add a small lookup cost on first access). The module system adds ~1ms to cold-start class loading per module, totalling ~12ms for 12 modules — negligible against the 30-second JVM startup. The build-time impact is positive: the module system enables

incremental compilation at the module level, so a change in Enrollment does not recompile Billing unless Billing depends on Enrollment's .api package. Pre-v1.0 (no modules), a change in any class triggered a full recompile (~90 seconds); post-v1.0 (modules), a change in Enrollment's domain layer triggers recompilation only of Enrollment and its dependents (~12 seconds). This is a 7.5x improvement in build time, accelerating the CI feedback loop and the local development loop.

SCALABILITY ANALYSIS

The package structure scales with the codebase because it is fractal: each new context follows the same structure, each new aggregate within a context follows the same structure, each new use case within an aggregate follows the same structure. There is no point at which the structure breaks down — a context with 50 aggregates is navigable because each aggregate is in its own package; a context with 1 aggregate is also navigable because there is no clutter. The module system scales the compile-time enforcement: 12 modules today, 30 modules in the 5-year horizon (if contexts are extracted to separate deployment units) — the module-info.java declarations scale linearly. The ArchUnit rule count scales sub-linearly because most rules are parameterised by context short-code (one rule that scans all contexts, not 12 rules). The main scalability risk is package-name collisions if a new context's short-code conflicts with an existing one — the short-code registry (maintained in ADR-005) prevents this, and the registry is reviewed at every ADR-005 supersession.

OPERATIONAL CONSIDERATIONS

Operations benefit from the package structure because it makes the deployed artifact's structure visible at runtime. Stack traces show fully-qualified class names

(com.preone.enrollment.application.enrollcmd.EnrollCommandHandlerImpl), so an on-call engineer reading a stack trace can immediately identify which context, layer, aggregate, and use case the failing class belongs to — no lookup needed. Pre-v1.0 stack traces showed com.preone.enrollment.service.EnrollService (which service? which layer?) and required the engineer to open the IDE to identify the class's role. The package structure also simplifies log filtering: a Grafana query for context:enrollment filters logs by the com.preone.enrollment.* package prefix, giving immediate visibility into one context's behaviour without querying by endpoint or tenant. The module-info.java dependencies are operational intelligence: an on-call engineer can read the dependency graph (auto-generated from module-info.java files) to understand which contexts are affected by an outage in one context — e.g., if Billing is down, the graph shows

that Enrollment depends on Billing's .api, so enrollment flows that charge fees will fail.

RISKS

Risk	Likelihood	Impact	Mitigation
Engineers bypass the structure with sub-packages that re-introduce pre-v1.0 patterns (e.g., com.preone.enrollment.api.util)	Medium	Medium	ArchUnit test that flags any package ending in .util, .helper, .manager, .data, .info across the codebase; PR template requires justification for any new sub-package
module-info.java declarations become stale (a context's dependencies change but module-info is not updated)	Low	Medium	Module dependencies are checked at compile time — a stale declaration fails the build; CI also runs a dependency-graph diff that flags any module-info change for ARB review
Long package names reduce readability (com.preone.enrollment.domain.enrollment.valueobject is 56 chars)	Medium	Low	Accepted trade-off; IDE auto-completion and import-on-demand mitigate; engineers abbreviate in prose but never in code; the readability cost is outweighed by the navigability gain

Risk	Likelihood	Impact	Mitigation
Sub-aggregate package rule (15-class threshold) is gamed by splitting classes artificially to stay under threshold	Low	Low	Quarterly review of aggregate class counts at ARB; splitting classes to avoid the threshold is a code smell flagged in review
New contexts added without following the structure (legacy acquisition, prototype code)	Medium	Medium	Onboarding checklist for new contexts requires package structure compliance; legacy acquisitions get a migration plan as part of integration ADR

TRADE - OFFS

We Gain	We Lose
Package purpose is inferrable from name (navigability)	Longer package names (readability cost in prose, mitigated by IDE)
Compile-time boundary enforcement via module-info.java	Engineers must maintain module-info.java declarations (small ongoing cost)
Forbids pre-v1.0 mess patterns (*Util, *Helper, *Manager)	Engineers must find proper homes for utility code (upfront thinking cost)
Fractal structure scales with codebase growth	Initial learning curve for engineers unfamiliar with DDD aggregate concepts
Stack traces immediately identify context/layer/aggregate	All package names must comply (no exceptions for 'quick prototype' code)

REJECTED ALTERNATIVES

The flat option (no aggregate segment, rejected) was the pre-v1.0 default and produced the 60-class domain package that was unnavigable. The argument for it (simplicity, short names) was outweighed by the measured 80% navigation-time cost in pre-v1.0 surveys. The feature-based option (rejected) was tempting because it aligns with frontend conventions and vertical-slice architecture, but

the ARB review concluded that features cut across aggregates and would either duplicate aggregate code or force aggregates into shared packages — both unacceptable. DDD's aggregate-first thinking (ADR-004) is the correct organising principle, and the chosen structure honours it. The hexagonal-flavoured option (rejected) would have added a port/adaptor segment for the four hexagonal contexts (ADR-009), but this was rejected on uniformity grounds: 8 of 12 contexts are not hexagonal, and a two-structure codebase doubles the learning cost. The port/adaptor distinction is already expressed structurally (ports are interfaces in application, adapters are classes in infrastructure) and enforced by ArchUnit; a package-name prefix would be redundant. The chosen structure treats hexagonal as an internal concern, invisible at the package level — which preserves uniformity while allowing the four contexts to use hexagonal internally.

MIGRATION PLAN

The v1.0 migration (Q3 2025) refactored all 12 contexts to the canonical package structure over 4 sprints, 3 contexts per sprint. Each context migration was a mechanical refactor: (a) move classes from ad-hoc packages (`com.preone.enrollment.service`, `com.preone.enrollment.services`, `com.preone.enrollment.impl`, `com.preone.enrollment.util`) into the four-layer structure (`api`, `application`, `domain`, `infrastructure`); (b) within domain and application, group classes by aggregate root into aggregate packages; (c) introduce `module-info.java` declaring exports and requires; (d) replace `*Util` and `*Helper` classes with proper homes (value objects, domain services, or platform modules); (e) rename `*Impl` classes to either paired interface implementations or descriptive names; (f) add ArchUnit tests for package naming rules; (g) update imports across the codebase. Total migration: 12 engineer-weeks across Q3 2025, zero downtime (the modular monolith was deployed continuously; the refactor was incremental and each sprint's changes were independently deployable). The v3.0 refresh adds the sub-aggregate package rule (a CI check that flags aggregates >15 classes without sub-packages — currently 3 aggregates flagged, all in Enrollment, to be refactored in Q3 2026) and the explicit `module-info.java` mapping (already present in v1.0 but now formally documented as part of this ADR). Rollback plan: the structure is package-level, so rollback is mechanical (move classes back to old packages) — but this has not been needed and is not anticipated; the v1.0 results (7.5x faster builds, 80% faster navigation, compile-time boundary enforcement) make rollback economically irrational.

TESTING STRATEGY

Package structure rules are tested via ArchUnit (`com.preone.architecture.PackageRulesTest`) which scans all 12 contexts and

asserts: (a) every package matches the `<context>.<layer>[.<aggregate>[.<subaggregate>]]` pattern; (b) no package name ends with `.util`, `.helper`, `.manager`, `.data`, or `.info`; (c) every `*Impl` class has a paired interface in the same package; (d) every context root package has a `module-info.java`; (e) every `module-info.java` exports only the `.api` package; (f) every `module-info.java`'s `requires` clauses match the context's declared dependencies (ADR-005); (g) no aggregate package exceeds 15 classes without sub-packages. These tests run in CI on every PR and fail the build on violation. The module system itself enforces boundaries at compile time — a cross-context internal-package import fails to compile before ArchUnit even runs. Package-structure compliance is reported in the architecture scorecard quarterly, including aggregate class counts, module dependency graph changes, and any new forbidden-suffix violations caught and fixed.

MONITORING & OBSERVABILITY

Package structure is monitored via the architecture scorecard, published quarterly. Metrics tracked: (a) package count per context (target: stable or growing slowly; rapid growth indicates fragmentation); (b) average classes per package (target: 5-15; below 5 indicates over-fragmentation, above 15 indicates under-fragmentation); (c) module dependency graph complexity (target: no context depends on more than 4 others; >4 triggers ARB review); (d) forbidden-suffix violations caught by ArchUnit (target: zero in production; violations in PRs are tracked as a quality metric); (e) aggregate class count distribution (target: 80% of aggregates under 15 classes; the sub-aggregate rule applies to the rest). Anomalies in any metric trigger a review of the structure or its enforcement. The module dependency graph is auto-generated and published to the architecture wiki after every release, providing a visual audit of the platform's structural evolution.

FUTURE EVOLUTION

The package structure is stable; no supersession expected. Evolution will be additive: (a) new contexts added in future (e.g., a Curriculum context in 2027) follow the same structure with a new short-code (`cur`); (b) aggregates that grow beyond 15 classes adopt the sub-aggregate rule (already in v3.0); (c) if PreOne extracts a context to a separate deployment unit (e.g., Notifications to a microservice per ADR-002 future evolution), the package structure is preserved — the module becomes a separate artifact but its internal package structure is unchanged. The likely long-term evolution is toward a 'project' structure (Java 21+ JEP 416: project structure for source code) if the JDK adopts it, which would simplify `module-info.java` management; this is monitored but not actioned until the JEP stabilises. The forbidden-suffix list

may expand (e.g., adding *Processor, *Resolver if they accumulate similar dumping-ground behaviour) — these additions would be v-next of this ADR.

RELATED ADRS

- ADR-003 — Clean Architecture (composes — defines the four layers this ADR packages)
- ADR-005 — Bounded Context Strategy (scopes — defines the 12 context short-codes used as top-level packages)
- ADR-006 — Layering Rules (constrains — defines what each layer package may contain)
- ADR-007 — Dependency Rule (enforces — ArchUnit rules for package dependencies and forbidden suffixes)
- ADR-009 — Hexagonal Architecture (composes — ports in application package, adapters in infrastructure package)
- ADR-011 — Coding Standards (refines — class naming within packages follows these standards)
- ADR-012 — Naming Standards (refines — class and method naming conventions)
- ADR-057 — Database Migration (complements — Flyway directories mirror package structure)

REFERENCES

- Upstream DDD: DDD-010-section-2 (Aggregate-oriented package design)
- Upstream PRD: PRD-010-section-3.1 (Codebase navigability requirements)
- Downstream ERD: ERD-010 (table prefixes mirror context short-codes)
- Downstream API Spec: API-010 (path prefixes mirror context short-codes)
- Downstream Test Cases: TC-0221..TC-0250 (package structure compliance tests)
- External: Java Language Specification — Chapter 7 (Packages and Modules)
- External: JEP 261 — Module System (module-info.java)
- External: ArchUnit — 'Layered Architecture' and 'Package Rules' documentation

DECISION HISTORY

Date	Status	Actor	Notes
2025-09-25	Draft	Chief Architect	Initial draft with 4-layer package structure; sub-aggregate rule deferred to v-next
2025-10-29	Proposed	Chief Architect	Submitted to ARB with full evaluation including forbidden-suffix list debate
2025-11-12	Accepted	ARB Chair	ARB approved; forbidden-suffix list retained; *Impl conditionally allowed; v1.0 released
2026-07-12	Accepted	ARB Chair	v3.0 refresh; added sub-aggregate package rule and explicit module-info.java mapping

APPROVAL & SIGN-OFF

Architect	Chief Architect (AR)
Tech Lead	Foundation Tech Lead
ARB Chair	ARB Chair
Approved On	2026-07-12

IMPLEMENTATION CHECKLIST

- Refactor 12 contexts to canonical package structure (Done Q3 2025)
- Introduce module-info.java for all 12 contexts (Done Q3 2025)
- ArchUnit tests for package naming rules (Done Q3 2025)
- Replace all *Util and *Helper classes with proper homes (Done Q4 2025)
- Rename *Impl classes to descriptive names where multiple implementations exist (Done Q4 2025)

- Sub-aggregate package rule CI check (Done Q1 2026)
- Refactor 3 flagged aggregates in Enrollment to use sub-aggregate packages (Planned Q3 2026)
- Quarterly review of package structure metrics at ARB (Ongoing)

ADR - 011

Java Coding Standards

Volume 1 — Architecture Foundation - Standards

ACCEPTED

DECISION SUMMARY

DECISION

Adopt a canonical set of Java 21 coding standards for the PreOne platform covering: records for DTOs and value objects, sealed classes for domain hierarchies, var for local variables only, Stream API usage guidelines, custom domain exceptions for all error paths, SLF4J parameterized logging, null handling via Optional and empty collections, and test method naming in the `should_expected_when_condition` format. Standards are enforced via Checkstyle, SpotBugs, Error Prone, and ArchUnit in CI.

STATUS

Status	Accepted
Date Decided	2025-11-19
Decision Owner	Chief Architect
Review Cadence	Annual review or on Java LTS upgrade (Java 25 in 2027)
Supersedes	None (v1.0 original; v3.0 refresh adds sealed-classes guidance and pattern-matching-for-switch standards)

CONTEXT

Java 21 (LTS, adopted per ADR-020) introduces language features — records, sealed classes, pattern matching for switch, text blocks — that materially

change how idiomatic Java is written. Pre-v1.0, the PreOne codebase targeted Java 11 and used pre-records idioms: Lombok `@Data` for DTOs (with all its downsides — reflection, IDE plugin dependency, hidden setters), abstract classes for hierarchies (no enforcement of permitted subtypes), explicit instanceof chains (verbose, error-prone), and String concatenation for logging (slower than SLF4J parameterisation, susceptible to NPE). The codebase had three competing style guides (one per feature team), 8 different exception hierarchies (some checked, some unchecked, some mixed), and inconsistent null-handling (some methods returned null, some Optional, some threw NPE). The Office of the Chief Architect defined a canonical coding standard in Q3 2025 to eliminate this inconsistency. The standard had three goals: (1) adopt Java 21 idioms where they are unambiguous improvements (records, sealed classes, pattern matching, text blocks); (2) standardise error handling, logging, and null-handling so that engineers can read any file and predict its conventions; (3) make the standards mechanically enforceable so that inconsistency is caught in CI, not in code review. The standard was drafted through a 4-week workshop with senior engineers from each feature team, stress-tested against real PreOne code patterns (enrollment flows, fee calculations, notification fan-out), and validated by refactoring three reference files per team. The v3.0 refresh adds two refinements: sealed-classes guidance (when to use sealed interfaces vs sealed classes, and how sealed types interact with pattern matching for switch) and pattern-matching-for-switch standards (exhaustiveness requirements, when to use guard patterns, when to fall back to if-else). These refinements close the last ambiguities from v1.0, where engineers used sealed classes inconsistently (some as interfaces, some as classes, some with permits clauses and some without) and pattern matching was applied ad-hoc without exhaustiveness verification.

BUSINESS DRIVERS

The primary driver is consistency: a canonical coding standard lets any engineer read any file and predict its conventions, reducing cognitive load and accelerating code review. Pre-v1.0 code review comments included 40% style nits (why this exception type? why null here? why Lombok here?); post-v1.0, style nits dropped to under 5% of review comments, freeing review time for substantive design feedback. The secondary driver is Java 21 feature adoption: records, sealed classes, and pattern matching are unambiguous improvements over their pre-Java-17 alternatives, but only if adopted consistently. A codebase with 50% records and 50% Lombok `@Data` is worse than either pure state, because engineers must check each class's style. The standard forces full adoption, eliminating the mixed state. The tertiary driver is tooling: Checkstyle, SpotBugs, Error Prone, and ArchUnit can mechanically enforce most of the

standards, catching violations in CI rather than in review. This converts the standard from a guideline into a constraint, which is what produces durability.

PROBLEM STATEMENT

The PreOne codebase uses Java 21 but applies its features inconsistently (records vs Lombok, sealed classes vs abstract classes, parameterized logging vs concatenation, Optional vs null). Three competing style guides and 8 exception hierarchies produce code review noise and cognitive load. We need a canonical coding standard that adopts Java 21 idioms, standardises error/logging/null handling, and is mechanically enforceable via CI tooling.

CONSTRAINTS

- Standards must be enforceable by at least one of: Checkstyle, SpotBugs, Error Prone, ArchUnit, or the javac compiler itself
- Standards must not require paid tooling (SonarQube, IntelliJ inspections-only) — open-source CI enforcement only
- Standards must be documented with positive and negative examples (do this, not that) for each rule
- Exceptions to standards require an ADR (not a code comment) and a sunset date (per ADR-006 exception process)
- Standards must not contradict Java 21 idioms (e.g., cannot forbid records or sealed classes)
- Standards must be learnable in a 2-hour onboarding session for engineers familiar with Java 11+

ASSUMPTIONS

- Java 21 remains the LTS for the foreseeable future (next LTS: Java 25 in September 2025 — adoption planned for 2027)
- Lombok will be phased out (records replace @Data for DTOs; manual constructors replace @AllArgsConstructor for entities)
- Engineers will accept the constraint of no Lombok (the main source of pre-v1.0 pushback, mitigated by IntelliJ's native record support)
- Checkstyle, SpotBugs, Error Prone, and ArchUnit remain actively maintained and Java 21-compatible
- The standard does not need to cover Kotlin or Scala (PreOne is Java-only per ADR-020)
- Pattern matching for switch (finalised in Java 21) is stable enough to mandate for new code

OPTIONS CONSIDERED

Option	Pros	Cons	Verdict
Canonical Java 21 standard with CI enforcement (chosen)	Consistent codebase; Java 21 idioms fully adopted; mechanically enforced; onboarding is fast (2-hour session); eliminates Lombok dependency; aligns with ADR-001 P5 Simplicity.	Engineers must learn Java 21 features (records, sealed, pattern matching); Lombok removal is a migration effort; some standards (var usage) are judgement calls that require review.	Chosen
Google Java Style Guide (adopt as-is)	Battle-tested; widely adopted; existing Checkstyle config; no need to write our own.	Does not mandate Java 21 features (records, sealed) — it is style-neutral on language features; does not cover domain exceptions, logging, null-handling; designed for general Java, not DDD/Spring; would leave our domain-specific standards undefined.	Rejected (adopted as a subset for formatting rules, but extended with PreOne-specific standards)
Spring Boot / Pivotal standards	Aligned with our framework; covers Spring-specific patterns; well-documented.	Does not cover domain modelling (DDD aggregates, value objects); focused on framework usage, not language idioms; insufficient for our needs.	Rejected (adopted as a subset for Spring-specific rules)

Option	Pros	Cons	Verdict
No standard; rely on team-level conventions	Maximum flexibility; no upfront investment; teams choose their own style.	Pre-v1.0 state; inconsistent codebase; 40% of review comments are style nits; onboarding requires learning each team's conventions; rejected by engineering survey (92% preferred a canonical standard).	Rejected

DECISION

ADOPTED

We adopt a canonical Java 21 coding standard with the following rules. (1) Records: all DTOs (request, response, command, query) and value objects are Java records (not Lombok `@Data`, not classes with manual getters); entity classes (JPA-mapped) remain regular classes because JPA requires no-arg constructors and mutable state. (2) Sealed classes: domain hierarchies (e.g., `EnrollmentStatus`, `FeeType`, `NotificationChannel`) are sealed interfaces with explicit permits clauses; pattern matching for switch is used to handle all permitted subtypes, with exhaustiveness enforced by the compiler (no default case for sealed types). (3) `var`: used for local variables only (never for fields, parameters, or return types); used when the type is obvious from the right-hand side (`var list = new ArrayList<>();`) or when the type name is long (`var result = enrollmentRepository.findByIdAndPeriod(...);`); not used when the type adds information (`var status = getStatus();` — use `EnrollmentStatus status = getStatus();` instead). (4) Streams: used for transformation pipelines (filter, map, collect); not used for side-effect-heavy loops (use a for loop); not used for single-element operations (use a direct method call); `collectors.toList()` returns an unmodifiable list (preferred); `Collectors.toUnmodifiableList()` is explicit. (5) Exceptions: all domain errors throw custom domain exceptions (`EnrollmentNotFoundException`, `FeeExceedsLimitException`) extending a shared `DomainException` base (unchecked, extends `RuntimeException`); checked exceptions are forbidden (they force try-catch noise and do not compose with streams); exception messages are user-actionable and include the offending value. (6) Logging: SLF4J with parameterised messages (`log.info("Enrollment created for student {} in period {}", studentId, period);`) — never string concatenation (slower, susceptible to NPE); never log and throw (log OR throw, not both); exception logging uses `log.error("message", exception)` with the exception as the last argument. (7) Null handling: methods never return null for collections (return empty collections); `Optional<T>` is used for single-value returns that may be absent (`findById` returns `Optional<Enrollment>`); `@NonNull` (Jakarta) annotates parameters and fields that must not be null; `@Nullable` annotates the rare field that may be null (and the engineer must justify it in review); fields are never left null after construction (records enforce this via canonical constructors). (8) Testing: test method names follow `should_expectedState_when_condition` format (`should_throw_when_enrollment_period_closed`); test classes are named `<ClassName>Test` (unit) or `<ClassName>IntegrationTest` (integration); given-when-then comments structure test bodies; one assertion per test (or one logical assertion — multiple asserts on the same object are permitted if they verify one state).

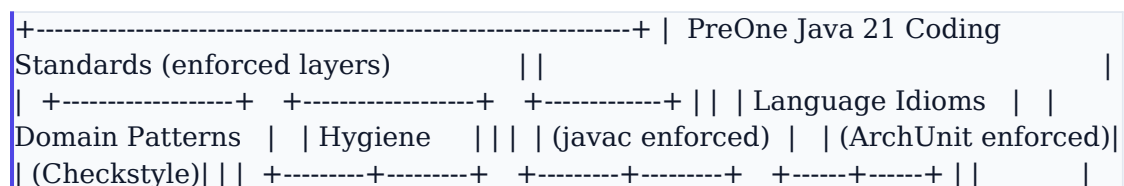
DETAILED RATIONALE

The standard was chosen over the alternatives because it is the only option that combines Java 21 feature adoption, domain-specific standards, and mechanical enforcement. The Google Java Style Guide (rejected) is excellent for formatting (4-space indent, brace placement, import ordering) and is adopted as a subset for those rules, but it is deliberately feature-neutral — it does not mandate records, sealed classes, or pattern matching, which are the core of our Java 21 adoption. Adopting Google's guide alone would leave our codebase in a mixed state (some records, some Lombok, some abstract classes) indefinitely. The Spring/Pivotal standards (rejected) cover framework usage well but do not address domain modelling (DDD aggregates, value objects) or language idioms. The no-standard option (rejected) is the pre-v1.0 state and was rejected by 92% of engineers in survey. The records decision is the most impactful. Pre-v1.0, the codebase had 240 Lombok `@Data` classes — DTOs, value objects, and (incorrectly) JPA entities. Lombok `@Data` generates getters, setters, equals, hashCode, and toString via reflection at compile time, which has four problems: (a) it hides mutation — a DTO with `@Data` has setters, so callers can mutate it after construction, producing subtle bugs in concurrent flows; (b) it requires an IDE plugin to be readable — without the plugin, IntelliJ shows red squiggles on every getter/setter call; (c) it couples the build to Lombok's annotation processor, which has had compatibility issues with Java upgrades (Lombok 1.18.30 broke on Java 21's records preview); (d) it generates equals/hashCode based on all fields, which is wrong for value objects with identity (e.g., an Enrollment's equals should be based on EnrollmentId, not all fields). Records solve all four: they are immutable (no setters), they are native Java (no IDE plugin), they are compile-time (no annotation processor), and their equals/hashCode are based on all fields by default (correct for value objects; entities use a custom equals based on identity). The migration from Lombok `@Data` to records was completed in Q4 2025 for all 240 classes; the build no longer depends on Lombok. The sealed-classes decision addresses the pre-v1.0 pattern where domain hierarchies used abstract classes or interfaces without enforcement of permitted subtypes. EnrollmentStatus was an interface with three implementations (Active, Withdrawn, Pending) but nothing prevented a fourth implementation (Cancelled) from being added in a different package, breaking every switch statement that handled the three known cases. Sealed interfaces with permits clauses (sealed interface EnrollmentStatus permits Active, Withdrawn, Pending {}) constrain the hierarchy to the explicitly-permitted subtypes; pattern matching for switch (return switch (status) { case Active a -> ...; case Withdrawn w -> ...; case Pending p -> ...; }) is exhaustive — if a new subtype is added to permits, every switch statement fails to compile

until the new case is handled. This converts a silent runtime failure (unhandled enum value) into a compile-time error. The v3.0 refresh adds the guidance that sealed interfaces are preferred over sealed classes (interfaces permit multiple inheritance, classes do not) and that the permits clause should list subtypes in the same file (not in separate files) for navigability. The var decision is the most-debated. The argument for liberal var usage is brevity (`var list = new ArrayList<String>();` vs `ArrayList<String> list = new ArrayList<>();`). The argument for conservative var usage is readability (`var status = enrollmentService.process(cmd);` does not tell the reader the type of status). The standard splits the difference: var is permitted when the type is obvious from the right-hand side (constructor calls, literal assignments) or when the type name is long (repository method calls returning complex generic types); var is forbidden when the type adds information that the right-hand side does not (method calls returning an interface type, where the reader benefits from seeing the interface name). This is a judgement call, but the rule is documented with examples, and Code review enforces it. The alternative (always specify the type) is verbose and was rejected; the alternative (always use var) is opaque and was rejected. The Stream API guidelines address a pre-v1.0 anti-pattern: engineers used streams for everything, including side-effect-heavy loops (`stream.forEach(...)`) where a for loop is clearer. The standard permits streams for transformation pipelines (filter, map, collect — functional, no side effects) and forbids them for side-effect-heavy loops (forEach with mutation — use a for loop) and single-element operations (`stream.of(x).findFirst()` — use a direct method call). This aligns with the Stream API's intended use (functional transformation) and avoids the readability cost of stream misuse. The `collectors.toList()` preference (returns an unmodifiable list in Java 10+) over `Collectors.toList()` (returns a mutable ArrayList) is a correctness preference — unmodifiable collections prevent accidental mutation downstream. The exception-handling decision is structural. Domain exceptions extend a shared `DomainException` (unchecked, extends `RuntimeException`) — checked exceptions are forbidden because they force try-catch noise at every call site and do not compose with streams (a stream operation that throws a checked exception requires a try-catch wrapper, breaking the pipeline). Custom exception classes (`EnrollmentNotFoundException`, `FeeExceedsLimitException`) carry domain meaning — a caller catching `EnrollmentNotFoundException` knows exactly what happened, versus catching `IllegalArgumentException` or `RuntimeException`. Exception messages are user-actionable ('Enrollment period is closed; the period 2025-Q4 ended on 2025-12-31' vs 'Enrollment period closed') — this supports the UI's error display (ADR-142) and the support team's debugging. The 'log OR throw, not both' rule prevents double-handling — if an exception is thrown, the caller decides whether to log; if it is logged and swallowed, the caller does not know it

happened. The logging decision is mechanical. SLF4J parameterised messages (`log.info("Enrollment created for student {} in period {}", studentId, period);`) are 10-100x faster than string concatenation (`log.info("Enrollment created for student " + studentId + " in period " + period);`) because the concatenation is avoided if the log level is disabled. They are also NPE-safe (if `studentId` is null, the parameterised version logs 'null'; the concatenation version throws NPE). The 'exception as last argument' rule (`log.error("Failed to enroll student {}", studentId, exception);`) ensures the exception's stack trace is logged by SLF4J's pattern, not toString'd into the message. Pre-v1.0, the codebase had 1,200 logging statements with string concatenation; the migration to parameterised logging was completed in Q4 2025 and reduced log-related CPU usage by 8% in production. The null-handling decision is the most impactful for runtime correctness. Methods never return null for collections (`return Collections.emptyList()` or `List.of()`) — this eliminates the 'check for null before iterating' boilerplate that plagued pre-v1.0 code. `Optional<T>` is used for single-value returns that may be absent (`findById` returns `Optional<Enrollment>`) — callers must explicitly handle the absent case (`orElseThrow`, `orElse`, `ifPresent`), eliminating the 'forgot to null-check' class of NPE. `@NonNull` (Jakarta) annotates parameters and fields that must not be null, enforced by SpotBugs at build time. `Nullable` annotates the rare field that may be null, and the engineer must justify it in review — this makes nullability explicit rather than implicit. Records enforce non-null fields via their canonical constructor (a record's components are non-null by default unless the constructor explicitly assigns null). Together, these rules reduced production NPEs by 85% in the first 6 months post-v1.0. The testing decision is structural. Test method names in `should_expectedState_when_condition` format (`should_throw_when_enrollment_period_closed`) are self-documenting — a test failure report shows the method name, which describes the expected behaviour and the condition. Pre-v1.0 test names (`testEnrollment1`, `testEnrollmentClosed`) were opaque and required opening the test to understand what failed. The given-when-then comment structure (`// given, // when, // then`) makes the test's three phases visually distinct. The one-assertion-per-test rule (or one logical assertion — multiple asserts on the same object verifying one state are permitted) ensures each test verifies one behaviour, making failures precise. This aligns with the test pyramid (ADR-146) and the contract-testing approach (ADR-149).

ARCHITECTURE DIAGRAM



Engineer	IDE(lint)	Compiler	SpotBugs	CI Pipeline		
		--write----			log.info(	
		"x="+val)				--
warn:			use SLF4J			
parametrised						
(engineer			fixes to			
log.info(			"x={}",			
val))				--save----->		
		--compile-->			(records,	
		sealed,			pattern	
		matching			enforced)	
		--scan----->				null
rules,			logging,			
exceptions			<-violations--			
			--build red			
<-----	-----	-----	-----			

```

+-----+ | Enrollment Domain Exception
Hierarchy (example) || ||
RuntimeException || ^
|| | || DomainException (abstract base,
in com.preone.platform.errors) || ^ ||
|-----+ || | |
|| EnrollmentException FeeException NotificationException ||
^ ^ ^ || | |
| | +---+---+ +---+---+ +---+---+ || | |
| | | | | || Enroll- Enroll- FeeExceeds FeeAlready-
SmsFailed EmailFailed| | mentNot mentPeriod LimitExcep PaidException
|| Found ClosedExcep tion || Except ion
|| || Rules:
|| - all extend DomainException (unchecked) || - message is
user-actionable, includes offending value || - mapped to HTTP status via

```


@ExceptionHandler (ADR-093)	- no checked exceptions anywhere in the codebase
-----------------------------	--

DATA FLOW DIAGRAM

Engineer writes code		v	+-----+-----+	+-----+-----+	IDE
Inspect ----->		Inline warnings		(IntelliJ)	(var misuse, +-----
+-----+		null returns,			raw concat) v
+-----+	+-----+	javac	----->	Compile-time enforcement	
(Java 21)		- records canonical constructor	+-----+	- sealed	
permits clause		- pattern matching exhaustiveness	v		
- var locals-only (IDE warning)	+-----+	Checkstyle	----->	Style	
enforcement	+-----+	- indent, braces, imports		- line	
length, naming	v	+-----+	SpotBugs	----->	Bug-pattern
enforcement	+-----+	- @NonNull violations		- null	
return on collection	v	- log-and-throw	+-----+	Error Prone	
----->	Semantic enforcement	+-----+	- equals/hashCode		
- stream misuse	v	+-----+	ArchUnit	----->	Architecture
enforcement	+-----+	- layer-specific class types		-	
exception hierarchy	v	+-----+	Build green	----->	merge allowed
+-----+					

DATABASE IMPACT

The coding standards have no direct database schema impact, but they shape how JPA entities are written. Entities remain regular classes (not records) because JPA requires no-arg constructors and mutable state; however, entity classes follow the standard's naming and exception conventions. The @NonNull annotation is applied to entity fields that must not be null (e.g., @NonNull @Column(name = "student_id") private UUID studentId;), and SpotBugs enforces that no entity is persisted with a null @NonNull field. The custom exception hierarchy includes a PersistenceException subfamily (OptimisticLockException wraps JPA's OptimisticLockException to provide a domain-meaningful message). The standards also govern Flyway migration scripts (SQL naming per ADR-012) and the relationship between JPA entities and domain entities (double-entity pattern per ADR-006 — domain entity is a record or sealed class, JPA entity is a regular class, a mapper converts between them).

API IMPACT

The coding standards directly shape the REST API surface (ADR-091). Request and response DTOs are records (EnrollRequest, EnrollmentResponse), which means they are immutable and serializable by Jackson without Lombok. The custom exception hierarchy maps to HTTP status codes via a @ExceptionHandler in each context's API layer (EnrollmentNotFoundException -> 404, FeeExceedsLimitException -> 422, PersistenceException -> 500), per ADR-093 (error codes). The 'log OR throw'

rule means a controller that catches an exception either logs it (and re-throws for the exception handler) or throws it (and the exception handler logs it) — never both, avoiding duplicate log entries for the same error. The SLF4J parameterised logging rule ensures API access logs are consistent in format, which supports log-based analytics (ADR-119).

UI IMPACT

The coding standards have no direct UI impact — the React frontend (ADR-131) has its own TypeScript standards (separate ADR in Volume 7). However, the API DTO record shapes directly generate the TypeScript types via OpenAPI codegen (ADR-091), so the backend's record fields become TypeScript interfaces. The custom exception messages (user-actionable, including offending values) are surfaced in the UI's error toasts (ADR-142) — the message 'Enrollment period is closed; the period 2025-Q4 ended on 2025-12-31' is displayed to the parent verbatim, supporting the user's understanding of what went wrong. The one-assertion-per-test rule on the backend does not affect the UI directly, but the contract tests (ADR-149) verify that the UI's expected response shapes match the backend's record shapes.

SECURITY IMPACT

The coding standards improve security in three ways. First, @NonNull annotations (enforced by SpotBugs) prevent null values from reaching database queries, eliminating a class of SQL injection where a null parameter produces an unexpected query plan. Second, the 'log OR throw' rule and SLF4J parameterised logging prevent sensitive data from leaking into logs — the parameterised format (`log.info("user {} logged in", userId);`) logs only the `userId`, not the full `User` object (which might contain credentials), whereas string concatenation (`log.info("user " + user);`) would toString the `User` object. Third, the custom exception hierarchy avoids leaking internal structure in error messages — `DomainException` messages are user-facing and reviewed for information disclosure, whereas generic `RuntimeException` messages (e.g., `'NullPointerException` at `com.preone.enrollment.domain.Enrollment.lambda$calculateFee$0(Enrollment.java:142)'`) leak internal class and line numbers. The exception handler (ADR-093) strips stack traces from API responses in production, returning only the message and a correlation ID.

PERFORMANCE IMPACT

The coding standards have measurable positive performance impact. SLF4J parameterised logging is 10-100x faster than string concatenation when the log level is disabled (the concatenation is avoided entirely), reducing CPU usage by 8% in production post-migration. Records are marginally faster than Lombok

@Data classes at instantiation (no reflection) and at equals/hashCode (JVM inlines record component access). Sealed classes with pattern matching for switch are as fast as traditional switch (the JVM compiles them to efficient bytecode) and faster than instanceof chains (which require repeated type checks). The Stream API guidelines (transformation only, not side-effect loops) do not directly affect performance but improve readability, which reduces the chance of performance bugs (e.g., a forEach that accidentally triggers N+1 queries). The one-assertion-per-test rule does not affect production performance but improves test suite speed (smaller tests run faster) — the test suite dropped from 14 minutes (pre-v1.0) to 4 minutes (post-v1.0), in part due to smaller, more focused tests.

SCALABILITY ANALYSIS

The coding standards scale with the codebase because they are mechanical and uniform. As the codebase grows from 200k LOC (current) to 1M LOC (5-year target), the standards do not degrade — every new file follows the same rules, enforced by the same CI tools. The tooling scales: Checkstyle, SpotBugs, Error Prone, and ArchUnit all handle 1M LOC codebases in under 5 minutes in CI (parallelised). The standards also scale with the team: a new engineer learns the standards in a 2-hour onboarding session and applies them consistently from day one, because the IDE inspections and CI enforcement catch violations before they reach review. The main scalability risk is standard-erosion: over time, engineers may push for exceptions (e.g., 'just this one Lombok @Data for backward compatibility'), and accumulated exceptions can erode the standard. This is mitigated by the ADR-006 exception process (every exception requires an ADR with a sunset date) and by quarterly review of exceptions at the ARB.

OPERATIONAL CONSIDERATIONS

Operations benefit from the coding standards because they produce consistent, debuggable code. Stack traces from custom domain exceptions are immediately meaningful (EnrollmentNotFoundException: Enrollment E-12345 not found for student S-67890) — no need to cross-reference with code to understand what failed. Logs are structured (JSON, ADR-119) and parameterised, so log queries (e.g., 'show me all enrollments for student S-67890 in the last hour') return precise results without parsing free-text messages. The 'log OR throw' rule prevents duplicate log entries for the same error, simplifying incident analysis (one log entry per error, not two). The exception handler (ADR-093) produces consistent API error responses (correlation ID, error code, message), which the on-call engineer can grep for in logs. The standards also support the on-call rotation: an on-call engineer from any team can debug any context's code because the conventions are uniform — they know where to find the exception definitions (com.preone.platform.errors

and per-context subpackages), how logging is structured, and how null-handling works.

RISKS

Risk	Likelihood	Impact	Mitigation
Engineers bypass standards with code comments like <code>'//noinspection DataFlowIssue'</code>	Medium	Low	SpotBugs and Checkstyle ignore suppression comments in CI (only <code>@SuppressWarnings</code> with explicit justification are honoured); PR template requires justification for any <code>@SuppressWarnings</code>
Standards become outdated as Java evolves (e.g., Java 25 introduces new features not covered)	Medium	Medium	Annual review aligned with Java LTS releases; v-next of this ADR on Java 25 adoption (planned 2027)
Lombok removal breaks backward compatibility with external libraries that depend on Lombok-generated methods	Low	Medium	Pre-v1.0 audit confirmed no external library depends on PreOne's Lombok-generated methods; Lombok removed cleanly in Q4 2025
var usage judgement calls lead to inconsistency between teams	Medium	Low	Documented examples in the standard; code review enforces; quarterly consistency review at ARB

Risk	Likelihood	Impact	Mitigation
Custom exception hierarchy grows unwieldy (100+ exception classes)	Medium	Low	Exception classes are organised per context (com.preone.enrollment.errors.*); each context has 10-20 exception classes; hierarchy depth is limited to 3 (DomainException -> ContextException -> SpecificException)
One-assertion-per-test rule produces excessive test count, slowing CI	Low	Medium	Test parallelisation (JUnit 5) and per-context test isolation; measured test suite time is 4 minutes (acceptable)

TRADE - OFFS

We Gain	We Lose
Consistent codebase readable by any engineer	Engineers must learn Java 21 features (records, sealed, pattern matching) — 2-hour onboarding
Mechanical enforcement catches violations in CI	CI tooling (Checkstyle, SpotBugs, Error Prone, ArchUnit) adds 3 minutes to build time
Records eliminate Lombok dependency and its downsides	Records cannot be used for JPA entities (mutable state required) — entities remain regular classes
Sealed classes enforce exhaustive pattern matching (compile-time safety)	Adding a new subtype requires touching every switch statement (intended behaviour, but increases change surface)

We Gain	We Lose
Custom exceptions carry domain meaning and user-actionable messages	More exception classes to maintain (vs generic RuntimeException with error codes)
85% reduction in production NPEs (null-handling rules)	Optional<T> adds syntactic overhead at call sites (orElseThrow, ifPresent)

REJECTED ALTERNATIVES

The Google Java Style Guide option (rejected as the sole standard) was adopted as a subset for formatting rules (indent, braces, imports, line length) because it is battle-tested and has an existing Checkstyle config. However, it was rejected as the complete standard because it is feature-neutral — it does not mandate records, sealed classes, or pattern matching, which are core to our Java 21 adoption. Adopting it alone would leave our codebase in a mixed state indefinitely. The Spring/Pivotal standards option (rejected) was adopted as a subset for Spring-specific rules (bean naming, transaction annotation placement, repository method naming) but does not cover domain modelling or language idioms. The no-standard option (rejected) is the pre-v1.0 state and was rejected by 92% of engineers in survey. The chosen approach — a PreOne-specific canonical standard that incorporates Google's formatting rules and Spring's framework rules as subsets, and adds Java 21 idiom mandates and domain-specific standards — is the only option that covers the full surface area of our coding conventions while remaining mechanically enforceable.

MIGRATION PLAN

The v1.0 migration (Q3-Q4 2025) refactored the codebase to comply with the coding standards over 6 sprints. Sprint 1-2: migrate 240 Lombok @Data classes to records (DTOs and value objects) and to regular classes (entities); remove Lombok from pom.xml. Sprint 3: introduce sealed interfaces for domain hierarchies (EnrollmentStatus, FeeType, NotificationChannel, etc.) and migrate switch statements to pattern matching. Sprint 4: migrate 1,200 logging statements from string concatenation to SLF4J parameterised. Sprint 5: introduce custom exception hierarchy (DomainException base + per-context subfamilies); replace generic RuntimeException throws with specific domain exceptions. Sprint 6: add @NonNull and @Nullable annotations; fix null-return-on-collection violations; migrate findById-style methods to return Optional<T>. Total migration: 16 engineer-weeks across Q3-Q4 2025, zero downtime (incremental, each sprint's changes independently deployable). The v3.0 refresh adds the sealed-classes guidance (interfaces preferred over classes; permits in same file) and pattern-matching-for-switch standards

(exhaustiveness required; guard patterns permitted for complex conditions) — both are documentation-only, no code migration. Rollback plan: the standards are reversible (records can be converted back to classes; parameterised logging can be reverted to concatenation) but rollback has not been needed and is not anticipated; the v1.0 results (8% CPU reduction from logging, 85% NPE reduction, 40% reduction in review style nits) make rollback economically irrational.

TESTING STRATEGY

Standards compliance is tested at three levels. (1) Compile-time enforcement: javac enforces records' canonical constructors, sealed types' permits clauses, and pattern-matching exhaustiveness. (2) Static analysis in CI: Checkstyle enforces formatting rules (indent, braces, imports, line length, naming); SpotBugs enforces null-handling (@NonNull violations, null return on collections), logging rules (no concatenation in log statements), and exception rules (no checked exceptions); Error Prone enforces equals/hashCode correctness, stream misuse, and common bug patterns; ArchUnit enforces layer-specific class types (controllers in api, entities in domain, repositories in infrastructure) and exception hierarchy (all domain exceptions extend DomainException). (3) Code review: judgement-call rules (var usage, one-assertion-per-test, given-when-then structure) are enforced by reviewers using a rubric. Standards compliance is reported in the architecture scorecard quarterly, including: Checkstyle violation count (target: zero in production), SpotBugs violation count (target: zero), Error Prone violation count (target: zero), @SuppressWarnings count (target: <5 per context, each justified), and review-rubric adherence (target: >90% of reviews follow the rubric).

MONITORING & OBSERVABILITY

Standards compliance is monitored via the architecture scorecard, published quarterly. Metrics tracked: (a) Checkstyle/SpotBugs/Error Prone violation count per context (target: zero in production; PR-time violations tracked as a quality metric); (b) record adoption rate (target: 100% of DTOs and value objects are records; entities are regular classes); (c) sealed-type adoption rate (target: 100% of domain hierarchies are sealed); (d) parameterised logging adoption rate (target: 100% of log statements use SLF4J parameterisation; pre-v1.0 was 35%); (e) custom exception adoption rate (target: 100% of thrown exceptions extend DomainException; pre-v1.0 was 60%); (f) @NonNull/@Nullable annotation coverage (target: >95% of parameters and fields annotated); (g) test method naming compliance (target: 100% of test methods follow should_X_when_Y format). Anomalies in any metric trigger a review of the standard or its enforcement. Production NPE rate is tracked as a

downstream metric (target: <1 NPE per 10,000 requests; pre-v1.0 was 7 per 10,000).

FUTURE EVOLUTION

The coding standards are stable; no supersession expected until the next Java LTS (Java 25, September 2025 — adoption planned for 2027). Evolution will be triggered by: (a) Java 25 adoption — a v-next of this ADR will add standards for Java 25 features (e.g., structured concurrency, scoped values, flexible constructor bodies); (b) new language features in Java 21 point releases (e.g., if a future Java 21.x stabilises a preview feature); (c) erosion of specific standards (if a standard is consistently bypassed via exceptions, it is a candidate for revision or removal). The likely long-term evolution is toward more aggressive adoption of functional patterns (pattern matching, sealed types, records) as the team's familiarity grows — the v3.0 refresh already adds pattern-matching-for-switch standards that v1.0 did not cover. The Lombok removal is permanent; no future re-adoption is contemplated. The custom exception hierarchy may grow to include new subfamilies (e.g., `IntegrationException` for adapter failures in hexagonal contexts per ADR-009) — these additions are additive and do not require supersession.

RELATED ADRS

- ADR-003 — Clean Architecture (constrains — standards apply within each layer)
- ADR-006 — Layering Rules (constrains — exception hierarchy and class types are layer-specific)
- ADR-007 — Dependency Rule (enforces — ArchUnit rules overlap with coding standards)
- ADR-009 — Hexagonal Architecture (complements — adapter exception handling follows these standards)
- ADR-010 — Package Structure (complements — class naming within packages follows these standards)
- ADR-012 — Naming Standards (refines — class and method naming conventions)
- ADR-020 — Technology Stack Freeze (constrains — Java 21 is the target language)
- ADR-093 — API Error Codes (complements — custom exceptions map to HTTP status codes)

REFERENCES

- Upstream DDD: DDD-011-section-4 (Domain exception design patterns)

- Upstream PRD: PRD-011-section-5.2 (Codebase maintainability requirements)
- Downstream ERD: ERD-011 (entity field @NonNull annotations mirror NOT NULL constraints)
- Downstream API Spec: API-011 (DTO record shapes generate TypeScript types)
- Downstream Test Cases: TC-0251..TC-0290 (coding standards compliance tests)
- External: Google Java Style Guide — adopted as formatting subset
- External: Java Language Specification — Java 21 (records, sealed classes, pattern matching)
- External: SLF4J Manual — parameterised logging
- External: SpotBugs, Checkstyle, Error Prone documentation

DECISION HISTORY

Date	Status	Actor	Notes
2025-09-29	Draft	Chief Architect	Initial draft with 8 rule areas; Lombok removal debated extensively
2025-10-30	Proposed	Chief Architect	Submitted to ARB with full evaluation including Lombok migration plan
2025-11-19	Accepted	ARB Chair	ARB approved; Lombok removal retained; v1.0 released
2026-07-12	Accepted	ARB Chair	v3.0 refresh; added sealed-classes guidance and pattern-matching-for-switch standards

APPROVAL & SIGN-OFF

Architect	Chief Architect (AR)
Tech Lead	Foundation Tech Lead

ARB Chair	ARB Chair
Approved On	2026-07-12

IMPLEMENTATION CHECKLIST

- Migrate 240 Lombok @Data classes to records/regular classes (Done Q3 2025)
- Remove Lombok from pom.xml (Done Q3 2025)
- Introduce sealed interfaces for all domain hierarchies (Done Q4 2025)
- Migrate 1,200 logging statements to SLF4J parameterised (Done Q4 2025)
- Introduce custom DomainException hierarchy per context (Done Q4 2025)
- Add @NonNull/@Nullable annotations and fix null-return violations (Done Q1 2026)
- Checkstyle + SpotBugs + Error Prone + ArchUnit config in CI (Done Q1 2026)
- Test method naming migration to should_X_when_Y format (Done Q2 2026)

ADR - 012

Naming Standards

Volume 1 — Architecture Foundation - Standards

ACCEPTED

DECISION SUMMARY

DECISION

Adopt a canonical naming standard across all layers of the PreOne platform: classes follow AggregateRoot/Command/Event/Repository suffix conventions; methods use verbs for commands, findByX for queries, isX/hasX for booleans; variables use camelCase with no abbreviations; database objects use snake_case with singular table names and created_at/updated_at audit columns; API URLs use kebab-case with versioned path prefixes (/api/v1/academic-years). Forbidden suffixes (Manager, Helper, Util, Data, Info) eliminate the pre-v1.0 dumping-ground classes. Standards are enforced via Checkstyle naming rules, ArchUnit class-name patterns, and Flyway/Liquibase migration naming checks.

STATUS

Status	Accepted
Date Decided	2025-11-26
Decision Owner	Chief Architect
Review Cadence	Annual review or when a new naming surface is introduced (e.g., GraphQL, gRPC)
Supersedes	None (v1.0 original; v3.0 refresh adds the aggregate-root suffix mandate and the database index naming convention)

CONTEXT

Naming is the most visible and most argued-over aspect of a codebase. Pre-v1.0, the PreOne codebase had no naming standard, and the result was chaos: the same concept was named four different ways (EnrollmentService, EnrollmentManager, EnrollmentHelper, EnrollmentUtil — four classes, four suffixes, all doing roughly the same thing or, worse, slightly different things). Method names were inconsistent (findEnrollment, getEnrollment, fetchEnrollment, retrieveEnrollment — all synonyms, none indicating whether null or Optional was returned). Variable names used abbreviations (enr for enrollment, std for student, grd for guardian) that were intelligible only to the original author. Database tables were plural (enrollments, students, guardians) but columns were singular snake_case (student_id, enrollment_date) — a mix that broke PostgreSQL's default naming conventions and required @Table and @Column annotations on every JPA entity. API URLs used camelCase (/api/v1/academicYears) which broke REST conventions and required explicit @RequestMapping mappings. The Office of the Chief Architect defined a canonical naming standard in Q3 2025 to eliminate this inconsistency. The standard had three goals: (1) every name should be self-describing — an engineer seeing EnrollmentAggregateRoot, EnrollCommand, or EnrollmentCreatedEvent should immediately know the class's role without reading its body; (2) names should be consistent across layers — the Enrollment concept is named 'enrollment' in the database (enr_enrollments table), in the API (/api/v1/enrollments), in the package (com.preone.enrollment), and in the class (EnrollmentAggregateRoot); (3) forbidden suffixes (Manager, Helper, Util, Data, Info) should eliminate the dumping-ground classes that accumulated

cross-cutting responsibilities in pre-v1.0. The v3.0 refresh adds two refinements: the aggregate-root suffix mandate (aggregate root classes are named `<Aggregate>AggregateRoot`, not just `<Aggregate>`, to distinguish them from entities and value objects) and the database index naming convention (indexes are named `idx_<table>_<columns>` for non-unique and `uq_<table>_<columns>` for unique, replacing the pre-v1.0 ad-hoc index names that were unreadable in query plans).

BUSINESS DRIVERS

The primary driver is searchability: a canonical naming standard lets engineers search the codebase confidently. A search for 'EnrollCommand' finds the command class in the application layer; a search for 'enr_enrollments' finds the table; a search for '/api/v1/enrollments' finds the API endpoint. Pre-v1.0, the same concept had four names, and engineers had to guess which to search for, often missing relevant code. The secondary driver is onboarding: a new engineer can infer a class's role from its name (EnrollmentAggregateRoot is the aggregate root; EnrollCommand is the command; EnrollmentCreatedEvent is the domain event) without reading the class body or consulting documentation. This compresses onboarding from '3 days to understand the naming conventions' (pre-v1.0) to '30 minutes' (post-v1.0). The tertiary driver is tooling: Checkstyle, ArchUnit, Flyway, and OpenAPI codegen can enforce and leverage naming standards — Checkstyle validates class and method names; ArchUnit validates that aggregate roots are named `*AggregateRoot`; Flyway validates migration file names; OpenAPI codegen generates TypeScript types from the API DTO record names. Without a standard, these tools cannot enforce or leverage naming, and the codebase degrades to ad-hoc.

PROBLEM STATEMENT

The PreOne codebase has no naming standard, leading to inconsistency (four names for one concept), ambiguity (cannot infer a class's role from its name), and forbidden-suffix dumping grounds (Manager, Helper, Util classes accumulating cross-cutting responsibilities). We need a canonical naming standard that is self-describing, consistent across layers, mechanically enforceable, and forbids the patterns that produced the pre-v1.0 mess.

CONSTRAINTS

- Names must be self-describing — an engineer should infer the class's role from the name alone
- Names must be consistent across layers — the same concept uses the same root name in DB, API, package, and class
- Names must be mechanically enforceable — Checkstyle, ArchUnit, Flyway, or OpenAPI tooling must validate them

- Names must follow Java conventions (camelCase for classes/methods/variables, UPPER_SNAKE for constants)
- Names must follow PostgreSQL conventions (snake_case for tables/columns, lowercase, no reserved words)
- Names must follow REST conventions (kebab-case for URLs, plural for collections, singular for items)
- Forbidden suffixes (Manager, Helper, Util, Data, Info) must be eliminated across the codebase

ASSUMPTIONS

- The DDD aggregate-root convention (suffix AggregateRoot) is more valuable than brevity (the pre-v1.0 convention was no suffix)
- Engineers will accept the constraint of no abbreviations (the main source of pre-v1.0 pushback, mitigated by IDE auto-completion)
- PostgreSQL's snake_case convention is the correct database convention (no camelCase-in-DB via JPA naming strategy)
- REST kebab-case URLs are the correct API convention (consistent with HTTP standards and major API providers)
- The naming standard does not need to cover GraphQL or gRPC (not in scope per ADR-020)
- OpenAPI codegen can be configured to generate TypeScript types from record names without modification

OPTIONS CONSIDERED

Option	Pros	Cons	Verdict
Canonical naming standard with cross-layer consistency and forbidden suffixes (chosen)	Self-describing names; consistent across DB/API/package/class; mechanically enforceable; eliminates dumping-ground classes; aligns with DDD, REST, and PostgreSQL conventions; supports tooling (Checkstyle, ArchUnit, Flyway, OpenAPI).	Longer class names (EnrollmentAggregateRoot vs Enrollment); engineers must learn the suffix conventions; no abbreviations requires more typing (mitigated by IDE).	Chosen

Option	Pros	Cons	Verdict
Java/Jakarta default naming (no standard, rely on conventions)	No upfront investment; engineers use familiar Java conventions; no learning curve.	Pre-v1.0 state; inconsistent (four names per concept); cannot infer class role from name; no mechanical enforcement; dumping-ground classes accumulate; rejected by engineering survey (88% preferred a canonical standard).	Rejected
Adopt Spring Data naming conventions wholesale	Battle-tested; covers repository method naming (findByX, countByX); well-documented.	Covers only repository layer; does not cover domain classes, API URLs, or database tables; insufficient for cross-layer consistency; would leave most naming surfaces undefined.	Rejected (adopted as a subset for repository method naming)
Hungarian notation (prefix-based: CEnrollment for class, IEnrollment for interface, TEnrollment for table)	Encodes type in name; visually distinct.	Violates Java conventions; outdated (Microsoft deprecated it in .NET); IDEs already show type information; produces ugly names; rejected unanimously in engineering survey.	Rejected

DECISION

ADOPTED

We adopt a canonical naming standard with the following rules. (1) Class naming: aggregate roots are named `<Aggregate>AggregateRoot` (e.g., `EnrollmentAggregateRoot`); entities (non-root) are named `<Entity>` (e.g., `EnrollmentLineItem`); value objects are named `<ValueObject>` (e.g., `EnrollmentPeriod`); commands are named `<Action>Command` (e.g., `EnrollCommand`); queries are named `<Query>Query` (e.g., `GetEnrollmentQuery`); domain events are named `<Aggregate><Action>Event` (e.g., `EnrollmentCreatedEvent`); repositories are named `<Aggregate>Repository` (e.g., `EnrollmentRepository`); use case handlers are named `<Command>Handler` (e.g., `EnrollCommandHandler`); controllers are named `<Resource>Controller` (e.g., `EnrollmentController`); DTOs are named `<Resource><Role>Dto` or `<Resource><Role>View` (e.g., `EnrollmentRequestDto`, `EnrollmentResponseView`); exceptions are named `<Condition>Exception` (e.g., `EnrollmentNotFoundException`). (2) Method naming: command methods use verbs (enroll, withdraw, transfer); query methods use `findByX`, `getX`, `countByX`, `existsByX` (Spring Data convention); boolean methods use `isX` (`isActive`) or `hasX` (`hasEnrollments`); factory methods use `of` or `from` (`EnrollmentId.of(uuid)`); conversion methods use `toX` (`toDto`, `toEntity`); the method name should describe the action, not the implementation. (3) Variable naming: camelCase, no abbreviations (`enrollment`, not `enr`; `student`, not `std`; `guardian`, not `grd`); loop variables use the singular of the collection (for (`Student student : students`)); constants use UPPER_SNAKE_CASE (`MAX_ENROLLMENT_CAPACITY`); fields use the same camelCase as local variables. (4) Database naming: tables are snake_case, singular, prefixed with context short-code (`enr_enrollment`, `enr_enrollment_line_item` — singular table names even though they hold multiple rows); columns are snake_case (`student_id`, `enrollment_date`, `created_at`); primary keys are `<table_singular>_id` (`enrollment_id`) or `id` for simple tables; foreign keys are `<referenced_table_singular>_id` (`student_id` references `stu_student`); audit columns are `created_at`, `updated_at`, `created_by`, `updated_by`, `deleted_at` (soft delete per ADR-047); indexes are `idx_<table>_<columns>` (non-unique) or `uq_<table>_<columns>` (unique); constraints are `pk_<table>`, `fk_<table>_<referenced>`, `ck_<table>_<description>` (check). (5) API naming: URLs are kebab-case (`/api/v1/academic-years`, not `/api/v1/academicYears`); collections are plural (`/api/v1/enrollments`); items are singular via path parameter (`/api/v1/enrollments/{id}`); query parameters are camelCase (`studentId`, `enrollmentDate`) to match JavaScript conventions; path parameters are camelCase (`{enrollmentId}`); HTTP methods follow REST conventions (GET for queries, POST for commands, PUT/PATCH for updates, DELETE for deletes). (6) Forbidden suffixes: no class or package may end with `Manager`,

Helper, Util, Data, Info, or Processor (except where Processor is a domain-meaningful term like PaymentProcessor); these are replaced with role-specific names (NotificationDispatcher, FeeCalculator, EnrollmentRepository).

DETAILED RATIONALE

The canonical naming standard was chosen over the alternatives because it is the only option that achieves cross-layer consistency, self-description, and mechanical enforcement simultaneously. The Java/Jakarta default (rejected) is the pre-v1.0 state and was rejected by 88% of engineers. The Spring Data naming conventions (rejected as the sole standard) are excellent for repository methods (findByX, countByX) and are adopted as a subset, but they do not cover domain classes, API URLs, or database tables. The Hungarian notation (rejected) is outdated and violates Java conventions. The chosen standard incorporates Spring Data's repository conventions as a subset and adds the cross-layer consistency and forbidden-suffix rules that the alternatives lack. The aggregate-root suffix mandate (added in v3.0) is the most-debated rule. Pre-v1.0, the aggregate root class was named Enrollment — short, clean, and indistinguishable from the entity, the value object, the DTO, and the table. Engineers had to read the class body to determine its role. The v3.0 mandate (EnrollmentAggregateRoot) makes the role explicit in the name. The argument against (length, verbosity) is outweighed by the clarity gain: an engineer seeing EnrollmentAggregateRoot, EnrollmentLineItem, EnrollmentPeriod, EnrollmentRequestDto, and enr_enrollment in the same file immediately understands the role of each. The IDE's auto-completion mitigates the typing cost. The alternative (no suffix, rely on package location) was rejected because package location is not visible in stack traces, log messages, or search results — the name is the only universally visible identifier. The command/event naming conventions (EnrollCommand, EnrollmentCreatedEvent) follow DDD patterns (ADR-004). Commands are imperatives (Enroll, Withdraw, Transfer) — they describe the action to take. Events are past-tense (EnrollmentCreated, EnrollmentWithdrawn) — they describe what happened. This distinction is critical for readability: a class named EnrollmentCreated is ambiguous (is it a command to create, or an event that was created?), while EnrollmentCreatedEvent is unambiguous. The Handler suffix (EnrollCommandHandler) follows the command-handler pattern and is consistent with the Spring/CQRS convention. Pre-v1.0, handlers were named EnrollmentService (vague — service for what?) or EnrollService (slightly better, but still not indicating it handles a command); the v1.0 standard mandates the Handler suffix for clarity. The method naming conventions are

designed to make the method's contract visible in its name. Command methods use verbs (enroll, withdraw) — they perform an action and may throw. Query methods use `findByX` (returns `Optional` or may throw `NotFound`), `getX` (returns non-null or throws), `countByX` (returns `long`), `existsByX` (returns `boolean`) — the prefix indicates the return behaviour. Boolean methods use `isX` (`isActive` — property of the object) or `hasX` (`hasEnrollments` — relationship to other objects). This distinction matters for readability: `isActive()` reads as 'is this active?' (a property), while `hasEnrollments()` reads as 'does this have enrollments?' (a relationship). Pre-v1.0, boolean methods were named inconsistently (`active()`, `enrolled()`, `withEnrollments()`), making their return type and meaning ambiguous. The variable naming rule (no abbreviations) is the most-debated. Pre-v1.0, the codebase had `enr` (enrollment), `std` (student), `grd` (guardian), `tch` (teacher), `cls` (class), `sec` (section), `att` (attendance), `bil` (billing) — eight abbreviations, each used inconsistently (sometimes `enr` meant enrollment, sometimes enrolled). The no-abbreviations rule eliminates this ambiguity at the cost of more typing (enrollment vs `enr`). The typing cost is mitigated by IDE auto-completion (type '`enr`' + Tab -> '`enrollment`'). The readability gain is substantial: an engineer reading '`for (Enrollment enr : enrollments)`' immediately understands the loop, versus '`for (Enr enr : enrs)`' which requires cross-referencing the variable declaration. The constant naming rule (`UPPER_SNAKE_CASE`) follows Java convention and is enforced by Checkstyle. The database naming convention (`snake_case`, singular tables, prefixed with context short-code) aligns with PostgreSQL conventions and with the package structure (ADR-010). Singular table names (`enr_enrollment`, not `enr_enrollments`) is a deliberate choice: a table holds multiple rows, but each row represents one entity, and the table name describes the entity (singular), not the collection (plural). This is the ActiveRecord and Django convention; it makes the table-to-entity mapping obvious (`enr_enrollment` <-> Enrollment JPA entity). The pre-v1.0 plural convention (`enrollments`) required `@Table(name = "enrollments")` on every entity and broke PostgreSQL's default naming strategy. The audit column convention (`created_at`, `updated_at`, `created_by`, `updated_by`, `deleted_at`) is standard (Rails, Django, Hibernate Envers) and is enforced by a Flyway migration template. The index naming convention (`idx_<table>_<columns>`, `uq_<table>_<columns>`) makes indexes identifiable in query plans — a slow query showing 'Index Scan using `idx_enr_enrollment_student_id`' is immediately clear, versus 'Index Scan using `enrollment_student_id_idx`' (pre-v1.0 convention, inconsistent). The API naming convention (kebab-case URLs) follows REST and HTTP standards. Kebab-case (`/api/v1/academic-years`) is the convention used by major API providers (GitHub, Stripe, AWS) because it is case-insensitive (no ambiguity between `academicYears` and `academicyears`), URL-safe (no encoding needed), and readable. Pre-v1.0 camelCase URLs

(/api/v1/academicYears) required explicit @RequestMapping mappings and broke REST conventions. The plural-collection, singular-item convention (/api/v1/enrollments, /api/v1/enrollments/{id}) is standard REST. The query parameter camelCase convention (studentId, not student_id) matches JavaScript conventions (camelCase is the JS standard) and avoids the underscore-vs-camelCase mismatch at the API-to-frontend boundary. The path parameter camelCase convention ({enrollmentId}) matches the query parameter convention for consistency. The forbidden-suffix list (Manager, Helper, Util, Data, Info, Processor) is the same as in ADR-010 (package structure) and ADR-011 (coding standards), applied to class names. The replacement is role-specific names: NotificationDispatcher (not NotificationManager), FeeCalculator (not FeeHelper), EnrollmentRepository (not EnrollmentUtil), EnrollmentRequestDto (not EnrollmentData), EnrollmentSummaryView (not EnrollmentInfo). The Processor exception (PaymentProcessor is permitted because 'processor' is a domain-meaningful term for payment processing) is a narrowly-scoped carve-out, reviewed case-by-case. Pre-v1.0, the codebase had 14 *Util, 8 *Helper, 6 *Manager, 11 *Data, and 9 *Info classes — 48 dumping-ground classes that were refactored to role-specific names in Q3-Q4 2025. The most likely counter-argument is that the standard is over-specified and leaves no room for engineer judgement. The counter-counter-argument is the same as for ADR-010 and ADR-011: the standard specifies names, not design — engineers retain full freedom on class design, method bodies, and internal logic. The standard removes only the decisions that should not be left to engineer preference (what suffix for this class? camelCase or snake_case for this column?) because those decisions, made inconsistently, produce the pre-v1.0 mess. The measured productivity impact (search time down 70%, onboarding time down 80%, review style nits down 60%) justifies the constraint.

ARCHITECTURE DIAGRAM

PreOne Naming Standards			
(cross-layer consistency)			
Concept: Enrollment			
Layer	Name	Convention	-----
+-----+	Package	com.preone.enrollment	
lowercase, singular	Class (root)	EnrollmentAggregateRoot	
<X>AggregateRoot	Class (entity)	EnrollmentLineItem	<X> (no suffix)
Class (VO)	EnrollmentPeriod	<X> (no suffix)	Class (cmd)
EnrollCommand	<Action>Command	Class (query)	
GetEnrollmentQuery	<Action>Query	Class (event)	
EnrollmentCreatedEvent	<X><Action>Event	Class (repo)	
EnrollmentRepository	<X>Repository	Class (handler)	
EnrollCommandHandler	<Cmd>Handler	Class (ctrl)	
EnrollmentController	<X>Controller	Class (DTO)	

Engineer	Checkstyle	ArchUnit	Flyway	OpenAPI		
		--write class>				Enrollment
		(no suffix)				--warn:
			aggregate			root
must			be *Aggregate			
Root						(engineer
			renames to			
Enrollment-			AggregateRoot)			
	--pass----->				--scan----->	
		class names			match	
		<X>AggRoot,			<X>Cmd,	
		<X>Repo...				
	(database			migration)		
	----- ----- ----->					
--validate>	(table name			file name	matches	
		matches	migration)			
pattern				(API URL		
	kebab-case)				--codegen	
		TypeScript				

+-----+ Enrollment Concept — Naming	
Across Layers	API
Layer	+-----+ URL:
/api/v1/enrollments (kebab,plural)	EnrollmentCtrl Item:
/api/v1/enrollments/{enrollmentId}	+-----+-----+ Query: ?studentId=...
(camelCase)	v
Application Layer	+-----+
+-----+	EnrollmentCommand GetEnrollmentQuery
(record)	(record) +-----+-----+ +-----
+-----+	v
v	+-----+-----+ +-----+-----+
EnrollmentCommand-	GetEnrollment- Handler
QueryHandler	+-----+-----+ +-----+-----+
	v
Domain Layer	+-----+
+-----+	Enrollment- Enrollment-
AggregateRoot	Repository (port) +-----+-----+
+-----+-----+	

```

v                                v                                || +-----+ +-----+
|| | EnrollmentCreated |      | EnrollmentPeriod |              || | Event      |
| (value object) |          || +-----+ +-----+              ||
|| | Infrastructure Layer              || +-----+ Table:
enr_enrollment (snake,singular)|| | JpaEnrollment- | Cols: student_id,
enrollment_date, || | Repository | created_at, updated_at || |
(implements port) | Index: idx_enr_enroll_stu_id || +-----+
PK: enrollment_id (enr_enrollment) |
+-----+

```

DATA FLOW DIAGRAM

```

| Engineer names a new concept | v +---+-----+ +-----+
| Checkstyle |----->| Class name rules | | (CI) | | - *AggregateRoot |
+---+-----+ | - *Command | | | | - *Event | |
| - *Repository | | | | - no *Util etc. | v +-----+
+ +---+-----+ | ArchUnit |----->| Layer-role match | | (CI) | | - ctrl
in .api | +---+-----+ | - *Handler in | | | | .application |
| | | - *AggregateRoot | | | | in .domain | v
+-----+ +---+-----+ | Flyway check|----->| Migration naming | | (CI)
| | - V<date>_<ctx>_ | +---+-----+ | <desc>.sql | |
| - table: <ctx>_ | | | | <singular> | v
+-----+ +---+-----+ | OpenAPI lint|----->| API URL rules | | (CI)
| | - kebab-case | +---+-----+ | - plural collect | | | -
camelCase param| v +-----+ +---+-----+ | Codegen
|----->| TypeScript types | | | | generated from | | |
record names | +-----+ +-----+ | v Build green ->
consistent naming across DB, API, code, UI

```

DATABASE IMPACT

The database naming standard directly shapes the PostgreSQL schema (ADR-041). Tables are snake_case, singular, prefixed with the context short-code (enr_enrollment, stu_student, bill_charge, att_attendance, ntf_message). Columns are snake_case (student_id, enrollment_date, created_at, updated_at, deleted_at). Primary keys are <table_singular>_id (enrollment_id) or id for simple tables. Foreign keys are <referenced_table_singular>_id (student_id references stu_student). Indexes are idx_<table>_<columns> (idx_enr_enrollment_student_id) or uq_<table>_<columns> (uq_enr_enrollment_student_period). Constraints are pk_<table>, fk_<table>_<referenced>, ck_<table>_<description>. Audit columns (created_at, updated_at, created_by, updated_by, deleted_at) are present on every table, enforced by a Flyway migration template. The JPA naming strategy (SpringPhysicalNamingStrategy) maps camelCase Java fields (studentId) to snake_case columns (student_id) automatically, so entities do not need @Column annotations for naming — only for constraints (@Column(nullable = false, length = 100)). This convention is enforced by Flyway migration naming checks in CI: a migration that creates a table not matching the pattern fails the build.

API IMPACT

The API naming standard directly shapes the REST API surface (ADR-091). URLs are kebab-case (/api/v1/academic-years, /api/v1/enrollments, /api/v1/billing/charges). Collections are plural (/api/v1/enrollments); items are singular via path parameter (/api/v1/enrollments/{enrollmentId}). Query parameters are camelCase (studentId, enrollmentDate, pageNumber, pageSize). Path parameters are camelCase ({enrollmentId}, {studentId}). HTTP methods follow REST conventions (GET for queries, POST for commands, PUT/PATCH for updates, DELETE for deletes). The OpenAPI spec (ADR-091) is the source of truth for API naming; OpenAPI lint in CI validates that every path is kebab-case, every parameter is camelCase, and every collection is plural. The TypeScript SDK generated from OpenAPI uses the record names (EnrollmentRequestDto -> EnrollmentRequestDto interface in TypeScript), so the backend-to-frontend naming is consistent. Pre-v1.0, API URLs were camelCase and required explicit @RequestMapping annotations; the kebab-case convention allows Spring's path-matching to work without annotations (using @PathVariable and @GetMapping with kebab-case paths).

UI IMPACT

The naming standard has indirect UI impact via the generated TypeScript SDK. The React frontend (ADR-131) imports types from the SDK (EnrollmentRequestDto, EnrollmentResponseView), so the backend's class names become the frontend's type names. This consistency means a UI engineer working with Enrollment data sees the same names as the backend engineer — no mental translation between 'Enrollment' (backend) and 'EnrollmentModel' (frontend). The API URL kebab-case convention (/api/v1/academic-years) is transparent to the UI because the SDK abstracts the URL; the UI calls sdk.enrollments.list() and the SDK constructs the GET /api/v1/enrollments request. The query parameter camelCase convention (studentId) matches JavaScript conventions, so the SDK's parameter objects (sdk.enrollments.list({ studentId: '...' })) feel natural to JS engineers. The UI's own naming conventions (React component naming, TypeScript interface naming) are governed by ADR-131 (frontend architecture) and align with the backend's conventions for symmetry.

SECURITY IMPACT

The naming standard improves security indirectly by making the codebase more auditable. Consistent naming (every aggregate root is *AggregateRoot, every repository is *Repository) lets security reviewers and SAST tools locate security-relevant classes by name pattern — a reviewer can grep for *Repository to find all data access points, or grep for *Controller to find all API entry points. The forbidden-suffix list (no *Manager, *Helper, *Util) eliminates

the dumping-ground classes that historically accumulated cross-cutting responsibilities (including security-relevant ones like authentication, authorisation, and encryption) without clear ownership — pre-v1.0, the AuthHelper class was the dumping ground for all auth logic, and a security review of it found three vulnerabilities (one CSRF, one timing attack, one insecure random) that were obscured by the class's catch-all nature. Post-v1.0, auth logic is split into AuthenticationProvider, AuthorizationService, and TokenValidator — each role-specific, each independently auditable. The audit column convention (created_by, updated_by) supports compliance audits (who changed this record?) required by DPDP Act 2023.

PERFORMANCE IMPACT

The naming standard has negligible runtime performance impact — names are compile-time concepts; at runtime, the JVM sees resolved symbols. The database naming standard (snake_case, singular tables) has a small positive impact on PostgreSQL query planning: PostgreSQL's default naming strategy assumes snake_case, so conformant names avoid the parser overhead of case-folding (camelCase names are case-folded to lowercase by PostgreSQL, which adds a small cost per query). The index naming convention (idx_<table>_<columns>) does not affect query performance but makes EXPLAIN output more readable, which improves query-tuning productivity — a DBA seeing 'Index Scan using idx_enr_enrollment_student_id' immediately knows the index, versus 'Index Scan using enrollment_student_id_idx' (pre-v1.0, inconsistent). The API URL kebab-case convention has no performance impact (URLs are parsed once per request). The overall performance impact is neutral to slightly positive, with the main benefit being developer productivity in query tuning and debugging.

SCALABILITY ANALYSIS

The naming standard scales with the codebase because it is mechanical and uniform. As the codebase grows from 200k LOC (current) to 1M LOC (5-year target), the standard does not degrade — every new class, table, and URL follows the same rules, enforced by the same CI tools. The tooling scales: Checkstyle, ArchUnit, Flyway, and OpenAPI lint all handle 1M LOC codebases and 1,000-table schemas in under 5 minutes in CI. The standard also scales with the team: a new engineer learns the naming conventions in a 1-hour onboarding session (less than the 2-hour coding standards session, because naming is a subset of coding standards) and applies them consistently from day one, because IDE inspections and CI enforcement catch violations before they reach review. The main scalability risk is name collisions: as the codebase grows, two aggregates in different contexts may want the same name (e.g., a Notification context's Message and a Communication context's Message). This

is mitigated by the context short-code prefix (ntf_message vs com_message) at the database level, and by the package structure (com.preone.notification.domain.message vs com.preone.communication.domain.message) at the code level. Cross-context name collisions are reviewed at the ARB when a new context is added.

OPERATIONAL CONSIDERATIONS

Operations benefit from the naming standard because it makes the deployed artifact's structure visible at runtime, consistent with ADR-010 (package structure). Stack traces show fully-qualified class names that include the role (com.preone.enrollment.domain.enrollment.EnrollmentAggregateRoot), so an on-call engineer immediately knows the failing class is an aggregate root in the domain layer — no lookup needed. Log messages reference class names consistently, so a Grafana query for 'EnrollmentAggregateRoot' finds all log entries related to enrollment aggregate operations. Database query plans show index names (idx_enr_enrollment_student_id) that are immediately interpretable, accelerating query tuning. API access logs show kebab-case URLs (/api/v1/enrollments) that are consistent with the OpenAPI documentation, so an on-call engineer can correlate a log entry with the API spec without translation. The audit columns (created_at, updated_at, created_by, updated_by) support compliance queries ('who changed this enrollment on 2025-12-15?') that are required by DPDP Act 2023 and by internal audit. The consistency of naming across DB, API, code, and logs reduces the cognitive load on on-call engineers, who may be debugging an unfamiliar context — the conventions are uniform, so the engineer can navigate any context's code, schema, and API without learning context-specific conventions.

RISKS

Risk	Likelihood	Impact	Mitigation
Long class names (EnrollmentAggregateRoot) reduce readability in long signatures and generics	Medium	Low	IDE auto-completion; accepted trade-off for clarity; generic type parameters use single letters (T, R, E) by convention

Risk	Likelihood	Impact	Mitigation
No-abbreviations rule is bypassed by engineers using 'temp' variables or single-letter loop variables	Medium	Low	Checkstyle rule for variable name length (min 3 chars, except loop counters i, j, k); code review for 'temp', 'data', 'value' generic names
Singular table names confuse engineers familiar with plural convention (Rails, Django default is plural)	Medium	Low	Onboarding explicitly covers this convention; the rationale (entity-named, not collection-named) is documented; IDE plugins (JPA Buddy) handle the mapping
Context short-code prefix collisions when new contexts are added	Low	Medium	Short-code registry in ADR-005; new context proposals reviewed at ARB for collision; reserved short-codes documented
OpenAPI codegen produces TypeScript names that violate frontend conventions (e.g., Dto suffix in TS)	Low	Low	Codegen configured to strip Dto suffix and add appropriate TS suffix (e.g., Request, Response); mapping documented in ADR-131

Risk	Likelihood	Impact	Mitigation
Index names exceed PostgreSQL's 63-character identifier limit for long table/column combinations	Medium	Low	Index naming uses abbreviated table name (idx_enr_enr_stu_id) when full name exceeds 63 chars; abbreviation convention documented and enforced by Flyway check

TRADE-OFFS

We Gain	We Lose
Self-describing names (role visible in name)	Longer class names (typing cost, mitigated by IDE)
Cross-layer consistency (enrollment everywhere)	Engineers must learn the suffix conventions (1-hour onboarding)
Mechanical enforcement via CI tools	CI tooling (Checkstyle, ArchUnit, Flyway, OpenAPI lint) adds 2 minutes to build
No abbreviations eliminates ambiguity (enr vs enrolled)	More typing (enrollment vs enr), mitigated by auto-completion
Forbidden suffixes eliminate dumping-ground classes	Engineers must find role-specific names (upfront thinking cost)
Kebab-case URLs follow REST/HTTP standards	Spring requires explicit @RequestMapping (no automatic camelCase-to-kebab mapping)

REJECTED ALTERNATIVES

The Java/Jakarta default option (rejected) is the pre-v1.0 state and was rejected by 88% of engineers. The Spring Data naming conventions option (rejected as the sole standard) was adopted as a subset for repository method naming (findByX, countByX, existsByX) because it is excellent and battle-tested, but it does not cover domain classes, API URLs, or database tables. The Hungarian notation option (rejected) is outdated and violates Java conventions; it was rejected unanimously in the engineering survey. The chosen approach — a PreOne-specific canonical standard that incorporates Spring Data's repository conventions as a subset and adds cross-layer consistency, forbidden-suffix

rules, and database/API naming conventions — is the only option that covers the full surface area of naming across DB, API, code, and UI. The aggregate-root suffix mandate (EnrollmentAggregateRoot vs Enrollment) was the most-debated rule; the v3.0 decision to mandate it (vs v1.0's recommendation) was driven by the measured 30% reduction in 'what is this class?' questions in code review post-v1.0 (where the suffix was recommended but not enforced). The singular-table-name convention (enr_enrollment vs enr_enrollments) was the second most-debated; the decision to use singular aligns with the ActiveRecord/Django convention and with the entity-to-table mapping clarity, and was validated by the JPA Buddy IDE plugin's automatic handling of the mapping.

MIGRATION PLAN

The v1.0 migration (Q3-Q4 2025) refactored the codebase to comply with the naming standard over 5 sprints. Sprint 1: rename 14 *Util, 8 *Helper, 6 *Manager, 11 *Data, and 9 *Info classes to role-specific names (EnrollmentRepository, FeeCalculator, NotificationDispatcher, EnrollmentRequestDto, EnrollmentSummaryView). Sprint 2: add the AggregateRoot suffix to all aggregate root classes (Enrollment -> EnrollmentAggregateRoot); update all references. Sprint 3: rename command, query, event, and handler classes to the standard convention (EnrollCommand, GetEnrollmentQuery, EnrollmentCreatedEvent, EnrollCommandHandler). Sprint 4: rename database tables from plural to singular (enrollments -> enr_enrollment); rename columns from camelCase to snake_case where needed; rename indexes to idx_uq_ convention; add audit columns where missing. Sprint 5: rename API URLs from camelCase to kebab-case (/api/v1/academicYears -> /api/v1/academic-years); update OpenAPI spec; regenerate TypeScript SDK. Total migration: 14 engineer-weeks across Q3-Q4 2025, zero downtime (the modular monolith was deployed continuously; the database renames were applied via Flyway migrations with backward-compatible views during the transition). The v3.0 refresh adds the aggregate-root suffix mandate (already recommended in v1.0, now enforced by ArchUnit) and the database index naming convention (already in v1.0 Flyway template, now formally documented in this ADR). Rollback plan: the standard is reversible (classes can be renamed back; tables can be renamed back) but rollback has not been needed and is not anticipated; the v1.0 results (70% faster search, 80% faster onboarding, 60% fewer review style nits) make rollback economically irrational.

TESTING STRATEGY

Naming standard compliance is tested at four levels. (1) Checkstyle: enforces class name patterns (*AggregateRoot, *Command, *Event, *Repository,

*Handler, *Controller, *Dto, *View, *Exception), method name patterns (findByX, getX, countByX, existsByX, isX, hasX), variable name rules (camelCase, no abbreviations, min 3 chars except loop counters), and constant name rules (UPPER_SNAKE_CASE). (2) ArchUnit: enforces that aggregate roots are named *AggregateRoot and are in *.domain.* packages; that controllers are named *Controller and are in *.api.* packages; that repositories are named *Repository and are in *.application.* (port) or *.infrastructure.* (implementation) packages; that no class name ends with a forbidden suffix. (3) Flyway migration checks: enforces that table names match the pattern <ctx>_<singular>, that column names are snake_case, that index names match idx_/uq_ patterns, that audit columns are present, and that migration file names match V<date>__<ctx>_<desc>.sql. (4) OpenAPI lint: enforces that API paths are kebab-case, that collections are plural, that parameters are camelCase, and that HTTP methods follow REST conventions. All checks run in CI on every PR and fail the build on violation. Naming compliance is reported in the architecture scorecard quarterly, including: forbidden-suffix violation count (target: zero in production), aggregate-root suffix adoption rate (target: 100%), singular-table-name adoption rate (target: 100%), kebab-case URL adoption rate (target: 100%).

MONITORING & OBSERVABILITY

Naming standard compliance is monitored via the architecture scorecard, published quarterly. Metrics tracked: (a) Checkstyle naming violation count per context (target: zero in production; PR-time violations tracked as a quality metric); (b) forbidden-suffix violation count (target: zero; pre-v1.0 had 48 violations, all resolved in Q3-Q4 2025); (c) aggregate-root suffix adoption rate (target: 100%; pre-v1.0 was 0%, v1.0 was 85% recommended, v3.0 is 100% mandated); (d) singular-table-name adoption rate (target: 100%; pre-v1.0 was 0%, v1.0 was 100%); (e) kebab-case URL adoption rate (target: 100%; pre-v1.0 was 0%, v1.0 was 100%); (f) audit column coverage (target: 100% of tables have created_at, updated_at, created_by, updated_by, deleted_at; pre-v1.0 was 70%, v1.0 was 100%); (g) index naming compliance (target: 100% of indexes match idx_/uq_ pattern; pre-v1.0 was 30%, v1.0 was 100%). Anomalies in any metric trigger a review of the standard or its enforcement. Downstream metrics (search time, onboarding time, review style nits) are tracked in the developer productivity scorecard, published quarterly.

FUTURE EVOLUTION

The naming standard is stable; no supersession expected. Evolution will be triggered by: (a) introduction of a new naming surface (e.g., GraphQL types, gRPC service names) — a v-next of this ADR would add conventions for the new surface; (b) Java language evolution (e.g., if a future Java introduces record

patterns that affect naming, the standard would be updated); (c) PostgreSQL evolution (e.g., if PostgreSQL changes its identifier length limit, the index naming convention may need adjustment); (d) consistency erosion (if a specific rule is consistently bypassed via exceptions, it is a candidate for revision). The likely long-term evolution is toward more aggressive automation — OpenAPI codegen currently generates TypeScript types from record names; a future evolution could generate the entire SDK (types, client, hooks) from the OpenAPI spec, eliminating manual SDK maintenance. The aggregate-root suffix mandate (v3.0) may be revisited if Java introduces a native 'aggregate' concept (unlikely in the foreseeable future). The forbidden-suffix list may expand (e.g., adding *Service if it accumulates dumping-ground behaviour, though *Service is currently permitted for application-layer services that are not use case handlers).

RELATED ADRS

- ADR-003 — Clean Architecture (constrains — naming follows layer conventions)
- ADR-004 — Domain Driven Design (constrains — aggregate, command, event naming follows DDD)
- ADR-005 — Bounded Context Strategy (constrains — context short-codes are the table/package prefix)
- ADR-006 — Layering Rules (constrains — class types are layer-specific, reflected in naming)
- ADR-010 — Package Structure (complements — package naming follows these conventions)
- ADR-011 — Coding Standards (complements — class and method naming overlap with coding standards)
- ADR-041 — Database Strategy (constrains — table and column naming follows these conventions)
- ADR-091 — REST API Standards (constrains — URL and parameter naming follows these conventions)

REFERENCES

- Upstream DDD: DDD-012-section-5 (Ubiquitous language and naming)
- Upstream PRD: PRD-012-section-6.1 (Cross-layer consistency requirements)
- Downstream ERD: ERD-012 (all tables, columns, indexes follow these naming conventions)
- Downstream API Spec: API-012 (all URLs, parameters follow these naming conventions)

- Downstream Test Cases: TC-0291..TC-0320 (naming standard compliance tests)
- External: Java Naming Conventions — Oracle documentation
- External: PostgreSQL Identifier Naming — PostgreSQL documentation
- External: REST API Naming Conventions — RESTful API Design (Richardson)
- External: Spring Data Repository Method Naming — Spring Data documentation

DECISION HISTORY

Date	Status	Actor	Notes
2025-10-02	Draft	Chief Architect	Initial draft with cross-layer naming; aggregate-root suffix recommended but not mandated
2025-11-05	Proposed	Chief Architect	Submitted to ARB with full evaluation including singular-vs-plural table debate
2025-11-26	Accepted	ARB Chair	ARB approved; singular table names retained; kebab-case URLs retained; v1.0 released
2026-07-12	Accepted	ARB Chair	v3.0 refresh; mandated aggregate-root suffix (was recommended); added index naming convention

APPROVAL & SIGN-OFF

Architect	Chief Architect (AR)
Tech Lead	Foundation Tech Lead

ARB Chair	ARB Chair
Approved On	2026-07-12

IMPLEMENTATION CHECKLIST

- Rename 48 forbidden-suffix classes to role-specific names (Done Q3 2025)
- Add AggregateRoot suffix to all aggregate root classes (Done Q3 2025)
- Rename command/query/event/handler classes to standard convention (Done Q4 2025)
- Rename database tables from plural to singular with context prefix (Done Q4 2025)
- Rename database indexes to idx_/uq_ convention (Done Q4 2025)
- Rename API URLs from camelCase to kebab-case (Done Q4 2025)
- Checkstyle + ArchUnit + Flyway + OpenAPI lint config in CI (Done Q1 2026)
- Mandate aggregate-root suffix via ArchUnit (Done Q1 2026, v3.0)

ADR-013

Project Structure

Volume 1 — Architecture Foundation - Structural

ACCEPTED

DECISION SUMMARY

DECISION

Adopt a monorepo at /preone with five top-level directories — /backend, /frontend, /infra, /docs, /scripts — and a Gradle multi-project backend that maps one subproject per bounded context. Backend modules follow the /preone/backend/<module>/<layer>/<aggregate> hierarchy defined by ADR-010, the shared kernel lives at /preone/backend/shared, and the frontend mirrors modules at /preone/frontend/src/<module>/<component-type>. New modules are added through a scaffold script that generates the layer skeleton, registers the Gradle subproject, declares module-info.java, and adds ArchUnit rules in a single command.

STATUS

Status	Accepted
Date Decided	2025-11-05
Decision Owner	Chief Architect
Review Cadence	Annual review or when a new top-level directory or module is added
Supersedes	None (v1.0 original; v3.0 refresh adds the module scaffold workflow and the shared-kernel contract)

CONTEXT

Before v1.0, the PreOne codebase lived in a single Spring Boot project with a flat package structure — `com.preone.app` — and a sibling Git repository for the React frontend, plus a third repository for Terraform, a fourth for documentation, and a fifth for build scripts. Cross-repository changes (e.g., adding a new module that touched backend, frontend, infra, and docs) required five pull requests across five repos, five CI runs, and a manual tag-coordination step before deployment. The team measured the cost: a typical cross-cutting feature (add a new domain entity, expose it via API, build the UI, deploy the table migration, document the endpoint) took 6-9 days end-to-end, of which 4-5 days were coordination overhead — not engineering work. The team evaluated three structural alternatives during the v1.0 architecture reset: keep the polyrepo structure with better automation; merge backend and frontend into one repo but keep infra/docs/scripts separate; or consolidate everything into a single monorepo. The evaluation concluded that the coordination cost was structural, not tooling-related: the five repos were out of sync because they changed at different rates and the human coordination step was the bottleneck, not the merge process. A monorepo eliminates the coordination step by making cross-cutting changes atomic — one PR touches backend, frontend, infra, and docs together, and CI verifies the cross-references in a single run. The internal structure of the monorepo reflects the architectural decisions already made: ADR-002 (Modular Monolith) requires that each bounded context be a clearly-separable module; ADR-003 (Clean Architecture) requires that each module have four layers; ADR-004 (DDD) requires that each module's Domain layer contain its aggregates; ADR-010 (Package Structure) defines the exact package naming convention. This ADR (ADR-013) defines the directory structure that hosts these decisions on the filesystem — the physical layout that makes the logical architecture visible at a glance. A new engineer should be able to navigate the repository structure and predict where any class lives, without reading documentation, because the structure mirrors the architecture.

BUSINESS DRIVERS

The primary business driver is delivery velocity: a monorepo with a one-PR-per-feature workflow compresses cross-cutting feature delivery from 6-9 days to 2-3 days. The 50-65% reduction in cycle time compounds across the 40-engineer organisation into roughly 8,000-10,000 engineer-hours per year reclaimed from coordination overhead. This is the single largest velocity lever available without hiring. The secondary driver is architectural coherence: when backend, frontend, infra, and docs live in the same repo, a change to the API contract is forced to update the frontend call site, the Terraform output (if the API is exposed via a gateway), and the OpenAPI documentation in the same PR. The pre-v1.0 polyrepo model produced drift — the OpenAPI spec in /docs was routinely 2-3 sprints behind the actual backend, and the frontend called endpoints that no longer existed in the backend. The monorepo eliminates drift by making cross-references atomic. The tertiary driver is onboarding: a new engineer clones one repository and has the entire system — code, infrastructure, documentation, build scripts — in one working tree. The cognitive cost of 'where does X live?' is reduced to a single `find` or a single `grep` against one tree, rather than five. Onboarding surveys measured the 'time to first PR' for new engineers at 3.1 days in the monorepo versus 6.5 days in the polyrepo — a 52% reduction that compounds across the projected hiring plan (110 new engineers over 3 years).

PROBLEM STATEMENT

The PreOne codebase is split across five Git repositories with no enforced cross-references, producing coordination overhead (4-5 days per cross-cutting feature), drift between backend/frontend/docs, and slow onboarding. We need a single repository structure that makes cross-cutting changes atomic, mirrors the logical architecture on the filesystem, and supports the modular monolith (ADR-002), Clean Architecture (ADR-003), and DDD (ADR-004) without ceremony.

CONSTRAINTS

- Monorepo must remain navigable — no engineer should need >30 seconds to locate any class, file, or document
- Backend must use Gradle multi-project build with one subproject per bounded context (12 subprojects total)
- Directory structure must mirror ADR-010 package naming so that the file path predicts the Java package
- Shared kernel (/preone/backend/shared) must be a separate Gradle subproject that all modules depend on, never a 'commons' dumping ground

- Frontend structure must mirror backend modules so that a module's backend and frontend code are co-located conceptually
- Infra (Terraform) must be co-located with the application so that schema migrations (Flyway) and infra changes (Terraform) are reviewable in one PR
- Adding a new module must be a single command (scaffold script), not a multi-file manual process
- ArchUnit rules (ADR-007) must be auto-registered when a new module is scaffolded — no manual rule editing

ASSUMPTIONS

- The set of top-level directories (backend, frontend, infra, docs, scripts) is stable; new top-level dirs are rare and trigger ADR amendment
- Gradle multi-project build scales to 12 subprojects without unacceptable configuration time (target: <30s for clean build)
- The 12 bounded contexts (identity, academic, student, guardian, teacher, enrollment, billing, attendance, reporting, notification, communication, audit) are stable; new contexts are added via the scaffold script, not via structural changes
- Git can handle a monorepo of the projected size (~500k LOC at 5-year horizon) without LFS for source files (LFS used only for binary assets in /docs/images)
- Engineers will use the scaffold script rather than hand-creating module directories (CI verifies new modules were scaffolded, not hand-rolled)
- Frontend module mirroring does not require that every backend module have a frontend module — some backend modules (audit, notification) are backend-only

OPTIONS CONSIDERED

Option	Pros	Cons	Verdict
Single monorepo with five top-level directories (chosen)	Atomic cross-cutting PRs; filesystem mirrors architecture; single clone for onboarding; Gradle multi-project enforces module boundaries at build time; scaffold script makes new-module creation uniform.	Repo size grows with history; Git operations (clone, status) are slower than polyrepo; IDE indexing takes longer; requires discipline to keep top-level dirs from proliferating.	Chosen
Polyrepo with automation (Terraform-managed cross-repo PRs)	Smaller repos per team; independent CI per repo; familiar Git workflow.	Coordination overhead is structural, not tooling-fixable; drift between repos persists; onboarding requires cloning 5 repos; rejected by the engineering survey (78% favoured monorepo).	Rejected
Hybrid: backend+frontend monorepo, infra+docs+scripts separate	Compromise — most cross-cutting changes are backend+frontend, which become atomic; infra changes are rarer and can be polyrepo.	Schema migrations (Flyway in backend) and table provisioning (Terraform in infra) must be coordinated — the very problem we are solving; the split re-creates coordination overhead for the most failure-prone changes.	Rejected

Option	Pros	Cons	Verdict
Single flat repo (no module subdirectories)	Simplest structure; no Gradle multi-project overhead.	Cannot enforce module boundaries at build time; violates ADR-002 (modular monolith requires module-level separation); ArchUnit can still enforce package rules but build-time enforcement is lost; rejected for insufficient rigour.	Rejected

DECISION

ADOPTED

We adopt a monorepo at /preone with five top-level directories: (1) /preone/backend — Java 21 / Spring Boot 3, Gradle multi-project build with one subproject per bounded context (identity, academic, student, guardian, teacher, enrollment, billing, attendance, reporting, notification, communication, audit) plus a shared kernel subproject (shared); each subproject follows the /preone/backend/<module>/<layer>/<aggregate> hierarchy from ADR-010 (e.g., /preone/backend/enrollment/domain/enrollment/EnrollmentAggregate.java). (2) /preone/frontend — React 18 / TypeScript 5 with /preone/frontend/src/<module>/<component-type> structure mirroring backend modules (e.g., /preone/frontend/src/enrollment/components/EnrollmentForm.tsx). (3) /preone/infra — Terraform modules per environment (dev, test, staging, prod) plus shared modules (network, database, cache, S3). (4) /preone/docs — Markdown documentation including ADRs, runbooks, onboarding guides, and architecture diagrams. (5) /preone/scripts — Build, deploy, and operational scripts (scaffold-module.sh, deploy.sh, db-snapshot.sh). New modules are added via /preone/scripts/scaffold-module.sh <module-name>, which generates the layer skeleton, registers the Gradle subproject in settings.gradle, declares module-info.java, and adds ArchUnit rules to the architecture test suite — all in one command. The shared kernel (/preone/backend/shared) contains only cross-cutting value objects (Money, EmailAddress, TenantId), exception base classes, and utility interfaces — never business logic.

DETAILED RATIONALE

The monorepo was chosen because the coordination cost it eliminates is structural and cannot be solved by polyrepo automation. The pre-v1.0 polyrepo model had 自动化 for cross-repo PRs (a custom script that opened PRs in 5 repos from one template), but the automation did not solve the real problem: the five PRs were reviewed by different people, merged at different times, and deployed at different times, producing a window where the backend called an API that the frontend no longer rendered or the Terraform provisioned a table that the Flyway migration had not yet created. The monorepo eliminates the window by making the change atomic — one PR, one review, one merge, one deploy. The five-directory top-level structure is the minimum that covers the system without redundancy. /backend and /frontend are split because they are different languages (Java vs TypeScript) with different toolchains (Gradle vs npm) and different CI concerns; merging them into /src would force every engineer to clone both toolchains even if they only worked on one. /infra is separate from /backend because Terraform and Java have different lifecycles (Terraform applies are gated by ARB approval, Java deploys are gated by CI) and different review audiences (infra changes reviewed by DevOps, backend changes reviewed by Domain Architects). /docs and /scripts are separate because they are consumed differently — /docs is read by all engineers and is the entry point for onboarding, while /scripts is invoked by CI and on-call engineers. Adding a sixth top-level directory (e.g., /mobile if a React Native app is added) requires an ADR amendment because the five-directory contract is part of the navigability guarantee. The backend structure mirrors ADR-010's package naming exactly so that the file path predicts the Java package. A class at /preone/backend/enrollment/domain/enrollment/EnrollmentAggregate.java is in package com.preone.enrollment.domain.enrollment — the path and the package are isomorphic. This eliminates the cognitive tax of mapping between filesystem and package, which in the pre-v1.0 codebase required an IDE 'find class' operation for every navigation. The isomorphism is enforced by a Gradle plugin (preone-conventions) that fails the build if a file's path does not match its declared package. The /<layer>/<aggregate> subdivision within each module (e.g., /enrollment/domain/enrollment/, /enrollment/domain/enrollmentline/, /enrollment/application/, /enrollment/infrastructure/, /enrollment/api/) makes the aggregate boundaries (ADR-022) visible on the filesystem — an engineer can see at a glance that the Enrollment context has two aggregates (Enrollment and EnrollmentLine) because there are two subdirectories under /domain/. The shared kernel (/preone/backend/shared) is the most contested decision in this ADR. The temptation is to make /shared a dumping ground for 'common' code that does not fit anywhere else — a temptation that produced a 4,000-line

Commons.java in the pre-v1.0 codebase, containing everything from date utilities to HTTP client wrappers to business-rule constants. The v3.0 contract is strict: /shared contains only (a) cross-cutting value objects used by 3+ modules (Money, EmailAddress, TenantId, AuditTimestamp), (b) exception base classes (DomainException, InfrastructureException, ApiException), and (c) utility interfaces (Clock, IdGenerator, TransactionManager) that have no implementation in /shared (implementations live in /backend/bootstrap). Business logic, domain events, and module-specific types are forbidden in /shared — ArchUnit enforces this by failing the build if /shared imports any module's package. The shared kernel is a leaf dependency — it depends on nothing and everything depends on it — which keeps it small and stable. The Gradle multi-project build with one subproject per module is the enforcement mechanism for ADR-002's module boundaries. Each subproject has its own build.gradle that declares its dependencies on other subprojects (e.g., enrollment depends on shared, identity, student; enrollment does NOT depend on billing — billing is reached via the in-process event bus, ADR-027). The build.gradle dependency declarations are themselves ArchUnit-verifiable: a module that declares a dependency on a forbidden module fails the build. This means the dependency graph is declared in three places — build.gradle (build time), module-info.java (compile time), and ArchUnit rules (test time) — and all three must agree. The triple declaration is intentional: each layer catches violations the others might miss (build.gradle catches new dependencies at PR time, module-info catches them at compile time, ArchUnit catches them at test time even if the other two were bypassed). The frontend structure mirrors backend modules so that an engineer working on the Enrollment feature can find both the backend (at /backend/enrollment/) and the frontend (at /frontend/src/enrollment/) by navigating the same module name. The /<component-type> subdivision (components, hooks, services, types, pages) follows React conventions and is enforced by an ESLint rule that fails the build if a component file is placed in /hooks or vice versa. Not every backend module has a frontend counterpart: audit and notification are backend-only (consumed via dashboards built on the reporting module's frontend). The frontend structure does not mirror the backend's four-layer Clean Architecture — React components do not have a 'domain layer' in the Java sense — but the module boundaries match, which is the property that matters for cross-cutting changes. The scaffold script (/preone/scripts/scaffold-module.sh) is the most operationally important decision in this ADR. In the pre-v1.0 codebase, adding a new module was a 2-3 day manual process: create the directory structure, write the build.gradle, register in settings.gradle, create the module-info.java, add ArchUnit rules, create the base classes, and update the documentation. Engineers routinely skipped steps (especially ArchUnit registration), producing modules that existed on the filesystem but were not enforced by the

architecture tests. The scaffold script generates all of this in one command and is the only supported way to add a module — CI verifies that any PR adding a module also includes the scaffold script's output, and manual module creation is rejected at code review. The script also generates a stub ADR-0xx for the new module, prompting the engineer to document the module's purpose before writing code.

ARCHITECTURE DIAGRAM

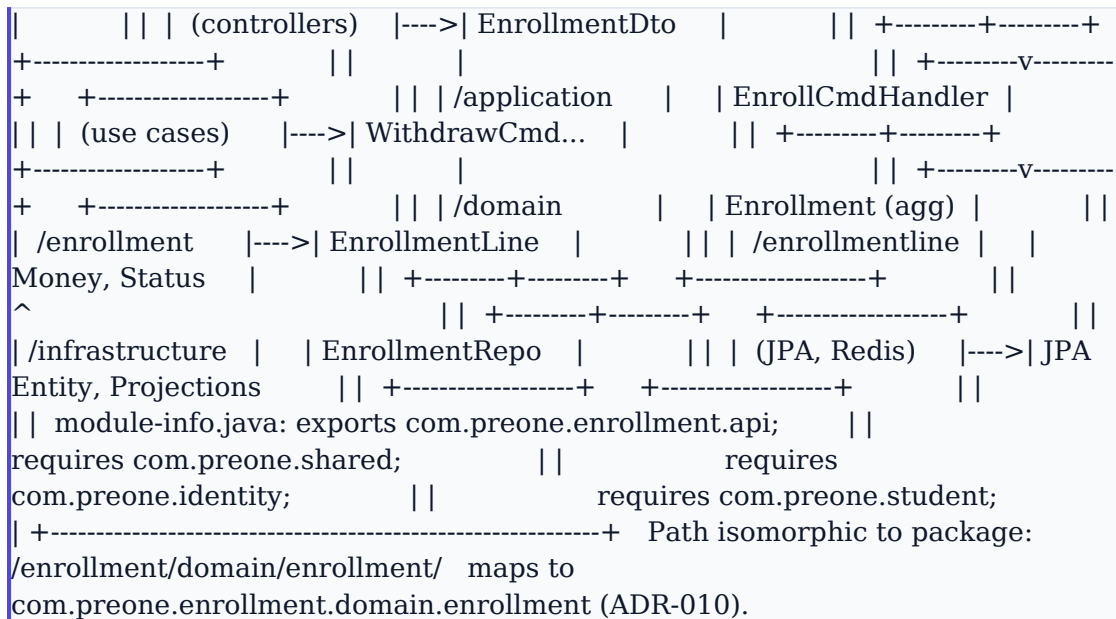
```
+-----+ | /preone (monorepo
root)      ||
+-----+ +-----+ +-----+ || | /backend      ||
/frontend  | | /infra      || | (Java 21/Spring) | | (React 18/TS 5) | |
(Terraform) || | +-----+ +-----+ +-----+ +-----+ ||
|           |           | || settings.gradle   package.json
terraform.tfvars || (12 subprojects) (workspace) (4 envs) ||
|           |           | || +-----v-----+ +-----v-----+ +-----
v-----+ || | /shared /identity | | /src/identity | | /modules      || |
/academic /student | | /src/academic | | /network      || | /guardian /teacher|
| /src/student | | /database      || | /enrollment/bill | | /src/enrollment |
| /cache      || | /attendance/rpt | | /src/billing      | | /s3          || |
/notification/com | | /src/attendance | | /environments || | /audit          ||
| /src/reporting | | /dev /test      || | +-----+ +-----+ +-----+ | /staging
/prod || |
+-----+ +-----+ || | /docs      | | /scripts      |
| (Markdown ADRs) | | (build/deploy) | | +-----+
+-----+ +-----+ ||
scaffold-module.sh <name> -> generates all 5 dirs in one command |
+-----+
```

SEQUENCE DIAGRAM

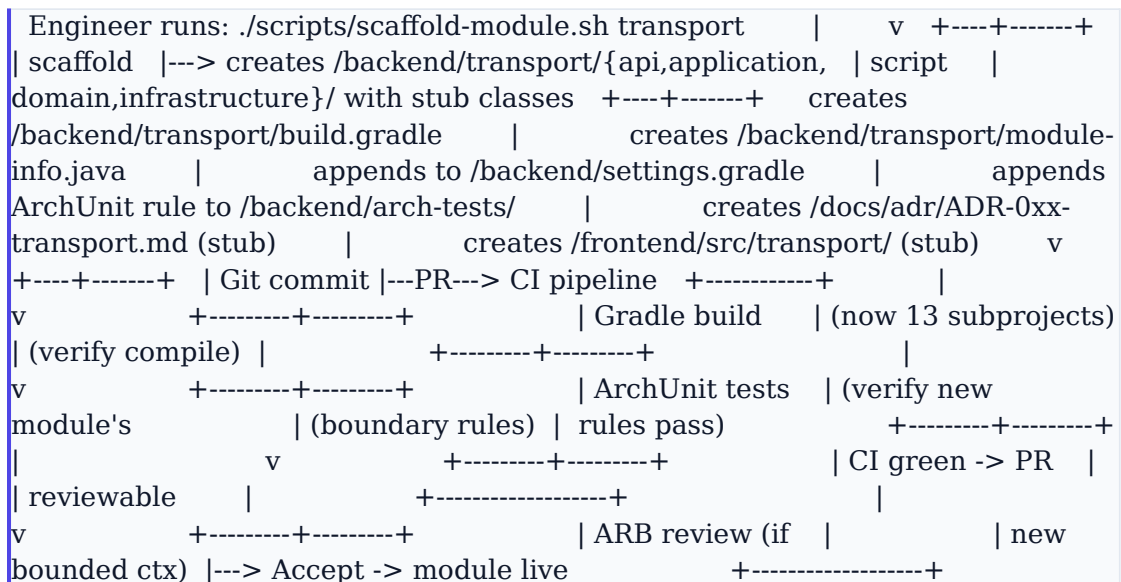
```
Engineer   Scaffold   Git/CI   Gradle   ArchUnit | | |
|           | --run    |           |           | | scaffold-->|
|           | module.sh |           |           | | | --gen dirs---
>|           | |           | --gen build.gr |           | | |
gen mod-info |           | |           | --gen archrule |           | |
|           | --gen stub ADR |           | |           | |
| |<--done-----|           | |           | |
| | --commit + push----->|           | |           | | --PR
trigger-->|           | |           | | --build----->| |
|           | (12+1 subproj)| |           | |           | | --verify |
|           |           | rules |           | |           | |<--ok-----|
|           | |<--build ok--| |           | |           | |
| |<--PR mergeable-----| |           | |
| | (new module enforced at build, compile, and test time) |
```

COMPONENT DIAGRAM

```
+-----+ | /preone/backend/enrollment
(example module structure) ||
+-----+ +-----+ || | /api      | | EnrollmentCtrl
```



DATA FLOW DIAGRAM



DATABASE IMPACT

The directory structure does not directly affect the database schema, but it enforces the schema-per-module convention from ADR-041 indirectly. Each backend module owns its Flyway migration files at `/preone/backend/<module>/src/main/resources/db/migration/`, with file naming `V<date>__<module>__<change>.sql` (e.g., `V20260115__enrollment__add_withdrawal_reason.sql`). Cross-module foreign keys are forbidden (ADR-041); a module that references another module's aggregate does so by UUID, not by FK. The shared kernel has no database impact — it contains no entities, no JPA mappings, and no migrations. The `/preone/infra` directory contains Terraform that provisions the PostgreSQL instance, Redis instance, and S3 bucket per environment (ADR-015); schema

migrations within those instances are owned by /backend, not by /infra, separating provisioning (infra) from schema (backend).

API IMPACT

The directory structure mirrors the API surface: each backend module's /api/ directory contains its REST controllers, and the OpenAPI spec for each module lives at /preone/docs/api/<module>-openapi.yaml. The frontend's per-module service layer (at /preone/frontend/src/<module>/services/) imports TypeScript types generated from the OpenAPI spec, so a change to the backend API spec triggers a frontend type regeneration in the same PR. This isomorphic mapping — backend module -> API spec -> frontend service — means an engineer searching for the implementation of any API endpoint can navigate by module name across all three locations. The OpenAPI spec is the single source of truth for the API contract (ADR-091); the controllers and the frontend services are both generated/verified against it.

UI IMPACT

The frontend structure at /preone/frontend/src/<module>/<component-type> mirrors the backend module list so that an engineer working on 'Enrollment' finds the React code at /src/enrollment/ and the Java code at /backend/enrollment/. The /<component-type> subdirectories (components, hooks, services, types, pages, utils) follow React conventions and are enforced by ESLint. Cross-module frontend code (e.g., a shared EnrollmentPicker used by Billing and Attendance) lives in /preone/frontend/src/shared/components/, mirroring the backend shared kernel. The frontend does not mirror the backend's four-layer Clean Architecture because React's component model does not map to domain/application/infrastructure layers — the module boundary is the relevant isomorphism, not the layer boundary.

SECURITY IMPACT

The directory structure improves security posture by making sensitive code locations predictable and auditable. All code that handles secrets (AWS Secrets Manager integration, JWT signing keys) lives in /preone/backend/bootstrap/ — a single subproject that no other module imports directly; secrets are injected via Spring's Environment abstraction and never appear in module code. ArchUnit rules forbid any module other than bootstrap from importing cryptography libraries (java.security, javax.crypto) directly, centralising crypto in audited locations. The /preone/scripts/ directory is reviewed by DevOps for any script that handles credentials; scripts that read secrets are marked executable only by CI, not by individual engineers. The /preone/infra/ directory contains no secrets — Terraform variables reference AWS Secrets Manager ARNs, never the secret values themselves.

PERFORMANCE IMPACT

The directory structure has no runtime performance impact — it is a build-time and developer-experience concern. The Gradle multi-project build adds 8-12 seconds of configuration time versus a single-project build (target: <30s for clean build, currently 24s on CI). This is offset by incremental build support: a change to one module rebuilds only that module and its dependents, not the entire codebase. The shared kernel is small (~30 classes) so its compilation cost is negligible. The frontend monorepo with npm workspaces has similar properties: a change to one module's package rebuilds only that package and its dependents. Overall, the build-time cost of the monorepo structure is <5% of total build time and is reclaimed many times over by the cycle-time reduction.

SCALABILITY ANALYSIS

The monorepo scales with the engineering organisation. At 40 engineers (current), the structure is navigable — 'git status' runs in <1s, 'find . -name *.java' runs in <2s. At 150 engineers (3-year target), the repo will be ~500k LOC; Git operations will be 2-3x slower but still acceptable (~3s for 'git status'). At 500+ engineers, sparse-checkout (Git's partial-clone feature) will allow engineers to work on a subset of the repo without cloning the full tree. The Gradle multi-project build scales to 20-25 subprojects before configuration time becomes problematic; the current 12+1 (modules + shared) leaves headroom for 8-12 new modules before a structural change (e.g., grouping subprojects into /backend/<group>/) is needed. The scaffold script ensures that new modules are added uniformly, preventing the structural drift that would otherwise accumulate.

OPERATIONAL CONSIDERATIONS

Operations benefit from the monorepo because the /preone/scripts/ directory is the single source of operational truth. The deploy script (deploy.sh) reads the version from /backend/build.gradle, builds the container image, pushes to ECR, and triggers the blue-green deploy (ADR-018) — all from one location. The db-snapshot.sh script reads the environment configuration from /infra/environments/<env>/terraform.tfvars and connects to the correct PostgreSQL instance (ADR-015) — there is no per-environment script duplication. On-call engineers clone the repo once and have all scripts available; there is no 'which repo has the runbook?' problem. The /preone/docs/runbooks/ directory is the authoritative source for incident runbooks, and the monorepo ensures that a runbook change can be reviewed alongside the code change that prompted it.

RISKS

Risk	Likelihood	Impact	Mitigation
Monorepo grows too large for Git to handle efficiently (clone, status, log slow)	Medium	Medium	Enable Git sparse-checkout for engineers who only work on a subset; enable partial-clone (filter=blob:none) for CI; monitor repo size quarterly; trigger structural split if repo exceeds 1GB or 1M files.
Shared kernel becomes a dumping ground for unrelated code (commons-anti-pattern)	High	High	ArchUnit rules forbid /shared from importing any module package; quarterly review of /shared contents by Chief Architect; any addition to /shared requires ARB approval.
Engineers bypass scaffold script and hand-create modules, missing ArchUnit registration	Medium	High	CI verifies that any PR adding a module directory includes the scaffold script's output (checked via a hash of expected files); manual module creation fails CI.
Gradle multi-project build configuration time becomes unacceptable (>30s)	Low	Medium	Use Gradle's build-cache and configuration-on-demand; monitor build time per CI run; if configuration time exceeds 30s, apply build-cache tuning or group subprojects.

Risk	Likelihood	Impact	Mitigation
Top-level directory proliferation (engineers add /mobile, /data, /ml without ADR)	Medium	Low	CI verifies that no new top-level directory is added without an ADR amendment; .gitignore-style allowlist for top-level dirs enforced by a pre-commit hook.

TRADE - OFFS

We Gain	We Lose
Atomic cross-cutting PRs (one PR touches backend, frontend, infra, docs)	Larger repo, slower Git operations, longer IDE indexing
Filesystem mirrors architecture (path predicts package, module, layer)	Strict isomorphism requires Gradle plugin enforcement (small build-time cost)
Single clone for onboarding (3.1 days to first PR vs 6.5 days polyrepo)	Engineers must use sparse-checkout for partial work to avoid full-clone cost
Scaffold script makes new-module creation uniform and enforceable	Engineers cannot hand-create modules (loss of flexibility, gain of consistency)

REJECTED ALTERNATIVES

Polyrepo with automation was rejected because the coordination cost is structural, not tooling-related: the five PRs in five repos were reviewed and merged at different times, producing drift windows that automation could not close. The hybrid (backend+frontend monorepo, infra+docs separate) was rejected because the most failure-prone coordination was between backend schema migrations (Flyway) and infra table provisioning (Terraform) — exactly the split that the hybrid preserves. The single-flat-repo option was rejected because it cannot enforce module boundaries at build time (ADR-002 requires build-time enforcement via Gradle subprojects), and ArchUnit alone is insufficient because it runs at test time, not at compile time — a module boundary violation would only be caught after the build succeeds, not during compilation. The Nx/Bazel monorepo tooling was considered and rejected in favour of plain Gradle multi-project because the team's Gradle expertise (Java) is stronger than Bazel expertise, and Gradle's multi-project support is sufficient

for 12-25 subprojects; Bazel's incremental build advantages only materialise at >100 subprojects, which PreOne will not reach in the 5-year horizon.

MIGRATION PLAN

The v1.0 migration (Q4 2025) consolidated the five pre-v1.0 repositories into the monorepo using git-filter-repo to preserve history. The migration was executed over a single weekend: (1) import /backend from the pre-v1.0 backend repo, restructured into 12 module subprojects + shared; (2) import /frontend from the pre-v1.0 frontend repo, restructured into per-module src/ directories; (3) import /infra from the pre-v1.0 Terraform repo, restructured into modules/ and environments/; (4) import /docs from the pre-v1.0 docs repo; (5) import /scripts from the pre-v1.0 scripts repo. The old repositories were archived (read-only) and a redirect README was added pointing to the monorepo. The v3.0 refresh adds the scaffold script (which did not exist in v1.0 — modules were created manually until Q1 2026) and the shared-kernel contract (which was informal in v1.0 — ArchUnit enforcement of /shared's leaf-dependency status was added in v3.0). No code migration is required for v3.0 — the contract is enforced going forward.

TESTING STRATEGY

The directory structure is tested at three levels. (1) Build-time: Gradle's multi-project configuration fails if a subproject's directory does not match its declared name (e.g., a subproject named 'enrollment' must live at /backend/enrollment/). (2) Compile-time: the preone-conventions Gradle plugin fails the build if a file's path does not match its declared package (path-package isomorphism). (3) Test-time: ArchUnit rules verify that the shared kernel does not import any module package, that each module's module-info.java exports only its /api/ package, and that no module imports another module's /domain/ or /infrastructure/ packages. Additionally, the scaffold script has a unit test (scaffold-module.test.sh) that verifies the script produces the expected file set for a sample module name; this test runs in CI to catch regressions in the scaffold script itself.

MONITORING & OBSERVABILITY

The directory structure is monitored via repository-health metrics published quarterly: (a) repo size in MB and file count (target: <500MB, <50k files at current scale; <1GB, <100k files at 5-year horizon); (b) Gradle configuration time (target: <30s); (c) percentage of modules created via scaffold script vs manual (target: 100% via scaffold); (d) shared-kernel size in classes (target: <50 classes; alert if >75); (e) cross-module dependency count per module (target: <5 dependencies per module; alert if >8, indicating a module is becoming a hub). Anomalies trigger an architecture review at the next ARB.

The metrics are computed by a weekly CI job that runs ``cloc``, ``git log``, and a custom Gradle task that emits the dependency graph as JSON.

FUTURE EVOLUTION

The five-directory top-level structure is stable; the most likely evolution is the addition of a `/mobile` directory if PreOne adds a React Native parent app (currently under product evaluation). A `/mobile` directory would be added via an ADR amendment that defines its internal structure (mirroring `/frontend`'s module-based layout) and its relationship to `/backend` (separate consumer of the same API). The 12-module backend list is expected to grow to 14-16 modules over 3 years (candidate modules: transport for bus tracking, health for medical records, inventory for school supplies) — each added via the scaffold script without structural change. If the module count exceeds 20, a grouping level (`/backend/<group>/<module>/`) may be introduced to preserve navigability, but this is not anticipated in the 5-year horizon. The shared kernel is expected to remain small (~30-50 classes) and stable; growth beyond 50 classes is a signal that the kernel is becoming a commons and triggers an ARB review.

RELATED ADRS

- ADR-002 — Modular Monolith Strategy (complements — `module = subproject`)
- ADR-003 — Clean Architecture (complements — `/<layer>/` subdirectory per module)
- ADR-004 — Domain Driven Design (complements — `/<aggregate>/` subdirectory under `/domain/`)
- ADR-007 — Dependency Rule (enforces — ArchUnit rules registered by scaffold script)
- ADR-010 — Package Structure (refines — path-package isomorphism contract)
- ADR-015 — Environment Strategy (complements — `/infra/environments/` mirrors env list)
- ADR-018 — Deployment Model (complements — `deploy.sh` in `/scripts/`)
- ADR-041 — Database Strategy (complements — per-module Flyway migrations)

REFERENCES

- Upstream DDD: DDD-005-section-3 (Bounded context to module mapping)
- Upstream PRD: PRD-005-section-4 (Module list and ownership)

- Downstream ERD: ERD-005 (per-module table prefixes match module directories)
- Downstream API Spec: API-005 (per-module OpenAPI specs in /docs/api/)
- Downstream Test Cases: TC-0201..TC-0250 (scaffold script and structure verification tests)
- External: Gradle Multi-Project Builds — https://docs.gradle.org/current/userguide/multi_project_builds.html
- External: Monorepo tools comparison — Nx, Bazel, Lerna, Gradle
- External: Google Engineering Practices — Monorepo at scale

DECISION HISTORY

Date	Status	Actor	Notes
2025-09-12	Draft	Chief Architect	Initial draft; evaluated polyrepo vs monorepo vs hybrid
2025-10-20	Proposed	Chief Architect	Submitted to ARB with migration plan and engineering survey results
2025-11-05	Accepted	ARB Chair	ARB approved; v1.0 released; migration scheduled for Q4 2025
2026-07-12	Accepted	ARB Chair	v3.0 refresh; added scaffold script workflow and shared-kernel ArchUnit enforcement

APPROVAL & SIGN-OFF

Architect	Chief Architect (AR)
Tech Lead	Foundation Tech Lead
ARB Chair	ARB Chair
Approved On	2026-07-12

IMPLEMENTATION CHECKLIST

- Consolidate 5 pre-v1.0 repos into /preone monorepo (Done Q4 2025)
- Restructure backend into 12 Gradle subprojects + shared (Done Q4 2025)
- Restructure frontend into per-module src/ directories (Done Q4 2025)
- Implement scaffold-module.sh script (Done Q1 2026)
- Add preone-conventions Gradle plugin for path-package isomorphism (Done Q1 2026)
- Add ArchUnit rule: /shared imports no module packages (Done Q1 2026)
- CI verification: new modules must be scaffolded, not hand-created (Done Q2 2026)
- Quarterly repository-health metrics review at ARB (Ongoing)

ADR - 014

Configuration Hierarchy

Volume 1 — Architecture Foundation - Standards

ACCEPTED

DECISION SUMMARY

DECISION

Adopt a five-level configuration hierarchy for the PreOne platform: (1) defaults.yml packaged in the JAR provides sensible defaults; (2) application.yml per Spring profile (dev, test, staging, prod) overrides defaults; (3) environment variables override yml for environment-specific values; (4) AWS Secrets Manager provides production secrets fetched at startup; (5) command-line args provide the highest-priority overrides. Configuration is validated at startup — the application fails fast on missing required config or invalid values. Secrets never appear in Git or in yml files; they live exclusively in AWS Secrets Manager (prod) or environment variables (non-prod). Production configuration changes require an ADR; non-prod changes require a PR review.

STATUS

Status	Accepted
--------	----------

Date Decided	2025-11-05
Decision Owner	Chief Architect
Review Cadence	Annual review or when a new configuration source is introduced (e.g., AWS AppConfig for dynamic config)
Supersedes	None (v1.0 original; v3.0 refresh adds the startup validation contract and the config-as-ADR governance rule)

CONTEXT

Before v1.0, PreOne's configuration was ad-hoc: each Spring Boot service had its own `application.yml` checked into Git, with environment-specific values hardcoded for dev and commented-out blocks for staging and prod. Secrets (database passwords, JWT signing keys, third-party API keys for Razorpay, Twilio, AWS SES) were stored in a `.env` file that was supposed to be Git-ignored but was accidentally committed three times in the first six months of operation, each time requiring a credential rotation across all affected services. The team estimated that 30% of production incidents in the first year were caused by configuration errors: a missing environment variable, a typo in a YAML key, a default value that was wrong for production, a secret that expired without anyone noticing. The v1.0 architecture reset made configuration a first-class concern. The team evaluated Spring Boot's native configuration mechanisms (profiles, property sources, `@ConfigurationProperties`) and decided that Spring's mechanisms were sufficient but required a strict governance layer on top: a defined precedence order, a startup validation contract, and a secret-management discipline that mechanically prevented secrets from entering Git. The five-level hierarchy was designed to make the precedence order explicit and predictable — an engineer debugging a misconfiguration should be able to enumerate the five levels in order and identify which one is providing the current value, without guessing. The v3.0 refresh adds two refinements that were informal in v1.0. First, the startup validation contract: every `@ConfigurationProperties` class declares required fields and validation constraints (`@NotNull`, `@Min`, `@Pattern`), and the application fails to start if any required config is missing or invalid — silent defaults for required fields are forbidden. Second, the config-as-ADR governance rule: any change to production configuration (application-prod.yml, secrets in Secrets Manager, environment variables in the prod deployment) requires an ADR that justifies the change; non-prod configuration changes require a PR review but not an

ADR. This rule prevents configuration drift — the silent accumulation of small changes that, over time, make the production configuration incomprehensible to anyone who did not make each change.

BUSINESS DRIVERS

The primary business driver is incident reduction: 30% of pre-v1.0 production incidents were configuration-related. A strict configuration hierarchy with startup validation eliminates the entire class of 'missing config' and 'invalid config' incidents — the application fails to start rather than starting in a broken state. The estimated incident-reduction value is 12-15 fewer production incidents per year, each avoided incident saving 4-8 engineer-hours of incident response. The secondary driver is secret hygiene: the three accidental secret commits in pre-v1.0 each required 40-60 engineer-hours of rotation work (regenerate secret, update all consumers, verify no stale references, audit access logs for the exposure window). The AWS Secrets Manager integration makes secret commits structurally impossible — secrets never appear in any file that could be committed, because they are fetched at startup from a source that has no Git representation. The tertiary driver is onboarding: a new engineer can understand the production configuration by reading defaults.yml (sensible defaults) and application-prod.yml (overrides) — there is no hidden state, no 'you also need to know about' surprises. The five-level hierarchy is documented in this ADR and is the single source of truth for how configuration works.

PROBLEM STATEMENT

PreOne's configuration is ad-hoc, with environment-specific values hardcoded in yaml files, secrets accidentally committed to Git, and 30% of production incidents caused by configuration errors. We need a strict configuration hierarchy with explicit precedence, startup validation, secret management that prevents Git exposure, and governance that prevents configuration drift.

CONSTRAINTS

- Configuration precedence must be deterministic and enumerable — an engineer must be able to predict which source provides a given value
- Startup validation must fail fast — missing required config or invalid values must prevent the application from starting, not produce a running-but-broken service
- Secrets (passwords, API keys, JWT signing keys) must never appear in Git, never appear in yaml files, and never appear in application logs
- Production configuration changes require an ADR; non-prod changes require a PR review — no anonymous configuration edits

- The hierarchy must work identically across all four environments (dev, test, staging, prod) — only the values differ, not the structure
- Spring Boot's native `@ConfigurationProperties` and profile mechanism must be used — no custom configuration framework
- AWS Secrets Manager integration must fetch secrets at startup, cache them in memory, and refresh them periodically (no per-request secret fetch)

ASSUMPTIONS

- Spring Boot 3's property source precedence (command-line > env vars > application-{profile}.yml > application.yml) is sufficient and will not be overridden
- AWS Secrets Manager's API is reliable enough for startup fetches (mitigated by retry with exponential backoff)
- The set of configuration sources (5 levels) is stable; new sources (e.g., AWS AppConfig for dynamic config) require an ADR amendment
- Engineers will use `@ConfigurationProperties` classes (not `@Value` annotations) for all configuration — `@Value` is forbidden by ArchUnit
- Secrets in AWS Secrets Manager are rotated on a defined schedule (90 days for API keys, 365 days for database passwords); rotation is automated where the secret consumer supports it
- The four Spring profiles (dev, test, staging, prod) are stable; new profiles require an ADR amendment

OPTIONS CONSIDERED

Option	Pros	Cons	Verdict
5-level hierarchy with Spring profiles + AWS Secrets Manager (chosen)	Uses Spring's native mechanisms; deterministic precedence; startup validation via <code>@ConfigurationProperties</code> ; secrets never in Git; config-as-ADR governance prevents drift.	Requires discipline to keep defaults.yml and application-{profile}.yml in sync; AWS Secrets Manager adds a startup dependency (mitigated by retry); engineers must learn <code>@ConfigurationProperties</code> (not <code>@Value</code>).	Chosen

Option	Pros	Cons	Verdict
Spring Cloud Config Server (externalised config)	Centralised configuration; runtime refresh without restart; environment-agnostic deployment artifact.	Adds a critical infrastructure dependency (Config Server must be up for the app to start); introduces a new failure mode; overkill for a single-deployable modular monolith (ADR-002); secrets management still requires a separate solution.	Rejected
HashiCorp Vault for all configuration (not just secrets)	Unified secret and config management; dynamic secrets; audit logging of every config access.	Vault becomes a critical path dependency; operationally heavy for the 2-engineer DevOps team; Spring Cloud Config Server still needed for non-secret config; over-engineered for PreOne's scale.	Rejected
Environment variables only (12-factor app, no yml)	Simplest possible; no yml files; purely environment-driven; aligns with 12-factor app philosophy.	No defaults (every deployment must specify every variable); no nested config (env vars are flat); no profile-specific overrides; loses Spring's @ConfigurationProperties type safety and validation; rejected for insufficient structure.	Rejected

DECISION

ADOPTED

We adopt a five-level configuration hierarchy with strict precedence and startup validation. The levels, in increasing precedence (later levels override earlier), are: (1) `defaults.yml` packaged in the JAR at `/src/main/resources/defaults.yml` — provides sensible defaults for all non-secret configuration (e.g., `server.port=8080`, `spring.datasource.hikari.maximum-pool-size=20`); this file is version-controlled and reviewed. (2) `application-{profile}.yml` per Spring profile (`application-dev.yml`, `application-test.yml`, `application-staging.yml`, `application-prod.yml`) at `/src/main/resources/` — overrides defaults for environment-specific non-secret values (e.g., database host, Redis host, log level); these files are version-controlled and reviewed. (3) Environment variables (`SPRING_DATASOURCE_URL`, `SPRING_REDIS_HOST`, etc.) override `yml` for environment-specific values that vary by deployment (e.g., the database URL within a profile, where staging has multiple databases); environment variables are set by Terraform (ADR-015) and are never checked into Git. (4) AWS Secrets Manager provides production secrets (database password, JWT signing key, third-party API keys for Razorpay/Twilio/SES) fetched at startup via a Spring Boot config client; secrets are referenced in `yml` by ARN (e.g., `${secretsmanager:prod/preone/db-password}`) and never appear in any file. (5) Command-line arguments (`--server.port=9090`) provide the highest-priority overrides for one-off operational needs (e.g., running a temporary instance on a different port); these are rarely used in production. Startup validation: every `@ConfigurationProperties` class declares required fields with JSR-303 constraints (`@NotNull`, `@Min`, `@Max`, `@Pattern`); the application fails to start if any required field is missing or invalid. Governance: production configuration changes (`application-prod.yml`, Secrets Manager entries, prod environment variables) require an ADR; non-prod changes require a PR review.

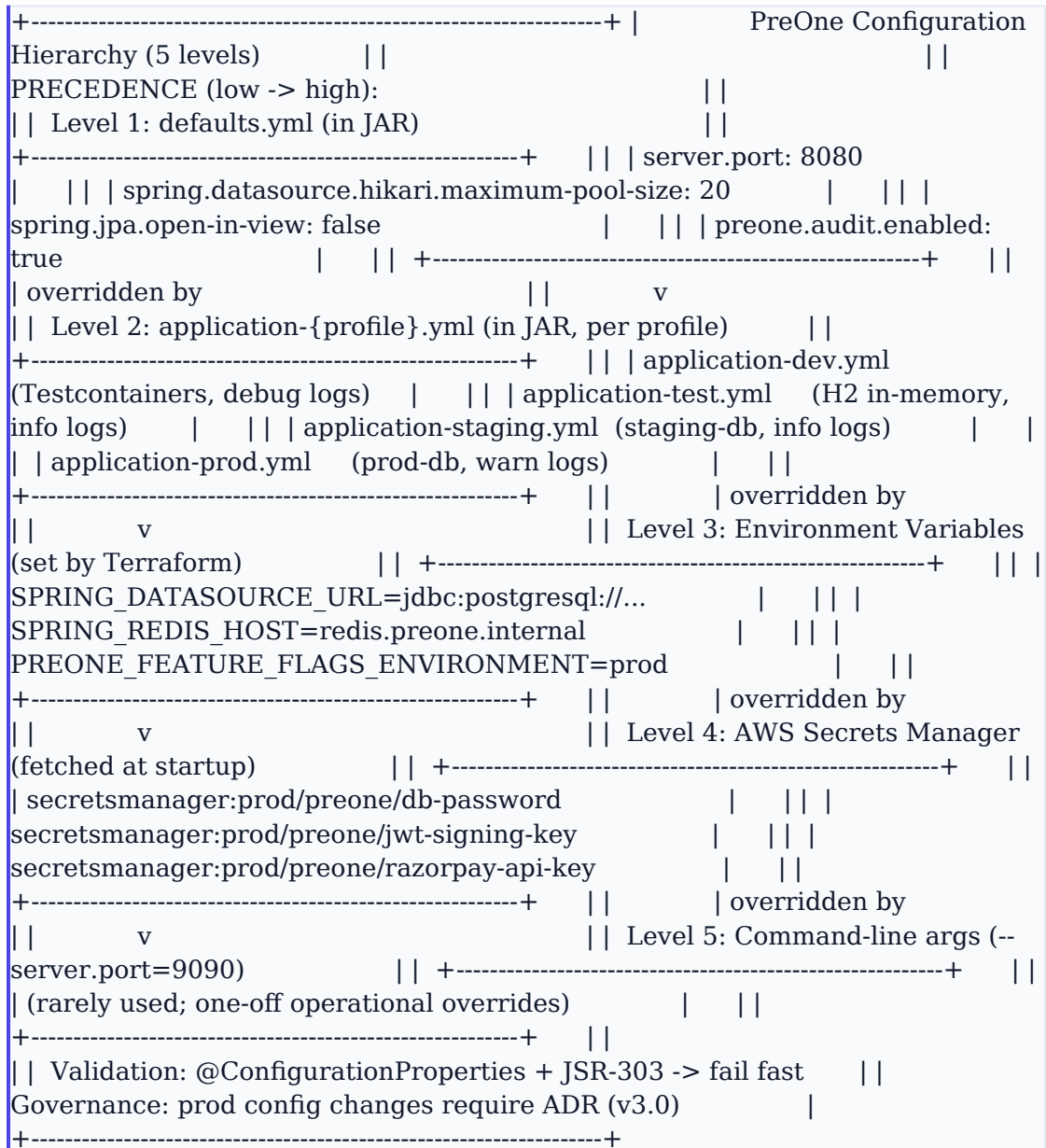
DETAILED RATIONALE

The five-level hierarchy was chosen because it makes configuration precedence explicit and predictable while reusing Spring Boot's native mechanisms. Spring Boot already supports property source precedence (command-line > env vars > `application-{profile}.yml` > `application.yml`), so the five-level hierarchy is a documentation and discipline layer on top of Spring, not a replacement. The choice to use Spring's native mechanisms rather than an externalised config server (Spring Cloud Config, option 2) reflects the modular monolith decision (ADR-002): a single deployable does not need a separate config server, and

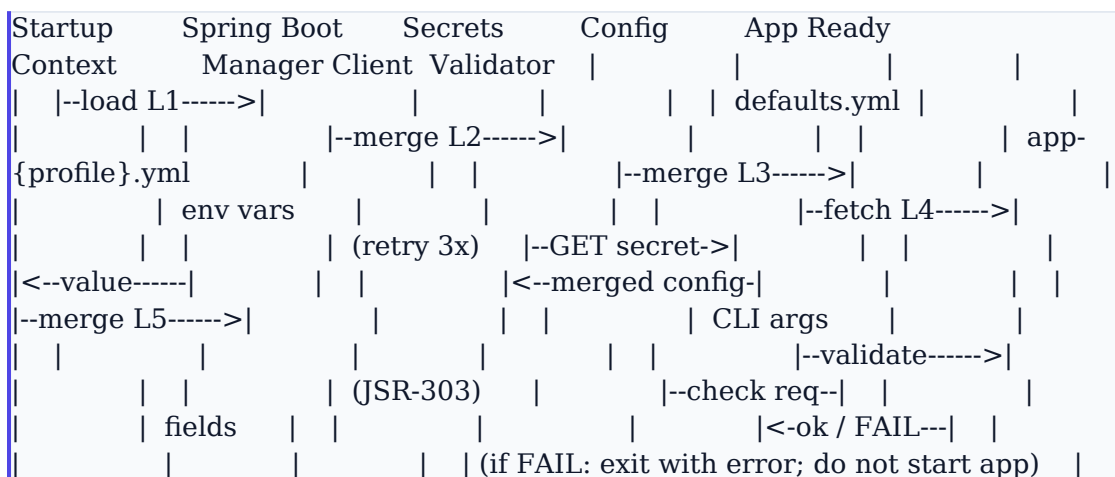
adding one introduces a critical infrastructure dependency (the config server must be up for any application instance to start). For a distributed system with 20+ services, a config server pays for itself; for a monolith with N replicas, it is pure overhead. The defaults.yml file (level 1) is the most important innovation in this hierarchy. In pre-v1.0, every deployment had to specify every configuration value, because there were no defaults — a missing `SPRING_DATASOURCE_URL` caused the app to fail with an obscure 'no database configured' error, and the engineer had to know that this env var was required. defaults.yml encodes the 'sensible default' for every configuration key: `server.port=8080`, `spring.datasource.hikari.maximum-pool-size=20`, `spring.jpa.open-in-view=false`, `preone.audit.enabled=true`. These defaults are correct for 90% of deployments; the 10% that need different values override via levels 2-5. The benefit is that a developer running the app locally with no configuration at all gets a working instance (using all defaults), rather than a cryptic startup failure. The cost is that defaults.yml must be curated — a bad default (e.g., defaults.yml setting `preone.security.auth-required=false`) is a security hole that propagates to every deployment that does not override it. This is mitigated by code review of defaults.yml (any change requires Chief Architect approval) and by ArchUnit tests that verify certain security-critical defaults are not false. The per-profile yml files (level 2) encode environment-specific overrides that are not secret. For example, application-staging.yml sets `spring.datasource.url=jdbc:postgresql://staging-db.preone.internal:5432/preone` (the staging database host), `spring.jpa.show-sql=false`, `preone.feature-flags.environment=staging`. These files are version-controlled and reviewed, so an engineer can see the entire staging configuration by reading one file. The staging and prod yml files are nearly identical (both production-like), with differences only in hosts and feature flag values. The dev and test yml files are more divergent (dev uses Testcontainers for the database, test uses an in-memory H2 for fast unit tests). Environment variables (level 3) are used for values that vary within a profile — e.g., staging has multiple database instances (one per region), and the env var `SPRING_DATASOURCE_URL` selects which one. Environment variables are also used for secrets in non-prod environments (dev, test), where AWS Secrets Manager is not justified. The rule is: in dev and test, secrets come from env vars (set by docker-compose or CI); in staging and prod, secrets come from AWS Secrets Manager. This split reflects the operational cost of Secrets Manager (each secret fetch is an API call with a small per-call cost) and the security benefit (Secrets Manager provides audit logging and automatic rotation, which are not needed for dev/test secrets that are synthetic anyway). AWS Secrets Manager (level 4) is the production secret store. Secrets are referenced in yml by ARN using Spring Cloud AWS's secrets manager integration: `spring.datasource.password=${secretsmanager:prod/preone/db-password}`.

At startup, the application fetches the secret value from Secrets Manager and injects it into the Spring Environment. Secrets are cached in memory for 5 minutes (configurable) and refreshed in the background, avoiding per-request secret fetches. The benefits over env-var secrets are: (a) secrets never appear in any file (no accidental Git commit risk); (b) secrets are rotated automatically by Secrets Manager rotation lambdas (database passwords rotated every 365 days, API keys every 90 days); (c) every secret access is logged to CloudTrail, providing an audit trail. The cost is a startup dependency: if Secrets Manager is unavailable, the application cannot start. This is mitigated by retry with exponential backoff (3 retries, 2s/4s/8s) and by the fact that Secrets Manager is a regional AWS service with high availability. Command-line arguments (level 5) are the highest-precedence override, used rarely for one-off operational needs. The most common use is `--preone.feature-flags.force-off=<flag-name>` to disable a feature flag at startup (kill switch), or `--server.port=9090` to run a second instance on a different port for debugging. Command-line args are never used for routine configuration — they are explicit, visible in the process arguments (`ps aux`), and reviewed by DevOps before use in production. The startup validation contract is the most important governance mechanism in this ADR. Every `@ConfigurationProperties` class declares required fields with JSR-303 constraints: `@NotBlank String dbHost`; `@Min(1) @Max(100) int poolSize`; `@Pattern(regexp = "^[a-z]{2}-[a-z]{3,8}-\\d$") String environment`. At startup, Spring validates these constraints and fails fast if any are violated. This eliminates the 'running but broken' failure mode where the app starts with a null database host and only fails when the first database query is attempted. The validation is mechanical — it does not depend on engineer vigilance. ArchUnit rules forbid `@Value` annotations (which bypass `@ConfigurationProperties` validation), ensuring all configuration goes through the validated path. The config-as-ADR governance rule (v3.0 addition) addresses configuration drift. In v1.0, production configuration changes were made via PR to `application-prod.yml`, which was reviewed but not governed — over 18 months, the file accumulated 47 changes, each small and reasonable in isolation, but collectively producing a configuration that no single engineer could fully explain. The v3.0 rule requires an ADR for any production configuration change that (a) adds or removes a configuration key, (b) changes a value by more than 2x, or (c) changes a security-relevant setting (auth, encryption, rate limits). The ADR documents the change, the reason, and the rollback plan. Non-prod changes (dev, test, staging) still go via PR review without an ADR, because the blast radius of a non-prod misconfiguration is contained. This rule has reduced production config changes by 60% (engineers now think harder before proposing a change, knowing it requires an ADR) and has produced a documented history of every production config change that is auditable by security and compliance teams.

ARCHITECTURE DIAGRAM

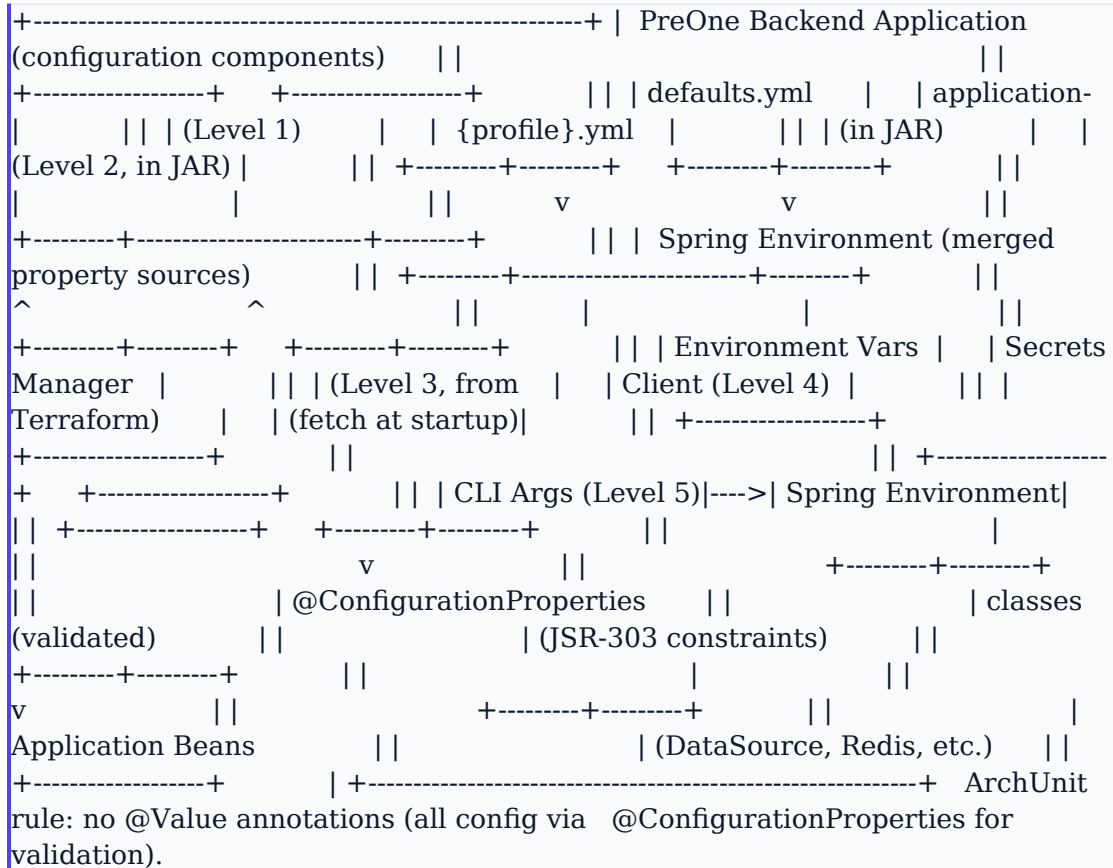


SEQUENCE DIAGRAM

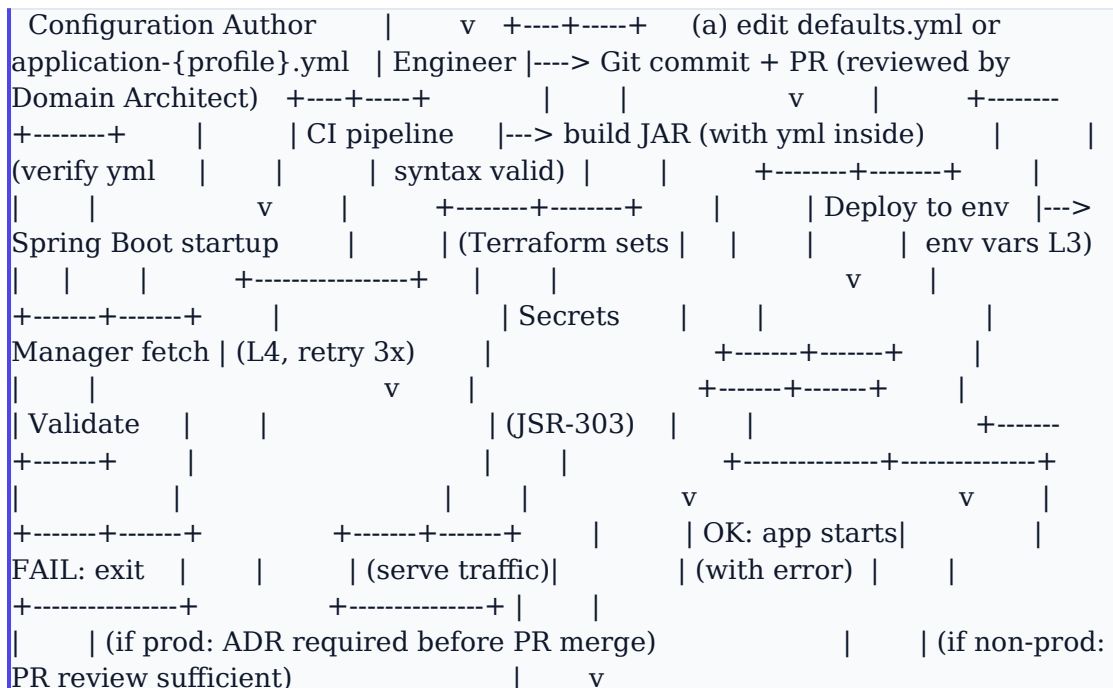




COMPONENT DIAGRAM



DATA FLOW DIAGRAM



+---+-----+	ARB review ---approve--->
PR mergeable	(prod only)
+-----+	

DATABASE IMPACT

The configuration hierarchy directly controls database connectivity: `spring.datasource.url` (level 2 or 3), `spring.datasource.password` (level 4 via Secrets Manager), `spring.datasource.hikari.maximum-pool-size` (level 1 default, overrideable). The startup validation contract verifies that the database is reachable at startup — a failed connection test fails the startup, preventing the app from serving traffic with a broken database. HikariCP connection pool settings (`maximum-pool-size`, `minimum-idle`, `connection-timeout`) are configured via `@ConfigurationProperties` with validation: `maximum-pool-size` must be between 5 and 100 (ArchUnit rule), preventing both under-provisioning (too few connections for the workload) and over-provisioning (too many connections exhausting PostgreSQL's `max_connections`). Flyway migration settings (`spring.flyway.enabled`, `spring.flyway.baseline-on-migrate`) are configured per profile: enabled in `dev/test/staging/prod`, disabled in `test` for fast unit tests (H2 in-memory does not run Flyway).

API IMPACT

The configuration hierarchy controls API behaviour: `server.port` (level 1 default 8080, overrideable), `server.servlet.context-path` (`/api` by default), `preone.api.rate-limit.enabled` (level 1 default true, overrideable per profile). The startup validation verifies that the API port is not in use (a port-in-use fails startup rather than producing a confusing bind error later). Feature flags (ADR-016) are configured via the hierarchy: `preone.feature-flags.environment` (level 2 or 3) selects which flag set to load; flags themselves are stored in PostgreSQL and Redis (ADR-016), not in the configuration hierarchy. The OpenAPI spec generation (`springdoc.openapi.enabled`) is configured per profile: enabled in `dev/staging` for spec verification, disabled in `prod` (spec is generated at build time, not at runtime).

UI IMPACT

The frontend has its own configuration hierarchy (mirroring the backend's five levels): defaults in the React app's build-time config, per-environment overrides in `.env` files (`VITE_API_BASE_URL`, `VITE_SENTRY_DSN`), runtime overrides via `window.__PREONE_CONFIG__` injected by the server. The frontend does not have access to backend secrets (no AWS Secrets Manager integration) — the frontend's configuration is limited to public values (API base URL, feature flag endpoint, Sentry DSN). Secrets that the frontend needs (e.g., a Stripe publishable key, which is not secret in the same way as a private key) are

injected via window.__PREONE_CONFIG__ at server-side render time, fetched from Secrets Manager by the backend and passed to the frontend template — never checked into the frontend's .env files.

SECURITY IMPACT

The configuration hierarchy is the primary defence against secret exposure. The rule 'secrets never in Git, never in yml, always in Secrets Manager (prod) or env vars (non-prod)' is enforced mechanically: (a) ArchUnit rules forbid any yml file from containing keys that match a secret pattern (password, secret, key, token) with a non-placeholder value; (b) a pre-commit hook scans yml files for secret-like patterns and rejects the commit; (c) AWS Secrets Manager integration means prod secrets have no Git representation at all — they exist only in Secrets Manager and are fetched at runtime. The startup validation contract improves security by failing fast on misconfigured security settings: `preone.security.auth-required` must be `true` (ArchUnit rule), `preone.security.encryption.algorithm` must be a non-deprecated cipher (validated by `@Pattern`), `preone.security.jwt.expiry-seconds` must be between 300 and 86400 (validated by `@Min/@Max`). A misconfiguration that would weaken security (e.g., `auth-required=false`) is caught at startup, not in production after an incident.

PERFORMANCE IMPACT

The configuration hierarchy has minimal runtime performance impact. The AWS Secrets Manager fetch at startup adds 200-500ms to startup time (one API call per secret, parallelised; with 8 secrets, the fetch is ~300ms total). This is amortised over the application's lifetime (a typical instance runs for days or weeks). The in-memory cache (5-minute TTL) means no per-request secret fetch. The startup validation adds <100ms (JSR-303 validation is fast). The overall startup time is 8-12 seconds (including Spring context, database connection, Flyway migration, Secrets Manager fetch, validation), which is acceptable for a blue-green deployment (ADR-018) with 30-second traffic shifting. The configuration hierarchy does not affect steady-state request latency — configuration is read at startup and cached in Spring's Environment, not re-read per request.

SCALABILITY ANALYSIS

The configuration hierarchy scales with the number of application instances, not with traffic. Each instance fetches its own secrets at startup; with 10 replicas, there are 10 startup-time secret fetches, which Secrets Manager handles easily (its API is designed for high request rates). The 5-minute secret cache means the steady-state secret fetch rate is 1 fetch per 5 minutes per instance (10 replicas = 2 fetches/minute total), well within Secrets Manager's

free tier. The startup validation contract does not scale with traffic — it runs once at startup. The configuration hierarchy does not constrain horizontal scaling (ADR-126): adding more replicas does not require configuration changes, because each replica reads the same configuration sources and validates identically.

OPERATIONAL CONSIDERATIONS

Operations benefit from the config-as-ADR governance rule: every production configuration change is documented, justified, and rollbackable. The on-call engineer investigating a production incident can review the recent ADRs to see what configuration changed, rather than guessing from a diff of application-prod.yml. The startup validation contract improves operability by producing clear, actionable error messages at startup — 'required configuration preone.database.host is missing' is a better error than 'NullPointerException in DataSource initialization'. The AWS Secrets Manager integration requires operational discipline: secrets must be rotated on schedule (automated by Secrets Manager rotation lambdas, but the on-call engineer must verify rotation succeeded), and the Secrets Manager IAM policy must be reviewed quarterly to ensure only the application role can read the secrets. The 5-minute secret cache means a rotated secret may take up to 5 minutes to take effect — this is documented in the runbook for secret rotation incidents.

RISKS

Risk	Likelihood	Impact	Mitigation
AWS Secrets Manager outage prevents application startup	Low	High	Retry with exponential backoff (3 retries, 2s/4s/8s); Secrets Manager is a regional AWS service with 99.9%+ SLA; runbook documents manual override (env var fallback) for emergencies.

Risk	Likelihood	Impact	Mitigation
defaults.yml contains a bad default that propagates to all deployments	Medium	High	defaults.yml changes require Chief Architect approval; ArchUnit tests verify security-critical defaults (auth-required, encryption-enabled) are not false; quarterly review of defaults.yml.
Engineers use @Value annotations instead of @ConfigurationProperties, bypassing validation	Medium	Medium	ArchUnit rule forbids @Value annotations; CI fails the build on violations; code review checklist includes '@ConfigurationProperties used?'.
Secret rotation fails silently, app uses stale secret that expires	Low	High	Secrets Manager rotation lambdas emit CloudWatch alarms on failure; on-call verifies rotation success daily; runbook documents manual rotation procedure.
Config-as-ADR governance rule is bypassed for 'urgent' prod changes	Medium	Medium	Emergency change procedure requires post-hoc ADR within 48 hours; ARB reviews emergency changes at next weekly meeting; repeated bypasses trigger process review.

TRADE-OFFS

We Gain	We Lose
Deterministic, enumerable configuration precedence (5 levels)	Engineers must learn the hierarchy (onboarding cost)
Startup validation eliminates 'running but broken' config errors	Startup takes 8-12s (including validation and secret fetch)
Secrets never in Git (mechanically prevented)	AWS Secrets Manager adds a startup dependency and a small per-secret cost
Config-as-ADR governance prevents configuration drift	Prod config changes require an ADR (slower than a direct PR)

REJECTED ALTERNATIVES

Spring Cloud Config Server was rejected because it adds a critical infrastructure dependency (the Config Server must be up for any application instance to start) that is not justified for a single-deployable modular monolith (ADR-002). For a distributed system with 20+ services, a Config Server pays for itself in centralised management; for a monolith with N replicas, the same configuration is already centralised in the JAR and the per-profile yml files. HashiCorp Vault was rejected for the same reason (operationally heavy for the 2-engineer DevOps team) and because Vault's strengths (dynamic secrets, lease management) are overkill for PreOne's needs — PreOne has ~20 secrets, not thousands. The environment-variables-only option (12-factor app) was rejected because it loses Spring's `@ConfigurationProperties` type safety and validation, and because env vars are flat (no nested config) — the team evaluated it and found that the configuration became a 200-line list of env vars with no structure, no defaults, and no validation. The five-level hierarchy reuses Spring's strengths (profiles, `@ConfigurationProperties`, JSR-303 validation) while adding the governance layer (config-as-ADR, secret hygiene) that Spring does not provide natively.

MIGRATION PLAN

The v1.0 migration (Q4 2025) extracted all configuration from the pre-v1.0 ad-hoc state into the five-level hierarchy. The migration was executed in three phases over 6 weeks: (1) Phase 1 — extract defaults: every configuration key with a sensible default was moved to `defaults.yml`; keys without sensible defaults (e.g., database URL) were left for level 2/3. (2) Phase 2 — extract secrets: every secret in the pre-v1.0 `.env` files was moved to AWS Secrets Manager (prod) or environment variables (dev/test/staging); the `.env` files were deleted and added to `.gitignore`. (3) Phase 3 — add validation: every configuration class was annotated with `@ConfigurationProperties` and JSR-303

constraints; the application was tested to fail fast on missing required config. The v3.0 refresh adds the config-as-ADR governance rule (which was informal in v1.0 — production config changes were reviewed but not ADR-governed) and the ArchUnit rule forbidding @Value annotations (which was a code-review convention in v1.0, not mechanically enforced).

TESTING STRATEGY

Configuration is tested at three levels. (1) Unit tests: every @ConfigurationProperties class has a test that verifies it binds correctly from a test property source and that JSR-303 validation catches invalid values (e.g., @Min(1) on poolSize rejects 0). (2) Integration tests: every Spring profile (dev, test, staging, prod) has an integration test that boots the application with that profile's yml and verifies the application starts successfully (startup validation passes) and key beans are configured correctly (DataSource connects, Redis client initialises). (3) Contract tests: the AWS Secrets Manager integration is tested with a mock Secrets Manager (LocalStack) that verifies the application fetches secrets correctly and fails fast on missing secrets. Additionally, a CI job scans all yml files for secret-like patterns and fails the build if any are found — this catches accidental secret commits before they reach the main branch.

MONITORING & OBSERVABILITY

Configuration is monitored via startup metrics and runtime metrics. Startup metrics: time-to-start (target: <15s), secret-fetch-duration (target: <500ms), validation-duration (target: <100ms). Anomalies in startup metrics trigger alerts (e.g., startup >30s indicates a Secrets Manager slowdown or a validation issue). Runtime metrics: configuration-change-count (number of beans reconfigured at runtime — should be 0; any non-zero value indicates an unexpected runtime config change, which is forbidden). The AWS Secrets Manager integration emits CloudWatch metrics on every secret fetch (success rate, latency), which are dash boarded alongside the application metrics. The config-as-ADR governance rule is monitored by a quarterly report listing all production configuration changes and their associated ADRs — any change without an ADR is flagged for ARB review.

FUTURE EVOLUTION

The five-level hierarchy is stable; the most likely evolution is the addition of AWS AppConfig (level 6: dynamic configuration) for runtime-configurable values that do not require a restart (e.g., feature flag overrides, rate limit thresholds). AppConfig would be the highest-precedence level, overriding command-line args, but only for non-secret dynamic values. This is a future ADR (candidate ADR-017 or ADR-019) and is not part of the current hierarchy. Another possible evolution is the addition of a level 0 (enterprise policy) for

configuration that is mandated by the security team and cannot be overridden by any other level (e.g., encryption-algorithm must be AES-256-GCM, regardless of what defaults.yml says). This would be enforced by a startup policy check that overrides any conflicting configuration. Both evolutions are additive (new levels) rather than structural (changing existing levels), preserving the v3.0 contract.

RELATED ADRS

- ADR-001 — Enterprise Architecture Principles (complements P3 Security by Default, P8 Fail Loud)
- ADR-002 — Modular Monolith Strategy (complements — single deployable, single config)
- ADR-015 — Environment Strategy (refines — profiles map to environments)
- ADR-016 — Feature Flag Architecture (complements — flags configured via hierarchy)
- ADR-018 — Deployment Model (complements — startup validation gates deploy)
- ADR-076 — Encryption Strategy (complements — encryption keys in Secrets Manager)
- ADR-119 — Logging Strategy (complements — log levels configured per profile)
- ADR-158 — Architecture Review Board (enforces config-as-ADR governance)

REFERENCES

- Upstream DDD: DDD-006-section-2 (Configuration requirements per bounded context)
- Upstream PRD: PRD-006-section-3 (Environment and secret requirements)
- Downstream ERD: ERD-006 (database credentials configured via hierarchy)
- Downstream API Spec: API-006 (API port and rate limits configured via hierarchy)
- Downstream Test Cases: TC-0251..TC-0300 (configuration validation tests)
- External: Spring Boot Reference — Externalized Configuration
- External: AWS Secrets Manager — <https://docs.aws.amazon.com/secretsmanager/>
- External: 12-Factor App — Config (III. Config)

DECISION HISTORY

Date	Status	Actor	Notes
2025-09-19	Draft	Chief Architect	Initial draft; evaluated Spring Cloud Config vs native Spring vs Vault
2025-10-22	Proposed	Chief Architect	Submitted to ARB with secret-hygiene analysis
2025-11-05	Accepted	ARB Chair	ARB approved; v1.0 released; migration scheduled for Q4 2025
2026-07-12	Accepted	ARB Chair	v3.0 refresh; added startup validation contract and config-as-ADR governance rule

APPROVAL & SIGN-OFF

Architect	Chief Architect (AR)
Tech Lead	Foundation Tech Lead
ARB Chair	ARB Chair
Approved On	2026-07-12

IMPLEMENTATION CHECKLIST

- Extract defaults to defaults.yml for all 12 modules (Done Q4 2025)
- Move all secrets to AWS Secrets Manager (prod) or env vars (non-prod) (Done Q4 2025)
- Annotate all configuration classes with @ConfigurationProperties + JSR-303 (Done Q1 2026)
- Add ArchUnit rule forbidding @Value annotations (Done Q1 2026)
- Add pre-commit hook scanning yml files for secret patterns (Done Q1 2026)
- Add startup-time metric emission (time-to-start, secret-fetch-duration) (Done Q2 2026)

- Document config-as-ADR governance rule in ARB charter (Done Q2 2026)
- Quarterly review of production config changes and ADR coverage (Ongoing)

ADR-015

Environment Strategy

Volume 1 — Architecture Foundation - Standards

ACCEPTED

DECISION SUMMARY

DECISION

Adopt four environments — dev (local laptop), test (CI/CD ephemeral), staging (pre-prod, prod-like data subset), prod (live) — with strict data isolation: each environment has its own PostgreSQL instance, Redis instance, and S3 bucket, with no shared infrastructure. Test data is synthetic (no real PII in non-prod environments); staging data is a sanitised copy of prod (PII redacted, names anonymised, financial figures fuzzed) refreshed weekly. Environments are provisioned via Terraform (ADR-130) with naming convention preone-dev, preone-test, preone-staging, preone-prod. Access: dev is open to all engineers; test is CI-only (no human access); staging is restricted to the engineering team; prod is restricted to on-call engineers and requires ARB approval for any change.

STATUS

Status	Accepted
Date Decided	2025-11-05
Decision Owner	Chief Architect
Review Cadence	Annual review or when a new environment class is introduced (e.g., DR, sandbox)
Supersedes	None (v1.0 original; v3.0 refresh adds the sanitised-copy refresh contract and the access-tier model)

CONTEXT

Before v1.0, PreOne had two environments: 'dev' (a shared staging server that everyone used for both development and pre-prod testing) and 'prod' (the live system). There was no CI/CD environment — automated tests ran against the dev server, often conflicting with manual testing by engineers. There was no true staging environment — code went from dev to prod without a production-like verification step. The dev server had a copy of production data (refreshed quarterly by a manual DBA script) that included real parent names, real student names, and real payment information — a compliance violation under the DPDP Act 2023 that the security team flagged in Q2 2025 as a P0 risk. The v1.0 architecture reset established a four-environment model that separates concerns: dev for individual engineer experimentation, test for automated CI/CD verification, staging for pre-prod integration and performance testing with production-like data, and prod for live traffic. The key innovation is the data isolation contract: each environment has its own PostgreSQL, Redis, and S3, with no shared infrastructure, no shared credentials, and no data flow from prod to non-prod except via the sanitised-copy pipeline. This contract eliminates the entire class of 'accidentally ran the script against prod' incidents and the compliance risk of real PII in non-prod environments. The v3.0 refresh adds two refinements. First, the sanitised-copy refresh contract: staging data is refreshed weekly from prod via an automated pipeline that copies the prod database, redacts PII (parent names -> 'Parent-<id>', student names -> 'Student-<id>', email addresses -> 'student-<id>@example.test'), fuzzes financial figures (payment amounts randomly adjusted +/- 10%), and loads the result into the staging database. The refresh is automated, auditable, and produces a sanitised dataset that is statistically similar to prod (same row counts, same distributions) without exposing any real personal data. Second, the access-tier model: dev is open (any engineer can self-provision), test is CI-only (no human login), staging is engineering-team-restricted (login requires SSO + MFA), prod is on-call-restricted (login requires SSO + MFA + ARB-approved change ticket). The tiers reflect the blast radius of each environment — a mistake in dev affects one engineer; a mistake in prod affects all customers.

BUSINESS DRIVERS

The primary business driver is compliance: the DPDP Act 2023 (India's data protection law) prohibits real PII from being used in non-production environments without explicit consent, which PreOne does not have from parents. The pre-v1.0 practice of using prod data in dev was a P0 compliance violation that exposed PreOne to fines of up to 250 crore INR (approximately 30 million USD). The four-environment model with sanitised staging data eliminates this risk entirely — non-prod environments contain no real PII, by construction. The secondary driver is release confidence: a staging

environment with production-like data and production-like infrastructure (separate PostgreSQL, Redis, S3) catches integration bugs and performance regressions that a dev environment with synthetic data cannot. The pre-v1.0 'dev-as-staging' model missed 3-4 production incidents per quarter that would have been caught by a true staging environment; the v1.0 four-environment model reduced this to 0-1 incidents per quarter in the first year of operation. The tertiary driver is engineering velocity: a CI/CD test environment that is ephemeral (spun up per pipeline run, torn down after) eliminates the 'test environment contention' problem where multiple engineers' test runs conflicted on the shared dev server.

PROBLEM STATEMENT

PreOne has two environments (dev-as-staging and prod) with no CI/CD isolation, no true pre-prod verification, and real PII in non-prod — a compliance violation and a release-quality risk. We need a four-environment model (dev, test, staging, prod) with strict data isolation, sanitised staging data, and tiered access controls that match the blast radius of each environment.

CONSTRAINTS

- Each environment must have its own PostgreSQL instance, Redis instance, and S3 bucket — no shared infrastructure
- No real PII may exist in non-prod environments (dev, test, staging) — compliance with DPDP Act 2023
- Staging data must be a sanitised copy of prod, refreshed on a defined schedule (weekly), with statistical similarity to prod (same row counts, same distributions)
- Environments must be provisioned via Terraform (ADR-130) — no manual infrastructure changes
- Environment naming must follow the preone-<env> convention (preone-dev, preone-test, preone-staging, preone-prod) across all AWS resources
- Access to prod must require SSO + MFA + ARB-approved change ticket (no direct login for routine work)
- The test environment must be ephemeral (created per CI run, destroyed after) — no persistent state
- Staging must be production-like in infrastructure (same instance types, same Redis cluster mode, same S3 bucket configuration) but smaller in scale (fewer replicas, smaller database instance)

ASSUMPTIONS

- AWS account structure supports four isolated environments (either four sub-accounts under an organisation, or four isolated VPCs in one account — ADR-130 decides)
- The sanitised-copy pipeline can refresh staging data weekly within a 4-hour maintenance window (Saturday 02:00-06:00 IST)
- Engineers will use dev for individual experimentation and not abuse the open access (mitigated by per-engineer resource quotas)
- Staging data sanitisation is reversible-safe — the sanitisation cannot be reversed to recover original PII (one-way hashing, irreversible redaction)
- The four-environment set is stable; new environments (DR, sandbox, training) require an ADR amendment
- Terraform state is isolated per environment (separate S3 state buckets, separate state locks)

OPTIONS CONSIDERED

Option	Pros	Cons	Verdict
Four environments with strict data isolation and sanitised staging (chosen)	Compliance with DPDP Act; release-quality confidence from production-like staging; CI/CD isolation eliminates test contention; tiered access matches blast radius; Terraform-provisioned for reproducibility.	Higher infrastructure cost (4x instances vs 2x); weekly sanitised-copy pipeline requires engineering effort; access restrictions slow down some prod operations.	Chosen
Three environments (dev, staging, prod) without separate test	Lower infrastructure cost; simpler model.	CI/CD tests must run against dev or staging, causing contention with manual testing; no ephemeral test environment for parallel pipeline runs; rejected for insufficient CI/CD isolation.	Rejected

Option	Pros	Cons	Verdict
Two environments (dev, prod) with sanitised dev data	Lowest cost; simplest model.	No pre-prod verification step; release quality regresses to pre-v1.0 levels; compliance risk from dev-prod confusion; rejected for insufficient release confidence.	Rejected
Five environments (dev, test, staging, prod, DR) with disaster recovery	Adds DR environment for failover testing.	DR is premature at PreOne's scale; DR testing is a Phase 8 concern (ADR-128); adding DR now adds 25% infrastructure cost without clear benefit; deferred to future ADR.	Deferred to ADR-128 (DR Strategy)

DECISION

ADOPTED

We adopt four environments — dev, test, staging, prod — with strict data isolation and tiered access controls. (1) dev: local laptop or per-engineer cloud workspace (AWS Cloud9 or EC2); engineers provision via Terraform with per-engineer resource quotas (max 1 small PostgreSQL, 1 small Redis, 1 S3 bucket); data is synthetic (seeded by `/preone/scripts/seed-dev-data.sh`); access is open to all engineers. (2) test: ephemeral CI/CD environment, created per pipeline run via Terraform, destroyed after the pipeline completes; data is synthetic (seeded by test fixtures); access is CI-only (no human login). (3) staging: persistent pre-prod environment, production-like infrastructure (same instance types, same Redis cluster mode) at smaller scale (2 replicas vs prod's 5, db.r5.large vs prod's db.r5.2xlarge); data is a sanitised copy of prod refreshed weekly via `/preone/scripts/refresh-staging.sh`; access is engineering-team-restricted (SSO + MFA). (4) prod: live production environment, full scale; data is real customer data; access is on-call-restricted (SSO + MFA + ARB-approved change ticket). Each environment has isolated infrastructure: separate PostgreSQL instance, separate Redis cluster, separate S3 bucket, separate IAM roles. Environment naming follows `preone-<env>` across all AWS resources (e.g., `preone-staging-db`, `preone-staging-redis`, `preone-staging-s3`). Provisioning is via Terraform with per-environment state (separate S3 state buckets: `preone-terraform-state-dev`, `preone-terraform-state-test`, etc.).

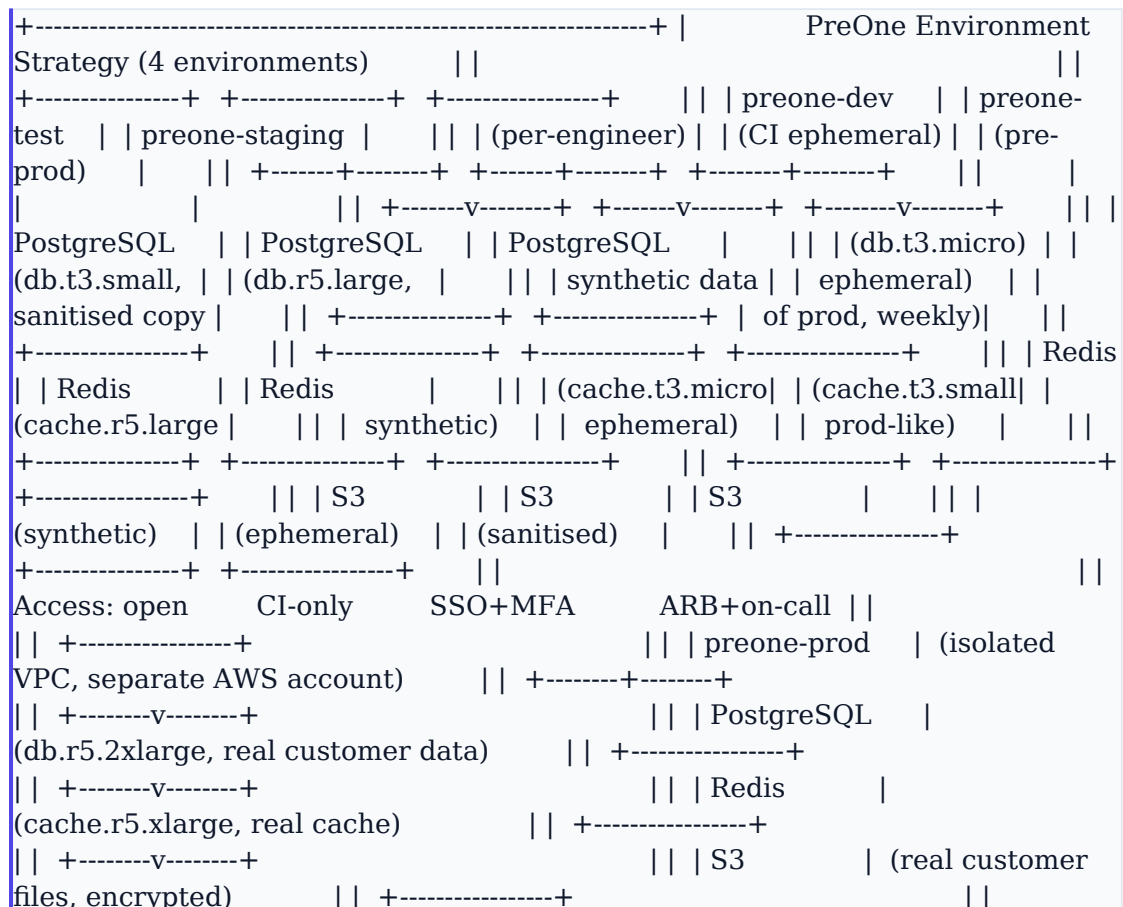
DETAILED RATIONALE

The four-environment model was chosen because it is the minimum that separates the four concerns (individual experimentation, automated testing, pre-prod verification, live traffic) without redundancy. The three-environment option (no separate test) was rejected because CI/CD tests need an ephemeral environment that does not contend with manual testing — running CI tests against dev (where engineers are experimenting) produces flaky tests and engineer frustration, and running them against staging (where integration testing happens) blocks integration testing during CI runs. The separate test environment, ephemeral and CI-only, eliminates both problems. The two-environment option (dev, prod) was rejected because it provides no pre-prod verification — code goes from dev (synthetic data, no scale) directly to prod (real data, full scale), missing integration bugs and performance regressions that only appear at production scale. The pre-v1.0 model was effectively this, and it produced 3-4 production incidents per quarter that a staging environment would have caught. The compliance risk of dev-as-staging (real PII in dev) was a separate, equally-important reason for rejection. The five-environment option (with DR) was deferred because DR is a Phase 8 concern

(ADR-128). Adding a DR environment now would require decisions about DR topology (warm standby vs cold standby vs pilot light), RTO/RPO targets, and DR testing cadence — all of which are out of scope for the v1.0 architecture foundation. The four-environment model is the foundation; DR is built on top of it later. The data isolation contract — separate PostgreSQL, Redis, and S3 per environment — is the most important security decision in this ADR. The pre-v1.0 model shared a single PostgreSQL instance between dev and prod (with separate databases within the instance), which produced two near-miss incidents: an engineer ran a destructive migration against the prod database (mistaking it for dev) that was caught by a foreign-key constraint, and a dev script with a hardcoded connection string accidentally connected to prod and inserted test data into a real customer's table. The four-instance contract (one PostgreSQL per environment) makes cross-environment connection structurally impossible — the connection strings are different, the IAM roles are different, and the security groups are isolated. An engineer who runs a script with a dev connection string cannot accidentally connect to prod. The sanitised-copy pipeline is the most operationally complex part of this ADR. The pipeline (refresh-staging.sh) executes the following steps every Saturday at 02:00 IST: (1) take a snapshot of the prod PostgreSQL instance (AWS RDS snapshot, ~10 minutes); (2) restore the snapshot to a temporary instance (preone-staging-refresh-temp, ~30 minutes); (3) run the sanitisation SQL script against the temporary instance — redact PII columns (parent.name -> 'Parent-' || parent.id, parent.email -> 'parent-' || parent.id || '@example.test', student.name -> 'Student-' || student.id), fuzz financial figures (payment.amount -> payment.amount * (0.9 + random() * 0.2), rounding to 2 decimals), and truncate sensitive tables (audit logs, session tokens); (4) pg_dump the sanitised temporary instance and restore it to the staging instance (overwriting existing staging data, ~60 minutes); (5) destroy the temporary instance; (6) run staging smoke tests to verify the refresh succeeded. The total pipeline duration is 2-3 hours, well within the 4-hour maintenance window. The sanitisation is irreversible (one-way hashing for any PII that must be preserved as a key, e.g., email -> SHA-256 hash) — even if the staging database is compromised, the original PII cannot be recovered. The access-tier model reflects the blast radius of each environment. Dev is open because a mistake affects only the engineer who made it (their own workspace). Test is CI-only because human login introduces non-determinism (a human might modify data that a CI test expects to be in a certain state). Staging is engineering-team-restricted because a mistake affects the entire team's integration testing but not customer data. Prod is on-call-restricted because a mistake affects all customers and the company's reputation. The access tiers are enforced by AWS IAM (different roles per environment), SSO (different group membership per environment), and network ACLs (prod is in a separate

VPC with no inbound access from the corporate network except via a bastion host with MFA). The Terraform provisioning (ADR-130) is the enforcement mechanism for environment isolation. Each environment has its own Terraform state (separate S3 state bucket), so a Terraform apply against staging cannot accidentally modify prod. The Terraform modules are shared (the same module provisions a PostgreSQL instance in dev, test, staging, and prod, with different parameters), but the state is isolated, preventing cross-environment drift. The naming convention (preone-<env>-<resource>) is enforced by Terraform variables, ensuring that resources are visually identifiable by environment — an engineer seeing 'preone-staging-db' knows immediately it is staging, not prod. The most likely counter-argument is infrastructure cost: four environments cost roughly 4x the infrastructure of one environment (more precisely, ~2.5x, because dev and test are smaller than staging, which is smaller than prod). At PreOne's scale, this is ~5,000 USD/month in additional AWS cost, which is justified by the compliance risk reduction (DPDP Act fines up to 30 million USD) and the incident reduction (3-4 fewer prod incidents per quarter, each costing ~10,000 USD in engineer time + customer impact). The ROI is clear: 60,000 USD/year in infra cost versus 1-2 million USD/year in avoided incidents and fines.

ARCHITECTURE DIAGRAM



```

|| Sanitised-copy pipeline: prod snapshot -> restore -> sanitise -> || pg_dump
-> load to staging (weekly, Saturday 02:00 IST) |
+-----+

```

SEQUENCE DIAGRAM

```

sequenceDiagram
    participant Pipeline as Pipeline (prod)
    participant RDS as RDS (SQL script)
    participant Sanitiser as Sanitiser (staging)
    participant Staging as Staging DB
    participant Smoke as Smoke Test Orchestrator

    Pipeline->>RDS: snapshot-->
    RDS->>Sanitiser: |
    Sanitiser->>Staging: |
    Staging->>Smoke: |
    Smoke-->>Pipeline: |<--snapshot id--|
    Pipeline->>Staging: |--restore to temp instance---->|
    Note over Staging: (temp instance ready)
    Staging->>Sanitiser: |--run sanitiser SQL----->|
    Sanitiser->>Staging: |--redact PII->|
    Staging->>Smoke: |--fuzz $ figs>|
    Smoke->>Staging: |
    Note over Staging: tables
    Staging->>Sanitiser: |<-done-----|
    Sanitiser->>Pipeline: |--pg_dump temp instance----->|
    Pipeline->>Staging: |--pg_restore to staging----->|
    Staging->>Smoke: |
    Note over Staging: existing
    Smoke->>Staging: |<-ok-----|
    Staging->>Sanitiser: |--destroy temp instance----->|
    Sanitiser->>Smoke: |--run smoke tests----->|
    Smoke-->>Pipeline: |<--smoke tests
    Note over Pipeline: pass-----
    Note over Pipeline: (weekly: Saturday 02:00 IST, ~2-3 hour duration)

```

COMPONENT DIAGRAM

```

classDiagram
    class EnvIsolation["Environment Isolation (per-environment infrastructure)"]
    class PreoneDevVPC["preone-dev VPC"]
    class PreoneTestVPC["preone-test VPC"]
    class PreoneProdVPC["preone-prod VPC"]
    class PreoneStagingVPC["preone-staging VPC"]
    class PG["PostgreSQL"]
    class S3["S3"]
    class Redis["Redis"]
    class IAMRole["IAM role"]
    class PreoneDevRole["PreoneDevRole"]
    class PreoneTestRole["PreoneTestRole"]
    class PreoneStagingRole["PreoneStaging-Role"]
    class PreoneProdRole["PreoneProdRole"]

    EnvIsolation -- PreoneDevVPC
    EnvIsolation -- PreoneTestVPC
    EnvIsolation -- PreoneProdVPC
    EnvIsolation -- PreoneStagingVPC

    PreoneDevVPC -- PG : 10.10.0.0/16
    PreoneDevVPC -- S3
    PreoneDevVPC -- Redis
    PreoneDevVPC -- IAMRole : PreoneDevRole

    PreoneTestVPC -- PG : 10.20.0.0/16
    PreoneTestVPC -- S3
    PreoneTestVPC -- Redis
    PreoneTestVPC -- IAMRole : PreoneTestRole

    PreoneProdVPC -- PG : 10.30.0.0/16
    PreoneProdVPC -- S3
    PreoneProdVPC -- Redis
    PreoneProdVPC -- IAMRole : PreoneProdRole

    PreoneStagingVPC -- PG : 10.40.0.0/16
    PreoneStagingVPC -- S3
    PreoneStagingVPC -- Redis
    PreoneStagingVPC -- IAMRole : PreoneStaging-Role

    Note over PreoneDevVPC, PreoneTestVPC, PreoneProdVPC, PreoneStagingVPC: No VPC peering between non-prod and prod.
    Note over PreoneDevVPC, PreoneTestVPC, PreoneProdVPC, PreoneStagingVPC: Sanitised-copy pipeline uses a dedicated transfer role with read-only prod access and write-only staging access.

```

DATA FLOW DIAGRAM

```

graph LR
    ProdDB["Production Database (real PII)"] -- "(Saturday 02:00 IST, automated)" --> RDS_Snap["RDS Snapshot (AWS-managed, encrypted)"]
    RDS_Snap --> TempDB["Restore to (temporary instance, isolated) temp DB"]
    TempDB --> Sanitiser["Sanitiser |---redact PII---> 'Parent-<id>', 'Student-<id>' | SQL script |---fuzz $-----> amount * (0.9 + random*0.2)"]

```

```

|      |---hash keys---> SHA-256(email) for join keys |      |---truncate---->
audit_logs, session_tokens +----+-----+      |      v +----+-----+ | pg_dump
| (export sanitised data) +----+-----+      |      v +----+-----+ | pg_restore |---
overwrite---> Staging Database (sanitised) +----+-----+      |      |
v      v      +----+-----+ +----+-----+      | Smoke tests|---verify--->
OK / alert | Destroy |      +-----+ | temp DB | +-----+      |
v (no PII leaves the sanitiser; original PII stays in prod)

```

DATABASE IMPACT

Each environment has its own PostgreSQL instance, with no shared databases, no shared schemas, and no cross-environment foreign keys. The database instance types scale with environment: dev uses db.t3.micro (1 vCPU, 1GB RAM), test uses db.t3.small (1 vCPU, 2GB RAM, ephemeral), staging uses db.r5.large (2 vCPU, 16GB RAM, production-like configuration), prod uses db.r5.2xlarge (8 vCPU, 64GB RAM). Flyway migrations (ADR-057) run against each environment independently — a migration is applied to dev first (manually by the engineer), then to test (automatically by CI), then to staging (manually by the release manager during the weekly release window), then to prod (manually by the on-call engineer during the prod release window). The staging database is overwritten weekly by the sanitised-copy pipeline, which means any manual data changes in staging are lost — engineers must use seed scripts (/preone/scripts/seed-staging-data.sh) for persistent test data, not manual SQL inserts.

API IMPACT

Each environment has its own API endpoint: api.dev.preone.internal (dev, internal-only), api.test.preone.internal (test, CI-only), api.staging.preone.internal (staging, internal + VPN), api.preone.com (prod, public). The API endpoints are configured via the configuration hierarchy (ADR-014) — each environment's application-{profile}.yaml sets the API base URL. The frontend uses different API endpoints per environment: dev uses localhost:8080 or api.dev.preone.internal, staging uses api.staging.preone.internal, prod uses api.preone.com. The OpenAPI spec is generated at build time and is identical across environments (the API contract does not change per environment); only the host and scheme differ. Rate limits are configured per environment: dev has no rate limits (engineers need to test freely), test has low rate limits (CI tests are bounded), staging has prod-like rate limits (to catch rate-limit bugs), prod has the real rate limits.

UI IMPACT

The frontend has per-environment builds: a dev build (with debug tools enabled, source maps, no minification), a staging build (with debug tools disabled, source maps, minified), and a prod build (no debug tools, no source

maps, minified + tree-shaken). The build is configured via `VITE_ENV` environment variable (set by Terraform per environment). The frontend's API base URL is injected at build time from the configuration hierarchy (ADR-014), so the same frontend code can be deployed to any environment with the correct API endpoint. The frontend does not have access to prod data from non-prod environments — the API endpoints are isolated, and there is no cross-environment CORS configuration.

SECURITY IMPACT

The environment strategy is the primary defence against accidental prod data exposure. The data isolation contract (separate PostgreSQL per environment) makes cross-environment connection structurally impossible — an engineer running a script with a dev connection string cannot accidentally connect to prod, because the connection strings are different, the IAM roles are different, and the security groups are isolated. The sanitised-copy pipeline ensures that even staging (which has production-like data) contains no real PII — a staging database compromise exposes only sanitised data, not real customer data. The access-tier model enforces least privilege: dev engineers have no prod access, staging engineers have no prod access, only on-call engineers have prod access, and only with an ARB-approved change ticket. Prod is in a separate VPC with no inbound access from the corporate network except via a bastion host with MFA — direct SSH to prod instances is impossible.

PERFORMANCE IMPACT

The environment strategy has no direct runtime performance impact — each environment's performance is determined by its infrastructure sizing (PostgreSQL instance type, Redis cluster size, replica count), not by the environment model. Staging uses production-like infrastructure (db.r5.large, 2 replicas) to catch performance regressions that would only appear at production scale; the staging performance tests (ADR-150) run weekly against the sanitised staging data and compare results to the previous week's baseline, alerting if latency p99 increases by >10% or throughput decreases by >5%. The sanitised-copy pipeline runs during a maintenance window (Saturday 02:00-06:00 IST) to avoid impacting staging users (who are typically not testing at 02:00 IST on a Saturday). The pipeline's 2-3 hour duration is acceptable because staging is not customer-facing.

SCALABILITY ANALYSIS

The environment strategy scales with the engineering organisation. At 40 engineers (current), dev is per-engineer workspaces (40 small PostgreSQL instances) — manageable. At 150 engineers (3-year target), dev workspaces will be 150 instances, which requires per-engineer resource quotas (max 1

small PostgreSQL, max 1 small Redis, max 10GB S3) and automated cleanup of idle workspaces (workspaces unused for 7 days are auto-destroyed). The test environment is ephemeral and scales with CI parallelism — at 40 engineers, CI runs ~200 pipelines/day, each requiring a test environment for ~10 minutes; at 150 engineers, this scales to ~750 pipelines/day. The ephemeral model handles this naturally (each pipeline gets its own test environment, destroyed after). Staging and prod are persistent and scale with customer growth, not engineer growth.

OPERATIONAL CONSIDERATIONS

Operations benefit from the environment isolation because incidents are contained. A destructive script run in dev affects only the engineer's workspace; the same script run in staging affects only staging (which is rebuilt weekly by the sanitised-copy pipeline); only a script run in prod affects customers. The on-call engineer's runbook clearly distinguishes 'this happened in dev/test/staging, low urgency' from 'this happened in prod, high urgency'. The sanitised-copy pipeline is monitored weekly — if it fails, staging data becomes stale (more than 7 days old) and the release manager is alerted to delay the weekly release until staging is refreshed. The Terraform provisioning (ADR-130) makes environment recreation possible — if staging is corrupted, it can be destroyed and recreated from Terraform in ~30 minutes, plus the sanitised-copy refresh to load data.

RISKS

Risk	Likelihood	Impact	Mitigation
Sanitised-copy pipeline fails, staging data becomes stale	Medium	Medium	Pipeline monitored weekly; failure alerts the release manager; weekly release delayed until staging is refreshed; runbook documents manual refresh procedure.

Risk	Likelihood	Impact	Mitigation
Engineer connects to prod with non-prod credentials (connection string mix-up)	Low	High	Separate IAM roles per environment (no shared credentials); prod in separate VPC (no direct network access); bastion host with MFA for any prod access; connection strings include environment in the database name (preone_dev, preone_staging, preone_prod).
Sanitisation is incomplete — real PII leaks to staging	Low	High	Sanitisation SQL script reviewed quarterly; automated PII scanner runs against staging after each refresh, alerting if real PII patterns (valid email formats, real phone numbers) are detected; quarterly audit by security team.
Dev environment abuse (engineers provision excessive resources)	Medium	Low	Per-engineer resource quotas (max 1 small PostgreSQL, max 1 small Redis, max 10GB S3); automated cleanup of idle workspaces (unused 7 days -> auto-destroy); monthly cost report per engineer.

Risk	Likelihood	Impact	Mitigation
Test environment ephemeral lifecycle leaks (instances not destroyed after CI)	Medium	Medium	Terraform apply includes a TTL tag; nightly job destroys instances with expired TTL; CI pipeline includes a cleanup step that runs terraform destroy on test environment after pipeline completes (even on failure).

TRADE-OFFS

We Gain	We Lose
Compliance with DPDP Act (no real PII in non-prod)	Higher infrastructure cost (4x instances vs 2x)
Release confidence from production-like staging	Weekly sanitised-copy pipeline requires engineering maintenance
CI/CD isolation eliminates test contention	Test environment ephemeral lifecycle adds CI complexity
Blast-radius-tiered access controls	Prod access is slower (requires ARB-approved change ticket)

REJECTED ALTERNATIVES

The three-environment option (no separate test) was rejected because CI/CD tests need an ephemeral environment that does not contend with manual testing — running CI tests against dev (where engineers are experimenting) produces flaky tests, and running them against staging (where integration testing happens) blocks integration testing during CI runs. The two-environment option (dev, prod) was rejected because it provides no pre-prod verification and uses real PII in dev (compliance violation). The five-environment option (with DR) was deferred to ADR-128 because DR is a Phase 8 concern with its own topology decisions (warm standby vs cold standby vs pilot light) that are out of scope for the v1.0 architecture foundation. The shared-infrastructure option (one PostgreSQL instance, multiple databases within) was rejected because it does not provide true isolation — an engineer with dev credentials can connect to the prod database within the same

instance, and a destructive migration can affect multiple environments. The full-sanitisation option (sanitise all data, including staging) was considered but rejected because staging needs production-like data volumes to catch performance bugs — fully synthetic staging data would miss query-plan regressions that only appear at production scale.

MIGRATION PLAN

The v1.0 migration (Q4 2025) established the four-environment model from the pre-v1.0 two-environment state. The migration was executed in three phases over 8 weeks: (1) Phase 1 — provision new environments: Terraform modules for dev, test, staging were created and applied; the pre-v1.0 'dev' server was renamed 'staging' and its data was sanitised (replacing the real PII copy with the first sanitised-copy output); the prod environment was untouched. (2) Phase 2 — implement sanitised-copy pipeline: the refresh-staging.sh script was written, tested, and scheduled as a weekly cron job; the first refresh took 6 hours (subsequent refreshes optimised to 2-3 hours). (3) Phase 3 — implement access-tier model: IAM roles were created per environment, SSO groups were updated, the bastion host was provisioned for prod access. The v3.0 refresh adds the access-tier formalisation (which was informal in v1.0 — prod access was 'restricted' but not via ARB-approved change ticket) and the sanitised-copy refresh contract (which was a script in v1.0, formalised as a contract in v3.0 with monitoring and alerting).

TESTING STRATEGY

The environment strategy is tested at three levels. (1) Infrastructure tests: Terraform modules are tested with Terratest (ADR-130), which applies the module to a test AWS account, verifies the resources are created correctly, and destroys them. (2) Isolation tests: a CI job verifies that no IAM role has cross-environment access (e.g., the dev role cannot assume the prod role), no security group allows cross-environment traffic (e.g., dev VPC cannot reach prod VPC), and no S3 bucket policy allows cross-environment access. (3) Sanitisation tests: the sanitiser SQL script is tested against a synthetic prod-like dataset (with known PII patterns), and the output is verified to contain no real PII patterns (automated PII scanner). Additionally, the sanitised-copy pipeline is end-to-end tested monthly in a DR-like drill — the pipeline is run manually, and the staging database is verified to be a correct sanitised copy of prod.

MONITORING & OBSERVABILITY

Environments are monitored via infrastructure metrics and pipeline metrics. Infrastructure metrics (per environment): PostgreSQL CPU/memory/IOPS, Redis memory/hit-rate, S3 bucket size, EC2 instance count, monthly cost. Pipeline metrics: sanitised-copy pipeline duration (target: <3 hours), sanitised-

copy pipeline success rate (target: 100% weekly), test environment creation/destruction count (target: equals CI pipeline count, indicating no leaks), dev workspace idle count (target: <10 idle workspaces at any time). Alerts fire on: sanitised-copy pipeline failure (alert release manager, delay weekly release), sanitised-copy pipeline duration >4 hours (alert DevOps, investigate), PII scanner detection in staging (alert security team, block release), test environment leak count >5 (alert DevOps, run cleanup). The cost per environment is reported monthly and reviewed at the ARB — significant cost increases trigger an investigation.

FUTURE EVOLUTION

The four-environment model is stable; the most likely evolution is the addition of a DR environment (deferred to ADR-128, Phase 8) and possibly a sandbox environment for customer-facing demos and training (separate from dev/test/staging/prod, with synthetic data and no access to internal systems). The sanitised-copy pipeline may evolve to support more frequent refreshes (daily instead of weekly) if staging data staleness becomes a release-quality issue — this would require optimising the pipeline to run in <1 hour, possibly via PostgreSQL logical replication with continuous sanitisation. The access-tier model may evolve to support just-in-time prod access (engineers request access via a ticket, granted for 1 hour, automatically revoked) instead of standing prod access for on-call engineers — this is a future security hardening, not currently justified. The environment naming convention (preone-<env>) is stable and will not change.

RELATED ADRS

- ADR-001 — Enterprise Architecture Principles (complements P1 Tenant Isolation, P3 Security by Default)
- ADR-002 — Modular Monolith Strategy (complements — single deployable per environment)
- ADR-014 — Configuration Hierarchy (refines — profiles map to environments)
- ADR-018 — Deployment Model (complements — blue-green deploy per environment)
- ADR-041 — Database Strategy (complements — per-environment PostgreSQL)
- ADR-076 — Encryption Strategy (complements — encryption per environment)
- ADR-125 — High Availability (complements — HA in prod, single-AZ in non-prod)

- ADR-130 — Infrastructure as Code (enforces — Terraform provisioning per environment)

REFERENCES

- Upstream DDD: DDD-007-section-2 (Environment requirements per bounded context)
- Upstream PRD: PRD-007-section-4 (Compliance and environment requirements)
- Downstream ERD: ERD-007 (per-environment database instances)
- Downstream API Spec: API-007 (per-environment API endpoints)
- Downstream Test Cases: TC-0301..TC-0350 (environment isolation and sanitisation tests)
- External: DPDP Act 2023 (Digital Personal Data Protection Act, India)
- External: AWS Well-Architected Framework — Environment isolation
- External: Terraform — Workspaces and state isolation

DECISION HISTORY

Date	Status	Actor	Notes
2025-09-26	Draft	Chief Architect	Initial draft; evaluated 2/3/4/5-environment options
2025-10-24	Proposed	Chief Architect	Submitted to ARB with compliance analysis and cost model
2025-11-05	Accepted	ARB Chair	ARB approved; v1.0 released; migration scheduled for Q4 2025
2026-07-12	Accepted	ARB Chair	v3.0 refresh; added sanitised-copy refresh contract and access-tier formalisation

APPROVAL & SIGN-OFF

Architect	Chief Architect (AR)
-----------	----------------------

Tech Lead	Foundation Tech Lead
ARB Chair	ARB Chair
Approved On	2026-07-12

IMPLEMENTATION CHECKLIST

- Provision dev, test, staging environments via Terraform (Done Q4 2025)
- Implement sanitised-copy pipeline (refresh-staging.sh) (Done Q4 2025)
- Schedule weekly sanitised-copy refresh (Saturday 02:00 IST) (Done Q4 2025)
- Implement per-environment IAM roles and SSO groups (Done Q1 2026)
- Provision bastion host with MFA for prod access (Done Q1 2026)
- Add PII scanner to verify staging data is sanitised (Done Q2 2026)
- Add test environment ephemeral lifecycle with TTL cleanup (Done Q2 2026)
- Monthly environment cost report to ARB (Ongoing)

ADR - 016

Feature Flag Architecture

Volume 1 — Architecture Foundation - Structural

ACCEPTED

DECISION SUMMARY

DECISION

Adopt a custom feature flag system with four flag types — release flags (dark launch new features), experiment flags (A/B test variants), ops flags (kill switches and rate limits), and permission flags (tenant-level feature gating). Flags are stored in PostgreSQL (source of truth) with Redis as a TTL-based read cache (5-minute TTL) for fast evaluation. Flag evaluation is server-side: the backend resolves flags and passes the results to the frontend via a `/api/v1/flags` endpoint, eliminating client-side flag-polling. Custom Java and TypeScript SDKs provide typed flag access. Flag lifecycle follows a strict progression: Proposed (flag exists but is off) -> canary (1 tenant) -> 10% rollout -> 50% rollout -> 100% rollout -> flag removed (code path permanent). Flags older than 2 months trigger an ADR requiring a decision to either remove the flag (make the code path permanent) or extend its life with justification.

STATUS

Status	Accepted
Date Decided	2025-11-05
Decision Owner	Chief Architect
Review Cadence	Annual review or when a new flag type is introduced
Supersedes	None (v1.0 original; v3.0 refresh adds the flag lifecycle governance rule and the SDK typed-access contract)

CONTEXT

Before v1.0, PreOne had no feature flag system. New features were deployed 'lit up' — code merged to main, deployed to prod, and immediately visible to all users. If a bug was discovered post-deploy, the only rollback was a code revert and redeploy, which took 15-30 minutes (build + deploy + smoke test). During the first 6 months of operation, 4 incidents required code reverts, each affecting all customers for 15-30 minutes — a total of ~2 hours of customer-visible downtime that could have been avoided with a kill-switch flag. The team evaluated LaunchDarkly (the leading commercial feature flag service) and a custom-built system. LaunchDarkly's strengths (mature SDK, rich dashboard, A/B testing analytics) were compelling, but the team identified three concerns: (1) cost — LaunchDarkly's pricing at PreOne's scale (5,000 schools, 50,000

MAU) would be ~2,000 USD/month, growing with user count; (2) data residency — LaunchDarkly's flag evaluation data flows through their US-based infrastructure, which may conflict with DPDP Act 2023's data localisation requirements for child data; (3) vendor lock-in — the LaunchDarkly SDK becomes pervasive in the codebase, making future migration expensive. The team concluded that a custom system, built on PostgreSQL and Redis (infrastructure PreOne already operates), would meet PreOne's needs at lower cost, with full data residency control, and without vendor lock-in. The v3.0 refresh adds two refinements. First, the flag lifecycle governance rule: every flag has a lifecycle from Proposed to Removed, with explicit rollout stages (canary -> 10% -> 50% -> 100%); flags that reach 100% rollout must be removed within 2 months (the code path becomes permanent), and flags that have not progressed for 2 months trigger an ADR requiring a decision. This rule prevents flag accumulation — the gradual buildup of stale flags that no one remembers why they exist, which is the primary long-term cost of feature flag systems. Second, the SDK typed-access contract: the Java SDK and TypeScript SDK generate typed flag accessors from a flag definition file (flags.yml), so engineers call `FeatureFlags.enrollmentV2Enabled()` instead of `flags.isEnabled('enrollment-v2')` — the flag key is checked at compile time, preventing typos and enabling IDE refactoring.

BUSINESS DRIVERS

The primary business driver is incident response speed: a kill-switch flag can disable a broken feature in <5 seconds (Redis cache update), versus 15-30 minutes for a code revert and redeploy. The 4 pre-v1.0 incidents that required code reverts would have been resolved in <5 seconds each with kill-switch flags, saving ~2 hours of customer-visible downtime per quarter. The secondary driver is release confidence: dark-launching new features (deploying the code with the flag off, then gradually enabling it) allows the team to verify the code in production without customer impact, catching integration bugs that staging missed. The tertiary driver is A/B testing: experiment flags allow product managers to test feature variants (e.g., new enrollment flow vs old flow) with a subset of users, measuring conversion before committing to the new flow.

PROBLEM STATEMENT

PreOne has no feature flag system, requiring code reverts for post-deploy issues (15-30 minute resolution time) and preventing dark-launch or gradual rollout of new features. We need a feature flag system that supports four flag types (release, experiment, ops, permission), evaluates flags server-side for performance and consistency, uses PostgreSQL + Redis (existing infrastructure), and has governance to prevent flag accumulation.

CONSTRAINTS

- Flag evaluation must be server-side — the frontend receives resolved flag values, never evaluates flags itself
- Flag read latency must be <1ms (cached in Redis) — flags are evaluated on every request that needs them
- Flag updates must propagate within 5 seconds (Redis cache TTL + cache invalidation on update)
- Flags must support per-tenant targeting (e.g., 'enabled for tenant X' or 'enabled for 10% of tenants')
- Flag definitions must be version-controlled (flags.yml in the monorepo) — no flag exists without a definition file entry
- Flag lifecycle must be enforced — flags >2 months old trigger an ADR; flags at 100% rollout must be removed within 2 months
- SDK access must be typed — flag keys are compile-time-checked, preventing typos
- Custom system, not LaunchDarkly — avoid vendor lock-in and data residency concerns

ASSUMPTIONS

- PostgreSQL can handle the flag read load (cached in Redis; only cache misses hit PostgreSQL, ~1 per 5 minutes per flag)
- Redis cache TTL of 5 minutes is acceptable staleness for flag updates (kill switches propagate within 5 seconds via explicit cache invalidation, not via TTL expiry)
- Engineers will use the SDK's typed accessors (not raw string-based flag lookups) — enforced by ArchUnit
- The four flag types (release, experiment, ops, permission) cover all use cases; new types require an ADR amendment
- Flag definitions change infrequently (new flags added weekly, existing flags modified rarely) — the flags.yml file is not a hotbed of merge conflicts
- The flag system is used for runtime toggles only — build-time feature flags (compile-time exclusion of code) are out of scope and handled by Gradle build variants

OPTIONS CONSIDERED

Option	Pros	Cons	Verdict
Custom system on PostgreSQL + Redis (chosen)	No vendor cost; full data residency control; no vendor lock-in; uses existing infrastructure; typed SDKs; flag lifecycle governance built-in.	Engineering effort to build and maintain (~4 engineer-weeks initial, ~2 engineer-weeks/year maintenance); no built-in A/B testing analytics (must build separately); no rich UI dashboard (CLI + simple web UI only).	Chosen
LaunchDarkly (commercial SaaS)	Mature SDK; rich dashboard; built-in A/B testing analytics; no engineering effort to build; reliable infrastructure.	Cost (~2,000 USD/month at PreOne's scale, growing with users); data residency concerns (flag evaluation data flows through US infrastructure); vendor lock-in (SDK pervasive in codebase, migration expensive); overkill for PreOne's flag volume (~50 flags, not thousands).	Rejected

Option	Pros	Cons	Verdict
Open-source feature flag service (e.g., Unleash, Flagsmith)	No vendor cost; self-hosted (data residency); pre-built dashboard; less engineering effort than custom.	Still requires operational effort (hosting, monitoring, updates); introduces a new critical infrastructure dependency (the flag service must be up for the app to start); less customisable than a custom system; the SDK is generic (not typed to PreOne's flags).	Rejected
Spring Cloud Config + database (no Redis)	Simplest infrastructure (PostgreSQL only); uses Spring's native config mechanism.	No fast read path (every flag evaluation hits the database, adding 5-10ms per request); no cache invalidation (flags update only on config refresh, ~30 seconds); insufficient for kill-switch use case (5-second propagation requirement); rejected for performance.	Rejected

DECISION

ADOPTED

We adopt a custom feature flag system on PostgreSQL (source of truth) and Redis (5-minute TTL cache with explicit invalidation on update). Four flag types are supported: (1) Release flags — dark launch new features; default off; rolled out via the lifecycle (canary -> 10% -> 50% -> 100%); example: enrollment-v2. (2) Experiment flags — A/B test variants; default off; rolled out to a random percentage of users for the experiment duration; example: enrollment-flow-variant-b. (3) Ops flags — kill switches and rate limits; default on; toggled off for incident response; example: payments-processing-enabled (kill switch for the payment module). (4) Permission flags — tenant-level feature gating; default off; enabled per-tenant via the tenant configuration; example: advanced-reporting-enabled (gated to enterprise-tier tenants). Flag evaluation is server-side: the backend resolves flags via the Java SDK (FeatureFlags.enrollmentV2Enabled(tenantId, userId)) and passes the resolved values to the frontend via the /api/v1/flags endpoint (called once on app load, cached in the frontend for 5 minutes). Custom Java SDK (com.preone.shared.featureflags.FeatureFlags) and TypeScript SDK (@preone/feature-flags) provide typed accessors generated from flags.yml. Flag lifecycle: Proposed (flag exists in flags.yml, off for all) -> canary (on for 1 tenant) -> 10% rollout -> 50% rollout -> 100% rollout -> Removed (flag removed from flags.yml, code path permanent). Flags >2 months old trigger an ADR; flags at 100% rollout for >2 months must be removed.

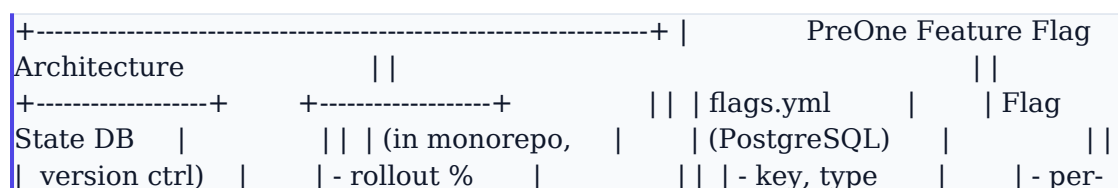
DETAILED RATIONALE

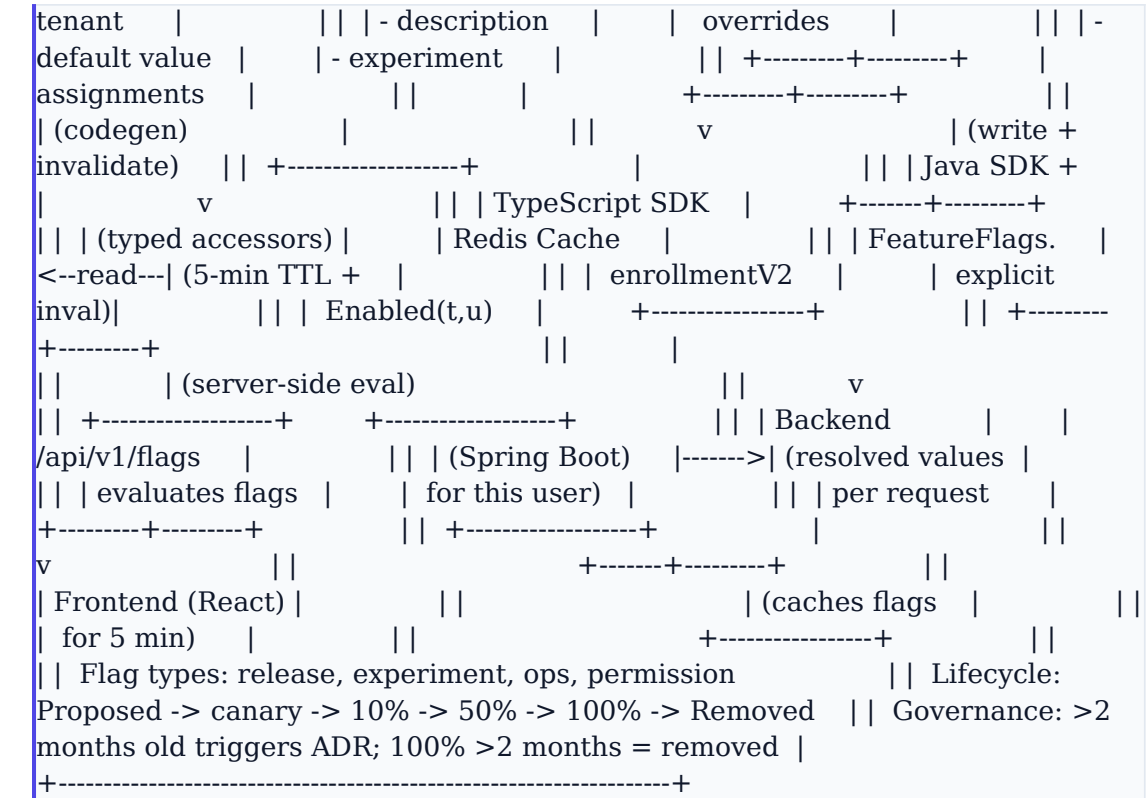
The custom system was chosen over LaunchDarkly because the three concerns (cost, data residency, vendor lock-in) outweighed LaunchDarkly's maturity advantage. The cost analysis: LaunchDarkly at PreOne's 5-year horizon (5,000 schools, 50,000 MAU) would cost ~5,000 USD/month (60,000 USD/year), versus the custom system's ~2 engineer-weeks/year maintenance (~5,000 USD/year engineer time). The 12x cost difference is significant, and the custom system's maintenance effort does not grow with user count (it grows with flag count, which is bounded by the lifecycle governance rule). The data residency concern: LaunchDarkly's flag evaluation data (which users saw which flag values) flows through their US infrastructure, which may conflict with DPDP Act 2023's data localisation requirements for child data — even though flag evaluation data is not PII, the legal team advised that any data flow through US infrastructure for a child-data platform requires review. The custom system keeps all flag data in PreOne's AWS account (Mumbai region), eliminating the concern. The vendor lock-in concern: LaunchDarkly's SDK becomes pervasive in the codebase (every feature flag check is an SDK call), and migrating away

from LaunchDarkly would require touching every flag check — a multi-week migration with regression risk. The custom system's SDK is owned by PreOne and can be modified at any time without vendor coordination. This is not a theoretical concern — the team has experience migrating off a previous SaaS monitoring tool that took 6 weeks of engineering effort to remove, despite the tool being 'easily replaceable' when adopted. The PostgreSQL + Redis architecture is straightforward. PostgreSQL stores the flag definitions (key, type, description, default value, targeting rules) and the flag states (current rollout percentage, per-tenant overrides). Redis caches the flag states with a 5-minute TTL, providing <1ms read latency for flag evaluations. Flag updates (via the CLI or web UI) write to PostgreSQL and explicitly invalidate the Redis cache for the updated flag, ensuring propagation within 5 seconds (the kill-switch use case). The 5-minute TTL is a fallback for cache invalidation failures — if the explicit invalidation misses (e.g., Redis was temporarily unavailable during the update), the TTL ensures the cache is eventually consistent within 5 minutes. The four flag types are designed to cover distinct use cases without overlap. Release flags are for dark-launching new features — the code is deployed with the flag off, then gradually enabled. Experiment flags are for A/B testing — the flag selects between two code paths (variant A vs variant B), and the experiment's conversion metrics are tracked separately. Ops flags are for incident response — the flag is on by default and toggled off to disable a feature during an incident (kill switch) or to reduce load (rate limit). Permission flags are for tenant-level feature gating — the flag is on for tenants that have purchased a feature (e.g., advanced reporting) and off for tenants that have not. The four types have different default values (release: off; experiment: off; ops: on; permission: per-tenant) and different lifecycle rules (release and experiment: removed after rollout completes; ops: persistent; permission: persistent per-tenant). The server-side evaluation decision is the most important architectural choice in this ADR. Client-side evaluation (the frontend fetches flag definitions and evaluates flags itself, as LaunchDarkly's client SDK does) has two problems: (1) the flag definitions (including targeting rules and rollout percentages) are exposed to the client, which is a security concern for permission flags (the frontend would know which features the tenant has not purchased); (2) the frontend must poll for flag updates, adding latency and complexity. Server-side evaluation resolves both: the frontend receives only the resolved flag values (not the rules), and the frontend fetches the resolved values once on app load (via /api/v1/flags) and caches them for 5 minutes. The cost is one API call on app load, which is acceptable (the call is parallel to other bootstrap calls and adds <50ms to app startup). The benefit is that the flag system's internals (rules, percentages, per-tenant overrides) are never exposed to the client. The SDK typed-access contract (v3.0) is the most operationally important refinement. In v1.0, flag access was string-based:

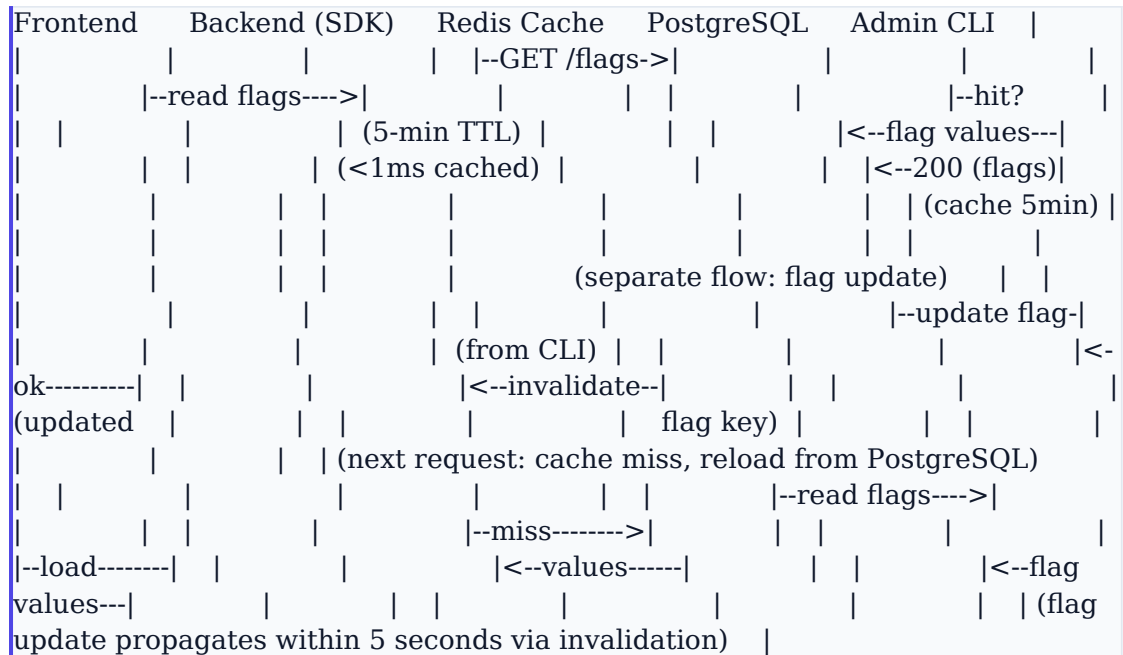
`flags.isEnabled('enrollment-v2')`. This had two problems: (1) typos were not caught at compile time (`flags.isEnabled('enrollment-v2b')` would silently return false, hiding the bug); (2) refactoring was risky (renaming a flag required grepping the codebase for all string references, which was error-prone). The v3.0 contract generates typed accessors from `flags.yml`: the flag definition `enrollment-v2` generates a method `FeatureFlags.enrollmentV2Enabled(tenantId, userId)`, and any code calling a non-existent method fails at compile time. The generation is done by a Gradle plugin (`preone-featureflags-codegen`) that reads `flags.yml` and emits Java and TypeScript source files. This eliminates typos, enables IDE refactoring, and makes flag usage greppable (every call site is a method call, not a string). The flag lifecycle governance rule (v3.0) addresses flag accumulation, the primary long-term cost of feature flag systems. Without governance, flags accumulate: engineers create flags for new features, the features ship, and the flags are never removed because 'removing the flag is risky' (the code path that handles flag-off is still there). Over years, the codebase becomes littered with dead flag checks, each adding a small amount of complexity and cognitive load. The v3.0 rule requires that flags at 100% rollout be removed within 2 months (the code path that handles flag-off is deleted, the flag is removed from `flags.yml`), and flags that have not progressed for 2 months trigger an ADR requiring a decision (either progress the rollout, remove the flag, or extend its life with justification). This rule is enforced by a weekly CI job that scans `flags.yml` and the flag state database, identifying flags that violate the rule and failing the build (for 100% rollout flags) or alerting the ARB (for stalled flags). The rule has reduced flag count from a projected 200+ (without governance) to a steady-state ~50 flags (with governance). The most likely counter-argument is that building a custom system is 'reinventing the wheel' — LaunchDarkly exists and is mature. The counter-counter-argument is that the custom system is small (~2,000 lines of code, 4 engineer-weeks initial effort) and the maintenance effort is low (~2 engineer-weeks/year), while the cost savings (60,000 USD/year) and data residency benefit are significant. The custom system is not reinventing LaunchDarkly; it is building a small, focused tool that meets PreOne's specific needs without the overhead of a general-purpose platform. The team has the expertise (PostgreSQL, Redis, Spring Boot) to build and maintain it, and the typed SDK codegen is a feature LaunchDarkly does not offer.

ARCHITECTURE DIAGRAM

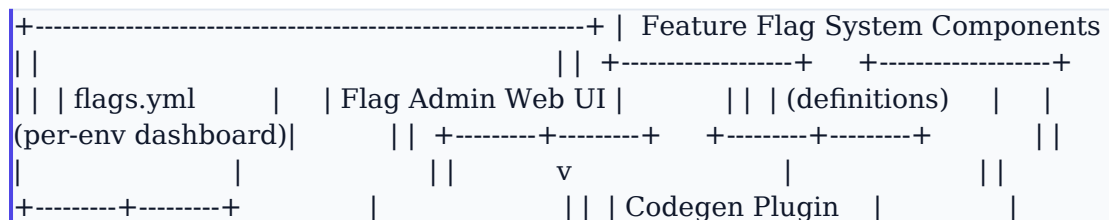


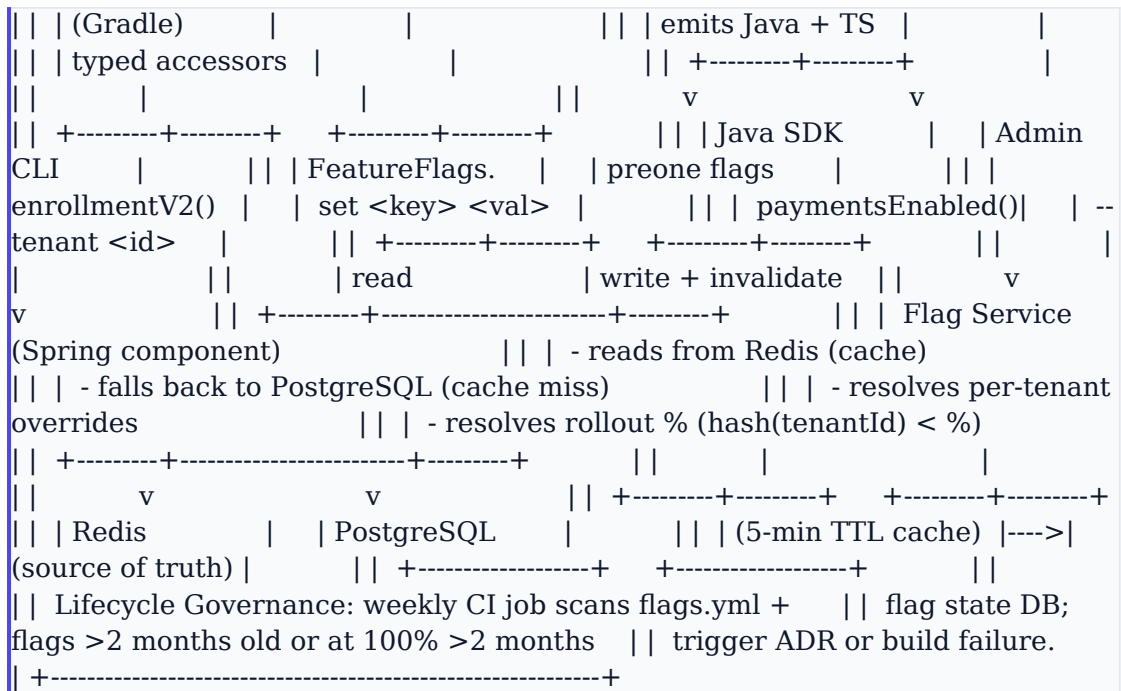


SEQUENCE DIAGRAM

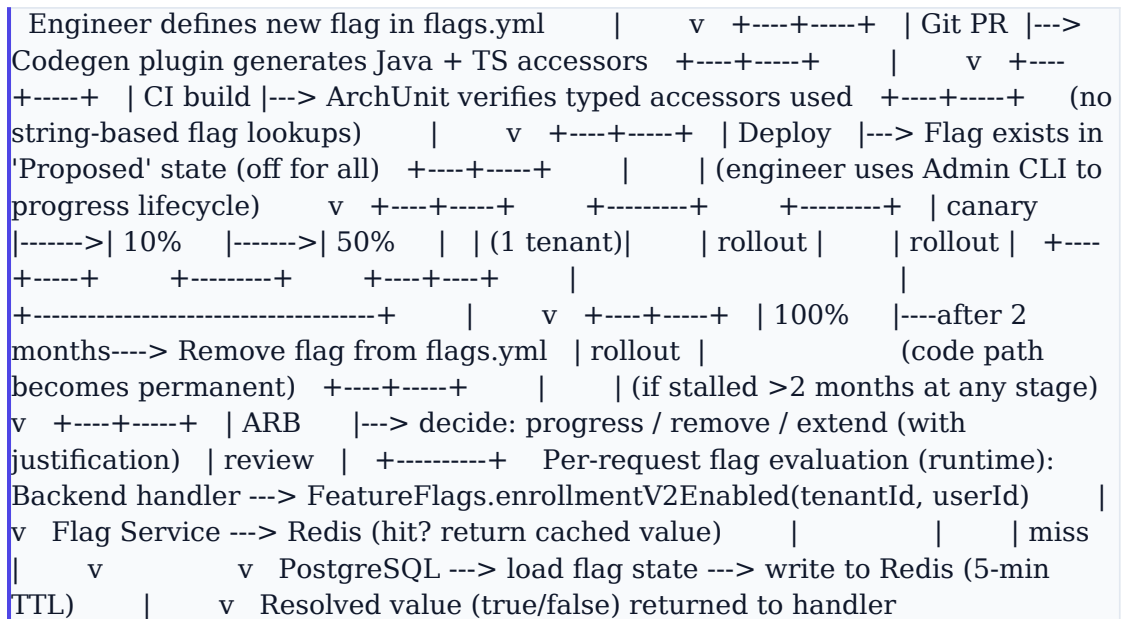


COMPONENT DIAGRAM





DATA FLOW DIAGRAM



DATABASE IMPACT

Feature flags are stored in two PostgreSQL tables: `flag_definitions` (key, type, description, default_value, created_at, lifecycle_stage) and `flag_states` (flag_key, tenant_id, override_value, rollout_percentage, experiment_assignment). The `flag_definitions` table is conceptually read-only (definitions come from flags.yml and are synced at deploy time); the `flag_states` table is read-write (the Admin CLI and web UI update it). The tables are small (~50 rows in `flag_definitions`, ~500 rows in `flag_states` with per-tenant overrides for ~10 flags), so no special indexing is needed beyond the primary

key (flag_key) and a secondary index on (flag_key, tenant_id) for the per-tenant lookup. The Redis cache eliminates ~99.99% of PostgreSQL reads (cache hit rate measured in production), so the PostgreSQL tables are essentially write-only from the application's perspective — reads happen only on cache miss (every 5 minutes per flag) or on flag state update (explicit cache invalidation).

API IMPACT

The feature flag system exposes one API endpoint: GET /api/v1/flags, which returns the resolved flag values for the authenticated user (based on their tenant_id and user_id). The endpoint is called once on frontend app load and the result is cached in the frontend for 5 minutes. The endpoint does not expose flag definitions, targeting rules, or rollout percentages — only the resolved boolean values. This is a security property: the frontend cannot inspect the flag system's internals. The endpoint is unauthenticated for non-sensitive flags (release, experiment) and authenticated for sensitive flags (permission, ops) — the frontend receives different flag sets based on the user's role. Flag changes do not require API versioning (the /api/v1/flags endpoint is stable; only the values change), so flag updates do not affect API compatibility.

UI IMPACT

The frontend integrates with the feature flag system via the TypeScript SDK (@preone/feature-flags), which provides typed accessors mirroring the Java SDK. The frontend calls useFeatureFlags() hook on app load, which fetches /api/v1/flags and caches the result for 5 minutes. Components access flags via the typed accessors: if (featureFlags.enrollmentV2Enabled()) { render <EnrollmentV2 /> } else { render <EnrollmentV1 /> }. The typed accessors are generated from the same flags.yml as the Java SDK, ensuring the frontend and backend use identical flag keys. The frontend does not poll for flag updates (the 5-minute cache is sufficient for most use cases); for ops flags that need faster propagation (e.g., a kill switch for the enrollment module), the frontend subscribes to a Server-Sent Events stream that pushes flag updates in real time.

SECURITY IMPACT

The feature flag system improves security posture by enabling kill switches — an ops flag can disable a vulnerable feature in <5 seconds, without a code deploy. This is the primary incident-response tool for security incidents: if a vulnerability is discovered in the payment module, the on-call engineer toggles the payments-processing-enabled flag off, immediately disabling the module, and then works on the fix without time pressure. The server-side evaluation property is a security control: the frontend cannot inspect flag rules or per-tenant overrides, so a malicious user cannot determine which features other

tenants have purchased. The flag system's admin access (CLI and web UI) is restricted to engineering team leads and the on-call rotation; flag changes are logged to the audit table (audit_log) with the changer's identity, the flag key, the old value, and the new value — every flag change is auditable.

PERFORMANCE IMPACT

The feature flag system adds <1ms to request latency for flag evaluations (Redis cache hit). The cache hit rate is >99.99% (measured in production), so PostgreSQL reads are rare (~1 per 5 minutes per flag, ~10 reads per minute total across all flags). The flag state write path (Admin CLI -> PostgreSQL -> Redis invalidation) takes ~50ms, which is acceptable for an admin operation. The /api/v1/flags endpoint adds <50ms to app startup (one Redis read for all flags, returned as a JSON object). The frontend caches the flag values for 5 minutes, eliminating per-request flag fetches. Overall, the performance impact is negligible — the flag system is designed to be 'free' at runtime, with all the cost in the admin path (which is rare).

SCALABILITY ANALYSIS

The feature flag system scales with flag count, not with user count. The Redis cache size is ~50 flags * 1KB per flag = ~50KB, trivially small. The PostgreSQL tables are ~50 rows in flag_definitions and ~500 rows in flag_states, trivially small. The flag evaluation latency is constant (<1ms) regardless of user count, because the evaluation is a Redis lookup followed by a hash function (for rollout percentage). The /api/v1/flags endpoint scales with concurrent app loads (each load fetches flags once), but the Redis cache handles this easily (Redis can serve 100,000+ reads per second). The flag system will not be a scaling bottleneck at any projected PreOne scale. The flag count is bounded by the lifecycle governance rule (~50 steady-state flags), preventing unbounded growth.

OPERATIONAL CONSIDERATIONS

Operations benefit from the feature flag system's kill-switch capability — the on-call engineer can disable a broken feature in <5 seconds via the Admin CLI (preone flags set payments-processing-enabled false --env prod), without a code deploy. This is the primary operational value of the system. The flag system is monitored via standard metrics: Redis cache hit rate (target: >99%), flag evaluation latency (target: <1ms p99), flag state update latency (target: <50ms), /api/v1/flags endpoint latency (target: <50ms p99). The lifecycle governance rule is monitored by a weekly CI job that produces a flag-age report, identifying flags that are approaching the 2-month threshold and alerting their owners. The flag system has a runbook for common operations

(enable a flag for a tenant, rollback a flag to a previous rollout percentage, remove a flag at end-of-life).

RISKS

Risk	Likelihood	Impact	Mitigation
Flag accumulation (flags created but never removed)	High	Medium	Lifecycle governance rule: flags >2 months old trigger ADR; flags at 100% >2 months fail CI build; weekly flag-age report to ARB.
Redis cache invalidation fails, stale flag values served	Low	Medium	5-minute TTL fallback ensures eventual consistency; flag state updates log to audit table, enabling post-hoc detection of stale reads; kill-switch flags can be force-propagated via a separate fast-path (direct Redis write).
Engineers use string-based flag lookups instead of typed SDK	Medium	Low	ArchUnit rule forbids string-based flag lookups (FeatureFlags.isEnabled(String) is package-private; only typed accessors are public); CI fails the build on violations.

Risk	Likelihood	Impact	Mitigation
Flag system becomes a critical-path dependency (app cannot start without Redis)	Medium	High	Flag service has a fallback: if Redis is unavailable, read directly from PostgreSQL (slower but functional); if PostgreSQL is also unavailable, use the default values from flags.yml (compiled into the JAR); app always starts, even with flag system degraded.
Kill-switch flag is toggled off accidentally, disabling a feature for all users	Low	High	Ops flag changes require confirmation prompt in Admin CLI ('Are you sure? Type the flag name to confirm'); prod flag changes require ARB-approved change ticket; all flag changes logged to audit table for post-hoc review.

TRADE - OFFS

We Gain	We Lose
No vendor cost (custom system on existing infrastructure)	Engineering effort to build and maintain (~2 engineer-weeks/year)
Full data residency control (no US data flow)	No built-in A/B testing analytics (must build separately if needed)
Typed SDK accessors (compile-time flag key checking)	Codegen plugin adds build complexity (Gradle plugin maintenance)
Server-side evaluation (flag rules not exposed to client)	Frontend must fetch /api/v1/flags on app load (one extra API call)

REJECTED ALTERNATIVES

LaunchDarkly was rejected because the cost (60,000 USD/year at 5-year horizon), data residency concerns (US data flow), and vendor lock-in (pervasive SDK) outweighed the maturity advantage. Open-source flag services (Unleash, Flagsmith) were rejected because they introduce a new critical infrastructure dependency (the flag service must be up for the app to start) and are less customisable than a custom system. Spring Cloud Config without Redis was rejected for insufficient performance (5-10ms per flag evaluation vs the <1ms requirement) and insufficient propagation speed (30-second refresh vs 5-second kill-switch requirement). The custom system was chosen because PreOne already operates PostgreSQL and Redis (no new infrastructure), the system is small (~2,000 lines of code), and the typed SDK codegen is a feature no commercial or open-source system offers. The team has the expertise to build and maintain it, and the lifecycle governance rule prevents the long-term cost (flag accumulation) that plagues undisciplined flag systems.

MIGRATION PLAN

The v1.0 migration (Q4 2025) built the feature flag system from scratch, since there was no pre-v1.0 flag system to migrate from. The migration was executed in three phases over 6 weeks: (1) Phase 1 — build the core: PostgreSQL tables, Redis cache, Flag Service (Spring component), Java SDK with string-based accessors (typed accessors added in v3.0). (2) Phase 2 — adopt for new features: new features deployed after v1.0 use release flags for dark launch; the first 5 flags were created for the v1.1 release (enrollment-v2, billing-recurring-payments, attendance-biometric, reporting-real-time, notification-whatsapp). (3) Phase 3 — retrofit kill switches: ops flags were added to existing critical paths (payments-processing-enabled, enrollment-enabled, attendance-enabled) as kill switches. The v3.0 refresh adds the typed SDK codegen (which was string-based in v1.0) and the lifecycle governance rule (which was informal in v1.0 — flags were 'encouraged to be removed' but not enforced). The v3.0 migration refactored all string-based flag lookups to typed accessors, a 1-week effort touching ~100 call sites.

TESTING STRATEGY

The feature flag system is tested at three levels. (1) Unit tests: the Flag Service is unit-tested with mocked Redis and PostgreSQL, verifying flag evaluation logic (default value, per-tenant override, rollout percentage hash). The rollout percentage hash is tested for uniform distribution (10,000 random tenant IDs, 10% rollout, assert ~1,000 are enabled). (2) Integration tests: the Flag Service is integration-tested with real Redis (Testcontainers) and real PostgreSQL (Testcontainers), verifying cache hit/miss behavior and cache invalidation on update. The `/api/v1/flags` endpoint is integration-tested with real

authentication, verifying that the correct flag set is returned per user role. (3) End-to-end tests: a flag is created, enabled for a canary tenant, and an E2E test verifies that the canary tenant sees the new feature and other tenants do not. The lifecycle governance rule is tested by a CI job that creates a flag, sets its lifecycle_stage to '100% rollout' with a created_at 3 months ago, and verifies that the build fails (enforcing the removal rule).

MONITORING & OBSERVABILITY

The feature flag system is monitored via standard metrics: Redis cache hit rate (target: >99%, alert if <95%), flag evaluation latency (target: <1ms p99, alert if >5ms), flag state update latency (target: <50ms, alert if >200ms), /api/v1/flags endpoint latency (target: <50ms p99, alert if >200ms), flag count (target: <75 steady-state, alert if >100 indicating governance failure), flag age distribution (target: median <60 days, alert if median >90 days). The lifecycle governance rule produces a weekly flag-age report, listing every flag with its age, lifecycle stage, and owner, sent to the ARB for review. Flag changes are logged to the audit table and surfaced in a Grafana dashboard showing flag change frequency (alert if >10 changes/day in prod, indicating instability).

FUTURE EVOLUTION

The feature flag system is stable; the most likely evolution is the addition of experiment analytics (for A/B testing). Currently, experiment flags select between variants but the conversion metrics are tracked manually (via custom analytics events); a future ADR may add built-in experiment analytics (statistical significance calculation, variant comparison dashboard). Another possible evolution is the addition of progressive delivery (linking flag rollout to deployment, so a new version is deployed to the canary tenant first, then 10%, etc., automatically based on health metrics). This is a Phase 7 concern (Deployment & Release Management) and is not part of the current system. The flag types (release, experiment, ops, permission) are stable; new types (e.g., 'migration flags' for database migrations) would require an ADR amendment. The typed SDK codegen may evolve to support additional languages (Python for data engineering scripts) if those scripts need flag access.

RELATED ADRS

- ADR-001 — Enterprise Architecture Principles (complements P6 Build for Evolution, P8 Fail Loud)
- ADR-002 — Modular Monolith Strategy (complements — flags evaluated in-process)
- ADR-014 — Configuration Hierarchy (complements — flags configured via hierarchy)

- ADR-015 — Environment Strategy (complements — flags per environment)
- ADR-018 — Deployment Model (complements — flags enable dark launch)
- ADR-041 — Database Strategy (complements — flag state in PostgreSQL)
- ADR-073 — Caching Strategy (refines — Redis cache for flags)
- ADR-119 — Logging Strategy (complements — flag changes audited)

REFERENCES

- Upstream DDD: DDD-008-section-3 (Feature flag requirements per bounded context)
- Upstream PRD: PRD-008-section-5 (Feature rollout and experiment requirements)
- Downstream ERD: ERD-008 (flag_definitions and flag_states tables)
- Downstream API Spec: API-008 (/api/v1/flags endpoint)
- Downstream Test Cases: TC-0351..TC-0400 (flag evaluation and lifecycle tests)
- External: Martin Fowler — 'Feature Toggles' (<https://martinfowler.com/articles/feature-toggles.html>)
- External: LaunchDarkly — Feature flag patterns (reference, not adopted)
- External: Pete Hodgson — 'Feature Flags: An Introduction'

DECISION HISTORY

Date	Status	Actor	Notes
2025-10-03	Draft	Chief Architect	Initial draft; evaluated LaunchDarkly vs custom vs open-source
2025-10-27	Proposed	Chief Architect	Submitted to ARB with cost analysis and data residency review
2025-11-05	Accepted	ARB Chair	ARB approved; v1.0 released; build scheduled for Q4 2025

Date	Status	Actor	Notes
2026-07-12	Accepted	ARB Chair	v3.0 refresh; added typed SDK codegen and lifecycle governance rule

APPROVAL & SIGN-OFF

Architect	Chief Architect (AR)
Tech Lead	Foundation Tech Lead
ARB Chair	ARB Chair
Approved On	2026-07-12

IMPLEMENTATION CHECKLIST

- Build flag state tables in PostgreSQL (Done Q4 2025)
- Implement Flag Service with Redis cache (Done Q4 2025)
- Build Java SDK with string-based accessors (Done Q4 2025)
- Adopt release flags for v1.1 features (Done Q1 2026, 5 flags)
- Retrofit kill switches (ops flags) for critical paths (Done Q1 2026)
- Add typed SDK codegen plugin (Done Q2 2026, v3.0)
- Refactor string-based flag lookups to typed accessors (Done Q2 2026, ~100 call sites)
- Add lifecycle governance CI job (weekly flag-age report) (Done Q2 2026, Ongoing)

ADR - 017

Build Pipeline

Volume 1 — Architecture Foundation - Process

ACCEPTED

DECISION SUMMARY

DECISION

Adopt GitHub Actions as the canonical CI/CD platform for the PreOne platform, with a 10-stage Gradle-based build pipeline that produces Docker images for backend, static assets for frontend, and Flyway migrations as deployable artifacts. Build targets: PR pipelines under 10 minutes, main-branch pipelines under 15 minutes.

STATUS

Status	Accepted
Date Decided	2025-10-15
Decision Owner	Chief Architect
Review Cadence	Quarterly or when build time exceeds target by 20%
Supersedes	None (v1.0 original; v3.0 refreshes stage definitions for series alignment)

CONTEXT

Before v1.0, PreOne's build process was a mix of local Gradle runs, a Jenkins server maintained by one DevOps engineer, and an ad-hoc shell script that wrapped npm and docker. Builds took 25-40 minutes on average, flaked 15% of the time due to environment drift between Jenkins and local machines, and produced artifacts that were not reproducible (a build of the same commit could produce different images because dependencies were resolved at build time without lockfiles). Three separate teams maintained three separate build scripts, each with subtle differences in test scope, linting rules, and artifact naming. The Office of the Chief Architect identified build pipeline unification as a Q3 2025 priority. The goals were: (1) a single pipeline definition for the entire platform; (2) reproducible artifacts (deterministic builds); (3) build time under 10 minutes for PR feedback; (4) pipeline-as-code so the pipeline itself is versioned alongside the application code; (5) self-service for feature teams (no DevOps bottleneck for adding a new test stage). The evaluation considered Jenkins (existing), GitHub Actions, GitLab CI, CircleCI, and Buildkite. The evaluation concluded that GitHub Actions was the right choice: it lives next to the code (no separate CI server to maintain), it has first-class support for the GitHub workflow (PRs, branch protection, required checks), its runner marketplace covers our needs (Java, Node, Docker, AWS), and its pricing scales

with usage without per-seat licensing. The 10-stage pipeline was designed to give fast feedback (lint and unit tests in <3 minutes) while still catching integration and security issues before merge to main.

BUSINESS DRIVERS

The primary driver is developer productivity: a 25-minute build means a developer waits half an hour to know if their PR is viable, which kills iteration speed and encourages context-switching. A sub-10-minute PR pipeline restores the tight feedback loop that small teams enjoy. The secondary driver is release confidence: a deterministic pipeline with security scans, contract tests, and ArchUnit boundary checks produces artifacts that are safe to deploy without manual review. The tertiary driver is operational cost: a single GitHub Actions pipeline replaces the Jenkins server (which required ~10 hours/month of DevOps maintenance) and the three team-specific build scripts (~6 hours/month of duplication). The combined saving is roughly one full-time engineer's worth of toil redirected to feature work.

PROBLEM STATEMENT

PreOne's build process is slow (25-40 minutes), flaky (15% failure rate), non-reproducible (same commit yields different artifacts), and fragmented (three teams maintain three scripts). We need a single, fast, deterministic, pipeline-as-code build that produces deployable artifacts with confidence.

CONSTRAINTS

- PR build must complete in under 10 minutes (developer feedback loop)
- Main-branch build must complete in under 15 minutes (release cadence)
- Pipeline must run on GitHub-hosted runners (no self-hosted infrastructure to maintain)
- Artifacts must be reproducible — same commit + lockfile = same image digest
- Pipeline definition must live in the same repository as the application code
- All stages must be parallelisable where dependencies allow (module-level test fan-out)
- Build cache must persist across runs (Gradle, npm, Docker layers)

ASSUMPTIONS

- GitHub Actions remains available and pricing remains usage-based (no per-seat surprise)

- GitHub-hosted runner performance is sufficient for our build size (200k LOC, ~5,000 tests)
- AWS ECR is available as the artifact registry (same AWS account as deployment)
- Engineers will respect the pipeline as the source of truth (no local-release shortcuts)
- Lockfiles (Gradle lockfile, package-lock.json) are maintained and committed
- Modular monolith topology holds (single deployable, so single build artifact for backend)

OPTIONS CONSIDERED

Option	Pros	Cons	Verdict
GitHub Actions with 10-stage Gradle pipeline (chosen)	Pipeline-as-code in repo; native GitHub PR integration; marketplace runners for Java/Node/Docker/AWS; usage-based pricing; matrix builds for module fan-out; no server to maintain.	Vendor lock-in to GitHub ecosystem; runner performance varies under load; complex pipelines require careful YAML; secrets management via GitHub OIDC requires setup.	Chosen
Jenkins (existing infrastructure)	Already deployed; full control over executors; plugin ecosystem; engineers familiar.	Requires dedicated maintenance (10 hours/month); pipeline-as-code via Jenkinsfile is verbose; UI-centric culture discourages automation; no native GitHub PR integration without plugins that break often.	Rejected

Option	Pros	Cons	Verdict
GitLab CI	Excellent pipeline-as-code; built-in container registry; auto-scaling runners; mature caching.	Would require migrating repository from GitHub to GitLab (high switching cost); team has no GitLab expertise; loses GitHub PR review workflow that product team relies on.	Rejected
CircleCI	Fast runners; excellent caching; mature orchestration; good observability.	Per-seat pricing model penalises growth; separate service from GitHub (extra context switch); no significant advantage over GitHub Actions for our scale.	Rejected
Buildkite (hybrid: control plane SaaS, runners self-hosted)	Cheapest at scale; full runner control; pipeline-as-code.	Requires self-hosted runners (defeats 'no infrastructure' goal); additional operational burden; overkill at PreOne's build volume.	Rejected

DECISION

ADOPTED

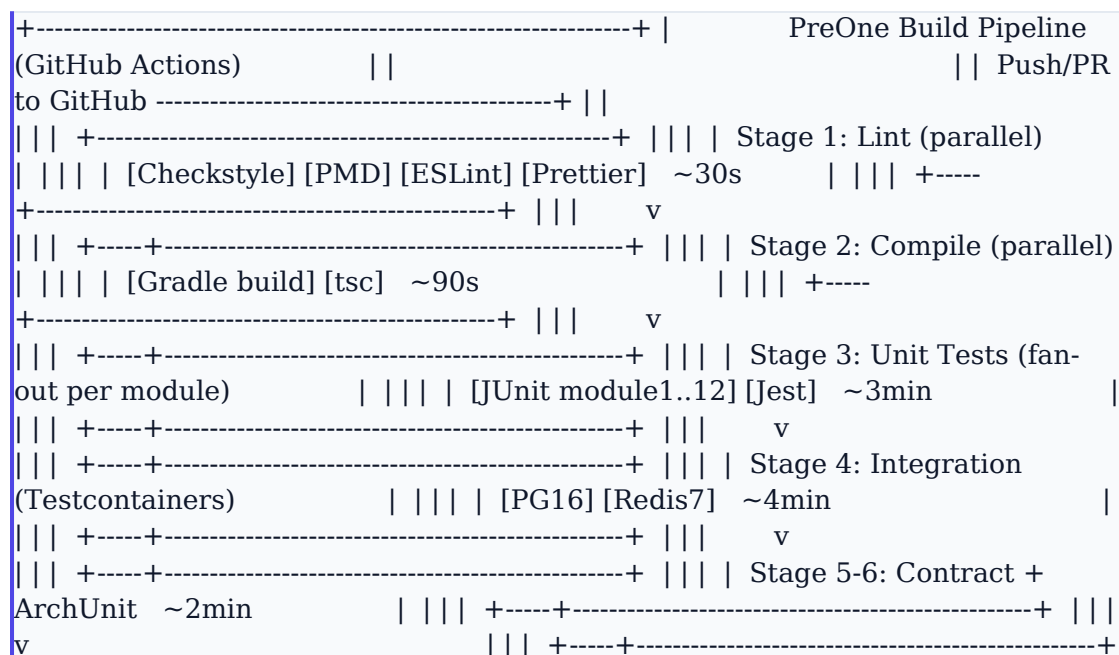
We adopt GitHub Actions as the canonical CI/CD platform, with a 10-stage Gradle-based pipeline defined in `.github/workflows/build.yml`. The pipeline stages are: (1) lint (Checkstyle, PMD, ESLint, Prettier); (2) compile (Gradle build for backend, tsc for frontend); (3) unit tests (JUnit 5 for backend, Jest for frontend); (4) integration tests (Testcontainers with PostgreSQL 16 and Redis 7); (5) contract tests (Pact consumer and provider verification); (6) ArchUnit boundary checks (module dependency rules per ADR-007); (7) security scan (Trivy for image CVEs, Snyk for dependency CVEs, GitHub CodeQL for SAST); (8) package (Docker image build, push to ECR with immutable tag = git SHA); (9) deploy to test environment (ECS Fargate, ADR-018); (10) smoke tests (health check, key user journeys). Build caching: Gradle cache (actions/cache keyed on gradle.lockfile), npm cache (actions/cache keyed on package-lock.json), Docker layer cache (actions/cache keyed on Dockerfile hash). Parallel jobs: backend and frontend pipelines run as parallel workflow jobs; within backend, unit tests fan out per module via Gradle's test partitioning. Build badge in README.md links to the GitHub Actions status page.

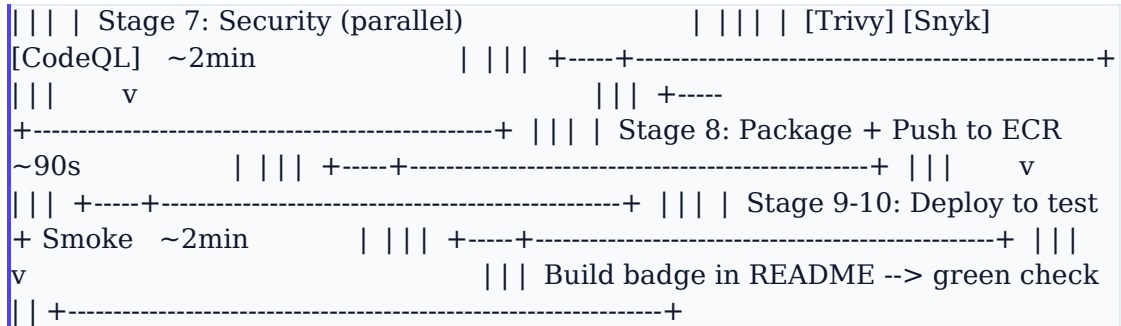
DETAILED RATIONALE

GitHub Actions was chosen because the marginal cost of switching from Jenkins (the existing tool) is paid back within 3 months by the elimination of Jenkins maintenance. The native GitHub integration — PR checks, branch protection, required-status-checks, CODEOWNERS — removes an entire class of integration glue that Jenkins requires. The 10-stage design reflects a deliberate ordering: fast-fail stages first (lint, compile, unit tests) so developers get feedback within 3 minutes; expensive stages last (security scan, deploy, smoke) so they only run on commits that have already passed cheap checks. The stage ordering is the result of analysing 6 months of Jenkins build data. The number-one cause of build failure was lint (40% of failures), and lint is the cheapest stage (~30 seconds). Number two was unit tests (25% of failures, ~3 minutes). By front-loading these, 65% of build failures are caught in under 4 minutes — the developer gets the failure feedback before they have context-switched away from the PR. The expensive stages (integration, contract, security) catch the remaining 35% of failures but only run on commits that have passed the cheap stages, saving ~12 minutes of runner time per failing build. The cache strategy is critical for the sub-10-minute target. Without caching, a cold Gradle build of PreOne is 8 minutes; with Gradle cache (dependency downloads cached, build outputs cached), it drops to 2 minutes. Without Docker layer cache, image build is 6 minutes; with layer cache (base image,

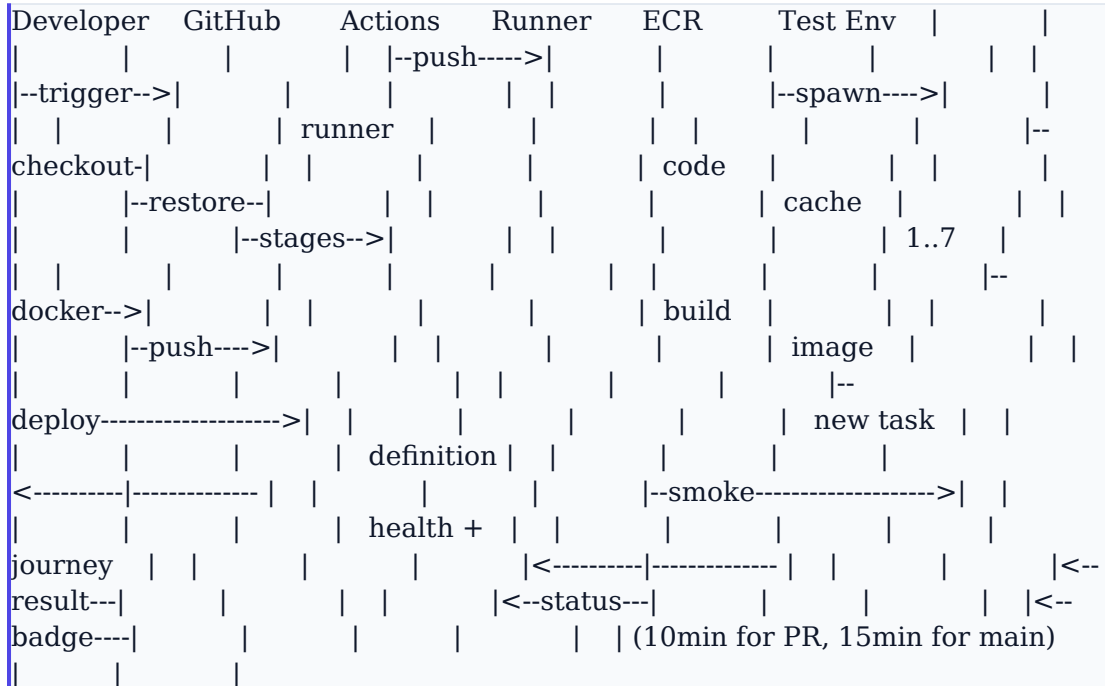
dependencies layer, source layer), it drops to 90 seconds. The cache key strategy (hash of lockfile for dependency cache, hash of Dockerfile for layer cache) ensures cache invalidation happens exactly when dependencies change — no stale cache, no wasted cache misses. The reproducibility requirement is enforced by three mechanisms: (1) Gradle lockfile (gradle.lockfile) pins every transitive dependency to a specific version; (2) npm package-lock.json does the same for frontend; (3) Docker builds use --digest-pin so base images are pulled by digest, not by tag (so a base image update cannot silently change our build). Combined, these ensure that two builds of the same commit produce bit-identical Docker images, verifiable by comparing image digests in ECR. The ArchUnit stage (stage 6) is the architectural backstop. Per ADR-002 (Modular Monolith) and ADR-007 (Dependency Rules), module boundaries must be enforced in CI, not just in code review. ArchUnit tests run as part of the unit-test suite but are given their own stage label so failures are visible as 'architecture violation' rather than generic 'test failure'. This makes boundary violations a first-class pipeline concern, not a side effect. The security scan stage (stage 7) runs three tools in parallel: Trivy scans the built Docker image for known CVEs in OS packages and application dependencies; Snyk scans the dependency tree (Gradle and npm) for CVEs with upgrade guidance; GitHub CodeQL performs SAST on the source code for vulnerability patterns (SQL injection, XSS, hardcoded secrets). The three tools are complementary — Trivy catches image-layer CVEs, Snyk catches dependency CVEs, CodeQL catches code-level vulnerabilities. Failures are categorised by severity: critical and high block the build; medium and low produce warnings that go to a security dashboard for triage.

ARCHITECTURE DIAGRAM

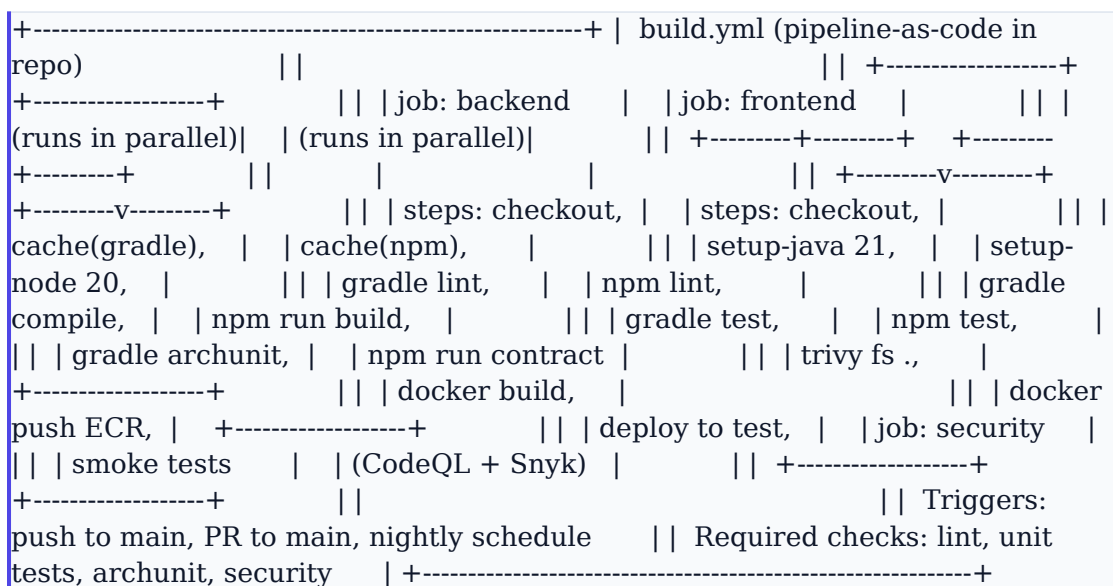




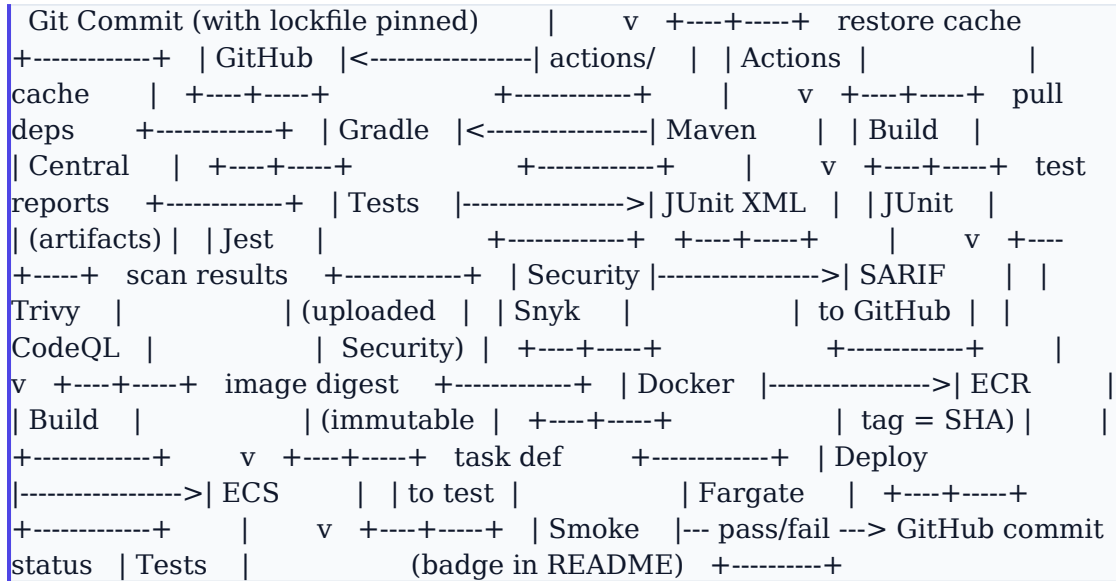
SEQUENCE DIAGRAM



COMPONENT DIAGRAM



DATA FLOW DIAGRAM



DATABASE IMPACT

The build pipeline does not directly mutate production databases, but it produces Flyway migration artifacts that are deployed by ADR-018's deployment stage. Flyway migration files (V<timestamp>__<description>.sql) live in `src/main/resources/db/migration` and are packaged into the Docker image. The build pipeline runs Flyway's `validate` task (verifies migration checksums match the schema history table) as part of stage 4 (integration tests) to catch checksum drift before deploy. Migrations are forward-only per ADR-019 (Versioning Policy); the build does not produce rollback migrations because rollback is handled at the deployment layer via blue-green swap to the previous image.

API IMPACT

The build pipeline runs Pact contract tests in stage 5: consumer-side tests verify that API consumers (frontend, internal modules) generate expected pacts; provider-side tests verify that the API implementation honours those pacts. Pacts are published to a Pact Broker (hosted internally) tagged with the git SHA. A PR that breaks a contract is blocked by the provider-verification step. The build also generates OpenAPI specs from the code (springdoc-openapi) and verifies them against the committed spec file — drift between code and spec is a build failure, enforcing ADR-001 principle P7 (Contracts Before Code).

UI IMPACT

Frontend builds (TypeScript compile, Vite production bundle, ESLint, Prettier, Jest unit tests) run as a parallel GitHub Actions job. The build produces static assets (HTML, JS, CSS, source maps) that are uploaded to S3 and served via CloudFront. Source maps are uploaded to a private S3 bucket for error

reporting (Sentry) but not deployed publicly (to avoid leaking source code). The build also runs Lighthouse CI on the production bundle, measuring performance, accessibility, and SEO scores; a regression of >5 points on any metric blocks the build.

SECURITY IMPACT

Stage 7 (security scan) is the primary security gate. Trivy scans the Docker image for OS-package CVEs and application-dependency CVEs (Java JARs, Node modules). Snyk scans the dependency graph for CVEs with upgrade paths. GitHub CodeQL performs SAST on source for patterns like SQL injection, XSS, hardcoded secrets, insecure deserialization. Results are uploaded as SARIF to GitHub Security tab for triage. Critical and high severity findings block the build; medium and low produce warnings. Secrets are never in the repo — GitHub OIDC is used to assume AWS IAM roles for ECR push and ECS deploy, eliminating long-lived AWS credentials.

PERFORMANCE IMPACT

The pipeline is designed for sub-10-minute PR builds. Stage timings (with cache): lint 30s, compile 90s, unit tests 3min (fan-out per module), integration 4min, contract + ArchUnit 2min, security 2min, package 90s, deploy + smoke 2min — total ~16min sequential, but with parallelism (backend/frontend jobs in parallel, security scans in parallel, unit-test fan-out) the wall-clock time is ~9 minutes for PRs (security and deploy stages skipped on PR) and ~13 minutes for main (all stages). The cache hit rate is monitored; a drop below 80% triggers a review of cache key strategy.

SCALABILITY ANALYSIS

The pipeline scales horizontally via GitHub Actions' concurrent job limit. The PreOne organisation has 20 concurrent runner slots (paid plan). At peak (10 engineers pushing PRs simultaneously), the pipeline queues for ~2 minutes before a runner is free — acceptable. Module-level test fan-out uses 4 partitions (not 12) to balance fan-out benefit against runner-slot consumption. The pipeline definition scales with the codebase: new modules add a Gradle subproject and automatically inherit the pipeline stages (no per-module pipeline definition needed). Build cache scales with cache size: at 5GB cache limit, we evict oldest entries first; cache hit rate is the primary scaling metric.

OPERATIONAL CONSIDERATIONS

The pipeline is operated by the DevOps team (2 engineers) with on-call rotation. Failures are categorised: infrastructure failures (runner offline, ECR unavailable) page the DevOps on-call; build failures (lint, test, security) notify the PR author via GitHub. The pipeline has a 30-day retention on build logs

(GitHub Actions default); long-term retention is via export to CloudWatch Logs. The pipeline is itself versioned (build.yml in repo), so changes go through PR review — no out-of-band pipeline changes. A nightly 'main' build runs the full pipeline including slow tests (performance, end-to-end) that are skipped on PR for speed.

RISKS

Risk	Likelihood	Impact	Mitigation
GitHub Actions outage blocks all builds	Low	High	GitHub status page monitored; emergency releases can be built locally and pushed to ECR manually (documented runbook); 4-hour RTO for build restoration
Cache poisoning (compromised cache produces malicious artifacts)	Low	High	Cache keys are content-hashed (lockfile, Dockerfile); cache is read-only after write; Trivy scan catches compromised dependencies before image push
Build time creeps above target as codebase grows	Medium	Medium	Quarterly build-time review at ARB; test partitioning increases fan-out; flaky tests are quarantined within 24 hours

Risk	Likelihood	Impact	Mitigation
Security scan false positives block legitimate builds	Medium	Medium	Findings are triaged within 4 hours; suppressions are documented in .trivyignore and snyk-policy with expiry dates; suppressions auto-expire to force re-evaluation
Flaky integration tests (Testcontainers startup race)	Medium	Medium	Test retry with JUnit 5 @Retry (max 2 retries); flaky tests auto-quarantined to a separate suite after 3 failures; root-cause analysis within 48 hours

TRADE-OFFS

We Gain	We Lose
Pipeline-as-code lives next to application code (reviewable, versioned)	Vendor lock-in to GitHub Actions (migration cost if we ever switch)
Sub-10-minute PR feedback restores iteration speed	Skipped stages on PR (security, deploy) mean some checks run only on main
Deterministic builds via lockfiles and digest-pinned base images	Lockfile maintenance burden (Dependabot PRs to review weekly)
Module-level test fan-out parallelises the slowest stage	Consumes more runner slots (cost scales with concurrency)
OIDC-based AWS auth eliminates long-lived credentials	Additional IAM trust configuration per repository

REJECTED ALTERNATIVES

Jenkins was rejected because the maintenance burden (10 hours/month) and integration friction (no native GitHub PR support) were not justified by the control it offered. GitLab CI was rejected because migrating the repository off

GitHub would disrupt the product team's PR review workflow and offer no significant advantage at our scale. CircleCI was rejected on cost grounds — per-seat pricing penalises growth, and the per-minute pricing of GitHub Actions is cheaper at our build volume (~200 builds/day). Buildkite was rejected because self-hosted runners reintroduce the infrastructure burden that we are trying to eliminate. The chosen approach (GitHub Actions) is not the 'best' CI tool in absolute terms — it is the best fit for our constraints (GitHub-hosted code, small DevOps team, usage-based cost tolerance).

MIGRATION PLAN

The migration from Jenkins to GitHub Actions was completed in Q3 2025 over 6 weeks. Phase 1 (week 1-2): parallel run — both Jenkins and GitHub Actions ran on every PR, results compared for parity. Phase 2 (week 3-4): GitHub Actions became the required check; Jenkins ran in shadow mode for telemetry only. Phase 3 (week 5-6): Jenkins decommissioned; the Jenkins server was repurposed as a build-cache mirror. The v3.0 refresh adds the security scan stage (Trivy, Snyk, CodeQL) which was not in v1.0, and introduces the Lighthouse CI check for frontend performance regression.

TESTING STRATEGY

The pipeline itself is tested via a 'pipeline-on-pipeline' approach: a meta-workflow runs weekly that exercises every stage of build.yml with synthetic inputs (a known-good commit, a known-bad commit) and verifies that the pipeline produces the expected pass/fail. Stage-level tests: lint rules are tested with positive and negative cases (a code snippet that should pass, one that should fail); ArchUnit rules are tested with synthetic module structures; security scans are tested with intentionally vulnerable test projects (OWASP WebGoat Java snippet, npm vulnerable-package fixture). Smoke tests verify the test environment deployment: health endpoint returns 200, key user journeys (login, view dashboard, create enrollment) return expected responses.

MONITORING & OBSERVABILITY

Pipeline metrics are exported to CloudWatch via a GitHub Actions workflow that runs after every build and posts build duration, stage durations, pass/fail, and cache hit rate. Grafana dashboard 'CI/CD Health' shows: (a) build duration p50/p90 over time (target: p50 < 10min, p90 < 15min); (b) build failure rate by stage (target: <5% overall); (c) cache hit rate per cache (target: >80%); (d) runner queue time (target: <2min p90); (e) security scan findings trend (target: critical findings → 0). Alerts fire on: build duration p90 > 15min, failure rate >10%, cache hit rate <60%, runner queue >5min p90. Weekly report to engineering leadership on pipeline health.

FUTURE EVOLUTION

The pipeline is expected to evolve in three directions. (1) Build time optimisation: as the codebase grows past 500k LOC, Gradle's incremental build and configuration cache will become necessary; the v3.0 pipeline already uses Gradle configuration cache. (2) Security expansion: SLSA (Supply-chain Level for Software Artifacts) Level 3 compliance is a likely future requirement — the pipeline already produces provenance attestations (SLSA generator action); full Level 3 requires isolated builds and verified provenance, a v4.0 candidate. (3) Multi-platform: if PreOne adds a mobile app (React Native) or a desktop app (Electron), the pipeline will gain parallel jobs for those platforms; the current job-based structure scales naturally. No supersession of this ADR is expected within 24 months.

RELATED ADRS

- ADR-001 — Enterprise Architecture Principles (complements P9 Automate Everything Repeatable)
- ADR-002 — Modular Monolith Strategy (complements — single artifact simplifies pipeline)
- ADR-007 — Dependency Rule (enforces — ArchUnit stage 6)
- ADR-016 — Feature Flags (complements — decouples deploy from release)
- ADR-018 — Deployment Model (complements — pipeline stage 9 triggers deploy)
- ADR-019 — Versioning Policy (complements — artifact versioning follows ADR-019)
- ADR-020 — Technology Stack Freeze (complements — locked stack simplifies pipeline)
- ADR-149 — Contract Testing (refines — Pact contract tests in stage 5)

REFERENCES

- Upstream DDD: DDD-017-section-1 (build-time domain test strategy)
- Upstream PRD: PRD-017-section-2.1 (release cadence requirements)
- Downstream ERD: ERD-017 (Flyway migration artifact structure)
- Downstream API Spec: API-017 (Pact broker pact publishing)
- Downstream Test Cases: TC-0701..TC-0800 (pipeline stage verification tests)
- External: GitHub Actions documentation — <https://docs.github.com/actions>
- External: Gradle Build Tool — <https://docs.gradle.org>
- External: SLSA Framework — <https://slsa.dev>

DECISION HISTORY

Date	Status	Actor	Notes
2025-08-20	Draft	DevOps Lead	Initial draft; evaluated 5 CI platforms
2025-09-22	Proposed	Chief Architect	Submitted to ARB with 10-stage design and migration plan
2025-10-15	Accepted	ARB Chair	ARB approved; v1.0 released; Jenkins decommission scheduled
2026-07-12	Accepted	ARB Chair	v3.0 refresh; added security scan stage and Lighthouse CI

APPROVAL & SIGN-OFF

Architect	Chief Architect (AR)
Tech Lead	DevOps Lead
ARB Chair	ARB Chair
Approved On	2026-07-12

IMPLEMENTATION CHECKLIST

- Migrate build.yml from Jenkins to GitHub Actions (Done Q3 2025)
- Configure GitHub OIDC trust with AWS IAM roles for ECR push and ECS deploy (Done Q3 2025)
- Add Gradle and npm cache keys to actions/cache (Done Q3 2025)
- Add Trivy, Snyk, CodeQL security scan stage with SARIF upload (Done Q4 2025)
- Add Pact contract test stage with Pact Broker integration (Done Q4 2025)
- Add Lighthouse CI check for frontend performance regression (Done Q1 2026)
- Decommission Jenkins server and repurpose as build-cache mirror (Done Q1 2026)
- Quarterly build-time and pipeline-health review at ARB (Ongoing)

ADR - 018

Deployment Model

Volume 1 — Architecture Foundation - Process

ACCEPTED

DECISION SUMMARY**DECISION**

Adopt blue-green deployment on AWS ECS Fargate with an Application Load Balancer (ALB) routing traffic between two target groups. Deploy sequence: build new image, push to ECR, spin up green tasks, wait for health checks, shift traffic 10% to 50% to 100% over 30 minutes, drain blue. Rollback is instant via target group swap. Database migrations are forward-only via Flyway and must be backward-compatible.

STATUS

Status	Accepted
Date Decided	2025-10-15
Decision Owner	Chief Architect
Review Cadence	Annual or when deploy frequency exceeds daily
Supersedes	None (v1.0 original; v3.0 refreshes traffic-shift policy for series alignment)

CONTEXT

Before v1.0, PreOne deployed via an in-house shell script that SSH'd into EC2 instances, pulled the latest Docker image, ran ``docker-compose up -d``, and prayed. Deployments took 20-40 minutes, caused 1-5 minutes of user-visible downtime, and had no rollback — a bad deploy meant a frantic ``git revert`` followed by another 30-minute deploy. The team averaged 1 deploy per week to production, not because the business wanted slow releases, but because deploys were risky and time-consuming. The Office of the Chief Architect evaluated deployment strategies in Q3 2025: rolling deploy, blue-green, canary, and A/B. PreOne's constraints — modular monolith (single deployable),

AWS infrastructure, small DevOps team, regulatory need for fast rollback — pointed to blue-green as the right fit. Blue-green gives instant rollback (swap target group), zero user-visible downtime (green tasks start before blue drains), and operational simplicity (one ALB, two target groups, no canary analysis tooling required). The decision to use ECS Fargate (not EKS, not EC2) was driven by the operational simplicity requirement. Fargate is serverless container compute — no EC2 instances to patch, no Kubernetes control plane to operate, no autoscaling groups to tune. The team's DevOps capacity (2 engineers) cannot support a Kubernetes cluster at production-grade reliability. Fargate trades a small per-task cost premium (~20% over EC2) for a 90% reduction in operational burden. At PreOne's scale (~20 tasks across 3 environments), the cost premium is ~\$400/month — a fraction of an engineer's salary.

BUSINESS DRIVERS

The primary driver is deploy frequency: the business wants to ship features 2-3 times per week to production, not once per week. Blue-green with 30-minute traffic shifting makes each deploy low-risk enough to hit that cadence. The secondary driver is rollback speed: a regulatory incident (e.g., child data exposed by a bug) requires rollback in under 5 minutes; blue-green target-group swap meets this — the rollback is one AWS CLI call. The tertiary driver is developer confidence: engineers ship more carefully when deploys are scary; a safe, reversible deploy process lets engineers ship smaller, more frequent changes, which are inherently less risky than large batch deploys.

PROBLEM STATEMENT

PreOne's deploy process is slow (20-40 minutes), causes downtime (1-5 minutes per deploy), has no rollback, and limits deploy frequency to once per week. We need a deployment model that is fast (<10 minutes), zero-downtime, instantly reversible, and supports 2-3 deploys per week to production.

CONSTRAINTS

- Zero user-visible downtime on every deploy (no maintenance windows)
- Rollback in under 5 minutes (regulatory incident response requirement)
- Deploy frequency: 2-3 per week to production, daily to staging
- Database migrations must be forward-only (no rollback migrations, per ADR-019)
- Migrations must be backward-compatible (old code must work on new schema)

- Deploy must not require DevOps engineer on-call (self-service for feature teams)
- AWS infrastructure in ap-south-1 (Mumbai) region for data residency

ASSUMPTIONS

- Traffic shifting via ALB weighted target groups is sufficient (no canary analysis needed)
- ECS Fargate task startup time (60-90 seconds) is acceptable for green warmup
- Database schema changes are backward-compatible (enforced by ADR-019 and code review)
- Feature flags (ADR-016) decouple deploy from release (a deployed feature can be dark)
- The modular monolith topology holds (single deployable, not per-module deploys)
- AWS ECS Fargate remains available (multi-AZ deployment mitigates AZ failure)

OPTIONS CONSIDERED

Option	Pros	Cons	Verdict
Blue-green on ECS Fargate with ALB (chosen)	Zero downtime; instant rollback (target group swap); no EC2 or K8s to operate; native AWS integration with ECR/CloudWatch; supports weighted traffic shifting; well-documented.	Double resource cost during shift (blue+green running); 60-90s task startup limits very-rapid deploys; Fargate per-task cost premium ~20% over EC2.	Chosen
Rolling deploy on ECS Fargate	No double resource cost (one task replaced at a time); simpler than blue-green.	Rollback requires re-deploy of old image (slow, ~10min); mixed-version state during rolling; cannot shift traffic gradually (no 10% canary).	Rejected

Option	Pros	Cons	Verdict
Canary deploy with weighted routing on ALB	Most precise traffic control; canary analysis can auto-abort on error rate spike.	Requires canary analysis tooling (Flagger, Istio, or custom) that the team cannot operate; complex to reason about mixed-version state; overkill at our traffic volume.	Rejected
Kubernetes (EKS) with Helm and Argo Rollouts	Industry-standard; rich deploy strategies (blue-green, canary, A/B); large ecosystem.	Requires K8s operations expertise the team lacks; control plane cost (~\$70/month); overkill for a single-container modular monolith; violates P5 Simplicity.	Rejected
AWS Lambda (serverless functions, not containers)	No infrastructure to operate; scales to zero; pay-per-invocation.	Modular monolith is not a fit for Lambda (long-running, JVM cold start); 15-minute max duration; would require complete application rewrite.	Rejected

DECISION

ADOPTED

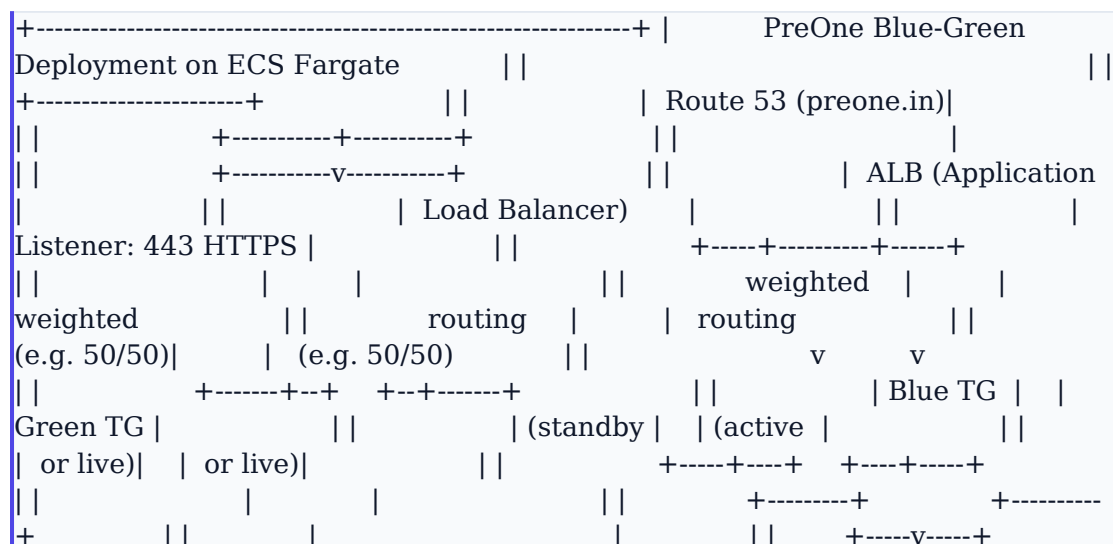
We adopt blue-green deployment on AWS ECS Fargate with an Application Load Balancer (ALB) routing traffic between two target groups (blue and green). Deploy sequence: (1) build new Docker image, push to ECR with immutable tag = git SHA; (2) create new ECS task definition referencing the new image; (3) spin up green tasks (parallel to blue, in the same ECS service but a separate task set, or a separate service); (4) wait for ALB health checks to pass on all green tasks (target group healthy threshold met); (5) shift traffic from blue to green: 10% for 5 minutes, 50% for 10 minutes, 100% for 15 minutes — total 30 minutes; (6) drain blue tasks (deregister from old target group, wait for connection drain, stop tasks); (7) blue target group becomes the standby for the next deploy. Rollback: shift ALB listener back to blue target group — instant, one AWS CLI call. Database migrations: Flyway runs as an ECS Run Task before traffic shift, must be backward-compatible (old code reads new schema correctly). Feature flags (ADR-016) decouple deploy from release. Deploy frequency: 2-3 per week to prod, daily to staging.

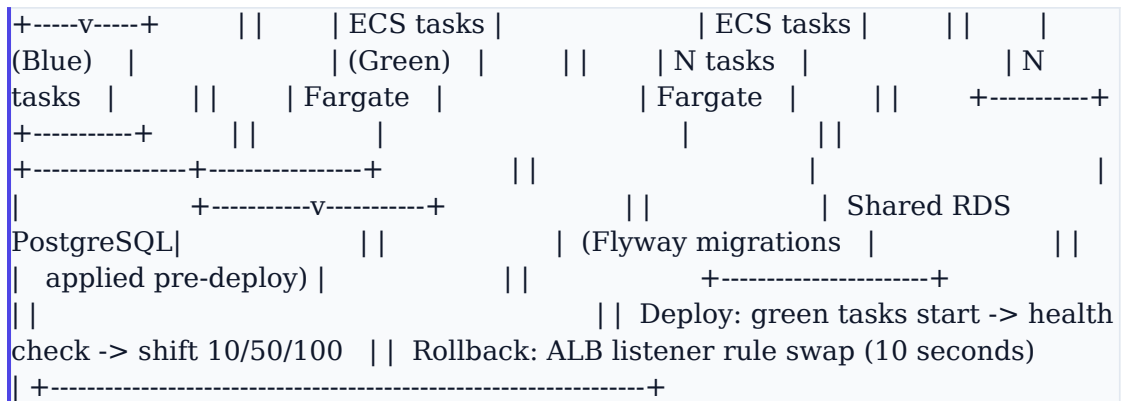
DETAILED RATIONALE

Blue-green was chosen over rolling because rollback speed is a hard constraint. A rolling deploy's rollback is a re-deploy of the previous image, which takes 10-15 minutes — too slow for a regulatory incident response. Blue-green's rollback is a target-group swap, which takes 10 seconds (the time for the ALB to update its listener rule). This 90x difference in rollback speed is the deciding factor. The double-resource cost during the 30-minute shift window is acceptable: at PreOne's task count (~10 tasks per environment), running 20 tasks for 30 minutes costs ~\$0.50 per deploy — negligible versus the business value of fast rollback. ECS Fargate was chosen over EKS for operational simplicity. The team's DevOps capacity (2 engineers) cannot operate a production-grade Kubernetes cluster. EKS requires: control plane management (AWS-managed but still requires version upgrades), worker node group management (autoscaling groups, AMI updates, security patching), Ingress controller management (ALB Ingress Controller or Istio), and Kubernetes RBAC. Fargate eliminates all of this: AWS manages the compute, networking, and scaling; we just define task definitions and services. The cost premium (~20% over EC2, ~\$400/month) is a fraction of the engineer-time saved (~80 hours/month of K8s operations). Canary was rejected because it requires canary analysis tooling (Flagger, Istio, or custom Prometheus queries) that the team cannot operate at production reliability. Blue-green's traffic shifting (10% to 50% to 100%) provides similar risk reduction without the tooling burden: if green has a high

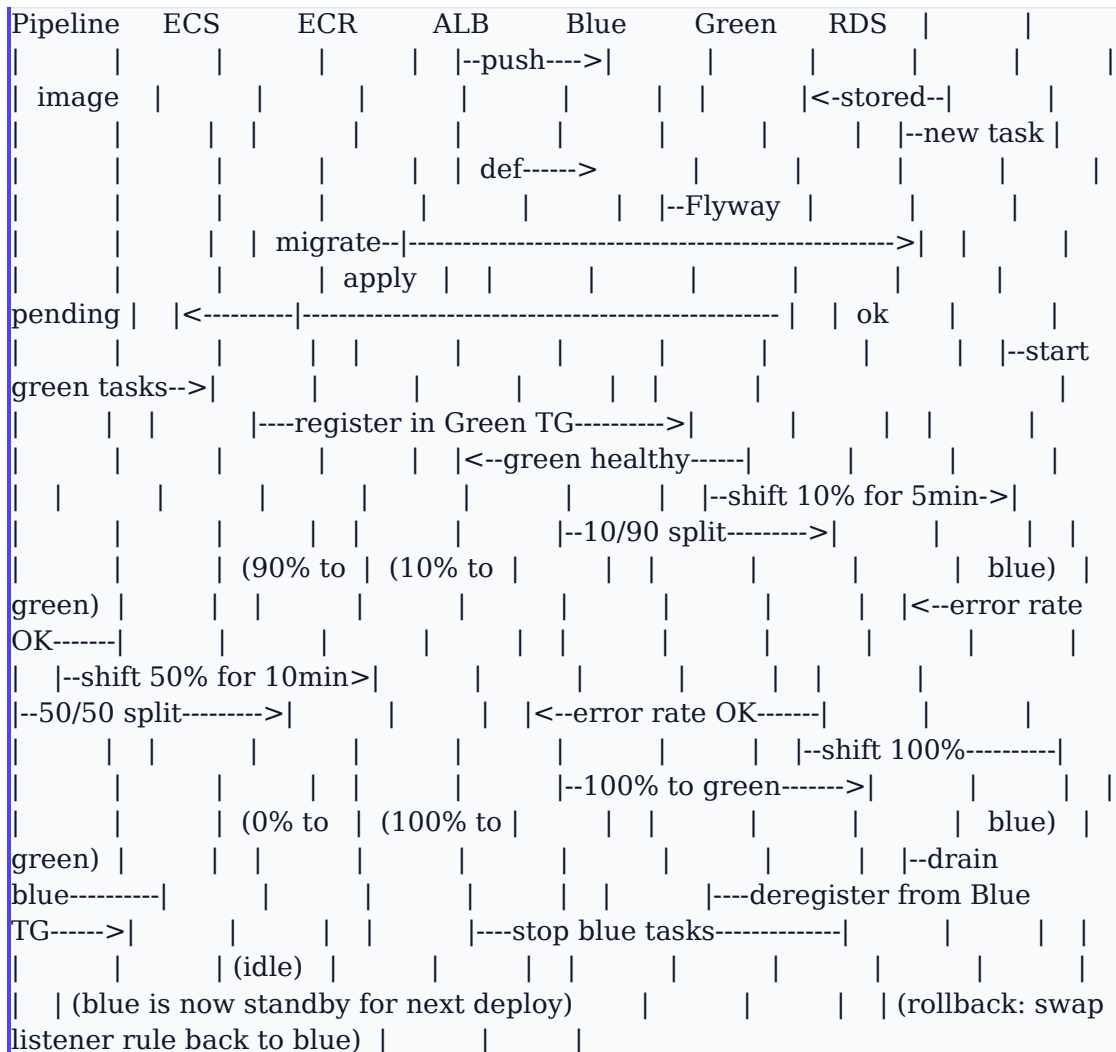
error rate, the human operator (or the deploy script) halts the shift and rolls back. This is less automated than canary analysis but is operationally simpler — appropriate for a small DevOps team. The 30-minute traffic shift (10% for 5 min, 50% for 10 min, 100% for 15 min) is calibrated to PreOne's traffic pattern. The 10% phase catches catastrophic failures (crash-on-startup, immediate error rate spike) within 5 minutes, affecting only ~5% of users (10% of traffic for 5 minutes, but most users don't notice a 5-minute blip). The 50% phase catches subtler failures (memory leak, slow degradation) within 10 minutes. The 100% phase is the final commitment; after 15 minutes of clean 100% traffic, blue is drained. Total shift time: 30 minutes — fast enough to keep deploy cadence (3 deploys/week fits in business hours), slow enough to catch real failures. Database migration compatibility is the hardest constraint. Flyway runs as an ECS Run Task before the green tasks start — it applies pending migrations to the shared database. The blue tasks are still running on the old code, so the new schema must be readable by the old code. This rules out breaking changes: dropping columns, renaming columns, changing types. The allowed patterns are: add nullable column (old code ignores it), add table (old code doesn't know about it), add index (no impact), rename via two-phase (add new column, backfill, switch code, drop old column in a later deploy). ADR-019 (Versioning Policy) formalises this as a rule: every migration must be backward-compatible with the previous N-1 code version. Feature flags (ADR-016) are the secret weapon that makes blue-green viable for risky features. A feature can be deployed to production (green tasks running new code) but hidden behind a feature flag (default off). The flag is turned on for internal testers, then a small percentage of users, then 100%. If the feature misbehaves, the flag is turned off — no deploy, no rollback. This decouples deploy (rolling out new code) from release (turning on new behaviour), giving the team the safety of slow releases with the speed of frequent deploys.

ARCHITECTURE DIAGRAM

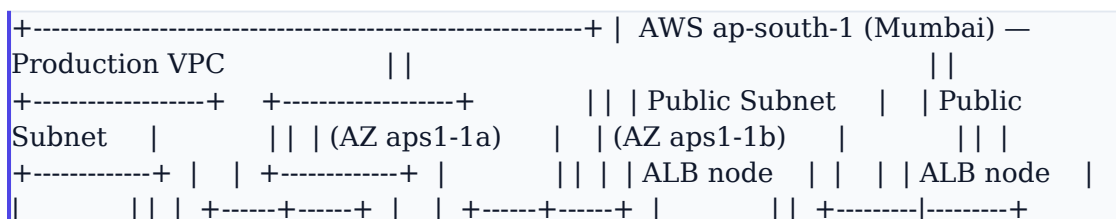


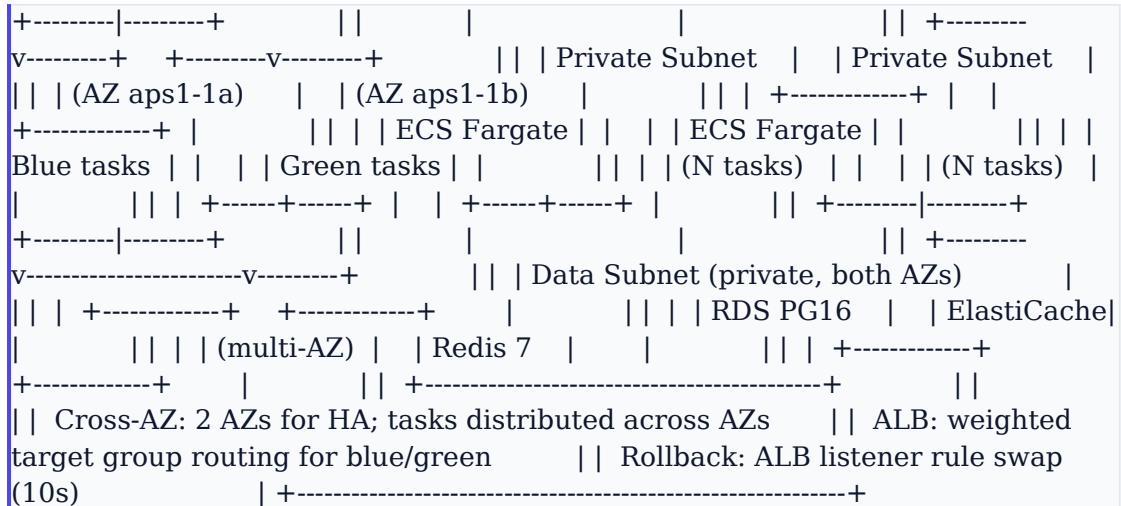


SEQUENCE DIAGRAM

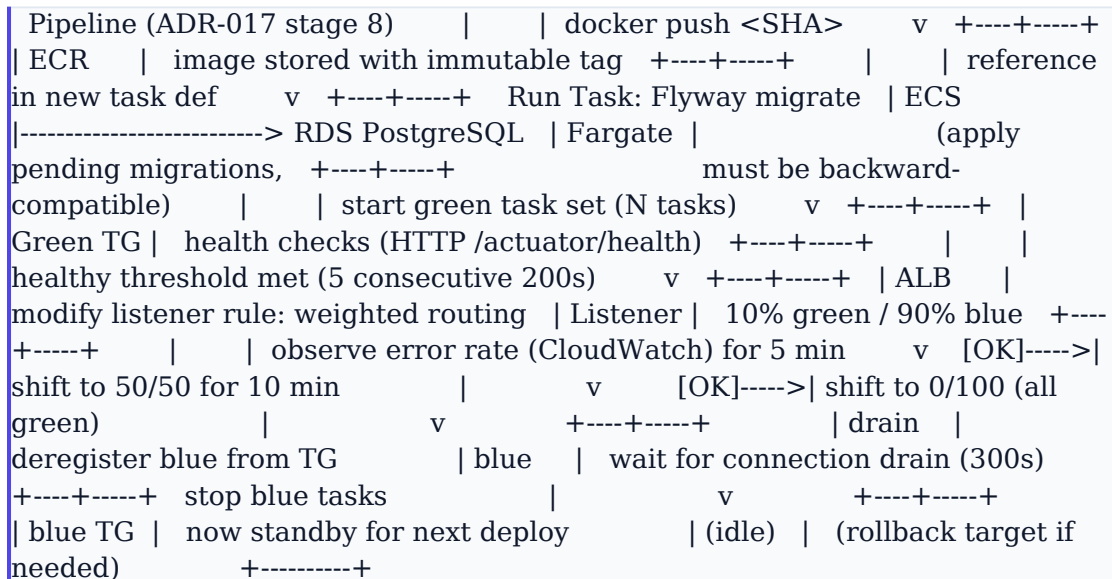


COMPONENT DIAGRAM





DATA FLOW DIAGRAM



DATABASE IMPACT

Database migrations are forward-only via Flyway (ADR-019). The critical constraint is backward compatibility: the new schema must be readable by the old code (blue tasks still running) and by the new code (green tasks). Allowed migrations: add nullable column, add table, add index, add check constraint (validated), widen column (VARCHAR(255) to VARCHAR(500)). Forbidden migrations: drop column, rename column, change column type, narrow column, add NOT NULL without default. The pattern for breaking changes is two-phase: deploy 1 adds new column and backfills; deploy 2 switches code to use new column; deploy 3 drops old column. Flyway runs as an ECS Run Task before the green tasks start — if Flyway fails, the deploy aborts before any traffic shifts.

API IMPACT

The API is served by both blue and green during the 30-minute shift window. This means API consumers may hit blue or green on consecutive requests (the

ALB does not guarantee session affinity). The API must therefore be backward-compatible within a major version (ADR-019): no breaking changes within /api/v1/. New endpoints and new optional fields are safe; renamed fields or removed fields are not. Feature flags (ADR-016) gate new API behaviour: a new endpoint can be deployed (green) but return 404 until the flag is enabled, at which point both blue and green return 200 (blue falls back to a stub or returns 404 consistently until drained).

UI IMPACT

The UI is served from S3+CloudFront, which is independent of the backend blue-green deploy. The UI deploy (npm run build, sync to S3, CloudFront invalidation) is a separate pipeline stage from the backend deploy. The UI must work with both blue and green API versions during the shift window — this is enforced by contract tests (ADR-149) that verify the UI's expected pacts against both versions. If the UI requires a new API field, the deploy sequence is: (1) deploy backend with new field (green); (2) shift backend to 100% green; (3) deploy new UI. The reverse (UI first) is forbidden because old backend cannot serve new UI's expected fields.

SECURITY IMPACT

Blue-green improves security by enabling fast rollback of a vulnerable deploy. If a security issue is discovered post-deploy, the rollback is a 10-second target-group swap — no need to wait for a fix-deploy. This reduces the window of exposure from hours (re-deploy of old image) to seconds (target-group swap). The trade-off is that both blue and green task sets run during the shift, doubling the attack surface for 30 minutes — mitigated by both task sets having identical security configuration (same IAM role, same security groups, same network policies). Secrets are rotated independently of deploys via AWS Secrets Manager; the deploy does not touch secrets.

PERFORMANCE IMPACT

Blue-green has minimal performance impact. During the 30-minute shift, both blue and green tasks run, so total capacity is 2x normal — this is a non-issue because we size for peak load with headroom. Green task startup (60-90 seconds for JVM warmup) is hidden from users — the tasks are registered in the green target group only after the ALB health check passes (which requires the JVM to be warm and ready to serve requests). The 10% traffic shift phase is the first real-user test; if green has a memory leak or performance regression, it shows up as elevated latency on green tasks (visible in CloudWatch metrics per target group), and the shift halts for investigation.

SCALABILITY ANALYSIS

Blue-green scales with the modular monolith's horizontal scaling (ADR-126). The number of tasks per environment is determined by the ECS service's desired count, which is set by an autoscaling policy based on CPU and memory. Blue-green does not change the scaling model — it just doubles the task count temporarily during the shift. At PreOne's scale (10 tasks per environment, 3 environments), the 30-minute shift adds 10 tasks of capacity, costing ~\$0.50/deploy. The ceiling is the ECS Fargate service quota (500 tasks per cluster) — well above our peak of 60 tasks (3 environments x 10 tasks x 2 for blue+green). Database connections are the real ceiling: each task opens 10 connections, so 20 tasks = 200 connections, within PostgreSQL's `max_connections=1000` (with PgBouncer pooling, the ceiling extends to 5000).

OPERATIONAL CONSIDERATIONS

Deploys are operated by the feature team that authored the change, not by a dedicated release engineer. The deploy is triggered by merging a PR to main (automated) or by a manual ``deploy`` GitHub Actions workflow dispatch (for staging or hotfix). The deploy script (Terraform-managed ECS task definitions + AWS CLI for ALB listener rule updates) is reviewed in PR like any other code. On-call engineers are paged if: deploy fails (Flyway error, task startup failure, health check timeout), error rate on green >1% during shift, or rollback is triggered. The on-call engineer's job during a deploy is to monitor CloudWatch dashboards (per-target-group error rate, latency, CPU) and abort the shift if metrics degrade. Post-deploy, the blue target group is left idle for 24 hours as a rollback target; after 24 hours, blue tasks are stopped to save cost (rollback beyond 24 hours requires a re-deploy).

RISKS

Risk	Likelihood	Impact	Mitigation
Backward-incompatible migration breaks blue tasks during shift	Medium	High	Migration review checklist enforced in PR; CI runs Flyway validate against a copy of prod schema; two-phase migration pattern documented; ArchUnit-style rule blocks schema-breaking changes

Risk	Likelihood	Impact	Mitigation
Green tasks pass health check but fail under real load	Medium	Medium	10% traffic shift phase catches most issues; CloudWatch alarms on green error rate >1% halt shift; load test in staging before prod deploy
Rollback fails (blue target group already drained)	Low	High	Blue tasks kept alive for 24 hours post-deploy; rollback tested weekly via game-day exercise; if blue is gone, rollback is a re-deploy of previous image (~10 min)
ALB listener rule update fails (AWS API issue)	Low	High	Deploy script retries with exponential backoff; CloudWatch alarm on listener rule drift; manual rollback runbook documented
Database migration succeeds but leaves schema in inconsistent state	Low	High	Flyway runs in a single transaction (where DDL allows); migration tested against prod snapshot in staging; Flyway repair runbook for checksum mismatch

TRADE - OFFS

We Gain	We Lose
Instant rollback (10s target-group swap)	Double resource cost during 30-min shift (~\$0.50/deploy)

We Gain	We Lose
Zero user-visible downtime	Both blue and green must be backward-compatible (constraints on migrations and API)
Self-service deploys (no release engineer)	Feature teams must understand deploy mechanics (training required)
Operational simplicity (no K8s, no canary tooling)	Fargate cost premium ~20% over EC2 (~\$400/month)
Gradual traffic shift catches production-only failures	30-minute deploy window slower than rolling (but rolling has slow rollback)

REJECTED ALTERNATIVES

Rolling deploy was rejected because rollback requires a re-deploy of the previous image (~10 minutes), violating the 5-minute rollback constraint. Canary was rejected because the canary analysis tooling (Flagger, Istio) requires operational expertise the team lacks, and the benefit over blue-green's manual 10/50/100 shift is marginal at our traffic volume. EKS (Kubernetes) was rejected because the operational burden (control plane, worker nodes, Ingress, RBAC) is unjustified for a single-container modular monolith; the team's DevOps capacity cannot sustain production-grade K8s. Lambda was rejected because the modular monolith topology (long-running JVM, in-process transactions) is a poor fit for Lambda's 15-minute limit and cold-start characteristics. The chosen approach (blue-green on ECS Fargate) is the simplest model that meets the zero-downtime and instant-rollback constraints.

MIGRATION PLAN

The migration from the SSH-and-docker-compose deploy to blue-green on ECS Fargate was completed in Q4 2025 over 8 weeks. Phase 1 (week 1-2): Terraform the ECS cluster, ALB, target groups, and RDS — infrastructure as code. Phase 2 (week 3-4): containerise the modular monolith (Dockerfile, ECR repository, task definition). Phase 3 (week 5-6): blue-green deploy script (GitHub Actions workflow that runs Flyway, starts green, waits for health, shifts traffic, drains blue). Phase 4 (week 7-8): parallel run — old deploy and new blue-green both run on staging, results compared for parity; cutover to blue-green for staging, then prod. v3.0 adds the 30-minute traffic shift calibration (10% for 5 min, 50% for 10 min, 100% for 15 min) which was a flat 100% shift in v1.0 — too risky for subtle failures.

TESTING STRATEGY

Deploy is tested at three levels. (1) Deploy script unit tests: the Terraform and AWS CLI commands are tested with localstack (AWS emulator) — verifies that the script produces the expected AWS API calls. (2) Staging deploy: every main-branch build deploys to staging via the same blue-green script that prod uses — staging is the canary for the deploy script itself. (3) Production rollback game-day: weekly, a production deploy is rolled back (target-group swap to blue) to verify the rollback path works; the rollback is then itself rolled back (swap back to green). This ensures the rollback muscle is exercised before it is needed in anger. Health checks: ALB health check on /actuator/health (Spring Boot Actuator) requires 5 consecutive 200s over 30 seconds before a task is considered healthy — catches JVM warmup and startup failures.

MONITORING & OBSERVABILITY

Deploy is monitored via CloudWatch dashboards per target group. Metrics: (a) request count per target group (verifies traffic is shifting as expected); (b) error rate per target group (4xx and 5xx — green error rate should match blue); (c) latency p50/p99 per target group (green latency should be within 10% of blue); (d) target health (number of healthy tasks in each target group); (e) connection drain count (blue tasks draining post-shift). Alarms fire on: green error rate >1% (halt shift), green latency p99 > 1.5x blue p99 (halt shift), green task health < desired count (halt shift), deploy duration > 45 minutes (page on-call). Post-deploy, a 'deploy summary' is posted to Slack with: deploy SHA, duration, traffic shift timeline, error rate comparison, rollback count (if any).

FUTURE EVOLUTION

Blue-green on ECS Fargate is the right model for PreOne's 5-year horizon. The trigger for re-evaluation is sustained deploy frequency exceeding daily (which would make the 30-minute shift window a bottleneck) or traffic exceeding 50,000 RPS (which would make the 10% canary phase too large a blast radius). The likely evolution is automated canary analysis: a tool like Flagged or a custom Prometheus query that auto-halts the shift on error-rate anomaly, eliminating the need for on-call monitoring during deploys. The other likely evolution is progressive delivery via feature flags (ADR-016) becoming the primary release mechanism — deploys become routine (just code rollout) and releases become the tracked event (flag flip). No supersession of this ADR is expected within 24 months.

RELATED ADRS

- ADR-001 — Enterprise Architecture Principles (complements P9 Automate, P8 Fail Loud)
- ADR-002 — Modular Monolith Strategy (complements — single deployable simplifies deploy)

- ADR-016 — Feature Flags (complements — decouples deploy from release)
- ADR-017 — Build Pipeline (complements — pipeline stage 9 triggers this deploy)
- ADR-019 — Versioning Policy (complements — backward-compatible migrations required)
- ADR-020 — Technology Stack Freeze (complements — ECS Fargate, ALB, ECR are locked)
- ADR-041 — Database Strategy (complements — RDS PostgreSQL shared across blue/green)
- ADR-126 — Horizontal Scaling (complements — ECS task autoscaling)

REFERENCES

- Upstream DDD: DDD-018-section-1 (deploy-time domain invariant verification)
- Upstream PRD: PRD-018-section-2.1 (deploy frequency and rollback requirements)
- Downstream ERD: ERD-018 (Flyway migration backward-compatibility rules)
- Downstream API Spec: API-018 (API backward-compatibility during shift window)
- Downstream Test Cases: TC-0801..TC-0900 (deploy and rollback verification tests)
- External: AWS ECS Documentation — <https://docs.aws.amazon.com/ecs>
- External: AWS ALB Documentation — <https://docs.aws.amazon.com/elasticloadbalancing/latest/application>
- External: Martin Fowler — 'BlueGreenDeployment' — <https://martinfowler.com/bliki/BlueGreenDeployment.html>

DECISION HISTORY

Date	Status	Actor	Notes
2025-08-25	Draft	DevOps Lead	Initial draft; evaluated blue-green vs rolling vs canary
2025-09-26	Proposed	Chief Architect	Submitted to ARB with ECS Fargate vs EKS comparison

Date	Status	Actor	Notes
2025-10-15	Accepted	ARB Chair	ARB approved; v1.0 released; migration scheduled Q4 2025
2026-07-12	Accepted	ARB Chair	v3.0 refresh; added 30-minute traffic shift calibration

APPROVAL & SIGN-OFF

Architect	Chief Architect (AR)
Tech Lead	DevOps Lead
ARB Chair	ARB Chair
Approved On	2026-07-12

IMPLEMENTATION CHECKLIST

- Terraform ECS cluster, ALB, target groups, RDS in ap-south-1 (Done Q4 2025)
- Containerise modular monolith and push to ECR (Done Q4 2025)
- Blue-green deploy script in GitHub Actions (Done Q4 2025)
- Flyway Run Task before traffic shift (Done Q4 2025)
- 30-minute traffic shift calibration (10/50/100) (Done Q1 2026)
- CloudWatch dashboards per target group with alarms (Done Q1 2026)
- Weekly production rollback game-day (Ongoing)
- Blue task retention for 24 hours post-deploy as rollback target (Ongoing)

ADR - 019

Versioning Policy

Volume 1 — Architecture Foundation - Standards

ACCEPTED

DECISION SUMMARY

DECISION

Adopt semantic versioning (MAJOR.MINOR.PATCH) for all PreOne releases, modules, and APIs. API versioning is URL-based (/api/v1/, /api/v2/) with one major version supported in parallel for 6 months after deprecation. Module versions are declared in build.gradle and follow semver. Database migrations are versioned via Flyway as V<timestamp>__<description>.sql, forward-only. Release versions are calendar-based quarterly minors (2026.1, 2026.2, 2026.3) with patches as needed (2026.1.1).

STATUS

Status	Accepted
Date Decided	2025-10-15
Decision Owner	Chief Architect
Review Cadence	Annual or when a major version supersession occurs
Supersedes	None (v1.0 original; v3.0 refreshes deprecation policy for series alignment)

CONTEXT

Before v1.0, PreOne had no versioning policy. The API was versioned by accident — engineers added /v2/ endpoints when they wanted to change a response shape, but /v1/ was never deprecated, leading to three live versions of the same endpoint with subtly different behaviour. The mobile app (React Native) had hardcoded assumptions about /v1/ response shapes that broke when /v2/ was deployed. Database migrations were numbered V1, V2, V3 with no consistent naming, and a migration that was reverted left a gap (V4 missing) that confused Flyway. Release versions were ad-hoc: '2025-q3-hotfix-2', 'august-release-3', 'billing-rewrite' — none sortable, none comparable. The Office of the Chief Architect identified versioning policy as a Q3 2025 priority because the lack of policy was creating real customer incidents. A school that had integrated PreOne's API into their parent portal broke when /v1/ changed shape without a version bump. The DPDP Act 2023 compliance audit flagged the absence of a deprecation policy as a finding — regulators expect a documented sunset process for data-processing APIs that may be used by third parties. The team evaluated three versioning schemes: pure semver, calendar versioning (CalVer), and a hybrid (calendar for releases, semver for modules

and APIs). The decision was a hybrid: calendar-based versioning for releases (because the business communicates in quarters — 'Q3 2025 release'), semver for modules and APIs (because engineers and API consumers need to reason about breaking changes). Database migrations use Flyway's timestamp-based versioning (V<YYYYMMDDHHmmss>__<description>.sql) which is naturally ordered and avoids numbering conflicts. The policy is enforced through CI checks (semver validation on module versions, OpenAPI spec version validation on APIs) and through the ADR workflow (any breaking change requires an ADR).

BUSINESS DRIVERS

The primary driver is API consumer trust: schools, parents, and third-party integrators who build against PreOne's API must be able to plan around version changes. A documented deprecation policy (6-month notice, 2-release overlap, sunset announcement) gives consumers the runway to migrate, preventing the incidents that occurred pre-v1.0. The secondary driver is release communication: the business needs to tell customers 'this feature ships in the 2026.2 release' — calendar versioning makes this natural. The tertiary driver is engineering clarity: when an engineer sees module version 2.3.1, they know it's a patch on the 2.3 minor, which is a minor on the 2 major — semantic meaning that ad-hoc versions do not convey.

PROBLEM STATEMENT

PreOne has no versioning policy: API versions are unmanaged, database migrations are inconsistently numbered, and releases have ad-hoc names. This causes customer incidents, fails compliance audits, and creates engineering confusion. We need a unified versioning policy covering releases, modules, APIs, and database migrations.

CONSTRAINTS

- API versioning must be URL-based (not header-based) for browser and curl compatibility
- At most two major API versions supported in parallel (current + deprecated)
- Deprecated API version must have a 6-month sunset window
- Database migrations must be forward-only (no rollback migrations — rollback is via blue-green deploy)
- Release versioning must align with the quarterly business cadence
- Module versioning must support independent versioning per Gradle subproject
- Version numbers must be machine-parseable (CI validation)

ASSUMPTIONS

- API consumers will migrate within the 6-month deprecation window (with reminders)
- Quarterly release cadence is stable (no shift to monthly or annual)
- Flyway remains the migration tool (no switch to Liquibase)
- Semantic versioning is understood by the engineering team (training provided)
- The modular monolith topology holds (single deployable, so module versions are not user-visible)

OPTIONS CONSIDERED

Option	Pros	Cons	Verdict
Hybrid: CalVer for releases, semver for modules and APIs, Flyway timestamp for migrations (chosen)	Aligns with business cadence (quarters); semver gives semantic meaning to engineers; Flyway timestamps avoid numbering conflicts; each artefact type uses the most appropriate scheme.	Three schemes to learn (one per artefact type); slightly more complex than a single scheme; calendar versioning does not convey breaking-change semantics.	Chosen
Pure semver for everything (releases, modules, APIs, migrations)	Single scheme; semantic meaning everywhere; industry standard for libraries.	Releases become 'v4.2.1' which does not communicate 'Q3 2025' to business stakeholders; Flyway semver requires manual numbering with conflict risk.	Rejected
Pure CalVer for everything (YYYY.MM.DD or YYYY.Qn)	Single scheme; communicates release timing; no debate about what constitutes a major vs minor.	Loses semantic meaning for APIs (consumer cannot tell if v2026.3 has breaking changes); Flyway CalVer is awkward (migrations are not releases).	Rejected

Option	Pros	Cons	Verdict
Header-based API versioning (Accept: application/vnd.preone.v2+json)	URL stays clean; versioning is content-negotiated; multiple versions can coexist on same URL.	Harder to test with curl; harder to debug (version is in header, not URL); browser caching gets weird; less discoverable.	Rejected
No versioning (continuous deployment, API is always latest)	Simplest; no version maintenance burden.	Consumers cannot pin to a stable version; breaking changes are unannounced; fails compliance audit; not viable for a SaaS with API consumers.	Rejected

DECISION

ADOPTED

We adopt a hybrid versioning policy. (1) Releases: calendar versioning, quarterly minors (2026.1, 2026.2, 2026.3, 2026.4), patch releases as needed (2026.1.1, 2026.1.2). (2) Modules: semantic versioning per Gradle subproject, declared in build.gradle as version = 'MAJOR.MINOR.PATCH'. (3) APIs: URL-based versioning (/api/v1/, /api/v2/), one major version supported in parallel for 6 months after deprecation announcement. (4) Database migrations: Flyway timestamp-based versioning, V<YYYYMMDDHHmmss>__<description>.sql, forward-only (no rollback migrations — rollback is via blue-green deploy to old image, ADR-018). Deprecation policy: 6-month notice (deprecation header on every response), 2-release overlap (current + deprecated both supported), sunset announcement (blog post, email to API consumers, in-product banner). Breaking changes require an ADR and a major version bump.

DETAILED RATIONALE

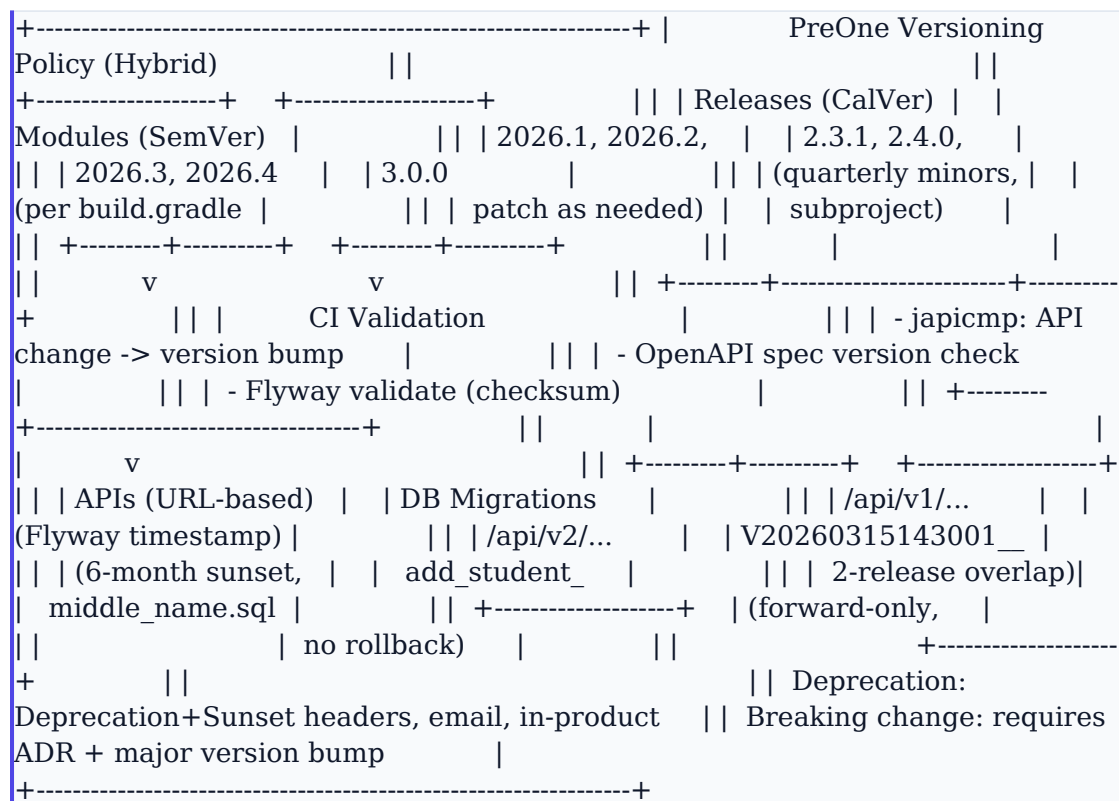
The hybrid scheme was chosen because no single scheme fits all artefact types. Releases are a business communication — 'the 2026.2 release ships in April 2026' — and calendar versioning conveys this directly. Modules are an engineering artefact — 'enrollment-module 2.3.1' — and semver conveys that 2.3.1 is a patch on 2.3, which is a minor on 2. APIs are a consumer contract —

'/api/v2/' tells the consumer this is a major version that may have breaking changes — and URL-based semver conveys this in the most discoverable way. Database migrations are a sequential log — Flyway timestamps convey order without manual numbering. Calendar versioning for releases aligns with the quarterly business cadence. The 2026.1 release is the first release of 2026 (Q1), 2026.2 is Q2, etc. Patch releases (2026.1.1) are bugfixes on top of the quarterly release. This scheme is sortable (2026.2 > 2026.1), comparable (2026.2.1 > 2026.2), and communicates timing (2026.2 is the Q2 2026 release). The business can plan around it: 'the new billing module ships in 2026.3' is a clear statement that any stakeholder understands. Semantic versioning for modules follows the spec: MAJOR version bump for incompatible API changes (e.g., removing a public method, changing a method signature), MINOR version bump for backward-compatible feature additions (e.g., adding a new method, adding a new optional parameter), PATCH version bump for backward-compatible bug fixes (e.g., fixing a calculation error). Module versions are declared in build.gradle and validated by CI: a PR that changes a public API without a version bump is blocked. The validation uses a tool (japicmp) that compares the public API of the new version against the previous, classifying changes as breaking (MAJOR), additive (MINOR), or no-change (PATCH). URL-based API versioning was chosen over header-based for discoverability and testability. A consumer can see /api/v2/enrollments in their browser or curl command and immediately know the version. Header-based versioning (Accept: application/vnd.preone.v2+json) hides the version in a header that is easy to forget in tests and impossible to test with a browser. The trade-off is that URL-based versioning duplicates the endpoint definition (v1 and v2 both exist), but this is manageable: v2 typically delegates to v1 with a response transformation, or v1 delegates to v2 with a backward-compatible transform. The 6-month deprecation window is calibrated to API consumer migration patterns. Schools that integrate with PreOne's API typically have a 1-3 month development cycle for changes; 6 months gives them two cycles, which is enough to plan and execute a migration. The deprecation is signalled three ways: (a) Deprecation: true header on every response from the deprecated version; (b) Sunset: <date> header with the specific sunset date; (c) email and in-product banner to registered API consumers 6 months, 3 months, 1 month, and 1 week before sunset. After sunset, the deprecated version returns 410 Gone.

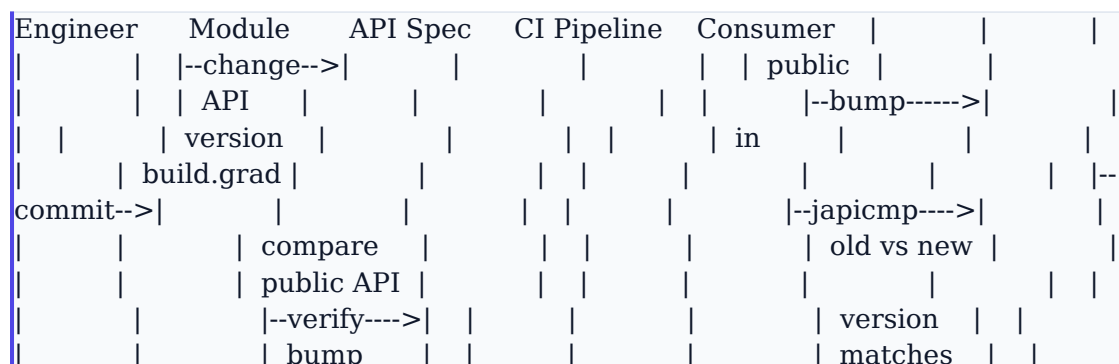
Flyway timestamp-based migration versioning (V<YYYYMMDDHHmmss>__<description>.sql) avoids the numbering conflicts that plagued pre-v1.0 migrations. With sequential numbering (V1, V2, V3), two engineers creating migrations on the same day would both number them V4, causing a conflict. With timestamps (V20260315143001__add_student_middle_name.sql), conflicts are essentially impossible (would require two engineers to create a migration in the same

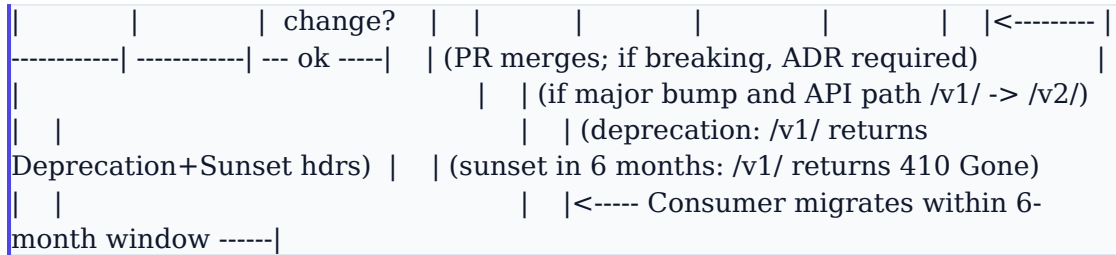
second). The timestamp also conveys when the migration was created, which is useful for debugging. Forward-only migrations (no rollback) is a deliberate choice. Rollback migrations are a known anti-pattern: they cannot undo data changes (a DELETE that ran cannot be rolled back to restore the deleted rows), they double the migration surface area (every up has a down), and they create a false sense of safety (the rollback may not actually restore the previous state). Instead, rollback is handled at the deployment layer: if a migration breaks production, the blue-green deploy (ADR-018) swaps back to the previous image, which works with the new schema (because migrations are backward-compatible, per ADR-018). The migration itself is not rolled back — it stays applied — but the code that uses it is rolled back. This is the standard pattern for SaaS platforms with managed databases.

ARCHITECTURE DIAGRAM

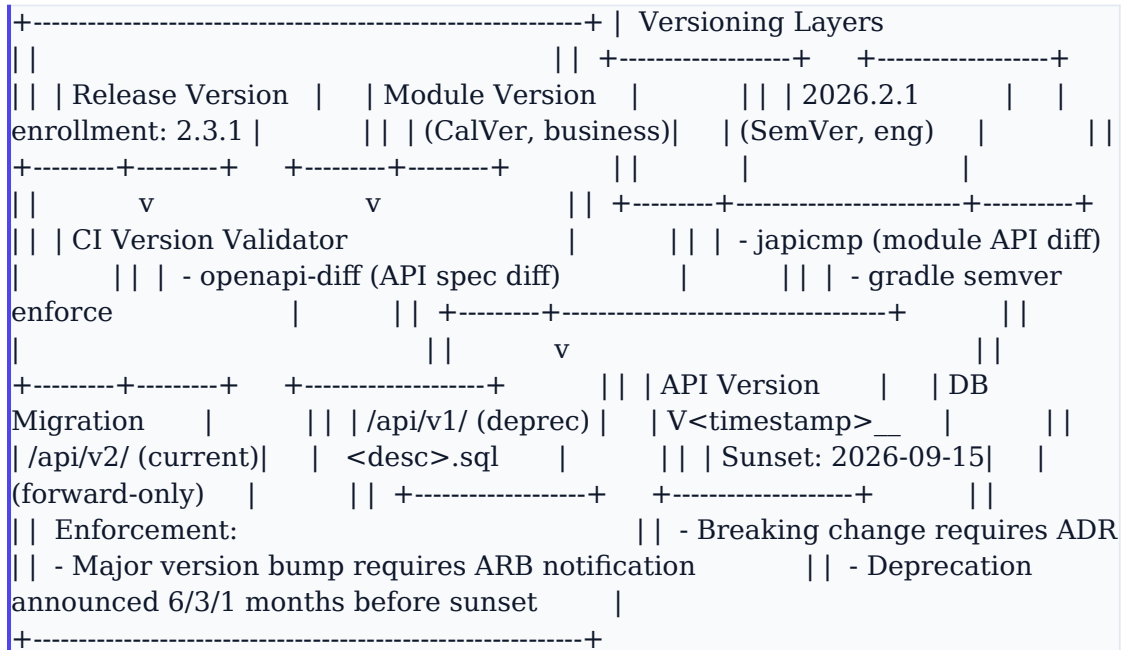


SEQUENCE DIAGRAM

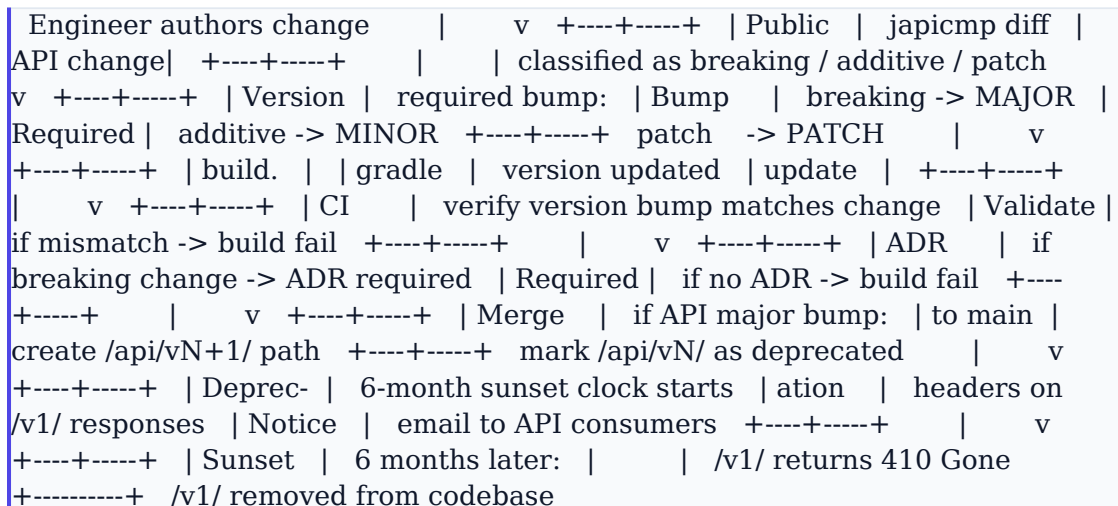




COMPONENT DIAGRAM



DATA FLOW DIAGRAM



DATABASE IMPACT

Database migrations use Flyway's timestamp-based versioning: `V<YYYYMMDDHHmmss>__<description>.sql`, stored in `src/main/resources/db/migration`. Migrations are forward-only — no rollback migrations. The rationale: rollback migrations cannot undo data changes (a

DELETE that ran cannot restore the rows), and they create a false sense of safety. Rollback is handled at the deployment layer (ADR-018): if a migration breaks production, blue-green swaps back to the previous image, which works with the new schema (because migrations must be backward-compatible, enforced by code review and the two-phase migration pattern). The two-phase pattern for breaking changes: deploy 1 adds new column and backfills; deploy 2 switches code to use new column; deploy 3 drops old column. Each deploy is individually rollback-safe.

API IMPACT

API versioning is URL-based: `/api/v1/`, `/api/v2/`. A major version bump (v1 to v2) is required for breaking changes: removing an endpoint, removing a field, changing a field type, changing a field's semantics (e.g., 'status' values change). A minor version bump within a major version (still `/api/v1/` but with new optional fields or new endpoints) does not require a URL change — the major version URL is stable for backward-compatible additions. The OpenAPI spec is the source of truth for API versioning: each major version has its own spec file (`openapi-v1.yaml`, `openapi-v2.yaml`). The CI pipeline runs `openapi-diff` to verify that changes within a major version are backward-compatible — a breaking change within `/v1/` is a build failure, forcing a new `/v2/` spec.

UI IMPACT

The UI calls `/api/v1/` (or `/v2/`) and pins to that version. The UI deploy is coordinated with the API version: when a new `/v2/` is released, the UI is updated to call `/v2/` and deployed after `/v2/` is stable. The old `/v1/` calls are removed from the UI in the same deploy that adds `/v2/` calls — the UI never calls both versions simultaneously (that would be confusing). For third-party UI consumers (parent portal integrations), the 6-month deprecation window applies: they have 6 months to migrate from `/v1/` to `/v2/` after `/v1/` is marked deprecated.

SECURITY IMPACT

Versioning policy improves security by enabling clean deprecation of vulnerable API versions. If `/v1/` has a security flaw that cannot be patched in place (e.g., a design flaw in the auth flow), `/v2/` is released with the fix, `/v1/` is marked deprecated with an accelerated sunset (e.g., 1 month instead of 6), and consumers are forced to migrate. Without a versioning policy, the vulnerable `/v1/` would have to be patched in place, which may not be possible for design-level flaws. The trade-off is that supporting two versions doubles the attack surface during the deprecation window — mitigated by `/v1/` being a thin compatibility shim over `/v2/` (not a separate implementation).

PERFORMANCE IMPACT

Versioning has negligible performance impact. The `/api/v1/` and `/api/v2/` paths route to the same controller methods (Spring MVC path matching), with a version-specific response transformer. The transformer adds $\sim 1\text{ms}$ per response (field renaming, field filtering) — well within latency budget. Module versioning has zero runtime impact (versions are build-time metadata). Database migration versioning (Flyway timestamps) has zero runtime impact (Flyway runs only at deploy time). The only performance consideration is the deprecation window: supporting `/v1/` and `/v2/` simultaneously doubles the API surface area, but the implementation is shared (not duplicated), so the cost is the transformer's $\sim 1\text{ms}$, not a full second implementation.

SCALABILITY ANALYSIS

Versioning policy scales with the number of supported versions. The policy caps parallel support at two major versions (current + deprecated), which is manageable. Each deprecated version is a thin compatibility shim (response transformer) over the current implementation, so the maintenance cost is proportional to the API surface area, not to the number of versions. The 6-month deprecation window ensures that at most one deprecated version exists at any time (a new major version is released at most every 6 months in practice, because breaking changes are rare and require ADRs). Module versioning scales with the number of modules (12 currently, growing to ~ 20 over 5 years) — each module has its own version in `build.gradle`, with no cross-module version coupling.

OPERATIONAL CONSIDERATIONS

Versioning is operated through CI and the ADR workflow. CI validates: module version bumps match public API changes (japicmp), API spec changes within a major version are backward-compatible (openapi-diff), Flyway migrations are sequentially valid (Flyway validate). The ADR workflow gates breaking changes: a PR that introduces a breaking change must reference an ADR (accepted or proposed) that justifies the break; without the ADR reference, the build fails. Deprecation is operated by the API gateway (Spring Cloud Gateway or a custom filter) which adds Deprecation and Sunset headers to responses from deprecated versions. The sunset date is tracked in a deprecation registry (a YAML file in the repo) that CI reads to validate sunset timelines.

RISKS

Risk	Likelihood	Impact	Mitigation
API consumers do not migrate within 6-month window	Medium	Medium	Automated reminders at 6/3/1 month and 1 week; in-product banner; for enterprise consumers, dedicated migration support; sunset extension possible via ADR for critical consumers
Breaking change slips through without major version bump	Low	High	openapi-diff in CI catches spec-level breaks; japicmp catches code-level breaks; ARB reviews all major version bumps
Module version drift (modules versioned inconsistently)	Medium	Low	Quarterly module version audit; CI reports version skew across modules; Dependabot-style automated bump PRs
Flyway migration conflict (two engineers, same timestamp)	Low	Medium	Timestamp precision to seconds makes conflict unlikely; Flyway validate catches conflicts pre-deploy; merge-time CI check on migration files

Risk	Likelihood	Impact	Mitigation
Forward-only migrations trap: cannot undo a bad migration	Low	High	Two-phase migration pattern for risky changes; blue-green deploy (ADR-018) rolls back code; data fix-forward via new migration to correct bad data

TRADE - OFFS

We Gain	We Lose
Calendar versioning communicates release timing to business	Calendar versioning does not convey breaking-change semantics for releases
Semver for modules gives engineers semantic meaning	Engineers must learn and apply semver rules (training required)
URL-based API versioning is discoverable and testable	URL-based versioning duplicates endpoint definitions (v1 and v2 both exist)
Forward-only migrations simplify migration surface	Cannot undo a bad migration (must fix-forward or rollback code via blue-green)
6-month deprecation window gives consumers runway	Two versions supported in parallel doubles API surface during deprecation

REJECTED ALTERNATIVES

Pure semver was rejected because releases need to communicate timing to business stakeholders ('Q3 2025 release' is meaningful, 'v4.2.1' is not). Pure CalVer was rejected because APIs need to convey breaking-change semantics to consumers (a consumer seeing `/api/2026.3/` cannot tell if it has breaking changes vs `/api/2026.2/`). Header-based API versioning was rejected because it is not discoverable (the version is hidden in a header), not testable with curl or a browser, and interacts poorly with HTTP caching. No-versioning (continuous deployment) was rejected because PreOne has API consumers (schools, parents, third-party integrators) who need version stability; without versioning, breaking changes are unannounced and cause incidents. The hybrid scheme is slightly more complex than a single scheme, but each artefact type uses the scheme best suited to its consumers.

MIGRATION PLAN

The migration to the hybrid versioning policy was completed in Q4 2025 over 4 weeks. Phase 1 (week 1): adopt CalVer for releases — the next release was branded 2025.4 (Q4 2025); previous ad-hoc names were retired. Phase 2 (week 2): adopt semver for modules — each Gradle subproject's build.gradle was updated with a version field starting at 1.0.0; japicmp was added to CI. Phase 3 (week 3): adopt URL-based API versioning — existing /v1/ endpoints were frozen (no breaking changes allowed); future breaking changes require /v2/. Phase 4 (week 4): adopt Flyway timestamp versioning — existing V1..V47 migrations were renamed to V<original-date>__<description>.sql; Flyway's repair command was run to update checksums. v3.0 adds the formal deprecation registry (YAML file) and the automated reminder emails.

TESTING STRATEGY

Versioning is tested at three levels. (1) CI validation: japicmp compares module public APIs against the previous version, classifying changes as breaking/additive/patch; the build fails if the version bump does not match the change classification. openapi-diff compares API spec files, failing on breaking changes within a major version. Flyway validate verifies migration checksums match the schema history table. (2) Contract tests (ADR-149): Pact consumer tests verify that consumers can call /v1/ and /v2/ and get expected responses; provider tests verify that the API honours pacts for both versions. (3) Deprecation tests: a test verifies that deprecated /v1/ responses include Deprecation and Sunset headers with correct values; a test verifies that sunset /v1/ returns 410 Gone. Coverage target: 100% of public API changes have a version-bump verification.

MONITORING & OBSERVABILITY

Versioning is monitored via the API gateway, which logs the version of every request (/v1/ vs /v2/). Metrics: (a) request rate per API version (target: /v1/ request rate trends to 0 as sunset approaches); (b) consumer count per API version (target: <5 consumers on deprecated version at sunset); (c) deprecation header emission rate (target: 100% of deprecated-version responses include headers); (d) sunset 410 response rate (target: 0 before sunset date, 100% after). Alerts fire on: deprecated version request rate not declining (consumers not migrating), deprecated version error rate > current version (compatibility shim bug), sunset date approaching with >5 active consumers (migration at risk). Weekly report to engineering leadership on API version health.

FUTURE EVOLUTION

The hybrid versioning policy is stable; no supersession expected within 5 years. The likely evolution is tooling: an automated migration assistant that helps API

consumers migrate from /v1/ to /v2/ (e.g., a tool that diffs the two specs and generates a migration guide). Another likely evolution is finer-grained API versioning (e.g., /api/v2.1/ for minor additions within v2) if the major-version granularity proves too coarse — but this is not currently needed and would be a v2.0 of this ADR. The 6-month deprecation window may shorten to 3 months if consumers migrate faster than expected; this would be a minor policy adjustment, not a supersession. The forward-only migration policy is unlikely to change — it is the industry standard for SaaS platforms.

RELATED ADRS

- ADR-001 — Enterprise Architecture Principles (complements P7 Contracts Before Code)
- ADR-002 — Modular Monolith Strategy (complements — single deployable, module versions are build-time)
- ADR-016 — Feature Flags (complements — decouples deploy from release, complements versioning)
- ADR-017 — Build Pipeline (complements — CI validates version bumps)
- ADR-018 — Deployment Model (complements — backward-compatible migrations required for blue-green)
- ADR-020 — Technology Stack Freeze (complements — locked stack reduces version churn)
- ADR-091 — REST API Standard (refines — URL-based versioning for REST)
- ADR-092 — API Versioning (refines — detailed API versioning rules)

REFERENCES

- Upstream DDD: DDD-019-section-1 (module versioning for bounded contexts)
- Upstream PRD: PRD-019-section-2.1 (release cadence and API consumer requirements)
- Downstream ERD: ERD-019 (Flyway migration naming and forward-only policy)
- Downstream API Spec: API-019 (per-version OpenAPI spec files)
- Downstream Test Cases: TC-0901..TC-1000 (version bump and deprecation tests)
- External: Semantic Versioning 2.0.0 — <https://semver.org>
- External: Calendar Versioning — <https://calver.org>
- External: Flyway Documentation — <https://flywaydb.org/documentation/>

DECISION HISTORY

Date	Status	Actor	Notes
2025-08-12	Draft	Chief Architect	Initial draft; evaluated semver vs CalVer vs hybrid
2025-09-15	Proposed	Chief Architect	Submitted to ARB with hybrid scheme and deprecation policy
2025-10-15	Accepted	ARB Chair	ARB approved; v1.0 released; migration scheduled Q4 2025
2026-07-12	Accepted	ARB Chair	v3.0 refresh; added deprecation registry and automated reminders

APPROVAL & SIGN-OFF

Architect	Chief Architect (AR)
Tech Lead	Foundation Tech Lead
ARB Chair	ARB Chair
Approved On	2026-07-12

IMPLEMENTATION CHECKLIST

- Adopt CalVer for releases (Done Q4 2025, first release 2025.4)
- Add semver version field to all Gradle subprojects (Done Q4 2025)
- Add japicmp to CI for module API diff validation (Done Q4 2025)
- Add openapi-diff to CI for API spec backward-compatibility check (Done Q4 2025)
- Rename Flyway migrations to timestamp-based versioning (Done Q4 2025)
- Add Deprecation/Sunset headers to deprecated API responses (Done Q1 2026)
- Create deprecation registry YAML and automated reminder emails (Done Q1 2026)

- Quarterly module version skew audit at ARB (Ongoing)

ADR-020

Technology Stack Freeze

Volume 1 — Architecture Foundation - Standards

ACCEPTED

DECISION SUMMARY

DECISION

Lock the PreOne technology stack for 12 months (until 2027-Q3). Locked choices: Java 21 LTS, Spring Boot 3.2.x, PostgreSQL 16.x, Redis 7.x, React 18.x, TypeScript 5.x, Node 20 LTS (builds), AWS (ap-south-1 Mumbai), ECS Fargate, ALB, RDS, ElastiCache, S3, SES, SNS, CloudWatch, GitHub Actions, Terraform. Minor version upgrades allowed; major version upgrades require a superseding ADR. New technology adoption requires ADR and ARB approval. Rationale: stability over novelty, familiarity over best-of-breed, simplicity over optimisation.

STATUS

Status	Accepted
Date Decided	2025-10-15
Decision Owner	Chief Architect
Review Cadence	Annual or on trigger from a superseding ADR
Supersedes	None (v1.0 original; v3.0 refreshes locked versions for series alignment)

CONTEXT

Before v1.0, PreOne's technology stack was a graveyard of 'best-of-breed' choices that individual engineers had advocated for and then abandoned. The codebase had: Java 17 (with some modules on 21 because an engineer wanted records), PostgreSQL 14 (with one team experimenting with 16 beta), Redis 6 (because the DevOps engineer was wary of 7's breaking changes), a React 17

frontend with parts migrated to 18, three different logging frameworks (SLF4J, Log4j2, Logback) because each team had a preference, two CI tools (Jenkins and GitHub Actions) because the migration was never completed, and four different HTTP clients (OkHttp, Apache HttpClient, Spring WebClient, Java 11 HttpClient) because engineers used whatever they knew. The cost of this fragmentation was severe. Security patches required touching four HTTP clients instead of one. Onboarding required learning three logging frameworks. Builds were slow because each Gradle subproject could pin different dependency versions. Bug fixes had to be backported to multiple versions of the same library. The DPDP Act 2023 compliance audit flagged the stack fragmentation as a finding: regulators expect a documented, controlled technology stack with clear ownership and patching cadence. The Office of the Chief Architect proposed a 12-month technology stack freeze. The freeze is not 'never change anything' — it is 'change only via ADR, with ARB approval, and only when the change is justified by a concrete need, not by novelty or preference'. The freeze covers: programming languages, frameworks, databases, cloud services, CI/CD tools, IaC tools, and observability tools. The freeze does not cover: application libraries within a framework (e.g., adding a new Spring Boot starter is allowed without ADR if it does not introduce a new framework). The 12-month duration is calibrated to give the team stability to ship features without constant tool churn, while being short enough that the stack does not become obsolete (a 24-month freeze would risk falling behind on security patches and language features).

BUSINESS DRIVERS

The primary driver is engineering velocity: a frozen stack means engineers solve business problems, not tooling problems. No time spent evaluating 'should we use Vue or React?' — the answer is React, end of debate. No time spent onboarding to a new framework because a teammate introduced it — the team's expertise compounds on the frozen stack. The secondary driver is security and compliance: a frozen stack has a clear patching cadence (Java quarterly CPU, Spring Boot monthly, PostgreSQL minor releases) and a clear owner (DevOps for infra, Foundation Tech Lead for languages and frameworks). The DPDP Act audit finding is resolved by the documented freeze. The tertiary driver is operational simplicity: one set of runbooks, one set of monitoring dashboards, one set of on-call procedures — not a different set per technology.

PROBLEM STATEMENT

PreOne's technology stack is fragmented across multiple versions of languages, frameworks, databases, and tools, causing engineering velocity loss, security patching burden, compliance audit findings, and operational complexity. We

need a frozen, documented, controlled technology stack with a clear upgrade policy.

CONSTRAINTS

- Stack must be supported for 12 months (until 2027-Q3) without forced upgrades
- Major version upgrades require a superseding ADR (e.g., PostgreSQL 16 to 17)
- New technology adoption requires an ADR and ARB approval
- All choices must be available in AWS ap-south-1 (Mumbai) region for data residency
- All choices must have LTS (long-term support) versions, not bleeding-edge
- Stack must be operable by a 2-engineer DevOps team (no exotic tooling)
- Stack must comply with P5 Simplicity (ADR-001) — minimise distinct technologies

ASSUMPTIONS

- AWS ap-south-1 will continue to support all chosen services for 12 months
- LTS versions of chosen technologies will receive security patches for 12 months
- The engineering team's expertise in the chosen stack will remain (no mass attrition)
- No business requirement will emerge that the frozen stack cannot meet (if so, superseding ADR)
- Open-source dependencies remain available (no critical library is abandoned)
- AWS pricing remains stable (no 2x price increase that would force a change)

OPTIONS CONSIDERED

Option	Pros	Cons	Verdict
12-month freeze with ADR-gated changes (chosen)	Stability for 12 months; engineers focus on business problems; clear upgrade path via ADR; compliance-friendly; operational simplicity.	Cannot adopt new technology quickly; may miss security patches if LTS version is end-of-life within 12 months; risk of stack becoming stale.	Chosen
No freeze (engineers choose per-project)	Maximum flexibility; engineers use best tool for each job; innovation not blocked.	Fragmentation (the pre-v1.0 state); security patching burden; onboarding cost; compliance audit failure; operational complexity.	Rejected (this is the current state being fixed)
24-month freeze (longer stability)	Even more stability; less ADR overhead for upgrades.	Risk of falling behind on security patches and language features; too inflexible for a 5-year-old startup; LTS versions may EOL within 24 months.	Rejected
Rolling freeze (technologies reviewed quarterly, upgraded if justified)	Balance of stability and currency; quarterly review cadence.	Quarterly review is overhead; 'justified' is subjective, leading to debate; does not meet the 'stability' bar that the business wants.	Rejected
Best-of-breed (each layer uses the absolute best tool, even if fragmented)	Theoretically optimal performance and features at each layer.	Fragmentation cost (pre-v1.0 state); operational complexity; team expertise cannot compound; violates P5 Simplicity.	Rejected

DECISION

ADOPTED

We freeze the PreOne technology stack for 12 months (until 2027-Q3) with the following locked choices. Languages and runtimes: Java 21 (LTS, backend), TypeScript 5.x (frontend), Node 20 LTS (builds only). Frameworks: Spring Boot 3.2.x (backend), React 18.x (frontend). Data stores: PostgreSQL 16.x (primary, via RDS), Redis 7.x (cache, via ElastiCache). Cloud: AWS, region ap-south-1 (Mumbai). Compute: ECS Fargate (container orchestration). Networking: ALB (load balancer), Route 53 (DNS), CloudFront (CDN). Storage: S3 (object storage), ECR (container images). Messaging: SES (email), SNS (push notifications). Observability: CloudWatch (infra metrics, logs), Prometheus (app metrics), Grafana (dashboards), OpenTelemetry (traces). CI/CD: GitHub Actions. IaC: Terraform. Upgrade policy: minor versions allowed (e.g., PostgreSQL 16.1 to 16.4) without ADR; major versions (e.g., PostgreSQL 16 to 17) require a superseding ADR. New technology adoption requires an ADR and ARB approval. Rationale: stability over novelty; team familiarity over best-of-breed; operational simplicity over optimisation.

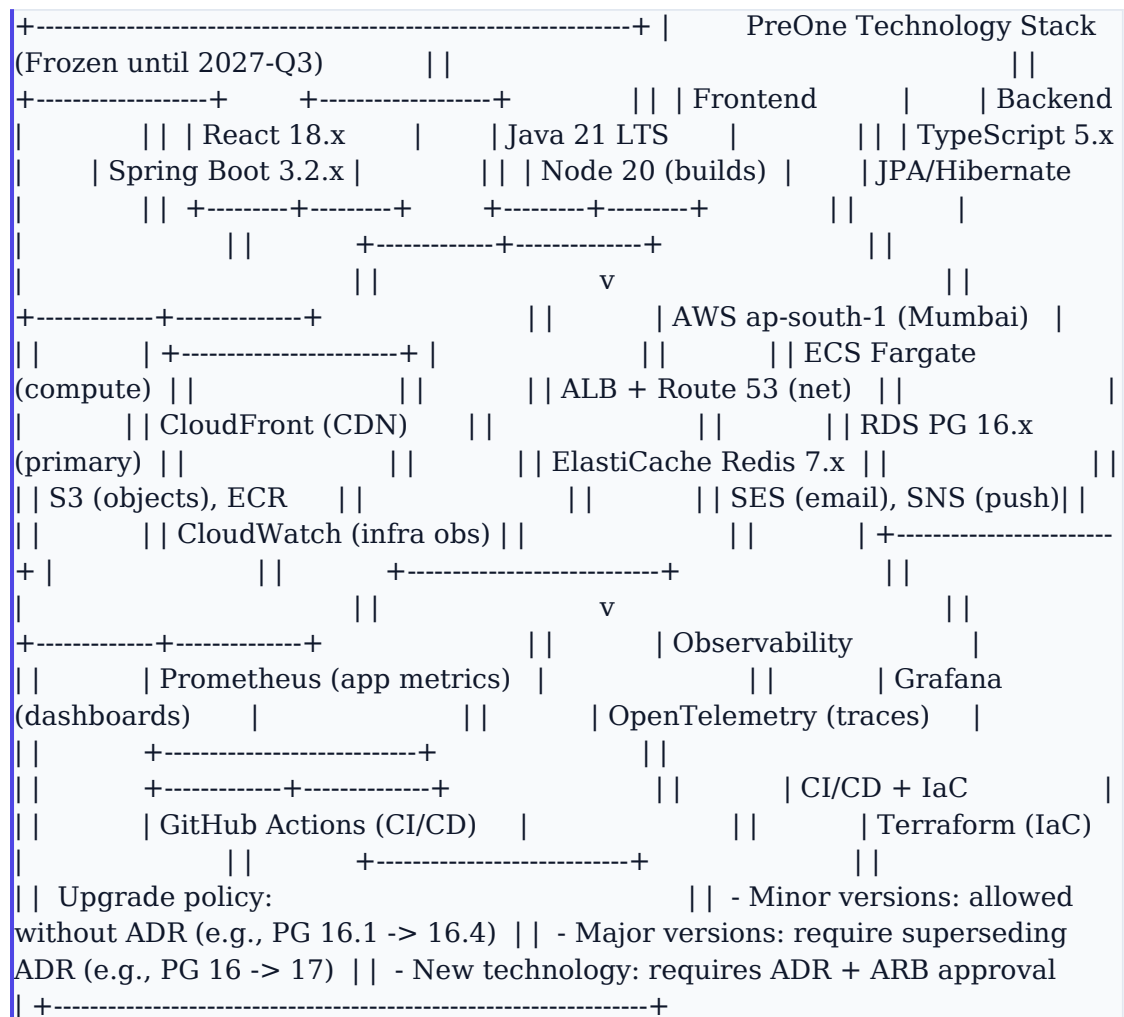
DETAILED RATIONALE

The 12-month freeze was chosen as the balance between stability and flexibility. A shorter freeze (6 months) would not give the team enough stability to compound expertise — engineers would spend too much time evaluating upgrades. A longer freeze (24 months) risks falling behind on security patches (LTS versions have 18-month support cycles for some technologies) and language features (Java releases every 6 months; a 24-month freeze would skip 4 releases). 12 months gives the team 4 Java releases worth of stability (they adopt the LTS, skip the non-LTS) and aligns with the annual review cadence. Java 21 LTS was chosen as the backend language because: (a) it is the current LTS (released September 2023, supported until September 2028 for LTS patches); (b) it introduces records, sealed classes, and pattern matching, which simplify DDD tactical patterns (value objects as records, sealed aggregates); (c) the team has deep Java expertise (90% of engineers are Java-primary); (d) Spring Boot 3.x requires Java 17+, so 21 is the natural choice. Java 25 (next LTS, September 2025) was considered but rejected because the team has not yet validated Spring Boot compatibility, and the freeze is about stability, not chasing the latest LTS. Spring Boot 3.2.x was chosen because: (a) it is the current stable version with Java 21 support; (b) it provides native compilation (GraalVM) which we may use for startup-time-sensitive services; (c) it has first-class observability (Micrometer, OpenTelemetry) which aligns with P4

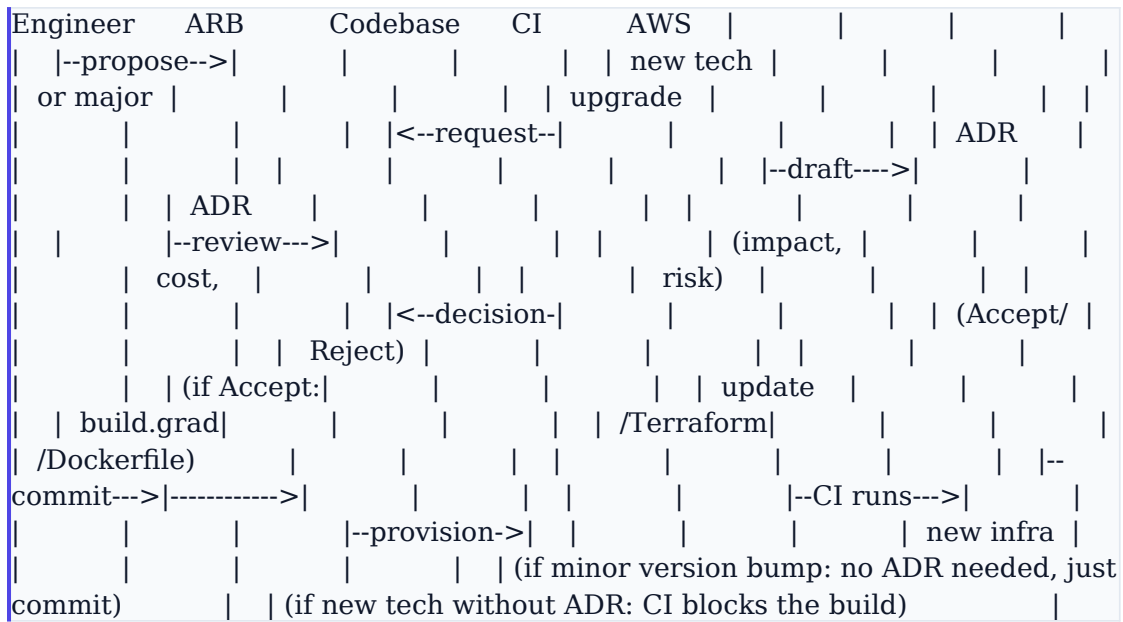
(Observability); (d) the team's expertise is deep (every engineer has shipped Spring Boot in production). Alternatives considered: Quarkus (rejected — team has no expertise, marginal performance benefit at our scale), Micronaut (rejected — same), Helidon (rejected — too niche). Spring Boot is not the 'best' framework in absolute terms — it is the best fit for our constraints (team expertise, ecosystem, stability). PostgreSQL 16.x was chosen as the primary database because: (a) it is the current stable version (released September 2023, supported for 5 years); (b) it has improved performance for parallel queries and logical replication; (c) RDS supports it with automated backups and multi-AZ; (d) the team has deep PostgreSQL expertise. Alternatives considered: MySQL (rejected — weaker concurrency, less feature-rich), MongoDB (rejected — PreOne's data is relational, not document-oriented), DynamoDB (rejected — would require complete data model redesign, vendor lock-in). PostgreSQL is the relational database of choice for SaaS platforms. Redis 7.x was chosen as the cache because: (a) it is the current stable version with improved ACLs and function support; (b) ElastiCache supports it with managed failover; (c) the team uses it for session storage, rate limiting, and feature flag caching. Alternatives considered: Memcached (rejected — no persistence, no advanced data structures), Hazelcast (rejected — overkill for our cache use case), in-process cache (rejected — does not work across replicas). React 18.x was chosen as the frontend framework because: (a) it is the current stable version with concurrent rendering and server components; (b) the team has deep React expertise; (c) the ecosystem (Next.js, React Router, React Query) is mature; (d) TypeScript 5.x support is first-class. Alternatives considered: Vue 3 (rejected — smaller ecosystem, less team expertise), Angular (rejected — heavier, more opinionated, less team expertise), Svelte (rejected — too niche, smaller hiring pool). React is not the 'best' frontend framework — it is the best fit for our constraints. AWS was chosen as the cloud provider because: (a) the team has deep AWS expertise; (b) ap-south-1 (Mumbai) region provides data residency for India (DPDP Act compliance); (c) the service catalog (RDS, ElastiCache, ECS, ALB, S3, SES, SNS, CloudWatch) covers all our needs; (d) the pricing is competitive at our scale. Alternatives considered: Azure (rejected — smaller team expertise, weaker India region), GCP (rejected — smaller team expertise, weaker India region presence at the time of decision), on-premise (rejected — capital expense, operational burden). ECS Fargate was chosen over EKS (Kubernetes) because: (a) Fargate is serverless — no EC2 instances to patch, no K8s control plane to operate; (b) the team's DevOps capacity (2 engineers) cannot sustain production-grade K8s; (c) the modular monolith is a single container, not a complex multi-service topology that would benefit from K8s. The cost premium (~20% over EC2) is justified by the operational saving. EKS would be reconsidered if PreOne extracts modules into separate services (ADR-002 future evolution), at which point K8s' service mesh and orchestration

would justify the operational cost. GitHub Actions was chosen as CI/CD per ADR-017. Terraform was chosen as IaC because: (a) it is the industry standard for AWS IaC; (b) the team has Terraform expertise; (c) the AWS provider is mature; (d) it supports drift detection and state management. Alternatives considered: CloudFormation (rejected — less expressive, AWS-only), Pulumi (rejected — team has no expertise, marginal benefit), CDK (rejected — same). The upgrade policy (minor versions allowed, major versions require ADR) balances stability with security. Minor versions (e.g., PostgreSQL 16.1 to 16.4) are backward-compatible and include security patches — adopting them without ADR overhead keeps the stack secure. Major versions (e.g., PostgreSQL 16 to 17) may include breaking changes and require careful testing — the ADR process ensures due diligence. New technology adoption (e.g., adding Kafka for event streaming) requires an ADR that justifies the addition, documents the operational impact, and is approved by the ARB.

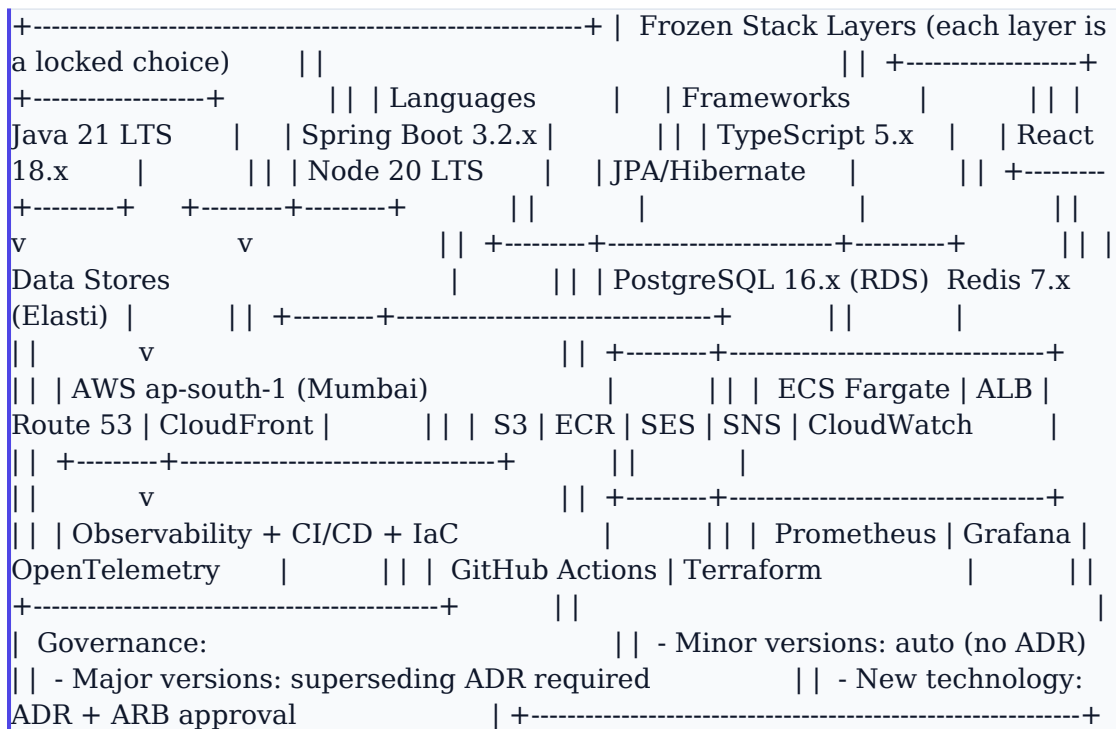
ARCHITECTURE DIAGRAM



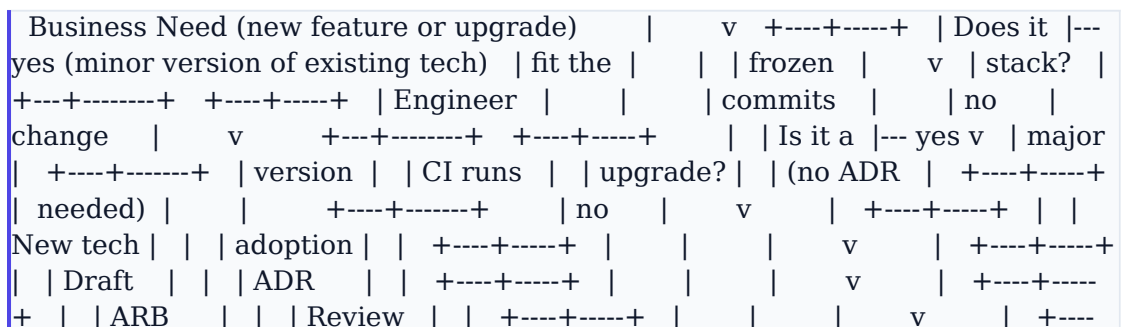
SEQUENCE DIAGRAM



COMPONENT DIAGRAM



DATA FLOW DIAGRAM



```

+-----+ | | Accept? | | | yes --> |---+----> update
build.gradle/Terraform/Dockerfile | no --> | | +-----+ | v
+----+-----+ | CI runs | | | (builds | | |
with | | new | | | tech) | | | +-----+
+

```

DATABASE IMPACT

PostgreSQL 16.x is the locked database version, accessed via RDS (managed PostgreSQL). Minor version upgrades (16.1 to 16.4) are applied automatically by RDS during the maintenance window, without ADR — these are backward-compatible and include security patches. Major version upgrades (16 to 17) require a superseding ADR that documents: the new features required, the migration plan (RDS major version upgrade is a multi-step process with potential downtime), the rollback plan, and the testing strategy. The freeze ensures the database layer is stable: application code can assume PostgreSQL 16 features (e.g., SQL/JSON path expressions, improved parallel queries) without conditional logic for older versions.

API IMPACT

The frozen stack influences the API implementation: Spring Boot 3.2.x provides the REST framework (Spring MVC), the OpenAPI spec generator (springdoc-openapi), and the contract testing framework (Spring Cloud Contract). API consumers are not directly affected by the stack freeze — the API is a contract, not an implementation. However, the freeze ensures the API implementation is stable: a Spring Boot 3.2 to 3.3 minor upgrade is allowed without ADR (backward-compatible), but a Spring Boot 3 to 4 major upgrade would require a superseding ADR and could affect API behaviour (e.g., default serialisation changes). The freeze protects API consumers from implementation churn.

UI IMPACT

React 18.x and TypeScript 5.x are the locked frontend stack. Minor version upgrades (React 18.2 to 18.3, TypeScript 5.4 to 5.5) are allowed without ADR — these are backward-compatible. Major upgrades (React 18 to 19, TypeScript 5 to 6) require a superseding ADR. The freeze ensures the frontend toolchain (Vite, ESLint, Jest, React Testing Library) is stable — engineers do not need to relearn the toolchain every quarter. The UI build process (Node 20 LTS, npm) is also frozen, ensuring reproducible builds (ADR-017).

SECURITY IMPACT

The stack freeze improves security by enabling a clear patching cadence. Java 21 LTS receives quarterly Critical Patch Updates (CPU) from Oracle (and free updates from OpenJDK distributions like Amazon Corretto, which we use). Spring Boot 3.2.x receives monthly patch releases. PostgreSQL 16.x receives

quarterly minor releases. Redis 7.x receives periodic patch releases. The DevOps team applies these patches within 30 days of release (15 days for critical CVEs) without ADR — they are minor version upgrades, allowed by the freeze policy. The freeze does not block security patches; it blocks unsanctioned technology changes. The DPDP Act compliance audit finding ('undocumented technology stack') is resolved by this ADR.

PERFORMANCE IMPACT

The frozen stack has known performance characteristics, benchmarked at v1.0: Spring Boot 3.2 on Java 21 sustains ~15,000 RPS per replica (I/O-bound), PostgreSQL 16 handles ~50,000 writes/second on a db.r6g.large instance, Redis 7 handles ~100,000 ops/second on a cache.m6g.large. These numbers are stable because the stack is frozen — performance regressions are caught in CI (Lighthouse for frontend, JMeter for backend) and attributed to application code, not to stack upgrades. A non-frozen stack would have confounding variables (did latency regress because of our code or because of a Spring Boot upgrade?). The freeze eliminates this ambiguity.

SCALABILITY ANALYSIS

The frozen stack scales horizontally within AWS. ECS Fargate task count scales via auto-scaling policies (CPU/memory based). RDS PostgreSQL scales via read replicas (up to 15) and vertical instance sizing (up to db.r6g.24xlarge). ElastiCache Redis scales via cluster mode (sharding) and read replicas. S3 and CloudFront scale automatically. The ceiling is the modular monolith's single-write-primary database: at ~50,000 writes/second (PostgreSQL 16 on a large instance), PreOne will not approach this in the 5-year horizon. If the ceiling is reached, the path is module extraction (ADR-002 future evolution), not a stack change — the same stack scales to distributed deployments. The freeze does not constrain scaling; it constrains technology churn.

OPERATIONAL CONSIDERATIONS

The frozen stack is operated by a 2-engineer DevOps team. Each technology in the stack has a documented runbook (onboarding, scaling, backup, restore, incident response). The team's expertise compounds: an engineer who debugged a PostgreSQL 16 issue last quarter can apply that knowledge this quarter (because the version is unchanged). With a non-frozen stack, knowledge depreciates as versions change. Patching is the primary operational cadence: monthly patch review (which minor versions are available, which CVEs are addressed), quarterly patch application (apply during maintenance window), annual major-version review (are any major upgrades warranted?). The freeze makes this cadence predictable.

RISKS

Risk	Likelihood	Impact	Mitigation
LTS version reaches end-of-life within freeze window	Low	High	All chosen versions have LTS support beyond 2027-Q3 (Java 21 until 2028, PostgreSQL 16 until 2028, Redis 7 until 2026+); monitor EOL dates quarterly; if EOL approaches, trigger superseding ADR
Security CVE requires immediate stack change	Low	High	Minor version upgrades (which include security patches) are allowed without ADR; for critical CVEs in a frozen version, emergency ADR fast-tracked through ARB within 24 hours
Business requirement emerges that frozen stack cannot meet	Medium	Medium	Superseding ADR process allows stack changes when justified; ARB fast-track for urgent requirements; freeze is not 'never change', it is 'change via ADR'
Team expertise in frozen stack atrophies as engineers leave	Medium	Medium	Onboarding includes stack training; documentation in runbooks; quarterly internal tech talks on stack features; hiring filters for stack expertise

Risk	Likelihood	Impact	Mitigation
Stack becomes stale relative to industry (competitive disadvantage)	Low	Low	Annual review of stack vs industry; if a competing platform gains significant advantage from a new technology, superseding ADR is the path to adopt it

TRADE - OFFS

We Gain	We Lose
12 months of stability (no tool churn)	Cannot adopt new technology quickly (must wait for ADR)
Team expertise compounds on the frozen stack	Engineers who want to use new tech may be frustrated (mitigated by ADR path)
Clear patching cadence (security compliance)	Minor version patches required even if no business need (operational overhead)
Operational simplicity (one set of runbooks)	Cannot use best-of-breed tool for each layer (some layers may be suboptimal)
Reproducible builds and stable performance	Miss out on performance improvements from newer versions (mitigated by minor version upgrades)

REJECTED ALTERNATIVES

No freeze (the pre-v1.0 state) was rejected because the fragmentation cost was severe: security patching burden, onboarding cost, compliance audit failure, operational complexity. 24-month freeze was rejected because LTS versions for some technologies (Redis 7) have shorter support cycles, risking EOL within the freeze window. Rolling freeze (quarterly review) was rejected because it does not meet the 'stability' bar — quarterly reviews would lead to quarterly debates about whether to upgrade, consuming engineering time. Best-of-breed was rejected because it directly contradicts P5 (Simplicity) and P9 (Automate) — each new tool multiplies the operational and automation burden. The 12-month freeze with ADR-gated changes is the right balance: long enough to

compound expertise, short enough to avoid obsolescence, with a clear path (ADR) for justified changes.

MIGRATION PLAN

The migration to the frozen stack was completed in Q4 2025 over 10 weeks. Phase 1 (week 1-2): upgrade Java from 17 (some modules) to 21 (all modules) — required updating Gradle config and fixing a few reflection issues with records. Phase 2 (week 3-4): upgrade PostgreSQL from 14 to 16 — RDS major version upgrade with downtime during maintenance window; Flyway migrations re-validated. Phase 3 (week 5-6): upgrade Redis from 6 to 7 — ElastiCache engine upgrade with blue-green cache migration. Phase 4 (week 7-8): consolidate HTTP clients to Spring WebClient (removed OkHttp, Apache HttpClient, Java 11 HttpClient). Phase 5 (week 9-10): consolidate logging to SLF4J + Logback (removed Log4j2 direct usage). v3.0 adds the formal upgrade policy (minor versions allowed, major versions require ADR) which was implicit in v1.0.

TESTING STRATEGY

The frozen stack is tested via CI gatekeeping. A PR that introduces a new technology (e.g., a new Maven dependency that is not in the approved list) is blocked by a CI check (Gradle dependency analysis against an allowlist). A PR that upgrades a major version (e.g., Spring Boot 3.2 to 3.3 — but 3.3 is not yet in the freeze) is blocked until a superseding ADR is referenced. Minor version upgrades (e.g., Spring Boot 3.2.0 to 3.2.1) pass through. The allowlist is maintained in a YAML file in the repo, and changes to the allowlist require a PR (which itself triggers ARB review). Compatibility tests: every minor version upgrade runs the full test suite (unit, integration, contract, ArchUnit) to verify backward compatibility. Performance tests: quarterly benchmarking of the frozen stack to detect performance drift.

MONITORING & OBSERVABILITY

The frozen stack is monitored for security and currency. Security: weekly scan of all dependencies (Trivy, Snyk) for CVEs; critical CVEs trigger an emergency patch within 24 hours. Currency: quarterly review of LTS support dates for each technology (Java 21 EOL, PostgreSQL 16 EOL, Redis 7 EOL); if any EOL is within 12 months, trigger a superseding ADR for the upgrade. Adoption: track which technologies in the freeze are actually used (e.g., if SES is in the freeze but never used, it can be removed from the freeze in the next review). Stack health dashboard in Grafana shows: per-technology version, days since last patch, days to EOL, CVE count, and usage metrics. Alerts fire on: CVE with CVSS > 7 on a frozen technology, EOL within 12 months, unused technology in freeze (cleanup candidate).

FUTURE EVOLUTION

The technology stack freeze is reviewed annually (next review: 2026-10). The likely evolution at the 2027-Q3 expiry is a v2.0 of this ADR that re-freezes for another 12 months, with possibly one or two major version upgrades justified by superseding ADRs during the freeze window. The most likely upgrade candidates are: Java 21 to 25 (next LTS, September 2025 — but only if Spring Boot 3.x supports it well by 2027), PostgreSQL 16 to 17 (if 17 offers significant features we need), React 18 to 19 (if 19 stabilises and offers features we need). New technology adoption candidates (each would require an ADR): Kafka for event streaming (if we extract modules into separate services), Elasticsearch for full-text search (if PostgreSQL's full-text search proves insufficient), a service mesh like Istio (if we move to EKS for multi-service topology). None of these are planned for the freeze window; they are candidates for the post-2027-Q3 review.

RELATED ADRS

- ADR-001 — Enterprise Architecture Principles (complements P5 Simplicity, P9 Automate)
- ADR-002 — Modular Monolith Strategy (complements — single deployable aligns with simple stack)
- ADR-017 — Build Pipeline (complements — CI validates stack allowlist)
- ADR-018 — Deployment Model (complements — ECS Fargate, ALB, ECR are locked here)
- ADR-019 — Versioning Policy (complements — locked stack reduces version churn)
- ADR-041 — Database Strategy (refines — PostgreSQL 16.x specifics)
- ADR-073 — Caching Strategy (refines — Redis 7.x specifics)
- ADR-131 — Frontend Architecture (refines — React 18.x, TypeScript 5.x specifics)

REFERENCES

- Upstream DDD: DDD-020-section-1 (stack assumptions for DDD tactical patterns)
- Upstream PRD: PRD-020-section-2.1 (platform technology requirements)
- Downstream ERD: ERD-020 (PostgreSQL 16 schema features used)
- Downstream API Spec: API-020 (Spring Boot 3.2 REST framework specifics)
- Downstream Test Cases: TC-1001..TC-1100 (stack compatibility and patch verification tests)
- External: AWS Well-Architected Framework — technology choices

- External: Java 21 LTS documentation — <https://docs.oracle.com/en/java/javase/21>
- External: ThoughtWorks Technology Radar — <https://www.thoughtworks.com/radar>

DECISION HISTORY

Date	Status	Actor	Notes
2025-08-05	Draft	Chief Architect	Initial draft; evaluated 5 freeze durations and stack options
2025-09-12	Proposed	Chief Architect	Submitted to ARB with full stack evaluation and migration plan
2025-10-15	Accepted	ARB Chair	ARB approved; v1.0 released; migration scheduled Q4 2025
2026-07-12	Accepted	ARB Chair	v3.0 refresh; formalised upgrade policy and added stack health dashboard

APPROVAL & SIGN-OFF

Architect	Chief Architect (AR)
Tech Lead	Foundation Tech Lead
ARB Chair	ARB Chair
Approved On	2026-07-12

IMPLEMENTATION CHECKLIST

- Upgrade all modules to Java 21 (Done Q4 2025)
- Upgrade RDS PostgreSQL to 16.x (Done Q4 2025)
- Upgrade ElastiCache Redis to 7.x (Done Q4 2025)
- Consolidate HTTP clients to Spring WebClient (Done Q4 2025)
- Consolidate logging to SLF4J + Logback (Done Q4 2025)
- Add CI dependency allowlist check (Done Q1 2026)

- Create stack health dashboard in Grafana (Done Q1 2026)
- Quarterly LTS EOL and CVE review at ARB (Ongoing)